

Learning Rust, SparkConnect, Apache Arrow and DataFusion with Sail

A Guided Architecture Book for Distributed Query Processing and Extensions

Alexy Khrabrov (chiefscientist.org)
Codex ChatGPT 5.5 and Claude Opus 4.7

May 2026

Contents

1	Reader Guide: How This Book Builds	8
1.1	Chapter Links	9
1.2	Concept Progression	9
1.3	Code Reading Strategy	11
1.4	What To Look For In Code Excerpts	11
2	Chapter 1: Architecture Overview	12
2.1	The Big Pieces	13
2.2	Spark Connect Is the Front Door	14
2.3	pysail Starts the Rust Server	15
2.4	Sail Uses DataFusion as the Query Kernel	16
2.5	Local Execution Is Direct DataFusion Execution	17
2.6	Cluster Execution Adds a Driver, Workers, Stages, and Shuffles	17
2.7	Shuffle Is Arrow Data Movement	19
2.8	Functions Are Both Planning-Time and Execution-Time Concerns	19
2.9	Where Extensions Want to Plug In	19
2.10	A First Reading Path Through the Code	21
2.11	Chapter Takeaways	22
3	Chapter 2: Rust Foundations in Sail	22
3.1	The Rust Shape of Sail	23
3.2	Shared Ownership With Arc	24
3.3	Trait Objects: dyn Trait	25
3.4	Local and Cluster Runners Share One Interface	25
3.5	Send, Sync, and 'static	26
3.6	Async Traits	26
3.7	Actors: The Control Plane in Rust	27
3.8	Typed Session Extensions	28
3.9	Builders and Mutators	29
3.10	Downcasting Extension Nodes	30
3.11	Error Types	31
3.12	A Mini Example: What Happens to a Query Plan Type	31
3.13	How Rust Shapes the Extension Proposal	32
3.14	Reading Exercises	32
3.15	Chapter Takeaways	33
4	Chapter 3: Spark Connect	33
4.1	Definitive Spark Connect References	33
4.2	The Main Files	34
4.3	The gRPC Service Surface	35
4.4	Sessions: Spark State Inside DataFusion State	36
4.5	Relations Become Sail Specs	37
4.6	SQL Is Also Routed Through the Front Door	39
4.7	Commands	39

4.8	The Core Execution Path	41
4.9	Response Streams and Arrow Batches	42
4.10	Reattachable Operations	43
4.11	Interrupts	44
4.12	Analyze Plan	45
4.13	Config	46
4.14	Data Types and Spark Compatibility	47
4.15	Errors Become Spark Exceptions	47
4.16	Artifacts and Python Distribution	48
4.17	Registering Functions and Data Sources	48
4.18	What Spark Connect Means for Extensions	48
4.19	Reading Exercises	50
4.20	Chapter Takeaways	51
5	Chapter 4: PySpark and pysail	51
5.1	The Main Files	52
5.2	Package Shape	52
5.3	Starting Sail From Python	54
5.4	The Global Runtime	56
5.5	Releasing the GIL	57
5.6	Connecting With PySpark	57
5.7	pysail Is Not a PySpark Fork	58
5.8	Python UDFs Enter Through Spark Connect	58
5.9	UDF Kinds	59
5.10	Loading a PySpark UDF Payload	60
5.11	Executing Python UDFs in Process	60
5.12	Python Conversion Code	61
5.13	Config for Python UDF Behavior	62
5.14	UDTFs and Python-Side Analysis	62
5.15	Python Data Sources	63
5.16	Testing PySpark Compatibility	63
5.17	Why PySpark Versions Matter	64
5.18	What pysail Means for Extensions	64
5.19	A Small End-to-End Example	65
5.20	Reading Exercises	66
5.21	Chapter Takeaways	66
6	Chapter 5: Apache Arrow	67
6.1	Definitive References	67
6.2	Where Arrow Lives In Sail	67
6.3	The Arrow Mental Model	69
6.4	Why Arc Shows Up Everywhere	70
6.5	RecordBatch As The Unit Of Execution	72
6.6	Spark Connect Output: Arrow IPC In Protobuf Clothing	72
6.7	Schema Conversion: Arrow Back To Spark Types	74

6.8	Extension Types: GeoArrow And Variant	75
6.9	PyArrow: The Python Boundary	76
6.10	UDF Kinds And Arrow-Native Execution	77
6.11	Distributed Shuffle: Arrow As The Exchange Unit	78
6.12	Row Round-Robin Repartition	79
6.13	Flight SQL: Arrow On The Network	80
6.14	Arrow IPC Versus Flight	81
6.15	A Small End-To-End Example	82
6.16	What Arrow Does Not Solve By Itself	83
6.17	Extension Design Pattern: Arrow-First Semantics	84
6.18	Common Mistakes When Learning Arrow In Sail	84
6.19	Reading Exercise: Follow One Batch	84
6.20	Reading Exercise: Follow One Python UDF	85
6.21	How This Prepares Us For DataFusion	85
6.22	Takeaways	86
7	Chapter 6: Apache DataFusion	86
7.1	Definitive References	86
7.2	Where DataFusion Lives In Sail	87
7.3	The Critical Function: <code>resolve_and_execute_plan</code>	90
7.4	<code>SessionContext</code> , <code>SessionState</code> , And <code>SessionConfig</code>	91
7.5	Typed Session Extensions	92
7.6	Sail's Query Planner	93
7.7	Logical Extension Nodes	94
7.8	Physical Extension Planning	95
7.9	Physical Plans And <code>ExecutionPlan</code>	96
7.10	Plan Properties: Partitioning, Boundedness, Emission	97
7.11	Logical Optimizers	97
7.12	Physical Optimizers	98
7.13	Function Mapping: Spark Names To DataFusion Expressions	99
7.14	Vectorized UDFs	100
7.15	Table Functions And Providers	100
7.16	DataFusion As Local Engine, Sail As Distributed Engine	101
7.17	Spark Semantics On A DataFusion Kernel	102
7.18	Extension Implications	103
7.19	Reading Exercise: Trace <code>range</code>	104
7.20	Reading Exercise: Trace <code>repartition</code>	104
7.21	Reading Exercise: Trace A Spark Function	105
7.22	Takeaways	105
8	Chapter 7: From Physical Plan To Job Graph	105
8.1	The Core Files	106
8.2	Local Runner Versus Cluster Runner	107
8.3	What A Job Graph Represents	110
8.4	<code>StageInputExec</code> : A Placeholder, Not An Operator	111

8.5	Stage Inputs And Input Modes	112
8.6	Output Distribution	113
8.7	How JobGraph::try_new Starts	113
8.8	Rewrite 1: Global Limit Needs One Input Partition	116
8.9	Rewrite 2: Collect-Left Hash Joins And Unmatched Rows	116
8.10	Recursive Stage Splitting	116
8.11	Partition Usage: Single Versus Shared	118
8.12	create_shuffle	119
8.13	Merge And Rescale Boundaries	120
8.14	Driver Stages	121
8.15	Topological Order	122
8.16	Replicas And Repeated Consumption	122
8.17	From Job Graph To Job Topology	122
8.18	Region Dependencies	123
8.19	Task Definitions	124
8.20	Job Output	125
8.21	A Worked Example: Hash Aggregate	125
8.22	A Worked Example: Global Limit	126
8.23	A Worked Example: Broadcast-Like Shared Input	127
8.24	Why This Design Works	127
8.25	Extension Implications	128
8.26	Reading Exercise: Trace A Repartition Boundary	128
8.27	Reading Exercise: Trace Job Acceptance	128
8.28	Takeaways	129
9	Chapter 8: Drivers, Workers, Tasks, And Streams	129
9.1	The Core Files	130
9.2	The Actor Runtime	131
9.3	Driver Actor: The Coordinator	132
9.4	Worker Actor: The Executor	133
9.5	Worker Launch And Registration	134
9.6	Worker Health	135
9.7	From Job To Task Regions	136
9.8	Task Attempts	137
9.9	Task Regions And Cascading Cancellation	138
9.10	Task Assignment	138
9.11	Scaling Workers	139
9.12	Building A Task Definition	140
9.13	Dispatching Tasks	140
9.14	Running A Task On A Worker	141
9.15	Why The Task Monitor Drains The Stream	143
9.16	Stream Accessor: Actors As Readers And Writers	143
9.17	Stream Locations	144
9.18	Stream Manager	144

9.19	Memory Streams And Replicas	145
9.20	Arrow Flight For Task Streams	145
9.21	Peer Tracking	146
9.22	Cleanup And Stream Tracking	147
9.23	Job Output	147
9.24	One Query Lifecycle	148
9.25	Failure And Retry Story	149
9.26	Why This Design Fits Rust	150
9.27	Extension Implications	150
9.28	Reading Exercise: Follow A Task To A Worker	150
9.29	Reading Exercise: Follow A Stream Fetch	151
9.30	Reading Exercise: Follow Worker Loss	151
9.31	Takeaways	151
10	Chapter 9: Shuffle And Data Movement	152
10.1	Code Map	152
10.2	The Vocabulary Of Movement	153
10.3	From Job Graph To Task Definition	155
10.4	The Runtime Rewrite	156
10.5	Writing Shuffle Output	158
10.6	Reading Shuffle Input	160
10.7	Locations Become Streams	161
10.8	Local Memory Streams	162
10.9	Arrow Flight As The Remote Shape	162
10.10	A Hash Shuffle Walkthrough	163
10.11	Other Movement Patterns	165
10.12	Failure, Attempts, And Early Termination	166
10.13	What Is Still Open	166
10.14	Extension Hooks	167
10.15	Reading Exercise	167
10.16	Takeaways	168
11	Chapter 10: The Sail Spec And Plan Resolver	168
11.1	Code Map	169
11.2	Why Sail Has A Spec Layer	170
11.3	The Full Planning Pipeline	171
11.4	Spark Connect To Sail Spec	173
11.5	SQL To Sail Spec	174
11.6	The Resolver Entry Point	175
11.7	Resolver State	175
11.8	Query Resolution	177
11.9	Reading Tables And Data Sources	178
11.10	Project Resolution And Rewriters	179
11.11	Attribute Resolution	180
11.12	Function Resolution	181

11.13	Commands Become Logical Plans Too	182
11.14	Logical Extension Nodes	183
11.15	Repartition As A Bridge To Distributed Execution	184
11.16	WithRelations And Plan IDs	185
11.17	Commands In Spark Connect Execution	186
11.18	A Worked Example: DataFrame Filter And Project	187
11.19	A Worked Example: Repartition By Customer	187
11.20	A Worked Example: Registered Python UDF	188
11.21	Extension Implications	189
11.22	A Proposed Resolver-Side Extension Shape	190
11.23	Design Rules For Future Extensions	191
11.24	Reading Exercise	191
11.25	Takeaways	192
12	Chapter 11: Functions, UDFs, And Codecs	192
12.1	Code Map	192
12.2	The Function Lifecycle	194
12.3	Spec Representation	195
12.4	Built-In Function Registry	196
12.5	Function Resolution Order	197
12.6	Registered PySpark UDFs	200
12.7	Inline PySpark UDFs	200
12.8	Scalar Python Execution	201
12.9	Grouped Aggregate Python UDFs	202
12.10	UDTFs And Stream UDFs	203
12.11	The Python Stream Bridge	205
12.12	The Remote Execution Codec	205
12.13	Encoding Scalar UDFs	206
12.14	Decoding Scalar UDFs	207
12.15	Encoding Aggregate And Window UDFs	207
12.16	Encoding Stream UDFs	208
12.17	Worker Sessions And Built-Ins	209
12.18	Distributed Example: Built-In Function	209
12.19	Distributed Example: Python Scalar UDF	210
12.20	Distributed Example: Python UDTF	211
12.21	Codec Design Lessons	213
12.22	Extension Implications	213
12.23	A Proposed Extension Codec Registry	214
12.24	A Proposed Function Registration Model	215
12.25	Reading Exercise	216
12.26	Takeaways	216
13	Chapter 12: Catalogs, Lakehouse Tables, And File Formats	217
13.1	Code Map	217
13.2	The Storage Boundary	218

13.3	Catalogs: Names Before Data	221
13.4	Catalog Providers	222
13.5	Table Formats	222
13.6	Session Format Registration	223
13.7	SourceInfo: A Read Request In One Struct	224
13.8	Reading A Named Table	225
13.9	Reading A Data Source	227
13.10	Listing Formats: Parquet As The Normal Case	227
13.11	SinkInfo: A Write Request In One Struct	228
13.12	Write Resolution	228
13.13	Column Matching And Generated Columns	231
13.14	FileWriteNode And Physical Planning	231
13.15	Catalog Commands As Physical Plans	234
13.16	Delta Lake	234
13.17	Iceberg	235
13.18	Lakehouse Extension Planners	235
13.19	Row-Level Metadata Columns	238
13.20	Python Data Sources	238
13.21	Example: Parquet Read	239
13.22	Example: Delta CTAS	239
13.23	Example: Delta MERGE	240
13.24	Extension Lessons	240
13.25	Reading Exercises	241
13.26	Key Takeaways	242
14	Chapter 13: Extension Architecture: From Proposal To Design	242
14.1	Code Map	243
14.2	What The Proposal Is Really Asking For	243
14.3	Two Boundaries, Not One	244
14.4	Existing Patterns Worth Keeping	245
14.5	Current Gaps	247
14.6	Design Goal	248
14.7	The Core Trait	248
14.8	Extension Registry	250
14.9	Function Registration	251
14.10	Optimizer Rules	251
14.11	Extension Physical Planners	252
14.12	Physical Codec Extensions	252
14.13	Driver And Worker Symmetry	253
14.14	Python Entry-Point Discovery	254
14.15	Spark Connect As The Plan-Time Extension Surface	257
14.16	A Sedona-Style Worked Design	260
14.17	Table Formats As Extensions	263
14.18	Catalog Providers As Extensions	263

14.19	Conflict Policy	264
14.20	Enablement Policy	264
14.21	Versioning And ABI	265
14.22	Security And Trust	266
14.23	System Tables For Observability	266
14.24	Implementation Roadmap	267
14.25	Tests An Extension API Needs	268
14.26	A Mental Model For Future Contributors	270
14.27	How The Previous Chapters Fit Together	270
14.28	Final Takeaways	270

1 Reader Guide: How This Book Builds

This book is meant to be read in order, but it is also a code companion. Each chapter introduces one architectural layer, then later chapters reuse that layer when the distributed and extension stories become more demanding.

1.1 Chapter Links

Part	Chapters	What They Establish
System shape	1. Architecture Overview	The full query pipeline and the Rust patterns that make it possible.
	2. Rust Foundations	
Front doors	3. Spark Connect	How user intent enters Sail through PySpark, gRPC, protobufs, and Python packaging.
	4. PySpark and pysail	
Columnar runtime	5. Apache Arrow	The data model and query engine Sail builds on.
	6. Apache DataFusion	
Distribution	7. Physical Plan to Job Graph	How one DataFusion plan becomes distributed task execution and Arrow stream movement.
	8. Drivers, Workers, Tasks, and Streams	
	9. Shuffle and Data Movement	
Spark semantics	10. Sail Spec and Plan Resolver	How Spark-compatible names, expressions, functions, commands, tables, and writes become executable DataFusion objects.
	11. Functions, UDFs, and Codecs	
	12. Catalogs, Lakehouse Tables, and File Formats	
Extension design	13. Extension Architecture	How the previous patterns become a proposed extension architecture for discussion #2001.

1.2 Concept Progression

The chapters deliberately introduce concepts before relying on them:

Concept	Introduced	Elaborated	Used For Extensions
Spark Connect unresolved plans	Chapter 3	Chapters 4 and 10	Chapter 13, where extensions must preserve Spark-facing behavior.
Spark Connect extension messages	Chapter 3	Chapter 10	Chapter 13, where <code>Relation.extension</code> , <code>Command.extension</code> , and <code>Expression.extension</code> become the plan-time extension ABI.
Rust trait objects and <code>Arc</code>	Chapter 2	Chapters 6, 8, 11, and 12	Chapter 13, where execution-time extension capabilities are trait-object contributions.
Arrow <code>RecordBatch</code> streams	Chapter 5	Chapters 8 and 9	Chapters 11 and 13, where UDFs and custom operators must execute on Arrow batches.
DataFusion logical and physical plans	Chapter 6	Chapters 7 and 10	Chapter 13, where extensions add optimizer rules and physical planners.
Job graphs and stages	Chapter 7	Chapters 8 and 9	Chapter 13, where extension plans must survive distributed execution.
Typed session extensions	Chapter 2	Chapters 6, 11, and 12	Chapter 13, where the extension registry is proposed as a session service.
Function resolution and codecs	Chapter 11	Chapter 13	The core reason the execution-time extension boundary needs worker-side registration and serialization.
Table format registry	Chapter 12	Chapter 13	The strongest existing model for extension registration.
Two extension boundaries	Chapter 1	Chapters 3, 10, 11	Chapter 13, where plan-time (Spark Connect dispatch) and execution-time (DataFusion-shaped contributions) are

1.3 Code Reading Strategy

Each chapter has a code map, but these are the highest-leverage excerpts to read first:

Question	Best Code To Read
How does a Spark Connect request enter Sail?	<code>crates/sail-spark-connect/src/service/plan_executor.rs</code> and <code>crates/sail-spark-connect/src/server.rs</code>
How does Sail create a session?	<code>crates/sail-session/src/session_factory/server.rs</code>
How does Sail customize DataFusion planning?	<code>crates/sail-session/src/planner.rs</code>
How does a physical plan become distributed work?	<code>crates/sail-execution/src/job_graph/planner.rs</code> and <code>crates/sail-execution/src/job_runner.rs</code>
How do tasks run on workers?	<code>crates/sail-execution/src/task_runner/core.rs</code>
How does shuffle move Arrow batches?	<code>crates/sail-execution/src/plan/shuffle_write.rs</code> , <code>crates/sail-execution/src/plan/shuffle_read.rs</code> , and <code>crates/sail-execution/src/stream/</code>
How do Spark functions become DataFusion functions?	<code>crates/sail-plan/src/resolver/expression/function.rs</code> and <code>crates/sail-plan/src/function/</code>
How do Python UDFs execute?	<code>crates/sail-python-udf/src/udf/</code> and <code>crates/sail-python-udf/src/stream.rs</code>
How do custom functions and plans reach workers?	<code>crates/sail-execution/src/codec.rs</code>
How do catalogs and file formats plug in?	<code>crates/sail-catalog/src/manager/mod.rs</code> , <code>crates/sail-common-datafusion/src/datasource.rs</code> , and <code>crates/sail-session/src/formats.rs</code>
How do lakehouse row-level operations work?	<code>crates/sail-plan-lakehouse/src/lib.rs</code> , <code>crates/sail-delta-lake/src/table_format.rs</code> , and <code>crates/sail-logical-plan/src/merge.rs</code>

1.4 What To Look For In Code Excerpts

The best excerpts in this book are not chosen because they are short. They are chosen because they reveal a boundary:

- a protobuf boundary,
- a Python/Rust boundary,
- a Spark/DataFusion semantic boundary,
- a local/distributed execution boundary,
- a driver/worker serialization boundary,
- a catalog/table-format boundary,
- or an extension registration boundary.

When reading a snippet, ask what it converts from and what it converts to. Sail’s architecture is mostly a sequence of careful conversions.

Navigation: Start Chapter 1: Architecture Overview

2 Chapter 1: Architecture Overview

Sail is easiest to understand as two promises held together by one architecture.

The first promise is compatibility: existing PySpark code should be able to connect to Sail through Spark Connect and keep speaking the language of Spark SQL, DataFrames, functions, UDFs, and sessions. The second promise is performance and portability: the actual engine is Rust, Apache Arrow, and Apache DataFusion, with Sail adding Spark semantics, distributed planning, catalogs, Python interoperability, and cluster execution.

That means Sail is not “Spark implemented in Rust” in the narrow sense. It is a Spark-compatible front door over a DataFusion-centered query engine.

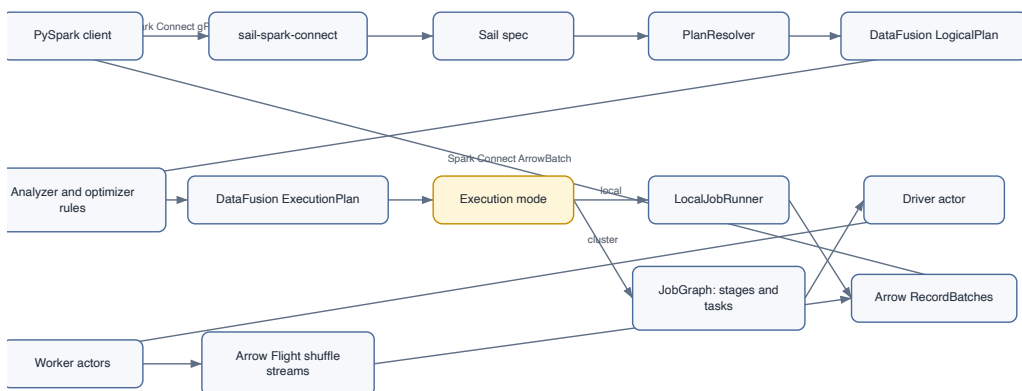


Figure 1: Flowchart 01.1

The rest of this book walks that diagram from left to right, then returns to the extension proposal in discussion #2001 and asks: where should a third-party DataFusion integration plug in so it works in both local and distributed execution?

2.1 The Big Pieces

Sail has a few major subsystems. Each one has a clean teaching role.

Subsystem	Main crates	What to learn there
Spark Connect front door	<code>sail-spark-connect</code> , <code>sail-python</code> , <code>python/pysail</code>	gRPC services, PySpark compatibility, Python-to-Rust server startup
Plan resolution	<code>sail-plan</code> , <code>sail-sql-parser</code> , <code>sail-sql-analyzer</code>	converting SQL and Spark relations into DataFusion logical plans
Session construction	<code>sail-session</code>	DataFusion <code>SessionConfig</code> , <code>SessionState</code> , custom planners, optimizer rules, and job runners
Query execution	<code>sail-execution</code>	local execution, cluster execution, job graphs, stages, tasks, drivers, workers, shuffles
Spark semantics	<code>sail-function</code> , <code>sail-logical-plan</code> , <code>sail-physical-plan</code> , <code>sail-logical-optimizer</code> , <code>sail-physical-optimizer</code>	custom functions, logical nodes, physical nodes, optimizer behavior
Data transport	Arrow, Arrow IPC, Arrow Flight	columnar batches across process and network boundaries
Catalogs and formats	<code>sail-catalog-*</code> , <code>sail-data-source</code> , <code>sail-plan-lakehouse</code>	table discovery, scans, writes, system tables, lakehouse integration

The most important mental model is this:

PySpark API call

- > Spark Connect protobuf relation or command
- > Sail spec
- > DataFusion logical plan
- > optimized DataFusion logical plan
- > DataFusion physical execution plan, with Sail extension nodes
- > local stream or distributed job graph
- > Arrow record batches
- > Spark Connect response stream

This is also the extension story. If an integration only hooks one of these layers, it will work only until a query crosses into another layer. A scalar function registered at planning time still has to be recognized when a physical plan is decoded on a remote worker. A logical optimizer rule that creates a custom extension node still needs a physical extension planner. A session option used by that rule has to be present in the `SessionConfig` that the planner reads.

That is exactly the problem described in discussion #2001.

2.2 Spark Connect Is the Front Door

The Spark Connect server is implemented by `SparkConnectServer` in `crates/sail-spark-connect/src/server.rs`. Its `execute_plan` method receives an `ExecutePlanRequest`, extracts the session ID and user ID, asks the `Sail SessionManager` for a `SessionContext`, and dispatches either a relation or a command.

The distinction matters:

- A relation is a query-producing tree: select, project, filter, join, aggregate, read, SQL relation, and so on.
- A command is an action or side effect: register a function, write data, create a view, start a stream, run a SQL command, merge into a table, and so on.

The central handoff is in `crates/sail-spark-connect/src/service/plan_executor.rs`. `handle_execute_relation` converts the Spark Connect relation into Sail's internal `spec::Plan`, then calls `handle_execute_plan`. That function asks `sail-plan` to resolve and plan the work, then asks the session's `JobService` to execute the resulting physical plan.

The output is a Spark Connect response stream. Sail reads `RecordBatch` values from `DataFusion` and serializes them into Spark Connect `ArrowBatch` messages using `Arrow IPC` in `crates/sail-spark-connect/src/executor.rs`.

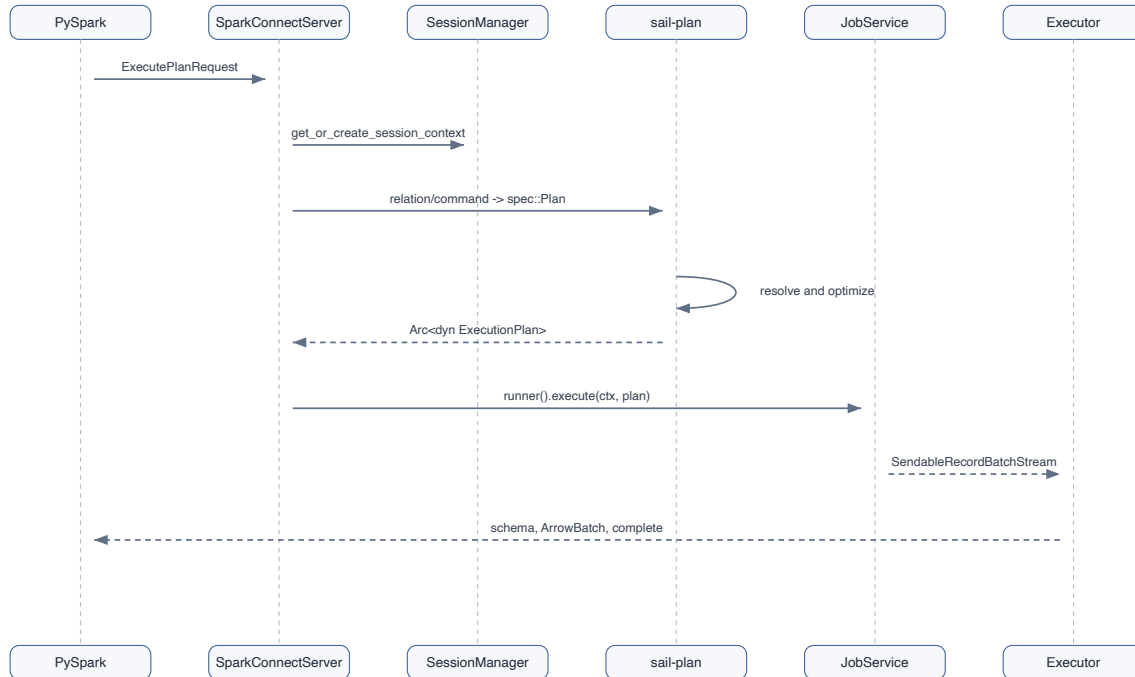


Figure 2: Sequence diagram 01.2

This is why Spark Connect deserves its own chapter. It is not just an API shim; it controls the session lifecycle, the response stream shape, the data type boundary, and the compatibility surface seen by PySpark.

2.3 pysail Starts the Rust Server

The Python package `pysail` is thin by design. The public Python class `python/pysail/spark/__init__.py::SparkConnectServer` delegates to a native PyO3 object.

The Rust side lives in `crates/sail-python/src/spark/server.rs`. It loads `AppConfig`, grabs the global Tokio runtime, binds a TCP listener, and starts the Spark Connect server in a background thread. The implementation explicitly releases the Python GIL while waiting for the server so Python UDFs are not blocked by the server thread.

This shape is important for the extension proposal. If third-party extensions are discovered from Python wheels, `pysail` startup is the natural discovery point. But the extension object has to cross from Python packaging into Rust planning and execution. Discussion #2001 proposes Python entry points such as:

```
[project.entry-points."pysail.extensions"]
sedona = "pysail_sedona:register"
```

That works as user experience only if the registered extension can contribute to the same Rust-side machinery used by the CLI and by custom embedders.

2.4 Sail Uses DataFusion as the Query Kernel

Sail's query planning entry point is `resolve_and_execute_plan` in `crates/sail-plan/src/lib.rs`.

It performs the key transitions:

1. Build a `PlanResolver`.
2. Resolve a Sail spec into a named `DataFusion LogicalPlan`.
3. Ask DataFusion's `SessionState` to optimize the logical plan.
4. Rewrite streaming plans when needed.
5. Ask the session query planner to create a physical `ExecutionPlan`.
6. Record initial logical, final logical, and final physical plans for explain output.

The important architectural choice is that Sail does not use DataFusion as a black box. It uses DataFusion's abstractions as the spine, then installs its own semantics around them.

In `crates/sail-session/src/session_factory/server.rs`, Sail creates a `SessionConfig` with custom extensions:

- Table format registry.
- Catalog manager.
- Activity tracker.
- Job service.
- Repartition buffer configuration.
- System table service.
- Delta table cache.

Then it creates a `SessionStateBuilder` with Sail's analyzer rules, optimizer rules, physical optimizer rules, and custom query planner.

That custom query planner is in `crates/sail-session/src/planner.rs`. `ExtensionQueryPlanner` builds a `DataFusion DefaultPhysicalPlanner` with Sail's extension planners:

```
lakehouse extension planners
-> system table physical planner
-> Sail ExtensionPhysicalPlanner
```

`ExtensionPhysicalPlanner` recognizes Sail logical extension nodes such as `range`, `show string`, `map partitions`, `monotonic IDs`, `Spark partition IDs`, `file writes`, `file deletes`, `streaming nodes`, `catalog commands`, `explicit repartition`, and `barriers`. It turns them into physical `ExecutionPlan` implementations from `sail-physical-plan` and related crates.

This is where discussion #2001 finds one of its sharp edges. Today, if `ExtensionPhysicalPlanner` does not recognize a logical extension node, it returns an internal error. DataFusion's extension planner convention is to return `Ok(None)` when a planner does not own a node, allowing later planners in the chain to try. For third-party planners, that difference controls whether composition works.

2.5 Local Execution Is Direct DataFusion Execution

When Sail is running locally, the `ServerSessionFactory` installs a `LocalJobRunner`. Its `execute` implementation in `crates/sail-execution/src/job_runner.rs` wraps the plan in tracing and then calls DataFusion's `execute_stream`.

That is the simplest possible execution path:

```
Arc<dyn ExecutionPlan>  
  -> execute_stream(plan, task_ctx)  
  -> SendableRecordBatchStream  
  -> Spark Connect response stream
```

Local mode is ideal for learning DataFusion because all of DataFusion's partitioned execution model is still present, but no distributed staging is needed. The physical plan executes in one process, and DataFusion's operators recursively call their children through the `ExecutionPlan` trait.

2.6 Cluster Execution Adds a Driver, Workers, Stages, and Shuffles

Cluster mode swaps in `ClusterJobRunner`. Instead of executing the physical plan directly, it sends a `DriverEvent::ExecuteJob` to a driver actor. The driver builds a distributed job graph and schedules tasks on workers.

The core data structure is `JobGraph` in `crates/sail-execution/src/job_graph/mod.rs`. The code comments are wonderfully plain: a job has stages, each stage has partitions, and tasks execute individual stage partitions. Each task can produce output split into channels.

The graph is built in `crates/sail-execution/src/job_graph/planner.rs`. `JobGraph::try_new` starts from a DataFusion physical plan and recursively splits it at distributed boundaries:

- `RepartitionExec` becomes a shuffle boundary.
- `ExplicitRepartitionExec` becomes a shuffle boundary.
- `CoalescePartitionsExec` becomes a shuffle boundary.
- `SortPreservingMergeExec` creates a merge input.
- Sail's `CoalesceExec` creates a rescale input.
- System tables and catalog commands become driver stages.

The planner also contains two distributed-correctness rewrites:

- Global limits are forced to have a single input partition when a limit or offset is present.
- Certain collected hash joins are rewritten into partitioned hash joins when unmatched build-side rows would otherwise require shared row-match state across distributed partitions.

This is one of Sail's best teaching examples. DataFusion physical plans already know about partitioning, but a distributed engine must interpret that partitioning as data movement, task placement, materialization, and reuse.

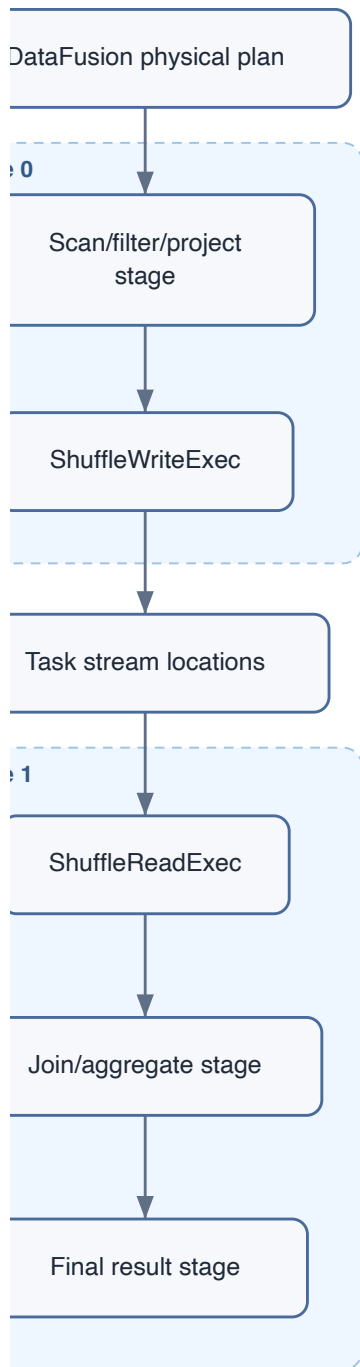


Figure 3: Flowchart 01.3

2.7 Shuffle Is Arrow Data Movement

Sail represents shuffle write and shuffle read as physical execution plan nodes.

`ShuffleWriteExec` in `crates/sail-execution/src/plan/shuffle_write.rs` executes its child for one input partition, partitions each `RecordBatch` into output channels using hash or round-robin partitioning, and writes those partitioned batches to task stream locations.

`ShuffleReadExec` in `crates/sail-execution/src/plan/shuffle_read.rs` has no children. For a given output partition, it opens the task stream locations it needs and merges the resulting record batch streams.

That design keeps the distributed runtime columnar all the way through:

`RecordBatch` stream

- > partition `RecordBatch` into channel batches
- > write channel batches
- > read channel batches from remote/local stream locations
- > merge streams
- > continue as `RecordBatch` stream

The public architecture docs describe Arrow Flight as Sail’s data plane for shuffle exchange and result return. The code-level point is that the logical idea of “shuffle” becomes ordinary `DataFusion` `ExecutionPlan` nodes that read and write Arrow batch streams.

2.8 Functions Are Both Planning-Time and Execution-Time Concerns

Sail has a Spark-compatible function layer in `crates/sail-plan/src/function`. Built-in scalar, generator, table, aggregate, and window functions are stored in static registries. The resolver uses those registries to turn unresolved Spark functions into `DataFusion` expressions and UDF objects.

But distributed execution adds another requirement: workers must be able to decode the physical plan they receive. That is why `crates/sail-execution/src/codec.rs` has explicit UDF and UDAF encode/decode logic. It can rebuild PySpark UDFs from serialized payloads, and it can re-resolve many built-in UDF names when decoding standard functions.

This is the most important extension lesson in the chapter:

Planning-time registry is necessary.

Distributed execution-time registry is also necessary.

If an extension contributes `ST_Intersects`, it is not enough for the planner to know the function. A remote worker decoding a physical plan also has to know how to reconstruct the same `ScalarUDF` or `AggregateUDF`. Discussion #2001 calls this out directly for Sedona-style extensions.

2.9 Where Extensions Want to Plug In

Discussion #2001 proposes a unified `SailExtension` trait. Its motivation is that real `DataFusion` integrations usually need several hooks at once:

- Scalar UDFs.
- Aggregate UDAFs.
- Window UDFs.

- Generator and table functions.
- Session config extensions.
- Logical optimizer rules.
- Physical optimizer rules.
- Physical extension planners.
- UDF/UDAF re-resolution during distributed physical-plan decoding.

The proposal’s motivating example is Apache SedonaDB. A spatial query might need `ST_*` scalar UDFs during plan resolution, session options during optimization, a logical optimizer rule to replace a cross join plus spatial predicate with a spatial join logical extension node, a physical planner to create `SpatialJoinExec`, and worker-side UDF re-resolution in a cluster.

This means the final chapter of the book should not treat extensions as a plugin convenience feature. Extensions are a stress test of the architecture. They ask whether Sail’s layers are composable in the same direction data actually flows.

Chapter 13 develops the proposal in two parts. Extensions cross **two boundaries** with different stability requirements:

- A **plan-time boundary** where a client expresses intent. It runs once per query and wants forward and backward wire compatibility, language neutrality, and a format that survives DataFusion and Arrow upgrades. Spark Connect’s `Relation.extension`, `Command.extension`, and `Expression.extension` messages are the natural channel.
- An **execution-time boundary** where workers run operators on Arrow batches. It runs once per batch, wants native dispatch and zero-copy access, and accepts version coupling in return. DataFusion FFI is the natural channel.

The same `SailExtension` object registers contributions to both. Some extensions only need one.

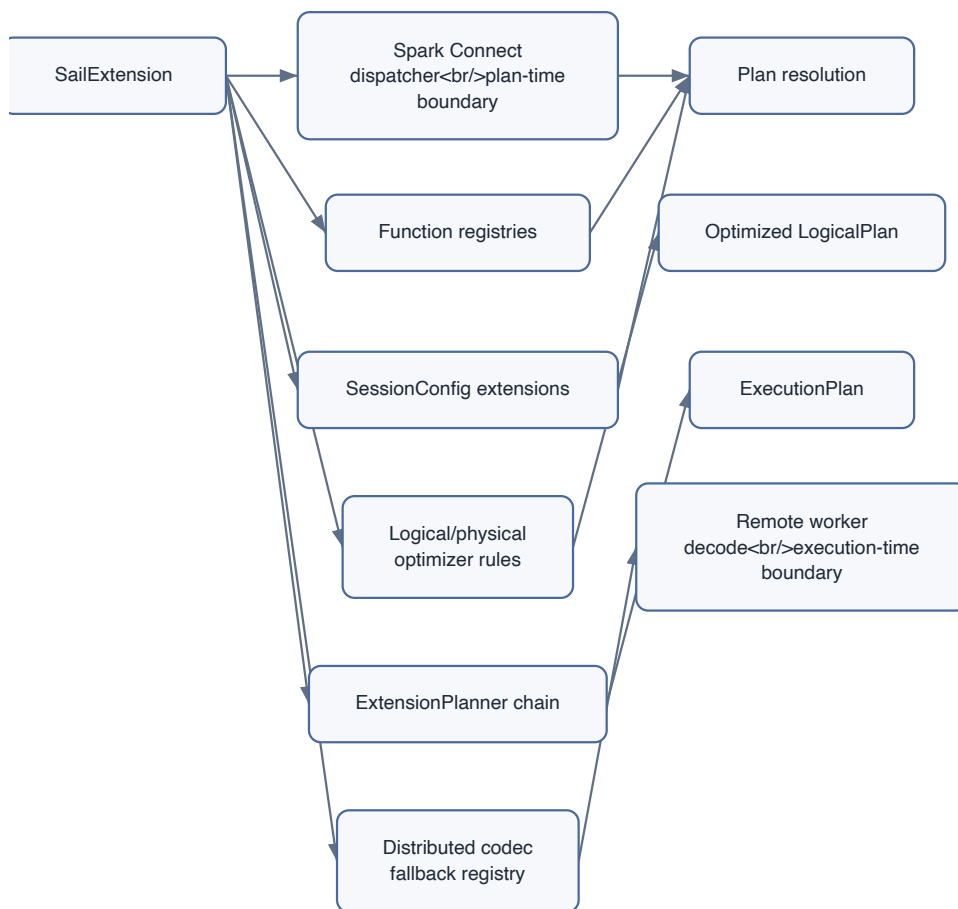


Figure 4: Flowchart 01.4

2.10 A First Reading Path Through the Code

For this chapter, read these files in order:

1. docs/concepts/architecture/index.md
2. docs/concepts/query-planning/index.md
3. crates/sail-spark-connect/src/server.rs
4. crates/sail-spark-connect/src/service/plan_executor.rs
5. crates/sail-plan/src/lib.rs
6. crates/sail-session/src/session_factory/server.rs
7. crates/sail-session/src/planner.rs
8. crates/sail-execution/src/job_runner.rs
9. crates/sail-execution/src/job_graph/mod.rs

10. `crates/sail-execution/src/job_graph/planner.rs`
11. `crates/sail-execution/src/plan/shuffle_write.rs`
12. `crates/sail-execution/src/plan/shuffle_read.rs`
13. `crates/sail-execution/src/codec.rs`

Do not try to understand every operator yet. Follow the type transitions:

```
ExecutePlanRequest
  -> SessionContext
  -> spec::Plan
  -> LogicalPlan
  -> ExecutionPlan
  -> SendableRecordBatchStream
```

Then follow the cluster-only transition:

```
ExecutionPlan
  -> JobGraph
  -> Stage
  -> StageInput
  -> Task
  -> ShuffleWriteExec / ShuffleReadExec
```

Once those two paths feel familiar, the rest of the book can zoom into each layer without losing the whole shape.

2.11 Chapter Takeaways

Sail's architecture is a layered translation pipeline. PySpark speaks Spark Connect. Spark Connect becomes Sail's internal spec. The spec resolves into DataFusion logical plans. DataFusion optimizes and physical-plans the query, with Sail adding Spark semantics through custom functions, logical nodes, physical nodes, optimizer rules, and session extensions. Local mode executes the physical plan directly. Cluster mode decomposes it into stages and tasks, moving Arrow record batches through shuffle streams.

The extension proposal in discussion #2001 matters because it turns this architecture inside out. A third-party integration must be able to contribute to every layer where its semantics appear. If Sail exposes only one hook, extensions will work in toy examples and fail when optimization, physical planning, or distributed execution enters the picture.

The next chapter should slow down and teach the Rust patterns that make this architecture possible: trait objects, Arc, async services, actor handles, DataFusion extension traits, and typed session extensions.

3 Chapter 2: Rust Foundations in Sail

This chapter is not a full Rust tutorial. It is a map of the Rust ideas you need in order to read Sail without feeling like every file is speaking a private dialect.

Sail is an unusually good Rust learning project because it uses Rust for the things Rust is good at: explicit ownership, cheap shared references, trait-based interfaces, asynchronous services,

structured errors, and safe concurrency. It is also a practical systems project, so these ideas show up under pressure. They are not decorative.

The core lesson is this:

Sail moves query plans and Arrow streams through a graph of typed interfaces. Rust makes those interfaces explicit.

When you see `Arc<dyn ExecutionPlan>`, `Box<dyn JobRunner>`, `SessionExtension`, or `ActorHandle<DriverActor>`, you are seeing the architecture in Rust form.

3.1 The Rust Shape of Sail

Chapter 1 described Sail as a pipeline:

Spark Connect request

- > Sail spec
- > DataFusion LogicalPlan
- > DataFusion ExecutionPlan
- > local stream or distributed job graph
- > Arrow RecordBatch stream

Rust gives each boundary a type. A few types appear again and again:

Rust pattern	Sail example	Why it matters
<code>Arc<T></code>	<code>Arc<AppConfig></code> , <code>Arc<dyn ExecutionPlan></code>	Shared ownership across async tasks, sessions, plans, and workers
<code>Box<dyn Trait></code>	<code>Box<dyn JobRunner></code> , <code>Box<dyn ServerSessionMutator></code>	Runtime choice among implementations
<code>Arc<dyn Trait></code>	<code>Arc<dyn ExecutionPlan></code> , <code>Arc<dyn QueryPlanner></code>	Shared polymorphic query operators
<code>async_trait</code>	<code>JobRunner</code> , <code>Actor</code> , gRPC service traits	Async methods in traits
<code>Result<T, E></code>	<code>PlanResult</code> , <code>ExecutionResult</code> , <code>SparkResult</code>	Explicit error paths across planning, execution, and protocol layers
Typed extensions	<code>SessionExtension</code>	Type-safe access to session services and configuration
Actor handles	<code>ActorHandle<DriverActor></code>	Message-passing control plane for distributed execution

These are the vocabulary words of the Sail codebase. The rest of the chapter explains each one through files you have already touched in the architecture overview.

3.2 Shared Ownership With Arc

`Arc<T>` means “atomically reference-counted pointer.” In practical terms, it lets multiple owners hold the same value safely across threads. Sail needs that constantly because sessions, runtimes, query plans, actors, and task contexts all outlive a single function call.

Look at `ServerSessionFactory` in `crates/sail-session/src/session_factory/server.rs`:

```
pub struct ServerSessionFactory {  
    config: Arc<AppConfig>,  
    runtime: RuntimeHandle,  
    system: Arc<Mutex<ActorSystem>>,  
    mutator: Box<dyn ServerSessionMutator>,  
    runtime_env: RuntimeEnvFactory,  
    catalog_cache_manager: Arc<CatalogCacheManager>,  
}
```

The factory does not own the global application config in a lonely way. It shares it. The session factory, runtime environment factory, catalog manager, worker manager, and driver setup can all receive clones of the same `Arc<AppConfig>`.

Cloning an `Arc` does not clone the underlying config. It increments a reference count:

```
let runtime_env = RuntimeEnvFactory::new(config.clone(), runtime.clone());
```

That line is small, but it is one of Rust’s most important performance habits. Large shared state can be passed cheaply while ownership stays explicit.

`DataFusion` plans use the same idea. A physical plan in `Sail` is usually:

`Arc<dyn ExecutionPlan>`

That reads as:

shared pointer to some concrete type that implements `DataFusion`’s `ExecutionPlan` trait

The concrete type might be a `DataFusion` operator, `ShuffleWriteExec`, `ShuffleReadExec`, `RangeExec`, `MapPartitionsExec`, `FileWriteExec`, or another `Sail` extension. The caller often does not need to know. It needs the `ExecutionPlan` interface.



Figure 5: Flowchart 02.1

The `Arc` part lets the plan be shared. The `dyn ExecutionPlan` part lets the plan be polymorphic.

3.3 Trait Objects: dyn Trait

Traits define behavior. Trait objects let Sail pick an implementation at runtime.

The JobRunner trait in `crates/sail-common-datafusion/src/session/job.rs` is the cleanest example:

```
#[tonic::async_trait]
pub trait JobRunner: StateObservable<JobRunnerObserver> + Send + Sync + 'static {
    async fn execute(
        &self,
        ctx: &SessionContext,
        plan: Arc<dyn ExecutionPlan>,
    ) -> Result<SendableRecordBatchStream>;

    async fn stop(&self, history: oneshot::Sender<JobRunnerHistory>);
}
```

A JobRunner takes a DataFusion physical plan and returns a stream of Arrow record batches. That is the interface. The implementation depends on execution mode.

In `ServerSessionFactory::create_job_runner`, Sail chooses:

```
let job_runner: Box<dyn JobRunner> = match self.config.mode {
    ExecutionMode::Local => Box::new(LocalJobRunner::new()),
    ExecutionMode::LocalCluster => { ... Box::new(ClusterJobRunner::new(...)) }
    ExecutionMode::KubernetesCluster => { ...
Box::new(ClusterJobRunner::new(...)) }
};
```

This is runtime polymorphism. The rest of the session does not need to branch on local versus cluster every time it executes a query. It just calls:

```
service.runner().execute(ctx, plan).await
```

The object behind `runner()` decides what that means.

The same pattern appears in the extension proposal. A future `SailExtension` trait would probably be used behind `Arc<dyn SailExtension>` because Sail must hold a list of unknown third-party extension implementations.

3.4 Local and Cluster Runners Share One Interface

`LocalJobRunner` and `ClusterJobRunner` are a small but powerful comparison.

The local runner executes the DataFusion physical plan directly:

```
Ok(execute_stream(plan, ctx.task_ctx())?)
```

The cluster runner sends the same plan to a driver actor:

```
self.driver
    .send(DriverEvent::ExecuteJob {
        plan,
        context: ctx.task_ctx(),
    },
```

```

        result: tx,
    })
    .await?;

```

Same trait. Same method signature. Very different behavior.

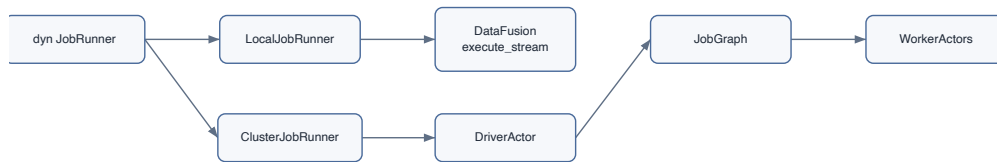


Figure 6: Flowchart 02.2

This is one of the most important Rust design moves in Sail: put the architectural decision behind a trait, then pass the trait object through the rest of the system.

3.5 Send, Sync, and 'static

You will often see trait bounds like this:

```
pub trait JobRunner: StateObservable<JobRunnerObserver> + Send + Sync + 'static
```

These are not noise.

Send means a value can be moved to another thread. Sync means references to it can be shared across threads. 'static means the value does not contain borrowed references that could expire while async tasks or background actors still need it.

Sail is full of async tasks, actor messages, gRPC handlers, and worker processes. If a service may be stored in a session, used by a task, or held across an `.await`, Rust needs to know it is safe to move and share.

The proposed extension API in discussion #2001 uses the same idea:

```
pub trait SailExtension: Send + Sync {
    fn name(&self) -> &str;
    ...
}
```

That bound is a design statement. Extensions are not just parser plugins. They may participate in planning and execution paths that cross async and distributed boundaries.

3.6 Async Traits

Rust traits do not natively support async methods in the most ergonomic way for this kind of code, so Sail uses `#[tonic::async_trait]` or `#[async_trait]`.

You see it in three central places:

- gRPC services, such as Spark Connect and worker services.
- JobRunner, where execution returns an async stream-producing result.
- Actor, where startup and shutdown may be async.

The Actor trait in `crates/sail-server/src/actor.rs` looks like this:

```
#[tonic::async_trait]
pub trait Actor: Sized + Send + 'static {
    type Message: Send + SpanAssociation + 'static;
    type Options;

    fn name() -> &'static str;
    fn new(options: Self::Options) -> Self;
    async fn start(&mut self, ctx: &mut ActorContext<Self>) {}
    fn receive(&mut self, ctx: &mut ActorContext<Self>, message: Self::Message) -
> ActorAction;
    async fn stop(self, ctx: &mut ActorContext<Self>) {}
}
```

Notice the split:

- `start` and `stop` are async because they may do setup or teardown work.
- `receive` is synchronous and should not block. If it needs async work, it spawns a task via the actor context.

That is a deliberate concurrency model. Actor message handling stays sequential, while longer async work is pushed into spawned tasks.

3.7 Actors: The Control Plane in Rust

Sail's distributed execution control plane uses actors. An actor owns state. Other code sends it messages through an `ActorHandle<T>`.

The generic actor system is in `crates/sail-server/src/actor.rs`:

```
pub struct ActorHandle<T: Actor> {
    sender: mpsc::Sender<MessageEnvelop<T::Message>>,
}
```

An `ActorHandle<DriverActor>` can send only `DriverActor` messages. An `ActorHandle<WorkerActor>` can send only `WorkerActor` messages. This gives the message-passing system compile-time shape.

The worker gRPC service shows the pattern. In `crates/sail-execution/src/worker/server.rs`, a `run_task` request is decoded into a typed `WorkerEvent::RunTask` and sent to the worker actor:

```
self.handle
    .send(event)
    .await
    .map_err(ExecutionError::from)?;
```

So the server's job is mostly translation:

gRPC request

- > typed request struct
- > domain event
- > actor message

The actor's job is stateful behavior:

receive event

- > update state
- > spawn tasks
- > send follow-up events
- > report status

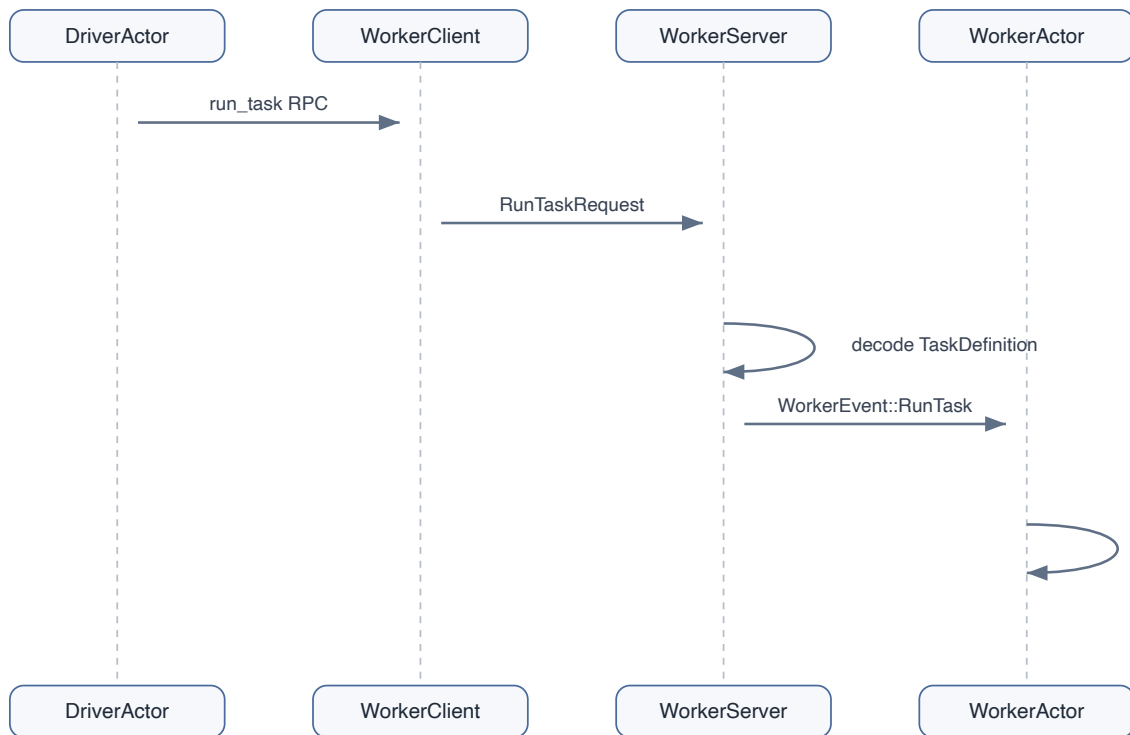


Figure 7: Sequence diagram 02.3

Rust helps here by making invalid message routes hard to express. You cannot accidentally send a `DriverEvent` to an `ActorHandle<WorkerActor>` without fighting the type system.

3.8 Typed Session Extensions

DataFusion's `SessionConfig` can store extensions. Sail wraps that in a small trait:

```
pub trait SessionExtension: Send + Sync + 'static {
    fn name() -> &'static str;
}
```

Then `SessionExtensionAccessor` provides typed lookup from `SessionContext`, `SessionState`, `TaskContext`, and `DataFusion`'s `Session` trait:

```
fn extension<T: SessionExtension>(&self) -> Result<Arc<T>>;
```

This turns session services into type-safe dependencies. For example, Spark Connect execution can ask the session for its `SparkSession` extension. Planning and physical execution code can ask for the catalog manager, table format registry, job service, activity tracker, repartition config, or system table service.

The pattern is:

```
register typed extension during session creation
-> retrieve typed extension where needed
-> fail clearly if missing
```

In `ServerSessionFactory::create_session_config`, `Sail` registers many extensions:

```
SessionConfig::new()
  .with_extension(create_table_format_registry()?)
  .with_extension(Arc::new(create_catalog_manager(...)?)?)
  .with_extension(Arc::new(ActivityTracker::new()))
  .with_extension(Arc::new(JobService::new(job_runner)))
  .with_extension(Arc::new(RepartitionBufferConfig::new(...)))
  .with_extension(Arc::new(self.create_system_table_service(info)?))
  .with_extension(Arc::new(DeltaTableCache::default()))
```

This matters for extensions because many third-party integrations need session state. Sedona-style spatial planning, for example, may need options that optimizer rules can read. The current `ServerSessionMutator` can mutate `SessionConfig`, `SessionStateBuilder`, and `RuntimeEnvBuilder`, but discussion #2001 argues that this is not enough because functions, codec re-resolution, and extension planner registration live elsewhere.

3.9 Builders and Mutators

`Sail` often uses builder-style APIs because `DataFusion` itself uses them. Session creation is the main example.

`ServerSessionFactory::create_session_state` builds a `DataFusion` session state:

```
let builder = SessionStateBuilder::new()
  .with_config(config)
  .with_runtime_env(runtime)
  .with_analyzer_rules(default_analyzer_rules())
  .with_optimizer_rules(default_optimizer_rules())
  .with_physical_optimizer_rules(get_physical_optimizers(...))
  .with_query_planner(new_query_planner());
let builder = self.mutator.mutate_state(builder, info)?;
Ok(builder.build())
```

The builder has two jobs:

- Accumulate configuration in a readable order.

- Give Sail one place to inject custom behavior before the immutable session state is built.

The mutator has a narrower purpose:

```
pub trait ServerSessionMutator: Send {
    fn mutate_config(...) -> Result<SessionConfig>;
    fn mutate_state(...) -> Result<SessionStateBuilder>;
    fn mutate_runtime_env(...) -> Result<RuntimeEnvBuilder>;
}
```

This is already an extension-like boundary. But it is embedder-oriented, not package/plugin-oriented. It does not solve plan-time function registries or worker-side UDF decoding. That is why discussion #2001 proposes a higher-level `SailExtension`.

3.10 Downcasting Extension Nodes

DataFusion has extension traits for custom logical and physical behavior. Sail uses them heavily.

In `crates/sail-session/src/planner.rs`, `ExtensionPhysicalPlanner` receives a generic `UserDefinedLogicalNode`:

```
async fn plan_extension(
    &self,
    planner: &dyn PhysicalPlanner,
    node: &dyn UserDefinedLogicalNode,
    logical_inputs: &[&LogicalPlan],
    physical_inputs: &[Arc<dyn ExecutionPlan>],
    session_state: &SessionState,
) -> Result<Option<Arc<dyn ExecutionPlan>>>
```

The planner then asks, one type at a time, whether the node is a Sail node:

```
if let Some(node) = node.as_any().downcast_ref::<RangeNode>() {
    ...
} else if let Some(node) = node.as_any().downcast_ref::<ShowStringNode>() {
    ...
} else if let Some(node) = node.as_any().downcast_ref::<MapPartitionsNode>() {
    ...
}
```

This is Rust’s way of combining an open interface with concrete behavior. The planner receives “some extension node.” It can only plan nodes it recognizes. Recognition happens through Any downcasting.

For readers, this explains a lot of Sail code:

```
trait object enters boundary
    -> as_any()
    -> downcast_ref::<ConcreteType>()
    -> concrete planning or execution logic
```

For extension authors, it explains why planner ordering matters. If one planner errors on unknown nodes instead of returning `Ok(None)`, later planners never get a chance.

3.11 Error Types

Sail has separate error layers:

- PlanError in sail-plan.
- ExecutionError in sail-execution.
- SparkError in sail-spark-connect.
- DataFusion's own DataFusionError.

The aliases are simple:

```
pub type PlanResult<T> = Result<T, PlanError>;
pub type ExecutionResult<T> = Result<T, ExecutionError>;
pub type SparkResult<T> = Result<T, SparkError>;
```

The point is not just style. Each layer needs to add context in its own vocabulary.

Planning errors talk about unsupported functions, invalid expressions, unresolved fields, and semantic analysis. Execution errors talk about task definitions, worker communication, job graphs, and DataFusion execution. Spark errors must become protocol-level statuses and Spark-compatible error responses.

When reading Sail, track where errors cross boundaries. A failed worker task should not leak as an arbitrary Rust panic. An unknown Spark function should become a planning error. A malformed protobuf request should become a Spark Connect status error.

3.12 A Mini Example: What Happens to a Query Plan Type

Here is the lifecycle of one important type:

```
Arc<dyn ExecutionPlan>
```

In local mode:

```
Arc<dyn ExecutionPlan>
-> LocalJobRunner::execute
-> datafusion::physical_plan::execute_stream
-> SendableRecordBatchStream
```

In cluster mode:

```
Arc<dyn ExecutionPlan>
-> ClusterJobRunner::execute
-> DriverEvent::ExecuteJob
-> JobGraph::try_new
-> Stage plans
-> serialized task definitions
-> worker execution
-> shuffle and result streams
```

Same Rust type, different execution strategy.

This is why Arc<dyn ExecutionPlan> is not just a pointer. It is the main currency between DataFusion and Sail's execution system.

3.13 How Rust Shapes the Extension Proposal

Discussion #2001 proposes a `SailExtension` trait that can contribute functions, optimizer rules, config extensions, physical planners, and distributed UDF re-resolution. Rust affects that proposal in several ways.

First, extensions will likely be trait objects:

```
Arc<dyn SailExtension>
```

That allows multiple independently implemented extensions to be registered in one session factory.

Second, extension contributions must be thread-safe:

```
Send + Sync + 'static
```

They may be shared across sessions, stored in configs, used during async planning, or needed on workers.

Third, extension contributions must cross several existing typed registries:

```
HashMap<String, Arc<ScalarUDF>>  
HashMap<String, Arc<AggregateUDF>>  
Vec<Arc<dyn OptimizerRule + Send + Sync>>  
Vec<Arc<dyn ExtensionPlanner + Send + Sync>>
```

Fourth, Python-discovered extensions create an ABI and packaging problem. Python entry points can discover a `pysail-sedona` package, but the object handed back into Rust must still match the exact Rust crate versions expected by `pysail`. Rust trait objects do not have a stable cross-version ABI. This is why discussion #2001 calls out version coupling between `pysail`, `datafusion`, `arrow`, `pyo3`, and the plugin wheel.

The Rust design question is therefore not “can we make a plugin trait?” That part is straightforward. The deeper question is “where does the trait object live, who owns it, how is it shared, and how do workers reconstruct the same extension-provided behavior?”

3.14 Reading Exercises

Read these files with one question in mind: what interface is this code defining, and what concrete implementation sits behind it?

1. `crates/sail-common-datafusion/src/session/job.rs`
 - Find the `JobRunner` trait.
 - Compare it with `LocalJobRunner` and `ClusterJobRunner`.
2. `crates/sail-execution/src/job_runner.rs`
 - Follow local execution from `execute_stream`.
 - Follow cluster execution into `DriverEvent::ExecuteJob`.
3. `crates/sail-server/src/actor.rs`
 - Identify the actor message type.
 - Look at how `ActorHandle<T>` preserves message typing.
4. `crates/sail-common-datafusion/src/extension.rs`

- Follow typed extension lookup from `SessionContext`, `SessionState`, and `TaskContext`.
5. `crates/sail-session/src/session_factory/server.rs`
 - List the session extensions registered in `create_session_config`.
 - Find where the query planner is installed.
 6. `crates/sail-session/src/planner.rs`
 - Find the extension planner chain.
 - Trace one downcast from logical extension node to physical execution node.

3.15 Chapter Takeaways

Rust makes Sail’s architecture visible. `Arc` shows what is shared. `Box<dyn Trait>` and `Arc<dyn Trait>` show where implementations are chosen dynamically. `Send`, `Sync`, and `'static` show which objects must survive async and threaded execution. `SessionExtension` shows how Sail adds typed services to DataFusion sessions. `Actors` show how Sail keeps distributed control-plane state behind message boundaries.

These patterns are also the foundation for the extension proposal. A useful Sail extension API will not be a single callback. It will be a set of Rust trait-object contributions that can be registered, shared, ordered, used during planning, and reconstructed during distributed execution.

The next chapter moves back to the front door: Spark Connect. We will follow a PySpark request through Sail’s gRPC service, session manager, relation and command handlers, Arrow response stream, and error model.

4 Chapter 3: Spark Connect

Spark Connect is Sail’s public front door for PySpark. When a PySpark user writes `spark.read.parquet(...).groupBy(...).count()`, Sail does not receive Python bytecode or a Spark JVM object. It receives Spark Connect protobuf messages over gRPC. The definitive starting point is Apache Spark’s own Spark Connect Overview, which describes Spark Connect as a decoupled client-server architecture using unresolved logical plans as the protocol.

That design gives Sail its “drop-in replacement” shape. The client can stay PySpark. The server can be Rust. The wire protocol between them is Spark Connect.

PySpark client

- > Spark Connect protobuf messages
- > Sail `SparkConnectServer`
- > Sail spec
- > DataFusion logical and physical plans
- > Arrow batches in Spark Connect responses

This chapter follows that front door in code. The goal is to understand what Spark Connect means architecturally, not just where the `.proto` files are generated.

4.1 Definitive Spark Connect References

Use these official references alongside this chapter:

- Spark Connect Overview: the Apache Spark documentation page for the client/server architecture, protobuf transport, gRPC, and Arrow result batches.
- Spark Connect architecture page: Apache Spark’s higher-level architecture explanation, including the connection flow, unresolved logical plans, Protocol Buffers, server-side optimization, and Arrow record batch result streaming.
- PySpark `SparkSession.builder.remote`: the official PySpark API for connecting to a Spark Connect server with an `sc://host:port` URL.
- PySpark Spark Session API reference: the Spark-session surface, including Spark Connect-only methods, artifacts, tags, interrupt methods, and the `client` attribute.
- PySpark API Reference: the official API index; it marks the Spark SQL, Pandas API on Spark, Structured Streaming, and DataFrame-based MLlib surfaces that support Spark Connect.
- Spark Connect protobuf definitions: the authoritative Spark repository location for `base.proto`, `relations.proto`, `expressions.proto`, `commands.proto`, `types.proto`, and related protocol files.

4.2 The Main Files

The Spark Connect layer lives mainly in `crates/sail-spark-connect`.

File	Role
<code>src/server.rs</code>	Implements the Spark Connect gRPC service
<code>src/service/plan_executor.rs</code>	Executes relations, commands, streaming operations, interrupts, and reattach/release
<code>src/service/plan_analyzer.rs</code>	Handles schema, explain, tree string, version, DDL parsing, streaming checks
<code>src/service/config_manager.rs</code>	Handles runtime Spark config operations
<code>src/service/artifact_manager.rs</code>	Placeholder for artifact upload/status support
<code>src/proto/plan.rs</code>	Converts Spark Connect relations and commands into Sail specs
<code>src/proto/expression.rs</code>	Converts Spark Connect expressions
<code>src/proto/data_type*.rs</code>	Converts Spark, Arrow, JSON, and DDL data types
<code>src/executor.rs</code>	Converts DataFusion <code>RecordBatch</code> streams into Spark Connect response batches
<code>src/session.rs</code>	Stores Spark-session state inside DataFusion’s <code>SessionContext</code>
<code>src/session_manager.rs</code>	Creates sessions with Spark-specific extensions
<code>src/error.rs</code>	Converts planning/execution errors into Spark-compatible gRPC statuses

The server implementation is generated-facing. The rest of the crate is translation-facing.

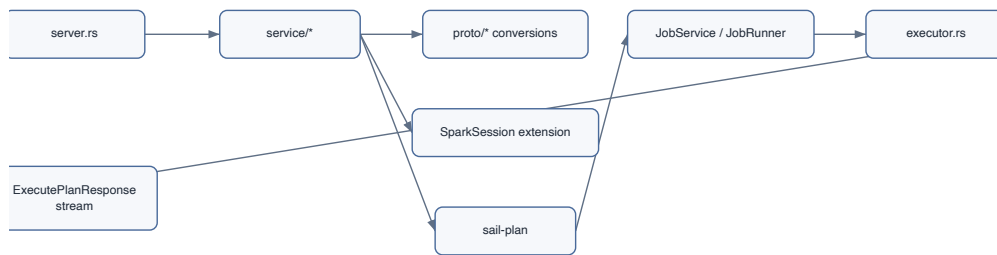


Figure 8: Flowchart 03.1

4.3 The gRPC Service Surface

SparkConnectServer in `crates/sail-spark-connect/src/server.rs` implements Spark’s generated SparkConnectService trait. The corresponding public protocol schema lives in Spark’s official Connect protobuf definitions, especially `base.proto` for the service and request/response envelope and `relations.proto/commands.proto` for plan payloads.

The service methods are the server’s public protocol surface:

- `execute_plan`
- `analyze_plan`
- `config`
- `add_artifacts`
- `artifact_status`
- `interrupt`
- `reattach_execute`
- `release_execute`
- `release_session`
- `fetch_error_details`
- `clone_session`

Some are fully implemented, some are partial, and some are explicit TODOs. That is normal for a compatibility server: Spark Connect is broad, and Sail implements the pieces needed by its Spark SQL/DataFrame compatibility goals first.

The official PySpark API exposes many of these protocol features through normal SparkSession methods. For example, the Spark Session API reference lists Spark Connect-only methods and client operations such as artifacts, progress handlers, tags, and operation interrupts.

The most important method is `execute_plan`.

```

async fn execute_plan(
    &self,

```

```

    request: Request<ExecutePlanRequest>,
) -> Result<Response<Self::ExecutePlanStream>, Status>

```

Its flow is simple and crucial:

1. Extract request metadata.
2. Get or create a session context.
3. Require that the request contains a plan.
4. Dispatch `Root(relation)` to relation execution.
5. Dispatch `Command(command)` to command execution.
6. Return an `ExecutePlanResponseStream`.

In code, the split is:

```

match op {
  plan::OpType::Root(relation) => {
    service::handle_execute_relation(&ctx, relation, metadata).await?
  }
  plan::OpType::Command(Command { command_type: command }) => {
    let command = command.required("command"?);
    handle_command(&ctx, command, metadata).await?
  }
  plan::OpType::CompressedOperation(_) => {
    return Err(Status::unimplemented("compressed operation plan"));
  }
}

```

Spark Connect calls the top-level operation a Plan, but Sail immediately separates it into the two categories that matter to a query engine:

Relation: produces a table-like result

Command: changes state, writes data, registers things, starts streams, or returns command metadata

4.4 Sessions: Spark State Inside DataFusion State

Every Spark Connect request carries a session ID and optional user context. On the client side, PySpark users create this remote session with `SparkSession.builder.remote("sc://host:port")`, documented in the official `builder.remote` API reference. Sail uses the request's session values to get a DataFusion `SessionContext`:

```

self.session_manager
    .get_or_create_session_context(session_id, user_id)
    .await

```

The Spark-specific session setup happens in `crates/sail-spark-connect/src/session_manager.rs`.

`SparkSessionMutator` implements `ServerSessionMutator`. During session creation, it adds two extensions to DataFusion's `SessionConfig`:

- `PlanService`, with Spark-style plan and catalog formatting.

- SparkSession, Sail's Spark-session state object.

```
Ok(config
  .with_extension(Arc::new(plan_service))
  .with_extension(Arc::new(spark)))
```

The SparkSession extension stores:

- session_id
- user_id
- Spark runtime config
- active executors for reattachable operations
- streaming query state

This is one of Sail's clever bridges. Spark Connect expects Spark session behavior. DataFusion expects a SessionContext. Sail installs Spark session semantics into DataFusion session state through typed extensions.

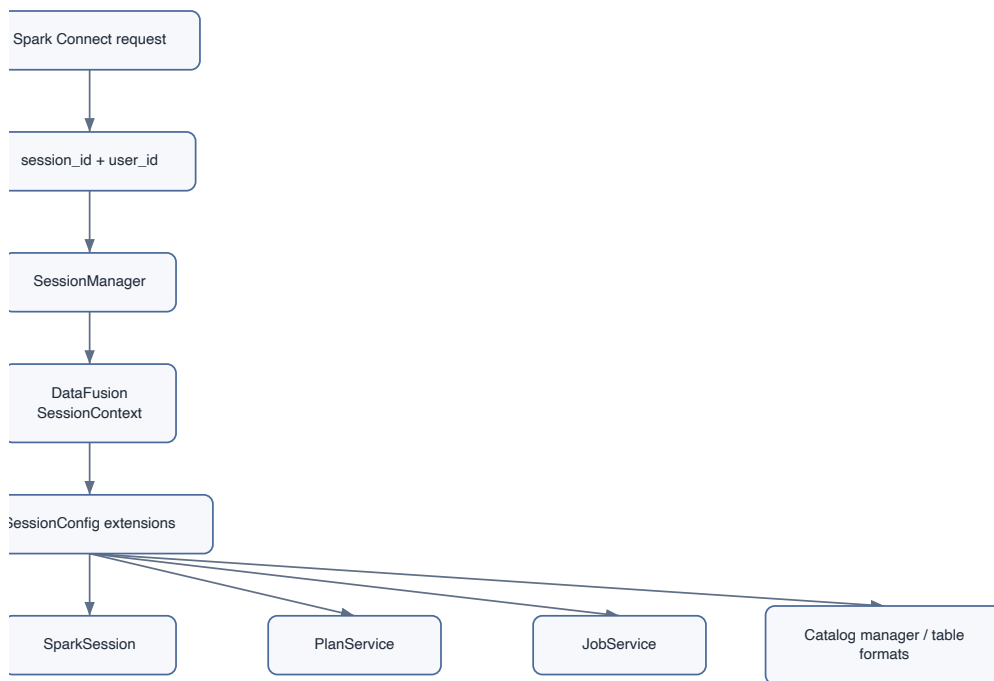


Figure 9: Flowchart 03.2

4.5 Relations Become Sail Specs

The key conversion file is `crates/sail-spark-connect/src/proto/plan.rs`.

Spark Connect relations are protobuf trees. Sail does not plan directly from those protobuf structs. It converts them into `sail_common::spec`. For the upstream protocol shape, read Spark's `relations.proto` in the official Spark Connect protobuf definitions.

The important implementation is:

```
impl TryFrom<Relation> for spec::Plan
```

It extracts:

- `RelationCommon`, including `plan_id`.
- `RelType`, the actual relation variant.
- A `RelationNode`, which is either a query node or a command node.

Then it returns:

```
spec::Plan::Query(spec::QueryPlan { ... })
```

or:

```
spec::Plan::Command(spec::CommandPlan { ... })
```

Examples of relation variants that become query nodes:

- `Read`
- `Project`
- `Filter`
- `Join`
- `SetOp`
- `Sort`
- `Limit`
- `Aggregate`
- `Sql`
- `Range`
- `LocalRelation`
- `Repartition`
- `MapPartitions`
- `CommonInlineUserDefinedTableFunction`

Some relation variants can become commands. This is why `TryFrom<Relation> for spec::Plan` returns a full `spec::Plan`, not only a `QueryPlan`.

The teaching point is that Spark Connect is not Sail's internal language. It is an external compatibility protocol. Sail's internal unresolved language is the Sail spec, because Sail also accepts SQL and needs one common representation for both.

Spark Connect Relation

- > `RelationNode`
- > `spec::QueryNode` or `spec::CommandNode`
- > `PlanResolver`
- > `DataFusion LogicalPlan`

4.6 SQL Is Also Routed Through the Front Door

Spark Connect can send a SQL relation. In `proto/plan.rs`, SQL text is parsed while converting the protobuf relation. This matches the official architecture description: the client sends unresolved intent, and the server analyzes and optimizes it, as described in Apache Spark's Spark Connect architecture page.

```
parse_one_statement(...)  
from_ast_statement(...)
```

This is an important compatibility detail. From the client's point of view:

```
spark.sql("select * from t")
```

is still a Spark Connect request. From Sail's point of view, it becomes a Sail spec through the same general conversion pipeline as DataFrame relations.

That gives Sail one downstream planning path:

```
Spark DataFrame relation  
  -> Sail spec  
SQL string  
  -> Sail SQL parser/analyzer  
  -> Sail spec  
Sail spec  
  -> DataFusion logical plan
```

The cost of this approach is compatibility work. Spark SQL has many grammar and semantic quirks. Sail has its own SQL parser and analyzer so the server can accept Spark-shaped SQL without embedding Spark itself.

4.7 Commands

The command dispatcher lives in `handle_command` in `server.rs`.

It routes Spark Connect command variants to service handlers. Examples:

- `RegisterFunction`
- `WriteOperation`
- `CreateDataframeView`
- `WriteOperationV2`
- `SqlCommand`
- `WriteStreamOperationStart`
- `StreamingQueryCommand`
- `GetResourcesCommand`
- `StreamingQueryManagerCommand`
- `RegisterTableFunction`
- `RegisterDataSource`
- `CheckpointCommand`
- `MergeIntoTableCommand`

The split matters because commands often execute eagerly. In `plan_executor.rs`, `ExecutePlanMode` has two variants:

```
enum ExecutePlanMode {  
    Lazy,  
    EagerSilent,  
}
```

Relations use `Lazy`: return a response stream and let the client consume result batches.

Commands often use `EagerSilent`: execute the plan immediately, drain the stream, and return no data unless Spark Connect expects a command result.

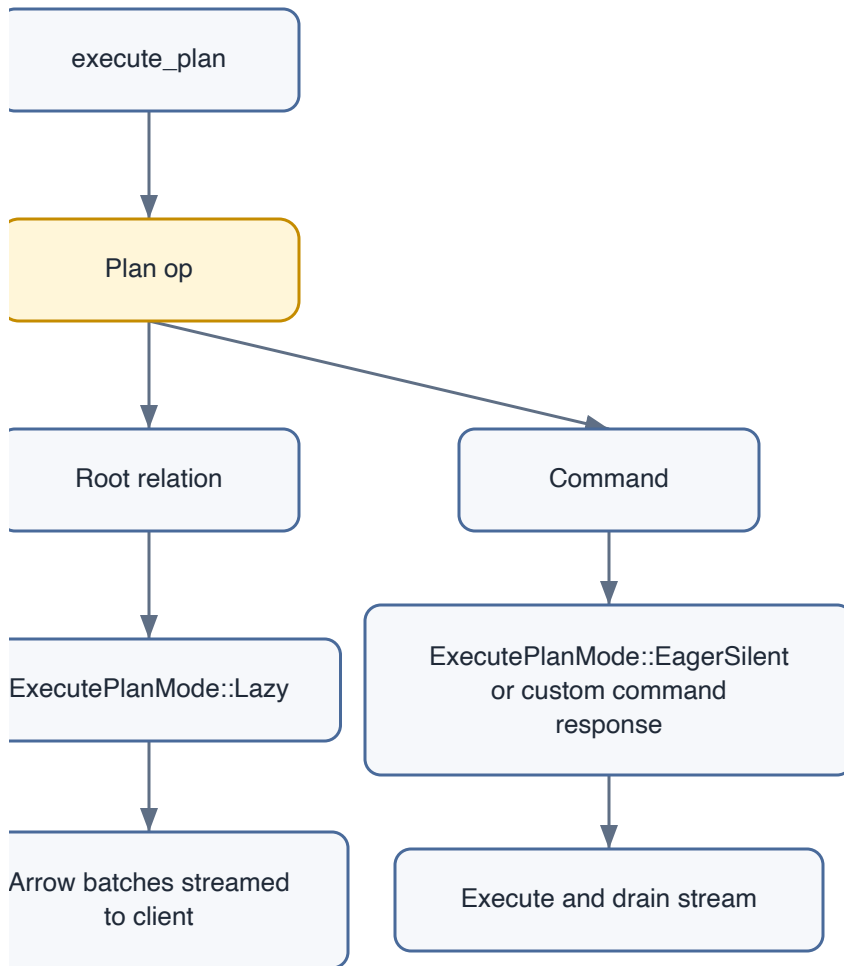


Figure 10: Flowchart 03.3

This is why a Spark statement like `CREATE TABLE` and a query like `SELECT * FROM t` use the same gRPC method but have different execution behavior inside Sail.

4.8 The Core Execution Path

The heart of execution is `handle_execute_plan` in `plan_executor.rs`. This is Sail's version of the official Spark Connect flow where a client sends an encoded unresolved logical plan and receives streamed Apache Arrow batches back over gRPC, described in the Spark Connect Overview.

It does four things:

1. Retrieve the SparkSession extension.
2. Retrieve the JobService extension.
3. Resolve and physical-plan the Sail spec.
4. Execute the physical plan through the session's job runner.

In code:

```
let spark = ctx.extension::<SparkSession>()?;
let service = ctx.extension::<JobService>()?;
let (plan, _) = resolve_and_execute_plan(ctx, spark.plan_config()?, plan).await?;
let stream = service.runner().execute(ctx, plan).await?;
```

That line `service.runner().execute(ctx, plan)` hides the local/cluster choice described in Chapter 2. Spark Connect does not care whether Sail is local, local-cluster, or Kubernetes-cluster. It receives a `SendableRecordBatchStream` either way.

```
spec::Plan
  -> resolve_and_execute_plan
  -> Arc<dyn ExecutionPlan>
  -> JobRunner::execute
  -> SendableRecordBatchStream
```

From there, Spark Connect's job is to translate a DataFusion/Arrow stream into Spark Connect response messages.

4.9 Response Streams and Arrow Batches

`crates/sail-spark-connect/src/executor.rs` owns the response-stream behavior. Apache Spark's Spark Connect docs call out this same result shape: query results are streamed to the client as Apache Arrow record batches rather than returned as one monolithic response.

The important output enum is:

```
pub enum ExecutorBatch {
    ArrowBatch(ArrowBatch),
    SqlCommandResult(Box<SqlCommandResult>),
    WriteStreamOperationStartResult(Box<WriteStreamOperationStartResult>),
    StreamingQueryCommandResult(Box<StreamingQueryCommandResult>),
    StreamingQueryManagerCommandResult(Box<StreamingQueryManagerCommandResult>),
    CheckpointCommandResult(Box<CheckpointCommandResult>),
    Schema(Box<DataType>),
    Complete,
}
```

A running executor first sends the schema, then Arrow batches, then a completion marker:

```
Schema
  -> ArrowBatch
  -> ArrowBatch
```

```
-> ...
-> Complete
```

The Arrow conversion uses Arrow IPC:

```
let cursor = Cursor::new(&mut output.data);
let mut writer = StreamWriter::try_new(cursor, batch.schema().as_ref())?;
writer.write(batch)?;
writer.finish()?;
```

That means Spark Connect sees result data as serialized Arrow streams, not row-by-row JSON or Python objects.

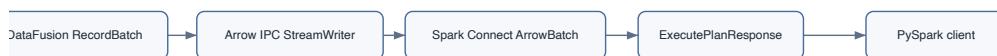


Figure 11: Flowchart 03.4

The row-count field is populated from `batch.num_rows()`, and empty result streams still emit an empty Arrow batch so the client receives schema-consistent output.

4.10 Reattachable Operations

Spark Connect supports reattachable execution. A client can disconnect and later resume reading from an operation. On the Python surface, related operation controls such as tags and interrupts are listed in the official Spark Session API reference.

Sail tracks this with:

- operation_id
- response IDs
- an executor buffer
- SparkSession's executor map

When `execute_plan` receives request options, it checks whether the operation is reattachable:

```
reattachable: is_reattachable(&request.request_options)
```

If the operation is lazy, `handle_execute_plan` creates an Executor, starts it, and registers it in SparkSession:

```
let executor = Executor::new(metadata, stream, heartbeat_interval);
let rx = executor.start()?;
spark.add_executor(executor)?;
```

The executor saves outputs in a bounded buffer. `reattach_execute` pauses the running executor if necessary, releases already acknowledged responses, and starts the executor again:

```

executor.pause_if_running().await?;
executor.release(response_id)?;
let rx = executor.start()?;

```

release_execute lets the client tell the server which buffered responses can be dropped.

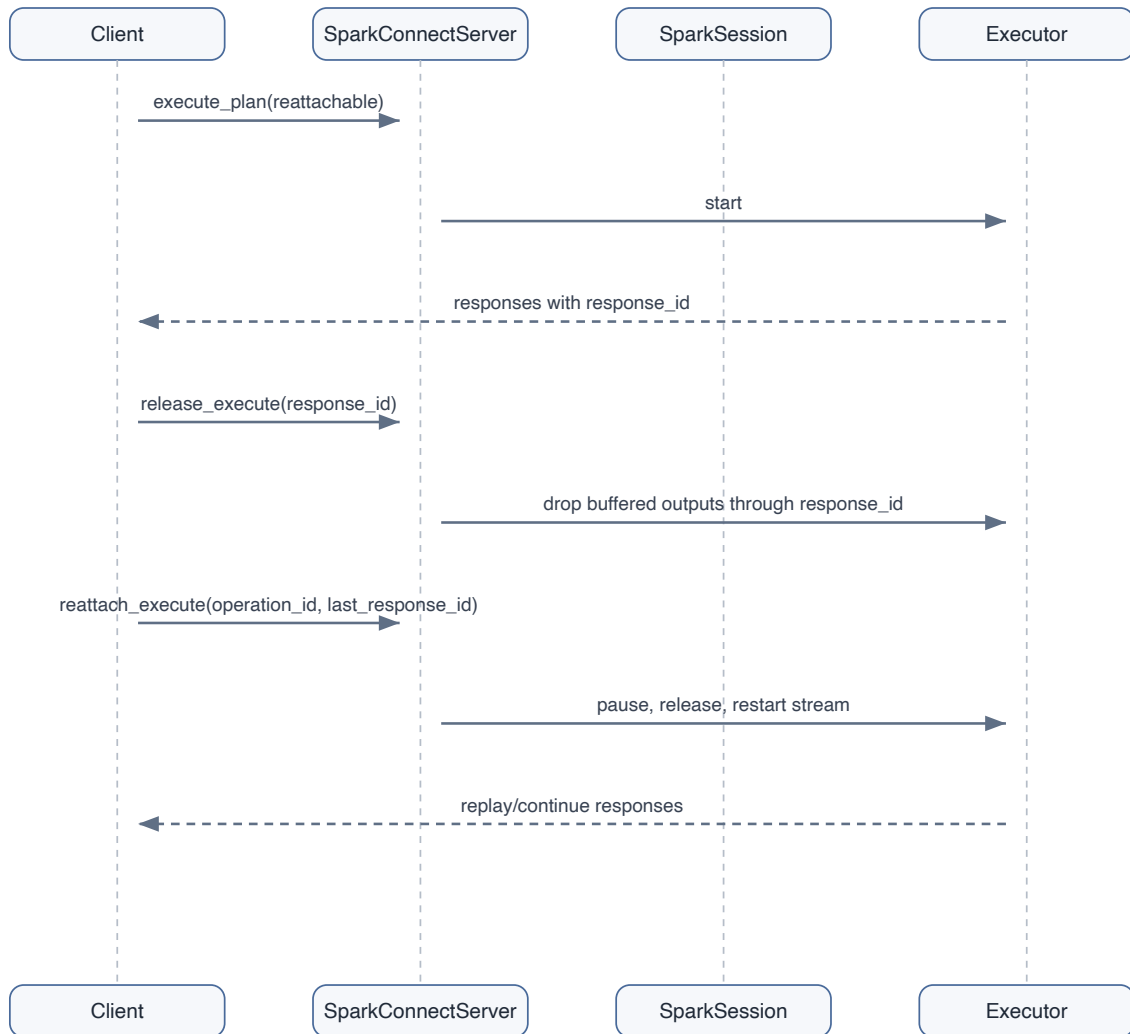


Figure 12: Sequence diagram 03.5

This is a protocol-level feature, but it shapes execution internals. Sail cannot simply return an anonymous stream and forget it. It must store enough operation state in the Spark session to pause, replay, release, or interrupt it.

4.11 Interrupts

The interrupt endpoint supports three modes, corresponding to Spark-session operation controls exposed in PySpark:

- Interrupt all operations in the session.
- Interrupt operations with a tag.
- Interrupt one operation ID.

The service functions remove matching executors from `SparkSession`, pause them if running, and return interrupted operation IDs.

This is another reason `SparkSession` is not just a bag of configuration. It is operational state for Spark Connect behavior.

`InterruptRequest`

- > find executor(s)
- > remove from session state
- > pause if running
- > return operation IDs

4.12 Analyze Plan

`analyze_plan` serves Spark client introspection calls. It does not usually execute data. It answers questions about a plan. These calls back the ordinary PySpark APIs whose Spark Connect support is documented throughout the PySpark API reference.

Implemented or partially implemented analysis handlers include:

- Schema
- Explain
- TreeString
- IsLocal
- IsStreaming
- SparkVersion
- DdlParse
- Persist
- Unpersist
- GetStorageLevel
- JsonToDdl

The schema path is worth reading:

```
let resolver = PlanResolver::new(ctx, spark.plan_config()?);
let NamedPlan { plan, fields } = resolver
  .resolve_named_plan(spec::Plan::Query(plan.try_into()?))
  .await?;
let schema = ...
to_spark_schema(schema)
```

This uses Sail's normal plan resolver, but stops at schema. That means analysis is semantically meaningful: schema answers come from the same resolution machinery that execution uses.

`Explain` also routes through Sail planning:

```
explain_string(
  ctx,
  spark.plan_config()?,
  spec::Plan::Query(plan.try_into()?),
  options,
).await
```

The Spark Connect layer therefore has two major plan paths:

```
execute_plan: convert -> resolve -> optimize -> physical plan -> execute
analyze_plan: convert -> resolve/explain/schema -> response
```

Extensions must work in both. If an extension function works during execution but schema analysis cannot resolve it, PySpark users will still see failures because clients often ask for schema before collecting data.

4.13 Config

The config endpoint manipulates Spark runtime configuration stored in `SparkSession`. The user-facing API is `SparkSession.conf`, documented as part of the official Spark Session reference.

Handlers in `config_manager.rs` implement:

- Get
- Set
- GetWithDefault
- GetOption
- GetAll
- Unset
- IsModifiable

All of these retrieve the typed `SparkSession` extension:

```
let spark = ctx.extension::<SparkSession>()?;
```

Then they delegate to `SparkSession` methods such as `get_config`, `set_config`, and `unset_config`.

This is separate from DataFusion's `SessionConfig` options. That distinction matters:

Spark runtime config:

- stored in `SparkSession` state
- visible through Spark Connect config API
- used to create `PlanConfig` during planning

DataFusion `SessionConfig`:

- created during session construction
- stores DataFusion options and typed extensions
- read by DataFusion planning/execution

Extensions often need both. A Spark-facing option might arrive through `spark.conf.set(...)`, but a DataFusion optimizer rule may need to read a typed extension or session option during planning.

4.14 Data Types and Spark Compatibility

Spark Connect forces Sail to translate types carefully. The official protocol definitions for types live in Spark's `types.proto` under the Connect protobuf definitions.

`proto/data_type_arrow.rs` maps Arrow fields and data types back into Spark Connect `DataType` messages. It handles ordinary Arrow types and extension cases such as:

- Spark UDT metadata.
- GeoArrow WKB extension types mapped to Spark geometry/geography.
- Variant extension types.

This conversion sits on the output side of planning and execution. DataFusion and Arrow may represent a type one way, but PySpark expects Spark Connect's data type model.

One subtle example is timestamps:

```
Arrow Timestamp(Microsecond, None)
  -> Spark TimestampNtz
```

```
Arrow Timestamp(Microsecond, Some(_))
  -> Spark Timestamp
```

The front door therefore constrains internal semantics. Sail can use Arrow/DataFusion internally, but it must preserve enough Spark meaning for the client.

4.15 Errors Become Spark Exceptions

Spark Connect clients expect Spark-shaped errors, not arbitrary Rust error strings.

`crates/sail-spark-connect/src/error.rs` converts many internal errors into `SparkError`, then into `tonic::Status`.

The mapping eventually produces a `SparkThrowable` with Spark/Java class names such as:

- `org.apache.spark.sql.AnalysisException`
- `org.apache.spark.sql.execution.QueryExecutionException`
- `java.lang.IllegalArgumentException`
- `java.lang.ArithmeticException`
- `org.apache.spark.api.python.PythonException`
- `java.time.DateTimeException`
- `java.lang.UnsupportedOperationException`

The status conversion intentionally uses Spark-compatible error details:

```
details.set_error_info(class, "org.apache.spark", metadata);
Status::with_error_details(Code::Internal, message, details)
```


It also truncates long gRPC messages to stay below metadata limits. That sounds mundane until you remember Python tracebacks can be long. Protocol compatibility includes boring survival details like this.

4.16 Artifacts and Python Distribution

Spark Connect includes artifact upload and status endpoints. In Sail, `artifact_manager.rs` currently returns TODO errors for add/status handling. The user-facing artifact APIs are listed in the official Spark Session API reference as `addArtifact` and `addArtifacts`.

This matters for the extension story. Spark’s artifact mechanism is one way clients distribute files, Python dependencies, or resources. Discussion #2001, however, focuses more directly on Python entry-point based extension discovery:

```
[project.entry-points."pysail.extensions"]
sedona = "pysail_sedona:register"
```

Those are different mechanisms:

Spark Connect artifacts:

- client sends files/resources to a session

Python entry-point extensions:

- installed Python package contributes Sail extension behavior

A complete extension system may eventually touch both, but they solve different problems.

4.17 Registering Functions and Data Sources

Spark Connect commands include function and data source registration.

`RegisterFunction` becomes a Sail command plan. The broader PySpark function registration APIs, including UDF and UDTF registration, are indexed in the official PySpark SQL API reference:

```
spec::CommandNode::RegisterFunction(udf.try_into())
```

Then it runs through normal planning and command execution.

`RegisterDataSource` has a more direct session-scoped path. The handler extracts the pickled Python data source class and registers a `PythonTableFormat` in the session’s `TableFormatRegistry`:

```
let format = Arc::new(PythonTableFormat::with_pickled_class(name.clone(),
command));
registry.register(format)
```

This is a small preview of extension behavior. A client can contribute behavior to a session, but the contribution is still routed through Sail’s typed session services and DataFusion planning interfaces.

4.18 What Spark Connect Means for Extensions

Discussion #2001 is not only about Rust-side plugin ergonomics. Spark Connect adds several extra requirements.

First, extensions must be visible during analysis as well as execution.

PySpark frequently asks for schema, explain output, local/streaming status, or type conversion before actual execution. An extension function has to resolve in `analyze_plan`, not just in `execute_plan`.

Second, extension configuration may arrive through Spark config.

If a user writes:

```
spark.conf.set("sail.sedona.join.index", "rtree")
```

then the extension must decide how that value becomes available to the optimizer or physical planner. Storing it only in `SparkSession` may not be enough if `DataFusion` rules expect `SessionConfig` extensions.

Third, extensions must preserve Spark Connect type compatibility.

A spatial extension may expose geometry/geography types. Sail already maps `GeoArrow` meta-data in the Arrow-to-Spark conversion path. A third-party extension must either reuse those conventions or provide a compatible type conversion story.

Fourth, distributed execution still matters.

Spark Connect receives one logical operation from PySpark, but Sail may execute it on remote workers. A function registered through Spark Connect must be available when the worker decodes and executes the physical plan.

Fifth, Spark Connect itself provides extension hooks.

The protocol defines `Relation.extension`, `Command.extension`, and `Expression.extension`, each typed as `google.protobuf.Any`. These let a client send an opaque payload that Sail can dispatch by `type_url`. Today Sail does not have a general dispatcher for these messages, but chapter 13 proposes them as the natural plan-time extension boundary: protobuf-versioned, language-neutral, and already crossing every query. In that framing the Rust trait surface becomes the execution-time boundary, and Spark Connect dispatch becomes the plan-time one.

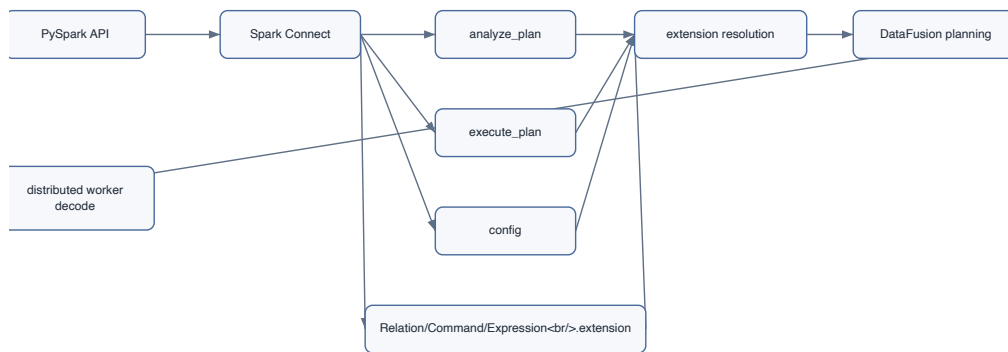


Figure 13: Flowchart 03.6

A good extension API must therefore cross the Spark Connect boundary, not sit behind it.

4.19 Reading Exercises

1. Read `crates/sail-spark-connect/src/server.rs`.
 - Find each gRPC method.
 - For `execute_plan`, identify where session lookup happens and where relation/command dispatch happens.
2. Read `crates/sail-spark-connect/src/service/plan_executor.rs`.
 - Follow `handle_execute_relation`.
 - Follow `handle_execute_plan`.
 - Compare `Lazy` and `EagerSilent`.
 - Find `reattach` and `release` handling.
3. Read `crates/sail-spark-connect/src/executor.rs`.
 - Find where `schema` is emitted.
 - Find where `RecordBatch` becomes `ArrowBatch`.
 - Find the executor buffer used for `reattachable` operations.
4. Read `crates/sail-spark-connect/src/proto/plan.rs`.
 - Follow `TryFrom<Relation>` for `spec::Plan`.
 - Pick one relation variant, such as `Project` or `Filter`, and trace the conversion into `spec::QueryNode`.
5. Read `crates/sail-spark-connect/src/service/plan_analyzer.rs`.
 - Follow `schema analysis`.
 - Follow `explain analysis`.
 - Notice which analysis requests are `TODOs` or `no-ops`.
6. Read `crates/sail-spark-connect/src/error.rs`.
 - Find how `PlanError` and `ExecutionError` become `SparkError`.
 - Find how `SparkError` becomes a gRPC `Status`.

4.20 Chapter Takeaways

Spark Connect is the compatibility contract between PySpark and Sail. It gives Sail a Spark-shaped protocol while letting the engine be Rust, Arrow, and DataFusion.

Inside Sail, Spark Connect requests are translated into Sail specs, resolved into DataFusion plans, executed through a JobRunner, and streamed back as Arrow batches. Sessions carry Spark-specific state through typed DataFusion extensions. Analysis, configuration, reattach/release, interrupts, and errors are all part of the compatibility surface.

For extensions, Spark Connect raises the bar. An extension must work during analysis, execution, configuration, output type conversion, error handling, and distributed worker execution. That is why the extension proposal cannot be just “let users register a UDF.” Spark Connect makes extension behavior user-visible before, during, and after query execution.

The next chapter moves from the protocol front door to the Python experience: `pysail`, PySpark, Python UDFs, Python data sources, and how Python packaging could become the extension discovery mechanism.

5 Chapter 4: PySpark and pysail

PySpark is the user experience Sail tries to preserve. `pysail` is the Python package that makes Sail feel like something a Python developer can install, start, test, and use from ordinary PySpark code.

The design is intentionally asymmetric:

PySpark remains the client API.
`pysail` starts and packages the Rust engine.
Spark Connect is the wire protocol between them.

That is why Sail can claim that no PySpark code rewrites are needed once the user connects to a Sail server. A PySpark program still imports `pyspark.sql.Session`; the difference is the remote URL:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.remote("sc://localhost:50051").getOrCreate()
spark.sql("SELECT 1 + 1").show()
```

The official PySpark entry point for this is `SparkSession.builder.remote`. Sail’s job is to provide a compatible Spark Connect server at that address.

5.1 The Main Files

File	Role
pyproject.toml	Python package metadata, build backend, optional dependencies, test matrices
python/pysail/spark/__init__.py	Public Python wrapper for SparkConnectServer
python/pysail/cli.py and python/pysail/__main__.py	Python entry points into the Sail CLI
crates/sail-python/src/lib.rs	PyO3 _native module registration
crates/sail-python/src/spark/server.rs	Native Python class that starts the Spark Connect server
crates/sail-python/src/globals.rs	Global runtime, config, telemetry, and environment snapshot
crates/sail-python-udf/*	Python UDF, UDAF, UDTF, Pandas, and Arrow execution support
crates/sail-plan/src/resolver/expression/udf.rs	Converts Spark Connect inline Python UDFs into DataFusion UDF expressions
python/pysail/tests/spark/conftest.py	How Sail's own tests create a PySpark client connected to Sail

The code divides into two worlds:

User-facing Python package:

pysail, pysail.spark, sail CLI

Engine-facing Rust/PyO3 bindings:

_native module, SparkConnectServer, Python UDF runtime

5.2 Package Shape

The Python package is defined in `pyproject.toml`.

It is named `pysail`, supports Python `>=3.10,<3.15`, and is built with `maturin`. That tells you the package is not pure Python. It ships a compiled Rust extension module:

```
[build-system]
requires = ["maturin>=1.0,<2.0"]
build-backend = "maturin"
```

The package entry point is:

```
[project.scripts]
sail = "pysail.cli:main"
```

So the installed `sail` command is a Python console script, but the Python script immediately delegates to Rust:

```
from pysail import _native

def main():
    _native.main(sys.argv)
```

The native module is built by `crates/sail-python/src/lib.rs`:

```
#[pymodule]
fn _native(m: &Bound<'_, PyModule>) -> PyResult<()> {
    flight::register_module(m)?;
    spark::register_module(m)?;
    m.add_function(wrap_pyfunction!(cli::main, m))?;
    m.add("_SAIL_VERSION", env!("CARGO_PKG_VERSION"))?;
    Ok(())
}
```

That module exposes:

- `pysail._native.main`
- `pysail._native.spark.SparkConnectServer`
- Flight-related bindings
- `_SAIL_VERSION`

The package layout is a useful lesson in Rust/Python hybrid projects:

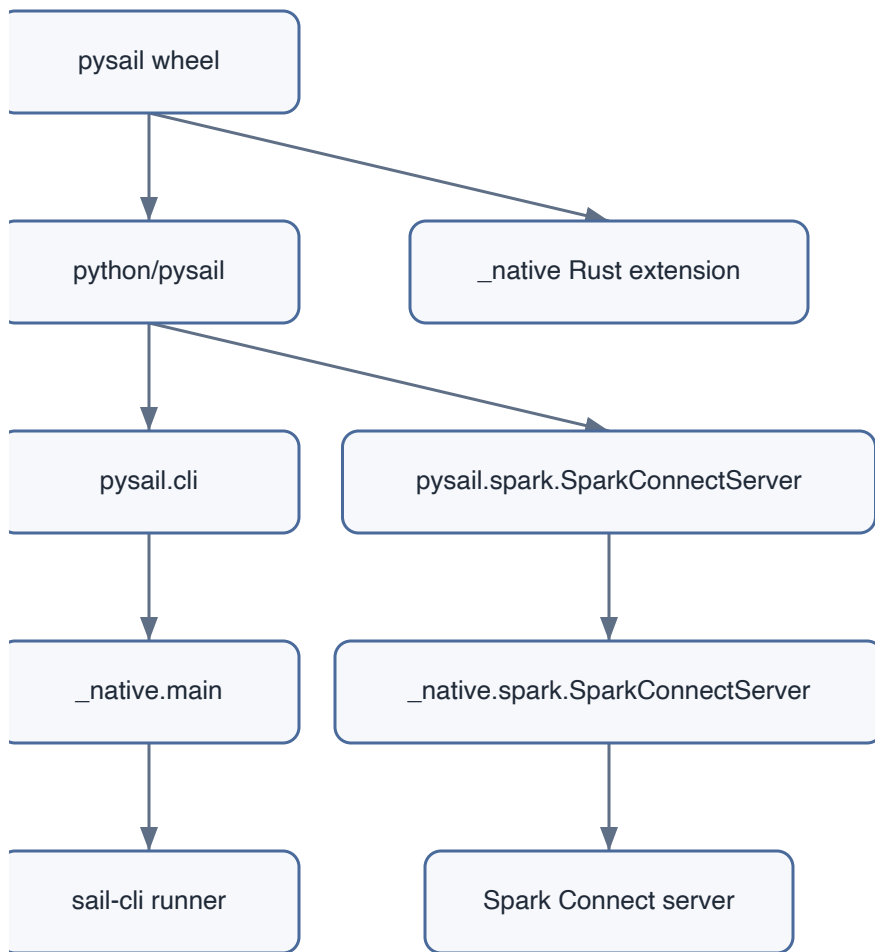


Figure 14: Flowchart 04.1

5.3 Starting Sail From Python

The public Python wrapper is tiny:

```

class SparkConnectServer:
    def __init__(self, ip: str = "127.0.0.1", port: int = 0) -> None:
        self._inner = _native.spark.SparkConnectServer(ip, port)

    def start(self, *, background=True) -> None:

```

```

        self._inner.start(background=background)

    def stop(self) -> None:
        self._inner.stop()

    @property
    def listening_address(self) -> tuple[str, int] | None:
        return self._inner.listening_address

```

The real work happens in Rust, in `crates/sail-python/src/spark/server.rs`.

The PyO3 class:

- loads AppConfig
- initializes or retrieves global runtime state
- binds a TCP listener
- starts the Spark Connect server
- records the actual listening address
- can run in the background or block the calling thread
- can shut down through a one-shot channel

The most important method is `start`:

```

let listener = self
    .runtime
    .primary()
    .block_on(TcpListener::bind(address))?;
self.state = Some(self.run(listener)?);

```

If the user passes port 0, the OS chooses an available port. The actual address is exposed through `listening_address`. Sail's tests use exactly that:

```

server = SparkConnectServer("127.0.0.1", 0)
server.start(background=True)
_, port = server.listening_address
yield f"sc://localhost:{port}"
server.stop()

```

That is the local development loop in one picture:

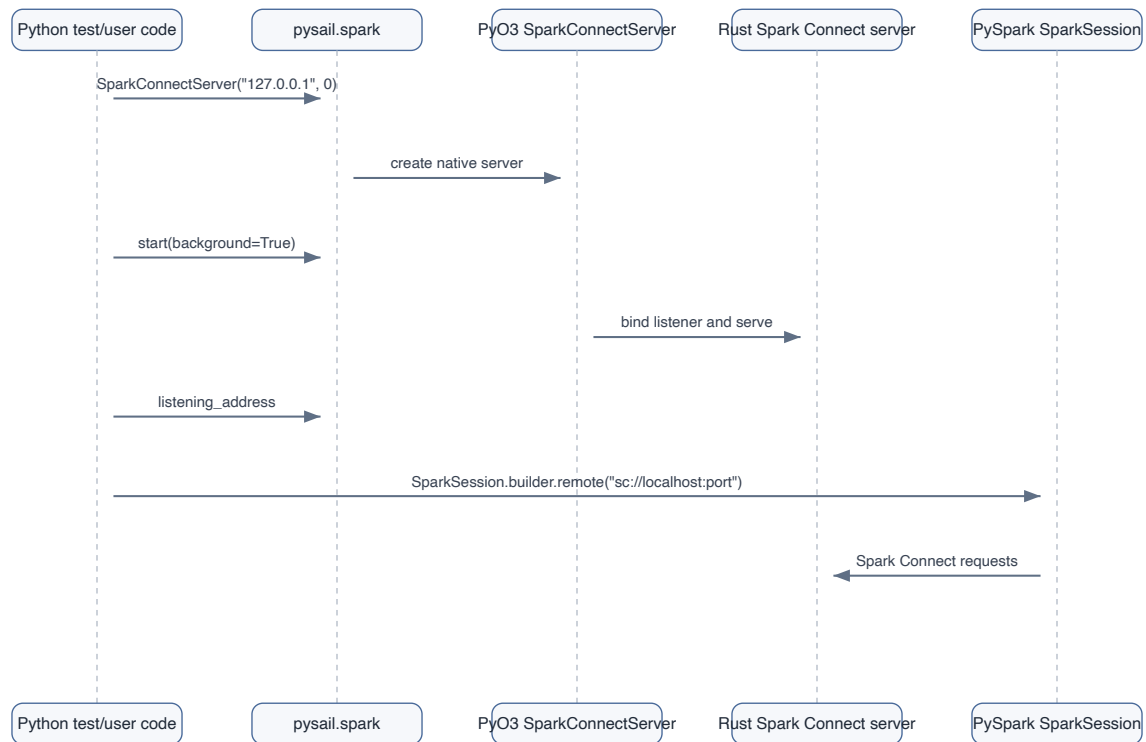


Figure 15: Sequence diagram 04.2

5.4 The Global Runtime

crates/sail-python/src/globals.rs contains GlobalState.

This is where pysail creates a global Sail runtime and initializes telemetry. It uses PyOnceLock so initialization happens once per Python interpreter:

```
static GLOBALS: PyOnceLock<GlobalState> = PyOnceLock::new();
```

GlobalState contains:

- a RuntimeManager
- an EnvironmentSnapshot

The environment snapshot matters because Sail configuration is environment-variable driven. Some environment variables are effectively static once the runtime and telemetry have been initialized. If they change afterward, pysail warns that the changes are ignored.

This is one of those systems details that looks small but saves debugging time. Python users often set environment variables inside notebooks or test processes. Sail has to explain when that is too late.

```
import pysail._native
-> load AppConfig
-> create runtime
```

```
-> initialize telemetry
-> snapshot Sail environment variables
```

5.5 Releasing the GIL

When Python calls into Rust and Rust blocks, Python's global interpreter lock can prevent other Python code from running. That is dangerous for Sail because Python UDFs may need to run while the server is active.

The server code explicitly uses `Python::detach`.

In `SparkConnectServerState::wait`, the comment says the method should be called within `Python::detach`; otherwise, the GIL is not released and Python UDFs will be blocked when the server handles client requests.

The blocking CLI path does the same:

```
py.detach(move || {
    sail_cli::runner::main(args)
})
```

This is an important Rust/Python boundary rule:

Long-running Rust server work should not hold the Python GIL.

Without that, Sail could start fine and then mysteriously deadlock or starve Python UDF execution.

5.6 Connecting With PySpark

Sail's own tests show the intended user pattern in `python/pysail/tests/spark/conftest.py`:

```
spark = SparkSession.builder.remote(remote).getOrCreate()
```

Then the test fixture configures the session:

```
session.conf.set("spark.sql.session.timeZone", "UTC")
session.conf.set("spark.sql.ansi.enabled", "true")
session.conf.set("spark.sql.execution.arrow.pyspark.enabled", "true")
```

These are ordinary PySpark calls. They go through Spark Connect and reach Sail's config/session machinery. The fixture then tests Sail through the normal PySpark surface: SQL, DataFrames, functions, catalog calls, writes, UDFs, streaming, and lakehouse features.

The official PySpark reference documents the broader Spark SQL API at `pyspark.sql`, and the main API index notes that Spark SQL, Structured Streaming, and DataFrame-based MLlib support Spark Connect through the Python API surface.

The user sees:

```
spark.range(10).where("id % 2 = 0").count()
```

Sail sees:

Spark Connect relation tree

- > Sail spec
- > DataFusion logical plan
- > DataFusion physical plan
- > Arrow result batches

5.7 pysail Is Not a PySpark Fork

This is subtle but central. pysail does not replace PySpark classes like DataFrame, Column, or SparkSession. It starts a server that PySpark can talk to.

That means compatibility is mostly tested at the protocol/API behavior level:

- Does PySpark emit a Spark Connect plan Sail can understand?
- Does Sail return the schema PySpark expects?
- Does Sail return Arrow batches PySpark can decode?
- Do errors look like Spark errors?
- Do config, UDF, catalog, write, and streaming operations behave like Spark?

This is why the test dependencies include `pyspark[connect]` in development and multiple Spark versions in test matrices:

```
[[tool.hatch.envs.test.matrix]]
spark = ["3.5.7", "4.0.1", "4.1.1"]
```

The engine is Sail. The client is still PySpark.

5.8 Python UDFs Enter Through Spark Connect

PySpark UDFs are user-provided Python functions. In Spark Connect, the function is serialized into the request and sent to the server.

Sail resolves those inline Python UDFs in `crates/sail-plan/src/resolver/expression/udf.rs`.

The resolver receives a `spec::CommonInlineUserDefinedFunction`, extracts:

- function name
- determinism
- distinct flag
- arguments
- serialized function payload

Then it builds a `PySparkUdfPayload` and wraps it in a `DataFusion ScalarUDF` or `AggregateUDF`.

For scalar UDFs:

```
let udf = PySparkUDF::new(
    PySparkUdfKind::Batch,
    get_udf_name(name, &payload),
    payload,
    deterministic,
    input_types,
```

```

        function.output_type,
        self.config.pyspark_udf_config.clone(),
    );
Ok(Expr::ScalarFunction(Expr::ScalarFunction {
    func: Arc::new(ScalarUDF::from(udf)),
    args: arguments,
}))

```

For grouped aggregate UDFs, Sail creates a `PySparkGroupAggregateUDF` and returns a `DataFusion` aggregate expression.

The key idea is:

```

Python function payload
-> Sail UDF payload
-> DataFusion ScalarUDF/AggregateUDF
-> executable physical plan

```

The official PySpark UDF APIs are:

- `pyspark.sql.functions.udf`
- `pyspark.sql.functions.pandas_udf`
- `pyspark.sql.functions.udtf`
- `pyspark.sql.DataFrame.mapInArrow`

5.9 UDF Kinds

`crates/sail-python-udf/src/udf/pyspark_udf.rs` defines the scalar UDF kinds Sail supports:

```

pub enum PySparkUdfKind {
    Batch,
    ArrowBatch,
    ScalarPandas,
    ScalarPandasIter,
    ScalarArrow,
    ScalarArrowIter,
}

```

The resolver maps Spark eval types to these internal UDF kinds:

Spark/PySpark style	Sail internal kind	Python-side data shape
regular row-oriented UDF	Batch	Python values
Arrow-optimized batch UDF	ArrowBatch	Arrow-backed batches
Pandas scalar UDF	ScalarPandas	<code>pandas.Series</code>
Pandas scalar iterator UDF	ScalarPandasIter	iterator of <code>pandas.Series</code>
Arrow scalar UDF	ScalarArrow	<code>pyarrow.Array</code>
Arrow scalar iterator UDF	ScalarArrowIter	iterator of <code>pyarrow.Array</code>

The official PySpark docs describe these APIs from the user's point of view. Sail's code answers the engine question: what kind of object should DataFusion execute when such a function appears in a query plan?

5.10 Loading a PySpark UDF Payload

`crates/sail-python-udf/src/cereal/pyspark_udf.rs` handles the serialized UDF payload format.

The payload builder writes:

- eval type
- selected Spark/PySpark config values
- input type metadata for some PySpark versions
- profiling flag
- argument offsets
- keyword argument names when supported
- the serialized Python command bytes

The payload loader calls into PySpark internals:

```
let serializer = PyModule::import(py, intern!(py, "pyspark.serializers"))?
    .getattr(intern!(py, "CPickleSerializer"))?
    .call0()?;
let tuple = PyModule::import(py, intern!(py, "pyspark.worker"))?
    .getattr(intern!(py, "read_udfs"))?
    .call1((serializer, infile, eval_type));
```

This is not an accident. To be compatible with PySpark UDF behavior, Sail reuses PySpark's own worker deserialization conventions. It wants the same Python wrapper behavior Spark users expect.

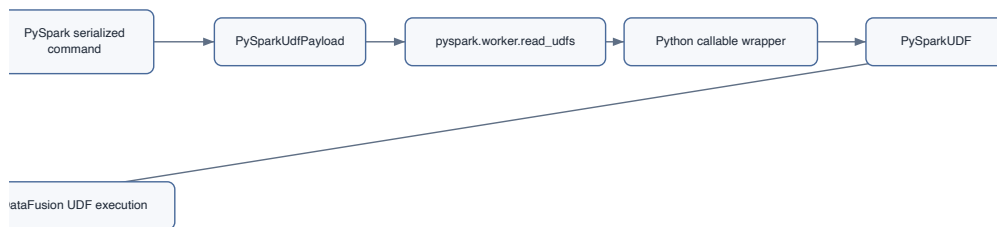


Figure 16: Flowchart 04.3

5.11 Executing Python UDFs in Process

`PySparkUDF` implements DataFusion's `ScalarUDFImpl`.

When DataFusion invokes it, Sail:

1. Converts DataFusion ColumnarValue arguments into Arrow arrays.
2. Attaches to Python.
3. Lazily loads or reuses the Python UDF wrapper.
4. Converts Arrow arrays to Python objects using PyArrow bridges.
5. Calls the Python function wrapper.
6. Converts the result back to Arrow ArrayData.
7. Casts it to the declared output type.

The core execution path is:

```
let args: Vec<ArrayRef> = ColumnarValue::values_to_arrays(&args)?;
let udf = Python::attach(|py| self.udf(py))?;
let data = Python::attach(|py| -> PyUdfResult<_> {
    let output = udf.call1(py, (args.try_to_py(py)?, number_rows))?;
    Ok(ArrayData::try_from_py(py, &output)?)
})?;
let array = cast(&make_array(data), &self.output_type)?;
Ok(ColumnarValue::Array(array))
```

That differs from JVM Spark. In JVM Spark, Python UDF execution typically involves a Python worker process and serialization between JVM and Python. Sail's Python UDF runs in the same process as the Rust execution engine, and Arrow memory can be shared through PyArrow bindings.

The Sail UDF performance docs summarize the motivation: use Pandas or Arrow UDFs when possible so wrapper overhead is amortized over batches, and use Arrow-native UDFs for the most direct Arrow sharing.

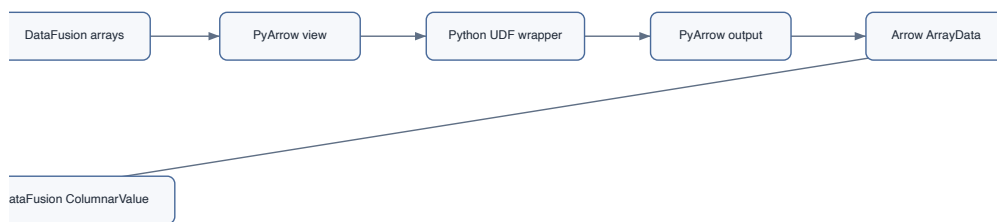


Figure 17: Flowchart 04.4

5.12 Python Conversion Code

The Python helper module embedded in Rust is `crates/sail-python-udf/src/python/spark.py`.

It contains conversion wrappers for:

- scalar Python values
- Pandas UDFs

- Arrow UDFs
- grouped map functions
- co-grouped map functions
- table functions
- Arrow table functions
- UDTF analysis

The Rust side loads that Python code from an embedded string:

```
const MODULE_SOURCE_CODE: &str = include_str!("spark.py");
```

Then `PySpark::module` initializes it once through a `PyOnceLock`.

This is a nice pattern: Sail can ship its Python-side UDF helpers inside the Rust extension module, so it does not need to locate a separate Python file at runtime.

5.13 Config for Python UDF Behavior

`PySparkUdfConfig` captures the Spark/PySpark settings that affect Python UDF behavior:

- session time zone
- Pandas grouped map column assignment
- safe Arrow conversion
- Arrow max records per batch
- Pandas conversion toggles
- int-to-decimal coercion
- binary-as-bytes behavior

It can also emit key-value pairs that PySpark's worker code understands:

```
"spark.sql.session.timeZone"
"spark.sql.execution.arrow.maxRecordsPerBatch"
"spark.sql.execution.pyspark.binaryAsBytes"
```

This shows another compatibility layer. The same Python function may behave differently depending on Spark configuration. Sail has to carry those settings from Spark Connect session state into the UDF payload and wrapper.

5.14 UDTFs and Python-Side Analysis

PySpark UDTFs can have an `analyze` static method. The official `udtf` documentation describes this as Python-side analysis that can return a dynamic schema.

Sail has hooks for this in `crates/sail-python-udf/src/python/spark.rs`:

```
pub fn analyze_udtf<'py>(<
    py: Python<'py>,
    handler: Bound<'py, PyAny>,
    arguments: Bound<'py, PyAny>,
) -> PyResult<Bound<'py, PyAny>> {
    Self::module(py)?
        .getattr(intern!(py, "analyze_udtf"))?
```

```

        .call1((handler, arguments))
    }

```

This matters because analysis happens before physical execution. A UDTF may determine its output schema from argument types or literal values. That means Python code can participate in planning, not just execution.

For the extension proposal, this is a preview of a broader rule:

Extensions may need hooks before execution starts.

5.15 Python Data Sources

Spark Connect can register Python data sources. Sail handles `RegisterDataSource` in `crates/sail-spark-connect/src/service/plan_executor.rs`.

The handler extracts the pickled Python data source class and registers a session-scoped `PythonTableFormat` in the `TableFormatRegistry`:

```

let format = Arc::new(PythonTableFormat::with_pickled_class(name.clone(),
command));
registry.register(format)

```

This is parallel to Python UDF registration:

```

Python behavior arrives through Spark Connect
-> Sail stores it in session-scoped registry
-> later scans can resolve and execute it

```

The important architectural point is session isolation. A registered Python data source belongs to that session's table format registry, not a global singleton shared by all users.

5.16 Testing PySpark Compatibility

Sail's Python tests are themselves a guide to compatibility.

The fixture in `python/pysail/tests/spark/conftest.py` either uses `SPARK_REMOTE` or starts a local Sail Spark Connect server. Then it creates a normal PySpark session:

```

SparkSession.builder.remote(remote).getOrCreate()

```

The tests cover:

- DataFrame behavior
- SQL behavior
- catalog behavior
- joins, aggregation, ordering, sampling, repartitioning
- functions
- Python UDFs, Pandas UDFs, Arrow UDFs, UDTFs
- data sources
- writes
- streaming
- Delta and Iceberg behavior

- TPC-H, TPC-DS, ClickBench plans/results

The test matrix explicitly checks different PySpark versions. That is because Spark Connect is a moving protocol and PySpark’s UDF behavior evolves. Sail has to track both API surface and wire behavior.

5.17 Why PySpark Versions Matter

The UDF payload builder contains version-specific logic:

```
let pyspark_version = get_pyspark_version()?;
...
if matches!(pyspark_version, PySparkVersion::V4_1)
    && matches!(eval_type, spec::PySparkUdfType::ArrowBatched)
{
    let schema_json = build_input_types_json(input_types)?;
    ...
}
```

That is a concrete example of why “Spark compatible” is not a single target. Spark 3.5, Spark 4.0, and Spark 4.1 differ in function support, UDF payload details, UDTF behavior, Arrow APIs, and type handling.

`pyproject.toml` reflects this with test dependencies and test matrices for multiple Spark versions.

5.18 What pysail Means for Extensions

Discussion #2001 proposes Python entry points such as:

```
[project.entry-points."pysail.extensions"]
sedona = "pysail_sedona:register"
```

This is a natural Python packaging experience:

```
pip install pysail pysail-sedona
```

Then, when `pysail` starts, it could discover installed extension packages and register them.

But this chapter should make the hard parts clear.

First, discovery is Python-level, but most extension hooks are Rust/DataFusion-level:

```
Python entry point
-> Rust extension registration
-> DataFusion UDFs, optimizer rules, extension planners, codecs
```

Second, version coupling is strict. A Python wheel that exposes Rust extension objects must match Sail’s arrow, datafusion, pyo3, and `pysail` versions. Rust trait objects are not a stable plugin ABI across arbitrary crate versions.

Third, worker execution must see the same extension behavior. Installing an extension in the client Python environment is not enough if cluster workers cannot decode the physical plan or reconstruct extension UDFs.

Fourth, analysis must work too. If a PySpark client asks for schema or explain output before execution, the extension must be registered before `analyze_plan` resolves the query.

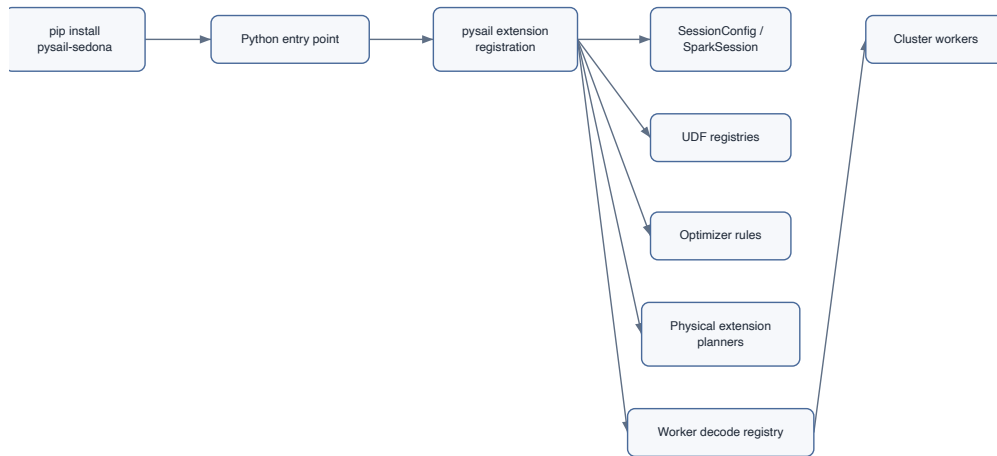


Figure 18: Flowchart 04.5

The pleasant user story is Pythonic. The engine story is Rust and distributed.

5.19 A Small End-to-End Example

Suppose the user writes:

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import udf
from pyspark.sql.types import IntegerType

spark = SparkSession.builder.remote("sc://localhost:50051").getOrCreate()

@udf(returnType=IntegerType())
def plus_one(x):
    return None if x is None else x + 1

spark.range(3).select(plus_one("id")).show()
  
```

Sail's path is:

```

PySpark creates Spark Connect relation containing inline Python UDF
-> sail-spark-connect converts relation to Sail spec
-> PlanResolver resolves CommonInlineUserDefinedFunction
-> PySparkUdfPayload is built
-> PySparkUDF becomes a DataFusion ScalarUDF
-> DataFusion physical plan executes
-> PySparkUDF invokes Python wrapper in process
  
```

- > output Arrow array returns to DataFusion
- > Spark Connect streams ArrowBatch results to PySpark

This is the whole Sail philosophy in miniature: keep the PySpark surface, translate through Spark Connect, execute in Rust/DataFusion/Arrow, and invoke Python only where Python semantics are actually needed.

5.20 Reading Exercises

1. Read `python/pysail/spark/__init__.py`.
 - Notice how small the public wrapper is.
 - Find the `listening_address` property.
2. Read `crates/sail-python/src/spark/server.rs`.
 - Follow `new`, `start`, `run`, and `stop`.
 - Find where `Python::detach` is used.
3. Read `crates/sail-python/src/globals.rs`.
 - Follow global runtime initialization.
 - Find environment-variable warning behavior.
4. Read `python/pysail/tests/spark/conftest.py`.
 - Follow the remote fixture.
 - Find `SparkSession.builder.remote`.
 - Note which Spark config values tests set by default.
5. Read `crates/sail-plan/src/resolver/expression/udf.rs`.
 - Trace a scalar UDF from Spark Connect spec to DataFusion `ScalarUDF`.
 - Trace a grouped aggregate UDF to `AggregateUDF`.
6. Read `crates/sail-python-udf/src/udf/pyspark_udf.rs`.
 - Find `PySparkUdfKind`.
 - Follow `invoke_with_args`.
7. Read `crates/sail-python-udf/src/cereal/pyspark_udf.rs`.
 - Find payload build and load.
 - Notice the PySpark version-specific logic.
8. Read `crates/sail-python-udf/src/python/spark.py`.
 - Skim the converter classes.
 - Find how Pandas and Arrow wrappers shape Python execution.

5.21 Chapter Takeaways

`pysail` is the Python package that makes Sail usable from Python, but PySpark remains the primary user API. `pysail` starts and packages a Rust Spark Connect server. PySpark connects to that server using `SparkSession.builder.remote`.

Python UDF support is where the layers meet most dramatically. PySpark serializes Python functions into Spark Connect plans. Sail turns those payloads into DataFusion UDFs. Execution invokes Python in process and exchanges Arrow memory through PyArrow bridges. Pandas and Arrow UDFs amortize Python overhead over batches, while Arrow-native functions can share Arrow data most directly.

For extensions, Python packaging gives an attractive discovery story, but the actual extension hooks must reach Rust planning, DataFusion execution, and distributed worker decoding. The final extension architecture has to make that Python-to-Rust bridge explicit.

The next chapter moves into Apache Arrow itself: arrays, schemas, record batches, IPC, PyArrow bridges, Arrow Flight, and why columnar memory is the common currency between Spark Connect, DataFusion, Python UDFs, and Sail's distributed shuffle.

6 Chapter 5: Apache Arrow

Apache Arrow is the data plane hiding in plain sight throughout Sail.

Spark Connect gives Sail a protocol for receiving unresolved Spark plans and returning results to Spark clients. DataFusion gives Sail an optimizer and an execution engine. PySpark compatibility gives Sail a Python surface area. The distributed runtime gives Sail a way to split work across workers. Arrow is the format that lets those pieces hand data to each other without constantly reinterpreting rows.

In Sail, Arrow is not merely an output serialization format. It is the shape of execution itself:

- DataFusion physical plans produce `RecordBatch` streams.
- Spark Connect responses carry Arrow IPC payloads.
- Python UDFs convert Rust Arrow arrays to PyArrow arrays and back.
- Shuffle partitions batches into per-task streams.
- Flight SQL encodes `SendableRecordBatchStream` as Arrow Flight data.
- Extension types preserve domain semantics such as geometry, geography, and variants.

This chapter is about learning Arrow through Sail's code. We will use the official Arrow documentation as the reference vocabulary, then map that vocabulary to the concrete Rust modules that make Sail work.

6.1 Definitive References

Keep these open while reading the chapter:

- Apache Arrow Columnar Format
- Arrow Flight RPC protocol
- PyArrow documentation
- PyArrow `RecordBatch` API
- DataFusion gentle Arrow introduction

The Arrow columnar format specification is the most important one. It defines the memory layout, data types, schemas, record batches, and IPC messages. The Flight specification explains the network layer built on Arrow IPC and gRPC. The PyArrow documentation matters because Sail crosses the Rust/Python boundary for PySpark UDFs.

6.2 Where Arrow Lives In Sail

Here are the main code paths for this chapter:

Area	Files	Arrow role
Spark Connect result streaming	<code>crates/sail-spark-connect/src/executor.rs</code>	Converts RecordBatch values into Spark Connect ArrowBatch messages using Arrow IPC
Spark schema conversion	<code>crates/sail-spark-connect/src/proto/data_type_arrow.rs</code>	Maps Arrow fields and data types back to Spark Connect data types
Python UDF conversion	<code>crates/sail-python-udf/src/conversion.rs</code>	Converts Rust Arrow objects to/from PyArrow objects
PySpark UDF execution	<code>crates/sail-python-udf/src/udf/pyspark_udf.rs</code>	Invokes Python functions with Arrow arrays and receives Arrow data
Distributed shuffle write	<code>crates/sail-execution/src/plan/shuffle_write.rs</code>	Partitions RecordBatch streams into shuffle outputs
Distributed shuffle read	<code>crates/sail-execution/src/plan/shuffle_read.rs</code>	Opens shuffle locations and merges RecordBatch streams
Arrow Flight SQL	<code>crates/sail-flight/src/service.rs</code>	Serves DataFusion output streams over Flight SQL
Physical plan examples	<code>crates/sail-physical-plan/src/range.rs</code>	Builds batches from Arrow arrays
Row round-robin repartition	<code>crates/sail-physical-plan/src/repartition.rs</code>	Uses Arrow compute kernels to split batches
GeoArrow extension type	<code>crates/sail-common/src/geoarrow/extension.rs</code>	Defines Arrow extension metadata for WKB geometry/geography

The short version is:

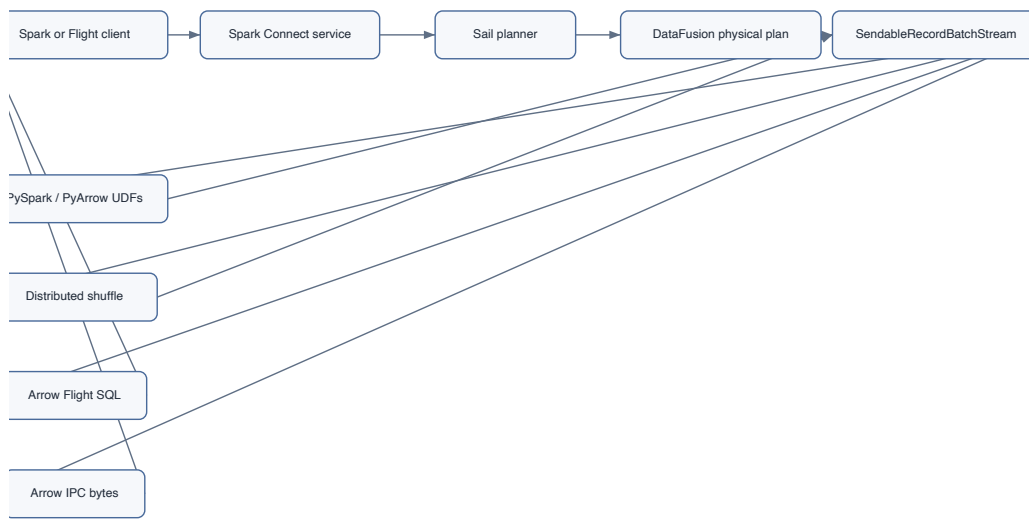


Figure 19: Flowchart 05.1

Arrow is the contract between almost every box in that diagram.

6.3 The Arrow Mental Model

Arrow is a columnar memory format. A table is not stored as a sequence of row objects. It is stored as arrays, one array per column, with a schema describing the column names, logical types, nullability, metadata, and nested structure.

The key types you see in Sail are:

Rust type	Meaning in Sail
<code>ArrayRef</code>	Shared reference to an immutable Arrow array
<code>ArrayData</code>	Low-level array buffers and metadata, useful at FFI boundaries
<code>DataType</code>	Arrow logical type such as <code>Int64</code> , <code>Utf8</code> , <code>Struct</code> , <code>Map</code> , <code>Timestamp</code>
<code>Field</code>	One named column, including data type, nullability, and metadata
<code>Schema</code>	Ordered collection of fields plus schema metadata
<code>SchemaRef</code>	Shared <code>Arc<Schema></code>
<code>RecordBatch</code>	Equal-length arrays plus a schema; the basic execution unit
<code>RecordBatchStream</code>	Async stream of <code>RecordBatch</code> values
<code>SendableRecordBatchStream</code>	Boxed, pinned, sendable <code>DataFusion</code> batch stream

The official Arrow format documentation describes a record batch as an ordered collection of arrays with the same length, described by a schema. `DataFusion`'s Arrow introduction adds the execution intuition: a `RecordBatch` is columnar inside, but externally it behaves like a row chunk that can be streamed, partitioned, and scheduled.

That dual nature explains why Arrow fits distributed query processing so well:

- Vectorized operators can process contiguous column arrays.
- The scheduler can move work in batch-sized units.
- Network protocols can serialize batches without inventing a new row format.
- Python UDFs can receive arrays instead of per-row Python objects.
- Metadata can carry logical semantics beyond the physical storage type.

6.4 Why Arc Shows Up Everywhere

In Sail's Rust code, Arrow objects are usually wrapped in `Arc`.

For example, `SchemaRef` is an `Arc<Schema>`, and `ArrayRef` is an `Arc<dyn Array>`. This reflects the Arrow memory model: arrays are immutable once built, so it is cheap and safe to share them across operators, streams, and async tasks.

When a physical operator needs to produce a new batch, it usually creates new array references and a shared schema:

```
let id_array: ArrayRef = Arc::new(Int64Array::from(x));
let batch = RecordBatch::try_new(projected_schema.clone(), vec![id_array])?;
```

That pattern appears in `RangeExec` in `crates/sail-physical-plan/src/range.rs`. The range source partitions a numeric range, builds an `Int64Array` for each chunk, then wraps the chunks in `RecordBatch` values.

The important lesson is that Sail does not need an internal row object for this operator. The execution unit is already Arrow-native:

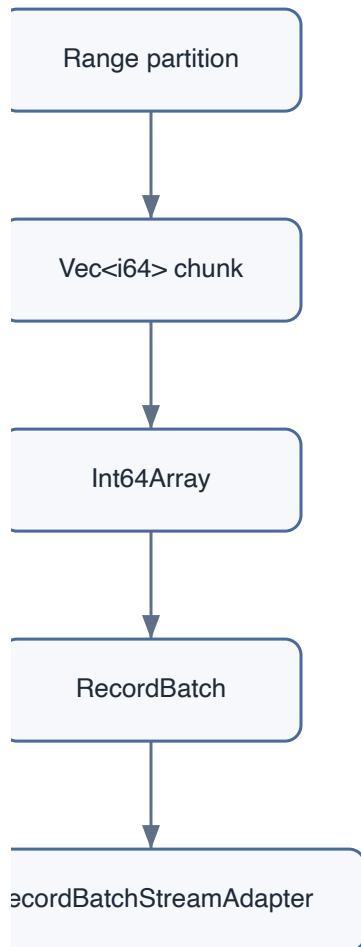


Figure 20: Flowchart 05.2

6.5 RecordBatch As The Unit Of Execution

Most query engines have a concept like “a batch of rows.” In Sail, that concept is concretely Arrow’s RecordBatch.

Look at the signature of DataFusion physical execution:

```
fn execute(  
    &self,  
    partition: usize,  
    context: Arc<TaskContext>,  
) -> Result<SendableRecordBatchStream>
```

This shape appears across Sail physical plan nodes. A plan node is executed for one output partition, and the result is a stream of Arrow batches. Sail can then compose operators by chaining streams.

RangeExec is a good first example:

1. Validate the requested partition.
2. Compute that partition’s range values.
3. Chunk values into RANGE_BATCH_SIZE.
4. Build Arrow arrays.
5. Build RecordBatch values.
6. Return a RecordBatchStreamAdapter.

The same type shows up at much larger boundaries:

- Spark Connect’s ExecutorTaskContext owns a SendableRecordBatchStream.
- Shuffle write consumes a SendableRecordBatchStream.
- Shuffle read returns a SendableRecordBatchStream.
- Flight SQL stores and later encodes a SendableRecordBatchStream.
- Python UDF execution turns ColumnarValue values into Arrow arrays.

Once you learn to recognize SendableRecordBatchStream, you can follow data through Sail.

6.6 Spark Connect Output: Arrow IPC In Protobuf Clothing

Spark Connect uses Protobuf messages for the control protocol, but tabular results are encoded as Arrow batches.

In `crates/sail-spark-connect/src/executor.rs`, the executor has this batch enum:

```
pub enum ExecutorBatch {  
    ArrowBatch(ArrowBatch),  
    SqlCommandResult(Box<SqlCommandResult>),  
    ...  
    Schema(Box<DataType>),  
    Complete,  
}
```

For query output, the important variants are:

- Schema(Box<DataType>)
- ArrowBatch(ArrowBatch)
- Complete

The executor first converts the DataFusion stream schema into a Spark schema:

```
let schema = to_spark_schema(context.stream.schema());
let out = ExecutorOutput::new(ExecutorBatch::Schema(Box::new(schema)));
```

Then it repeatedly reads Arrow RecordBatch values from the stream:

```
while let Some(batch) = context.next().await? {
    let batch = to_arrow_batch(&batch)?;
    let out = ExecutorOutput::new(ExecutorBatch::ArrowBatch(batch));
    tx.send(out).await?;
}
```

The conversion to Spark Connect's ArrowBatch is compact:

```
pub(crate) fn to_arrow_batch(batch: &RecordBatch) -> SparkResult<ArrowBatch> {
    let mut output = ArrowBatch::default();
    {
        let cursor = Cursor::new(&mut output.data);
        let mut writer = StreamWriter::try_new(cursor, batch.schema().as_ref())?;
        writer.write(batch)?;
        output.row_count += batch.num_rows() as i64;
        writer.finish()?;
    }
    Ok(output)
}
```

This is Arrow IPC streaming format. The official Arrow columnar documentation describes IPC streams as schema followed by record batch messages and optional dictionary batches. Sail uses StreamWriter to produce that payload and stores the bytes inside Spark Connect's ArrowBatch.data.

The relationship looks like this:

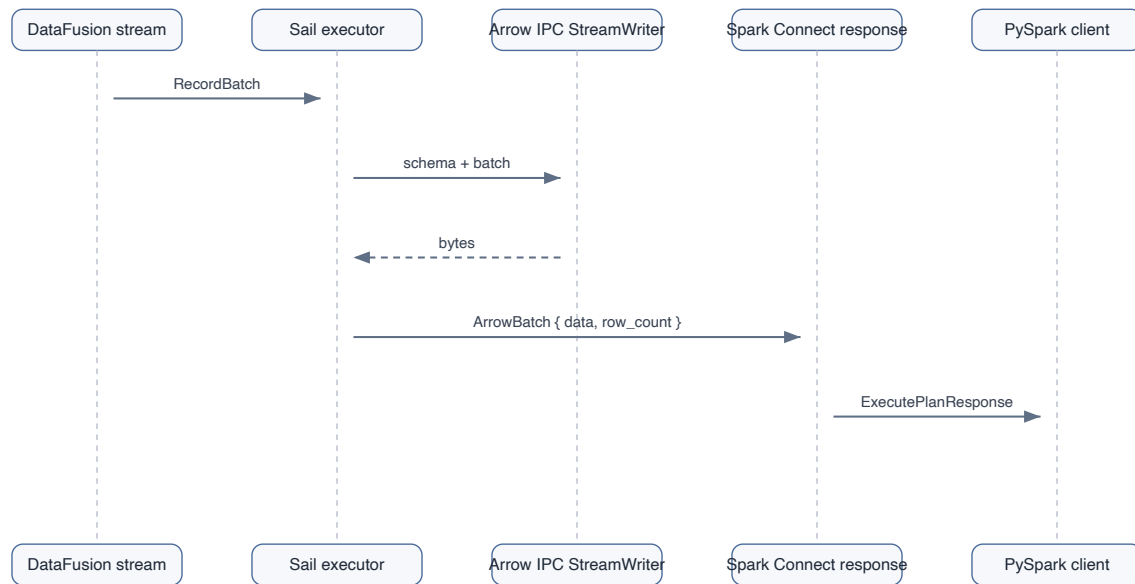


Figure 21: Sequence diagram 05.3

The executor also sends empty batches in two cases:

- As a heartbeat when no batch arrives before the configured interval.
- As an output placeholder when a query produces no rows.

That is why `ExecutorTaskContext::next` can return `RecordBatch::new_empty(self.stream.schema())`. The stream contract remains Arrow-shaped even when the batch contains zero rows.

6.7 Schema Conversion: Arrow Back To Spark Types

Sail must speak Arrow internally and Spark externally. The conversion from Arrow types to Spark Connect types lives in `crates/sail-spark-connect/src/proto/data_type_arrow.rs`.

The central conversion is:

```
impl TryFrom<adt::DataType> for DataType
```

It maps Arrow types such as:

- `adt::DataType::Boolean` -> Spark Boolean
- `adt::DataType::Int32` -> Spark Integer
- `adt::DataType::Int64` -> Spark Long
- `adt::DataType::Utf8` -> Spark String
- `adt::DataType::Date32` -> Spark Date
- `adt::DataType::Struct(fields)` -> Spark Struct
- `adt::DataType::List(field)` -> Spark Array
- `adt::DataType::Map(field, _)` -> Spark Map

The code also documents where the mapping is lossy or constrained. For example, Spark Char or VarChar may become Arrow string-like data and then come back as Spark String. Timestamp precision is another important compatibility choice: the conversion accepts microsecond timestamps and rejects other timestamp units in several branches.

This is a crucial extension lesson. Arrow is a strong physical and logical format, but it is not identical to Spark's type system. If an extension needs Spark-specific semantics, it must preserve them deliberately, often through:

- logical plan metadata,
- Arrow field metadata,
- Arrow extension types,
- or explicit conversion rules.

6.8 Extension Types: GeoArrow And Variant

Arrow supports extension types: a field can have a physical storage type plus metadata that names a higher-level logical meaning.

Sail already uses this idea in `crates/sail-common/src/geoarrow/extension.rs`:

```
pub struct GeoArrowWkbType {
    pub metadata: GeoArrowMetadata,
}

impl GeoArrowWkbType {
    pub const NAME: &'static str = "geoarrow.wkb";
}
```

The extension stores geometry/geography values as binary WKB, while metadata captures CRS and edge semantics:

```
pub struct GeoArrowMetadata {
    pub edges: Option<GeoArrowEdges>,
    pub crs: Option<GeoArrowCrs>,
}
```

The storage type check is strict:

```
match data_type {
    DataType::Binary | DataType::LargeBinary | DataType::BinaryView => Ok(()),
    data_type => Err(...),
}
```

Then the Spark Connect type conversion recognizes this extension:

```
} else if extension_type_name == Some(GeoArrowWkbType::NAME) {
    let ext = field.try_extension_type::<GeoArrowWkbType>()?;
    let meta: SparkGeoMetadata = ext.metadata.try_into()?;
    ...
}
```

Sail maps GeoArrow metadata to Spark geometry or geography. If edges is present, Sail treats the value as geography. If edges is absent, it treats the value as geometry. CRS metadata becomes Spark SRID values for supported CRS strings.

The same file recognizes `parquet_variant_compute::VariantType`, mapping that Arrow extension to Spark Variant.

This is the most concrete preview of the final extensions chapter. An extension does not have to invent a new internal data plane. It can often use Arrow's existing storage plus extension metadata:

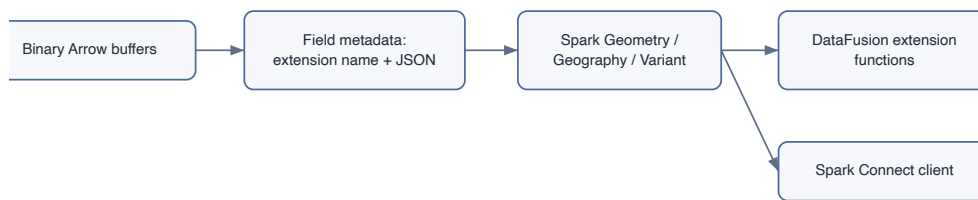


Figure 22: Flowchart 05.4

That architecture keeps data compatible with Arrow tooling while preserving domain meaning.

6.9 PyArrow: The Python Boundary

The Python side of Sail depends heavily on Arrow because PySpark UDFs should not have to receive rows one Python object at a time.

The conversion layer in `crates/sail-python-udf/src/conversion.rs` uses the `arrow_pyarrow` crate:

```
use arrow_pyarrow::{FromPyArrow, ToPyArrow};
```

Sail implements `TryToPy` for:

- `&DataType`
- slices of `DataType`
- slices of `ArrayRef`
- `Vec<ArrayRef>`
- `&Schema`
- `SchemaRef`
- `RecordBatch`

And it implements `TryFromPy` for:

- `ArrayData`
- `RecordBatch`

For arrays, the conversion uses the underlying Arrow ArrayData:

```
self.iter()
  .map(|x| x.into_data().to_pyarrow(py))
  .collect::<PyResult<Vec<_>>>()
```

That is a big deal. It means Sail is intentionally crossing the language boundary with Arrow arrays, not with ad hoc serialized Python values.

The UDF invocation path in `crates/sail-python-udf/src/udf/pyspark_udf.rs` makes that practical:

```
let args: Vec<ArrayRef> = ColumnarValue::values_to_arrays(&args)?;
let output = udf.call1(py, (args.try_to_py(py)?, number_rows))?;
let data = ArrayData::try_from_py(py, &output)?;
let array = cast(&make_array(data), &self.output_type)?;
```

The steps are:

1. Convert DataFusion ColumnarValue arguments to Arrow arrays.
2. Convert those Arrow arrays into PyArrow objects.
3. Call the Python function.
4. Convert the returned PyArrow object back into Arrow ArrayData.
5. Wrap it as an Arrow array.
6. Cast it to the declared output type.

The official PyArrow RecordBatch API is useful for understanding the Python objects Sail is inter-operating with. PyArrow exposes schemas, columns, row counts, zero-copy slices, filters, take, conversion from arrays, and IPC serialization methods. Sail's Rust side uses the same conceptual model, but with Rust ownership and type checks.

6.10 UDF Kinds And Arrow-Native Execution

The PySpark UDF kind enum includes several execution modes:

```
pub enum PySparkUdfKind {
    Batch,
    ArrowBatch,
    ScalarPandas,
    ScalarPandasIter,
    ScalarArrow,
    ScalarArrowIter,
}
```

The comment in the code calls out Spark 4.0 Arrow-native scalar UDF types. The important distinction is:

- Pandas UDFs use Arrow as an efficient transport to and from Pandas.
- Arrow-native UDFs can pass Arrow arrays more directly.

From an engine architecture perspective, this is exactly the direction Sail wants. Every conversion into Python objects costs time and memory. Every operator that can remain Arrow-native preserves vectorization and avoids unnecessary row materialization.

The performance guide at `docs/guide/udf/performance.md` makes the same point: Arrow UDFs avoid copying data into row-oriented Python objects and let the Rust engine and Python function share Arrow data through the Arrow/PyArrow boundary.

6.11 Distributed Shuffle: Arrow As The Exchange Unit

Distributed query processing needs exchanges. A join, aggregation, sort, or repartition may require data from one set of workers to move to another set of workers.

In Sail, the exchange unit is still `RecordBatch`.

The write side is `crates/sail-execution/src/plan/shuffle_write.rs`. `ShuffleWriteExec` wraps an input physical plan and a desired output partitioning. When `execute` is called for an input partition, it:

1. Opens one sink per shuffle output location.
2. Executes the child plan for the input partition.
3. Reads `RecordBatch` values from the child stream.
4. Partitions each batch by hash or row round-robin.
5. Writes partitioned batches to the corresponding sinks.
6. Closes the sinks.
7. Returns an empty batch stream as the execution result.

The core loop is:

```
while let Some(batch) = stream.next().await {
    let batch = batch?;
    let mut partitions: Vec<Option<RecordBatch>> = vec![None;
partition_sinks.len()];
    partitioner.partition(batch, |p, batch| {
        partitions[p] = Some(batch);
        Ok(())
    })?;
    ...
}
```

The read side is `crates/sail-execution/src/plan/shuffle_read.rs`. `ShuffleReadExec` has a set of read locations for each output partition. When executed, it opens all relevant task streams and merges them:

```
let futures = locations
    .iter()
    .map(|location| reader.open(location, schema.clone()));
let streams = try_join_all(futures).await?;
Ok(Box::pin(MergedRecordBatchStream::new(schema, streams)))
```

That gives downstream operators a normal `SendableRecordBatchStream` again. The shuffle itself is an implementation detail hidden between write and read nodes.

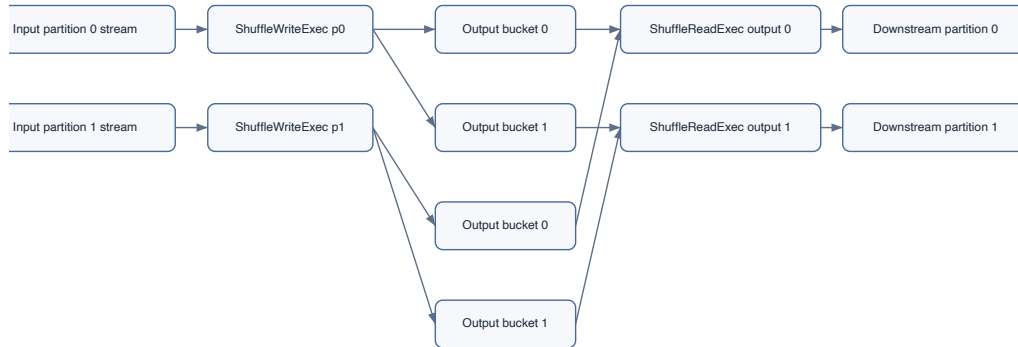


Figure 23: Flowchart 05.5

Notice the absence of a Sail-specific row format. The shuffle API talks in task stream locations, but the payload remains Arrow batches.

6.12 Row Round-Robin Repartition

Hash partitioning can delegate to DataFusion's `BatchPartitioner`. Sail also has a row round-robin partitioner in `crates/sail-physical-plan/src/repartition.rs`.

The logic is a useful Arrow lesson:

```

let schema = batch.schema();
let mut indices = vec![Vec::new(); self.num_partitions];
for row_index in 0..batch.num_rows() {
    let partition = (self.next_idx + row_index) % self.num_partitions;
    indices[partition].push(row_index as u32);
}
...
let indices_array: PrimitiveArray<UInt32Type> = partition_indices.into();
let columns = take_arrays(batch.columns(), &indices_array, None)?;
let partition_batch =
    RecordBatch::try_new_with_options(schema.clone(), columns, &options)?;
  
```

Sail does not loop over cells and rebuild rows. It builds an Arrow array of row indices and uses Arrow compute's `take_arrays` to select rows from every column. The output is one `RecordBatch` per non-empty destination partition.

This is the practical meaning of vectorized execution: even operations that are row-directed can often be implemented as array operations.

6.13 Flight SQL: Arrow On The Network

Sail also exposes a Flight SQL service in `crates/sail-flight/src/service.rs`. The official Arrow Flight protocol describes Flight as an RPC framework for high-performance Arrow data services, built on gRPC and Arrow IPC. It is organized around streams of Arrow record batches, with metadata methods for discovering and retrieving streams.

That maps cleanly to Sail's service.

For a SQL statement, `get_flight_info_statement`:

1. Parses the SQL string.
2. Converts it to Sail's plan representation.
3. Resolves and executes the plan.
4. Gets a `SendableRecordBatchStream`.
5. Stores the stream under a query handle.
6. Returns `FlightInfo` with a ticket.

Then `do_get_statement`:

1. Decodes the ticket into a query handle.
2. Removes the stored stream from state.
3. Reads the stream schema.
4. Encodes the stream with `FlightDataEncoderBuilder`.
5. Returns the encoded Flight data stream.

The encoding step is:

```
let output = FlightDataEncoderBuilder::new()
    .with_schema(schema)
    .build(output)
    .map(|result| result.map_err(|e| Status::internal(format!("encoding error: {e}"))));
```

The same DataFusion output can therefore leave Sail through two different protocol doors:

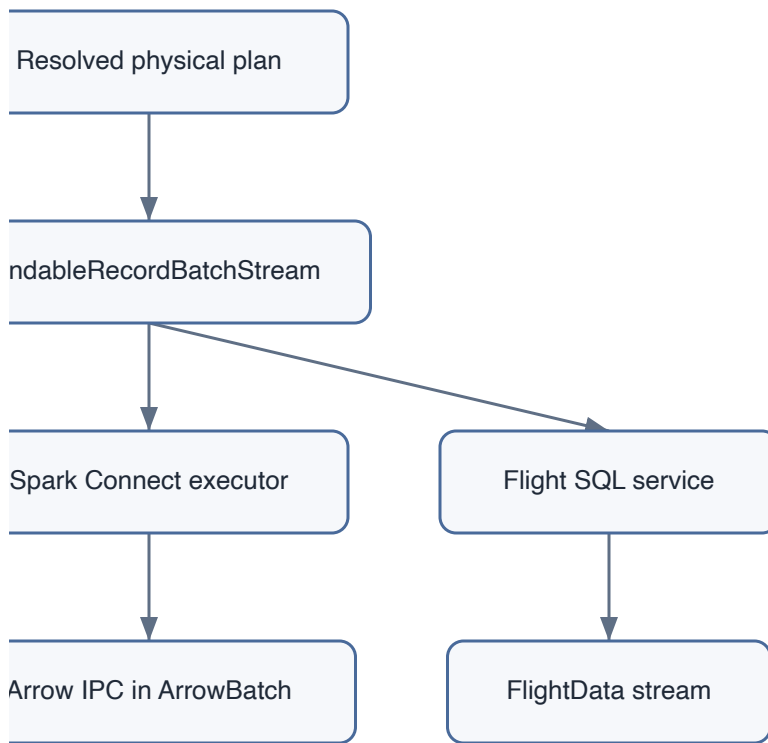


Figure 24: Flowchart 05.6

This is a powerful architectural property. Sail's execution engine does not need separate data representations for Spark Connect and Flight SQL. Protocols wrap the same Arrow stream abstraction.

6.14 Arrow IPC Versus Flight

It is easy to blur Arrow IPC and Arrow Flight. Sail uses both, so it helps to separate them:

Concept	In Arrow	In Sail
In-memory format	Column buffers, validity bitmaps, offsets, schemas	ArrayRef, SchemaRef, RecordBatch
IPC stream/file format	Serialized schema, dictionaries, record batches	Spark Connect ArrowBatch.data via StreamWriter
Flight	gRPC service protocol carrying Arrow data streams	SailFlightSqlService with FlightDataEncoderBuilder
PyArrow bridge	Python implementation of Arrow data structures	arrow_pyarrow conversions for UDFs

Think of it this way:

- Arrow memory format is how data exists while operators work.
- Arrow IPC is how batches are serialized.
- Arrow Flight is a service protocol for discovering and transferring streams.
- PyArrow is Python's Arrow implementation and API surface.

Sail benefits because these layers are designed to compose.

6.15 A Small End-To-End Example

Imagine a PySpark client runs:

```
spark.range(0, 5).selectExpr("id + 10 as value").collect()
```

The high-level flow is:

1. PySpark builds an unresolved Spark Connect plan.
2. Spark Connect sends that plan to Sail.
3. Sail resolves it into its internal plan representation.
4. DataFusion physical execution creates a SendableRecordBatchStream.
5. A source such as RangeExec produces an Arrow Int64Array.
6. Projection produces another Arrow array for value.
7. The executor reads each RecordBatch.
8. StreamWriter encodes each batch as Arrow IPC bytes.
9. Spark Connect returns ArrowBatch messages to the client.
10. The client decodes Arrow batches into Spark result rows.

The data path can be described without a single row class:

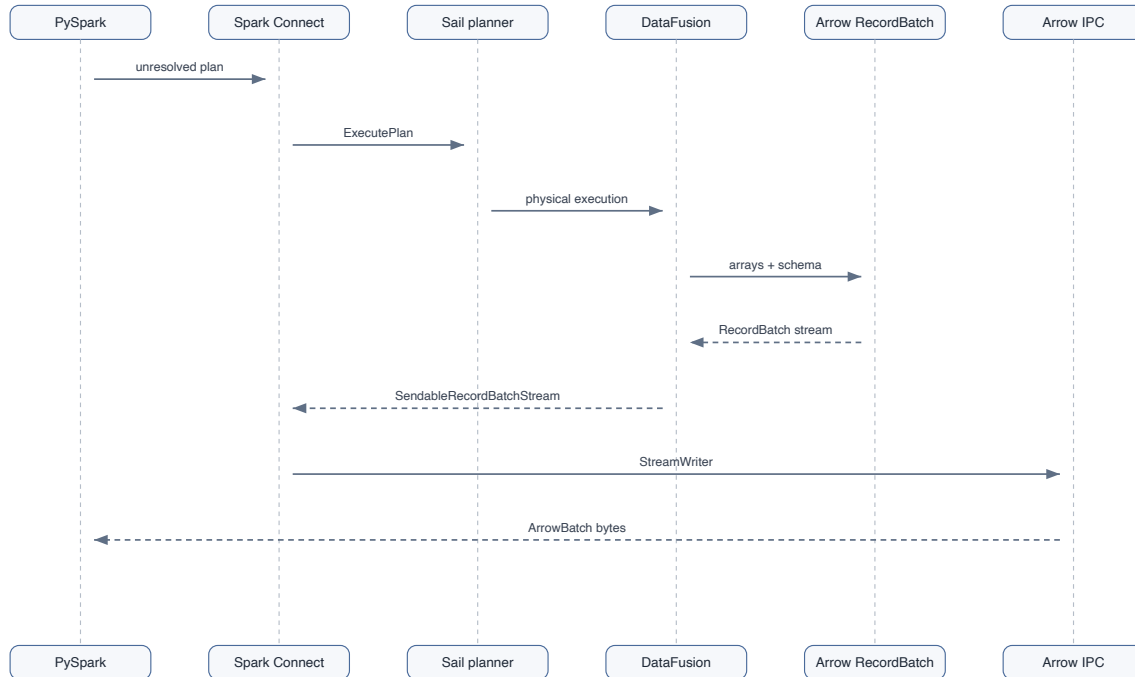


Figure 25: Sequence diagram 05.7

That is the central architecture of Sail result execution.

6.16 What Arrow Does Not Solve By Itself

Arrow gives Sail a powerful data representation, but it does not solve every problem.

Arrow does not decide:

- how Spark logical plans should map to DataFusion plans,
- how distributed tasks should be scheduled,
- how shuffle locations should be assigned,
- how Python worker lifetimes should be managed,
- how Spark-only semantics should survive type conversion,
- how extension packages should be loaded,
- how security and sandboxing should work for user code.

Those are Sail architecture problems. Arrow is the shared data language that makes the solutions simpler and faster.

This distinction matters for extensions. An extension proposal should not merely say “use Arrow.” It should identify:

- the Arrow storage type,
- the Arrow metadata or extension name,
- the DataFusion logical and physical expressions,
- the Spark Connect type mapping,

- the PyArrow/Python behavior,
- the distributed shuffle compatibility story,
- and the protocol boundary behavior.

6.17 Extension Design Pattern: Arrow-First Semantics

The GeoArrow code suggests a reusable pattern for future extensions:

1. Choose an Arrow physical representation.
2. Define metadata that captures the logical semantics.
3. Implement validation for allowed storage types.
4. Teach Spark Connect schema conversion to recognize the extension.
5. Add DataFusion expressions and functions that operate on the Arrow layout.
6. Preserve metadata through projections, UDFs, shuffle, and output protocols.
7. Add Python wrappers that expose the same semantics through PyArrow.

For example, a geospatial extension can store WKB as Binary and use `geoarrow.wkb` metadata for CRS and edge semantics. A variant extension can store variant-encoded bytes and preserve logical type identity with an Arrow extension name. A machine-learning vector extension might store `FixedSizeList<Float32>` plus metadata for dimension and metric assumptions.

The best extensions feel native to Arrow rather than bolted onto it.

6.18 Common Mistakes When Learning Arrow In Sail

6.18.1 Mistake 1: Thinking RecordBatch Means Row-Oriented

A `RecordBatch` is a batch of rows from the outside, but internally it is columnar. Operators should usually work column-by-column.

6.18.2 Mistake 2: Treating Schema Metadata As Decoration

Schema and field metadata can carry compatibility-critical information. Sail's UDT and GeoArrow conversions depend on metadata to recover Spark semantics.

6.18.3 Mistake 3: Copying Data At Language Boundaries

Python integration should use PyArrow whenever possible. Converting every row into Python objects defeats the point of using a columnar engine.

6.18.4 Mistake 4: Forgetting Empty Batches

Empty batches are still meaningful. They carry schema, heartbeats, and protocol state. Sail's Spark Connect executor intentionally emits empty batches in some cases.

6.18.5 Mistake 5: Ignoring Unsupported Type Mappings

Arrow and Spark type systems overlap but are not identical. Unsupported units, dictionary encodings, unions, and lossy string-like conversions must be handled explicitly.

6.19 Reading Exercise: Follow One Batch

To build intuition, trace one `RecordBatch` through the code:

1. Start in `crates/sail-physical-plan/src/range.rs`.
2. Find the `Int64Array::from(x)` call.
3. Follow the `RecordBatch::try_new` call.
4. Find where plan execution returns `SendableRecordBatchStream`.
5. Jump to `crates/sail-spark-connect/src/executor.rs`.
6. Find while `let Some(batch) = context.next().await?`.
7. Follow to `_arrow_batch`.
8. Look at `StreamWriter::try_new`, `writer.write`, and `writer.finish`.

Then repeat the exercise for Flight SQL:

1. Start in `crates/sail-flight/src/service.rs`.
2. Find `service.runner().execute(&ctx, plan)`.
3. Follow the stored stream into `SailFlightSqlState`.
4. Find `do_get_statement`.
5. Follow `FlightDataEncoderBuilder`.

After these two traces, you will understand the two biggest Arrow output paths in Sail.

6.20 Reading Exercise: Follow One Python UDF

For Python UDFs:

1. Start in `crates/sail-python-udf/src/udf/pyspark_udf.rs`.
2. Find `ColumnarValue::values_to_arrays`.
3. Follow `args.try_to_py(py)`.
4. Jump to `crates/sail-python-udf/src/conversion.rs`.
5. Find the `TryToPy` implementation for `&[ArrayRef]`.
6. Follow `into_data().to_pyarrow(py)`.
7. Return to `invoke_with_args`.
8. Follow `ArrayData::try_from_py`.
9. Observe the final `cast(&make_array(data), &self.output_type)`.

This is the Arrow/PyArrow bridge in miniature.

6.21 How This Prepares Us For DataFusion

The next chapter can now talk about DataFusion with the right foundation. DataFusion is not just a planner that happens to use Arrow. Its physical operators, stream interfaces, expression evaluation, partitioning, and UDF APIs are designed around Arrow batches.

When Sail adds Spark compatibility on top of DataFusion, it repeatedly answers one question:

How do we preserve Spark semantics while keeping the data path Arrow-native?

That question appears in:

- schema conversion,
- physical plan construction,

- Spark-specific expressions,
- Python UDF execution,
- shuffle exchanges,
- and future extension APIs.

6.22 Takeaways

Apache Arrow is Sail's data plane. The most important concrete type is `RecordBatch`, and the most important execution abstraction is `SendableRecordBatchStream`.

Spark Connect wraps Arrow IPC bytes in Protobuf responses. Flight SQL wraps Arrow streams in the Flight protocol. PySpark UDFs cross the Rust/Python boundary through PyArrow. Distributed shuffle partitions and merges Arrow batch streams. Extension types use Arrow metadata to carry domain-specific logical meaning without abandoning Arrow compatibility.

Once you see Sail as a system that plans Spark-compatible queries but executes Arrow-native streams, the architecture becomes much easier to reason about.

7 Chapter 6: Apache DataFusion

DataFusion is Sail's execution kernel.

Spark Connect gives Sail the client protocol. PySpark gives Sail a familiar Python API. Arrow gives Sail the in-memory format. DataFusion gives Sail the query engine: logical plans, expressions, optimizers, physical planning, partitioned execution, vectorized functions, file format integration, and `RecordBatch` streams.

Sail is not a thin wrapper around DataFusion, though. Sail is a Spark-compatible system built on DataFusion. That distinction matters. Spark compatibility requires Spark-shaped plans, Spark function semantics, Spark catalog behavior, Spark error behavior, Spark UDF behavior, Spark streaming concepts, and a distributed execution model. DataFusion supplies the engine primitives; Sail supplies the Spark interpretation and the distributed control plane.

The question for this chapter is:

How does Sail turn Spark-compatible plans into DataFusion execution without losing Spark semantics?

7.1 Definitive References

Keep these references nearby:

- DataFusion introduction
- DataFusion library user guide
- Building logical plans
- Using the DataFrame API
- Adding user-defined functions
- Extending operators
- Query optimizer

- ExecutionPlan API docs
- QueryPlanner API docs

The DataFusion introduction describes DataFusion as an extensible Rust query engine using Arrow's in-memory format. That one sentence captures why Sail can exist: DataFusion is fast enough and flexible enough to serve as the execution core for a Spark-compatible system.

Sail currently depends on DataFusion 53.1.0 in the workspace Cargo.toml. The exact APIs will evolve, but the architectural ideas in this chapter are stable: session state, logical plans, optimizer rules, physical plans, extension planners, and Arrow batch streams.

7.2 Where DataFusion Lives In Sail

The main files for this chapter are:

Area	Files	DataFusion role
Plan resolution and execution	<code>crates/sail-plan/src/lib.rs</code>	Converts Sail specs to DataFusion logical plans, optimizes them, and creates physical plans
Session construction	<code>crates/sail-session/src/session_factory/server.rs</code>	Builds <code>SessionConfig</code> , <code>SessionState</code> , runtime env, rules, query planner, and Sail session extensions
Custom query planner	<code>crates/sail-session/src/planner.rs</code>	Installs extension physical planners and maps Sail logical extension nodes to physical operators
Session extension helper	<code>crates/sail-common-datafusion/src/extension.rs</code>	Provides typed access to DataFusion session extensions from <code>SessionContext</code> , <code>SessionState</code> , and <code>TaskContext</code>
Logical extension nodes	<code>crates/sail-logical-plan/src/*.rs</code>	Defines Sail-specific logical nodes such as range, repartition, barrier, map partitions, and streaming nodes
Physical extension nodes	<code>crates/sail-physical-plan/src/*.rs</code>	Implements custom <code>ExecutionPlan</code> nodes
Physical optimizer rules	<code>crates/sail-physical-optimizer/src/*.rs</code>	Adds or adjusts DataFusion physical optimization behavior
Spark function mapping	<code>crates/sail-plan/src/function/*.rs</code>	Maps Spark function names and semantics to DataFusion expressions and UDFs
Function kernels	<code>crates/sail-function/src/*.rs</code>	Implements vectorized DataFusion UDFs for Spark-compatible functions
Table functions	<code>crates/sail-plan/src/function/table/range.rs</code>	Implements DataFusion <code>TableFunctionImpl</code> and <code>TableProvider</code> patterns

The short version:

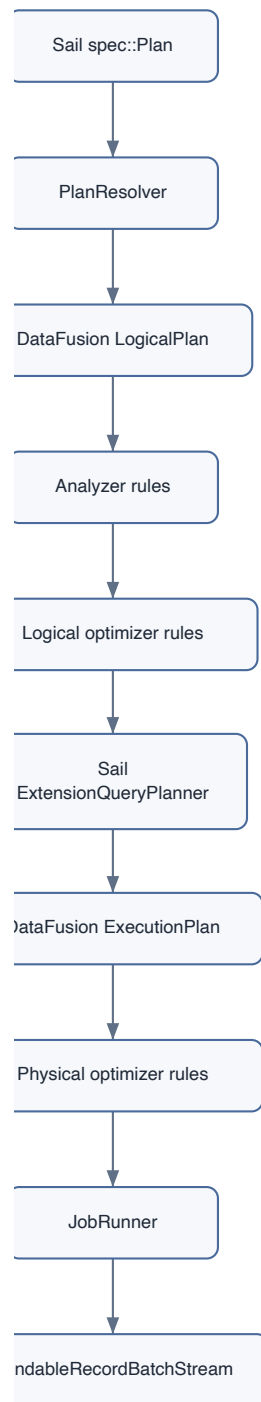


Figure 26: Flowchart 06.1

DataFusion owns the core plan abstractions in the middle. Sail owns the translation into those abstractions and the custom nodes around the edges.

7.3 The Critical Function: `resolve_and_execute_plan`

The most compact tour of Sail's DataFusion integration is `crates/sail-plan/src/lib.rs`.

The key function is:

```
pub async fn resolve_and_execute_plan(
    ctx: &SessionContext,
    config: Arc<PlanConfig>,
    plan: spec::Plan,
) -> PlanResult<(Arc<dyn ExecutionPlan>, Vec<StringifiedPlan>)>
```

This is the bridge from Sail's Spark-compatible plan spec to a DataFusion physical plan. It performs these steps:

1. Create a `PlanResolver`.
2. Resolve the Sail `spec::Plan` into a named DataFusion `LogicalPlan`.
3. Store the initial logical plan for explain output.
4. Ask the `SessionContext` to execute the logical plan into a `DataFrame`.
5. Pull the `SessionState` and logical plan back out of the `DataFrame`.
6. Run DataFusion logical optimization.
7. Rewrite streaming plans if necessary.
8. Use the session's query planner to create a physical plan.
9. Rename physical output fields if Spark field names require it.
10. Store the final physical plan for explain output.

In code:

```
let resolver = PlanResolver::new(ctx, config);
let NamedPlan { plan, fields } = resolver.resolve_named_plan(plan).await?;
let df = execute_logical_plan(ctx, plan).await?;
let (session_state, plan) = df.into_parts();
let plan = session_state.optimize(&plan)?;
let plan = session_state
    .query_planner()
    .create_physical_plan(&plan, &session_state)
    .await?;
```

This is a beautiful little funnel:

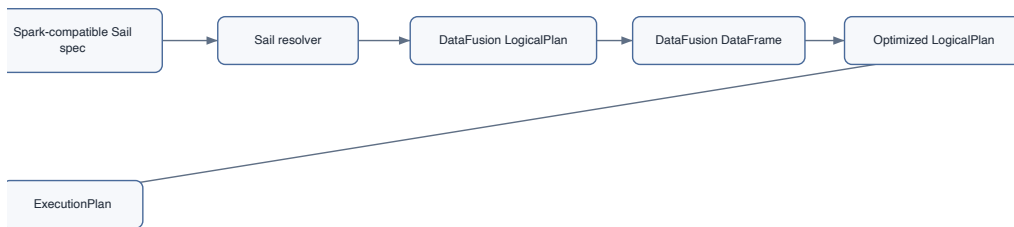


Figure 27: Flowchart 06.2

The `DataFrame` step may look surprising. Sail already has a logical plan, so why ask the context to execute it into a `DataFrame` and then split it apart? Because `SessionContext::execute_logical_plan` lets DataFusion apply its normal session machinery: analyzer behavior, catalog/function binding, and context state. Sail then takes back the plan and drives the rest of the pipeline.

7.4 SessionContext, SessionState, And SessionConfig

DataFusion’s session types are the engine’s ambient context:

- `SessionConfig` stores configuration options and extension objects.
- `SessionState` stores the config, runtime env, optimizer rules, function registries, catalog state, and query planner.
- `SessionContext` is the user-facing handle around session state.
- `TaskContext` is the execution-time context available to physical operators.

Sail constructs these deliberately in `crates/sail-session/src/session_factory/server.rs`.

The server session factory creates a `SessionContext` like this:

```

fn create(&mut self, info: ServerSessionInfo) -> Result<SessionContext> {
    let state = self.create_session_state(&info)?;
    let context = SessionContext::new_with_state(state);
    context.state_ref().write().register_udaf(first_value_udaf())?;
    Ok(context)
}

```

The registration of `first_value_udaf` is a telling detail. Sail does not simply enable all DataFusion defaults. It builds a session with selected behavior and then patches in assumptions required by its chosen optimizer rules.

The session config contains Sail services as DataFusion extensions:

```

SessionConfig::new()
    .with_create_default_catalog_and_schema(false)
    .with_information_schema(false)
    .with_extension(create_table_format_registry())?
    .with_extension(Arc::new(create_catalog_manager(...)?))

```

```

.with_extension(Arc::new(ActivityTracker::new()))
.with_extension(Arc::new(JobService::new(job_runner)))
.with_extension(Arc::new(RepartitionBufferConfig::new(...)))
.with_extension(Arc::new(self.create_system_table_service(info)?))
.with_extension(Arc::new(DeltaTableCache::default()));

```

That list is a capsule summary of Sail's architecture:

- Sail manages catalogs itself.
- Sail manages table formats itself.
- Sail tracks session activity.
- Sail installs a job service.
- Sail controls repartition buffering.
- Sail exposes system tables.
- Sail caches Delta table state.

DataFusion supplies the typed extension slot. Sail uses it as session-local dependency injection.

7.5 Typed Session Extensions

The helper in `crates/sail-common-datafusion/src/extension.rs` makes DataFusion extensions feel typed:

```

pub trait SessionExtension: Send + Sync + 'static {
    fn name() -> &'static str;
}

pub trait SessionExtensionAccessor {
    fn extension<T: SessionExtension>(&self) -> Result<Arc<T>>;
    fn runtime_env(&self) -> Arc<RuntimeEnv>;
}

```

Sail implements `SessionExtensionAccessor` for:

- `SessionContext`
- `SessionState`
- `&dyn Session`
- `TaskContext`

That matters because different parts of the engine are at different layers:

- Planning code often has `SessionContext` or `SessionState`.
- Table providers receive `&dyn Session`.
- Physical operators receive `TaskContext`.

The same pattern works everywhere:

```

let service = ctx.extension::<JobService>()?;
let registry = session_state.extension::<TableFormatRegistry>()?;
let config = task_context.extension::<RepartitionBufferConfig>()?;

```

This is one of Sail’s cleanest Rust patterns. DataFusion gives an untyped extension store; Sail wraps it with a trait that produces a useful error and a typed `Arc<T>`.

For the extension proposal later in the book, this pattern is a key precedent. Third-party integrations will need a way to store per-session services and configuration. Sail already has the mechanism; the open question is how to make registration public, ordered, discoverable, and distributed-safe.

7.6 Sail’s Query Planner

DataFusion lets a session install a custom query planner. Sail does this in `ServerSessionFactory::create_session_state`:

```
let builder = SessionStateBuilder::new()
    .with_config(config)
    .with_runtime_env(runtime)
    .with_analyzer_rules(default_analyzer_rules())
    .with_optimizer_rules(default_optimizer_rules())
    .with_physical_optimizer_rules(get_physical_optimizers(...))
    .with_query_planner(new_query_planner());
```

`new_query_planner` returns `ExtensionQueryPlanner` from `crates/sail-session/src/planner.rs`.

That planner is small, but strategically important:

```
impl QueryPlanner for ExtensionQueryPlanner {
    async fn create_physical_plan(
        &self,
        logical_plan: &LogicalPlan,
        session_state: &SessionState,
    ) -> Result<Arc<dyn ExecutionPlan>> {
        let mut extension_planners = new_lakehouse_extension_planners();
        extension_planners.push(Arc::new(SystemTablePhysicalPlanner));
        extension_planners.push(Arc::new(ExtensionPhysicalPlanner));
        let planner =
            DefaultPhysicalPlanner::with_extension_planners(extension_planners);
        planner.create_physical_plan(&logical_plan, session_state).await
    }
}
```

This is Sail’s physical planning strategy:

1. Use DataFusion’s `DefaultPhysicalPlanner`.
2. Add extension planners for lakehouse tables.
3. Add an extension planner for system tables.
4. Add Sail’s own catch-all extension planner.

DataFusion still handles normal logical plan nodes: projections, filters, aggregates, joins, sorts, limits, scans, and so on. Sail handles custom logical extension nodes that DataFusion does not know how to plan.

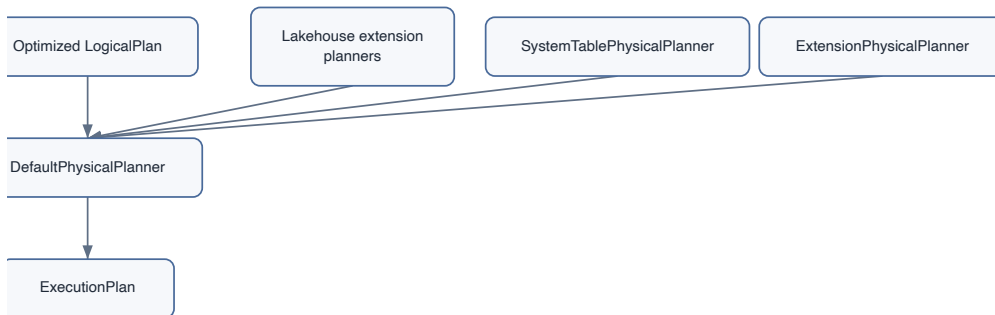


Figure 28: Flowchart 06.3

The extension proposal in discussion #2001 wants to generalize this seam. Today the list is hard-coded. A third-party extension API would let packages add their own extension planners without editing Sail core.

7.7 Logical Extension Nodes

DataFusion has built-in logical plan nodes, but it also supports user-defined logical nodes. Sail uses those for Spark concepts that do not map directly to a single built-in DataFusion logical plan node.

RangeNode in `crates/sail-logical-plan/src/range.rs` is the friendliest example:

```
pub struct RangeNode {
    range: Range,
    num_partitions: usize,
    schema: DFSchemaRef,
}
```

It implements `UserDefinedLogicalNodeCore`:

```
impl UserDefinedLogicalNodeCore for RangeNode {
    fn name(&self) -> &str {
        "Range"
    }

    fn inputs(&self) -> Vec<&LogicalPlan> {
        vec![]
    }

    fn schema(&self) -> &DFSchemaRef {
        &self.schema
    }
}
```

The node carries:

- Spark range parameters,
- target partition count,
- and a DataFusion schema.

It is logical because it says what should happen, not yet how to produce the Arrow batches.

Other Sail logical extension nodes include:

- ExplicitRepartitionNode
- BarrierNode
- MapPartitionsNode
- ShowStringNode
- SchemaPivotNode
- SortWithinPartitionsNode
- SparkPartitionIdNode
- streaming source/filter/limit/collector nodes
- file write/delete and row-level write nodes

These nodes are Spark compatibility pressure made visible. When Spark semantics fit DataFusion's built-in logical plan, Sail uses DataFusion's built-in logical plan. When they do not, Sail introduces a logical extension node and teaches the physical planner what to do with it.

7.8 Physical Extension Planning

ExtensionPhysicalPlanner is the function table from Sail logical extension nodes to physical execution nodes.

For RangeNode, it builds RangeExec:

```
if let Some(node) = node.as_any().downcast_ref::<RangeNode>() {
    let schema = UserDefinedLogicalNode::schema(node).inner().clone();
    let projection = (0..schema.fields().len()).collect();
    Arc::new(RangeExec::try_new(
        node.range().clone(),
        node.num_partitions(),
        schema,
        projection,
    )?)
}
```

For MapPartitionsNode, it uses the physical input and wraps it in MapPartitionsExec:

```
let [input] = physical_inputs else {
    return internal_err!("MapPartitionsExec requires exactly one physical input");
};
Arc::new(MapPartitionsExec::new(
    input.clone(),
    node.udf().clone(),
))
```



```
UserDefinedLogicalNode::schema(node).inner().clone(),
))
```

For `SortWithinPartitionsNode`, it does not need a custom physical operator. It uses DataFusion's `SortExec` with `preserve_partitioning`:

```
let sort = SortExec::new(ordering, input.clone())
    .with_fetch(node.fetch())
    .with_preserve_partitioning(true);
Arc::new(sort)
```

This is the architectural sweet spot:

- Use custom logical nodes to preserve Spark intent.
- Use built-in DataFusion physical operators whenever they already match.
- Use custom Sail physical operators only when necessary.

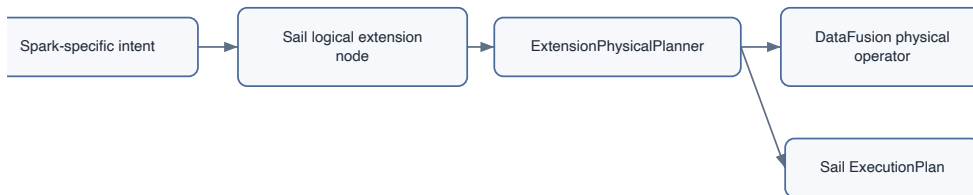


Figure 29: Flowchart 06.4

The result is less code than a from-scratch engine, but more semantic control than a simple SQL translation layer.

7.9 Physical Plans And ExecutionPlan

DataFusion physical operators implement `ExecutionPlan`. The most important method is:

```
fn execute(
    &self,
    partition: usize,
    context: Arc<TaskContext>,
) -> Result<SendableRecordBatchStream>
```

You saw this in Chapter 5, but now we can place it in DataFusion's architecture. An `ExecutionPlan` describes a partitioned physical computation. It knows its schema, children, properties, partitioning, boundedness, and how to execute one partition.

Sail custom physical nodes follow the same contract. `RangeExec`:

1. Checks the partition number.
2. Computes a partition-specific range.
3. Builds Arrow arrays and batches.

4. Returns a `RecordBatchStreamAdapter`.

`ExplicitRepartitionExec`:

1. Wraps an input plan.
2. Creates output channels per target partition.
3. Executes all input partitions cooperatively.
4. Sends partitioned `RecordBatch` values to receivers.
5. Returns a stream for the requested output partition.

`ShuffleWriteExec` and `ShuffleReadExec` in `sail-execution` also implement the same contract, which is why the distributed runtime can insert them into a `DataFusion` physical plan. `DataFusion`'s abstraction is local and partitioned; `Sail`'s distributed planner can split it into stages and reconnect it with shuffle nodes.

7.10 Plan Properties: Partitioning, Boundedness, Emission

`DataFusion` physical plans carry `PlanProperties`. `Sail` uses those properties carefully because distributed execution depends on them.

For example, `RangeExec` creates:

```
PlanProperties::new(  
  EquivalenceProperties::new(projected_schema.clone()),  
  Partitioning::RoundRobinBatch(num_partitions),  
  EmissionType::Both,  
  Boundedness::Bounded,  
)
```

That tells the optimizer and scheduler:

- the output schema's equivalence properties,
- the number and kind of output partitions,
- whether the operator emits incrementally or finally,
- and whether it is bounded.

`ShuffleWriteExec` uses different properties. It returns an empty stream from `execute`, but the side effect is writing shuffle data. Its properties reflect the input partition count rather than the shuffle output count, because each input partition execution writes many shuffle output streams.

These details matter. A distributed engine cannot safely insert exchanges, coalesce partitions, re-order joins, or execute streaming plans unless physical operators accurately describe themselves.

7.11 Logical Optimizers

`Sail` installs analyzer and optimizer rules in `crates/sail-session/src/optimizer.rs`:

```
pub fn default_analyzer_rules() -> Vec<Arc<dyn AnalyzerRule + Send + Sync>> {  
  sail_logical_optimizer::default_analyzer_rules()  
}  
  
pub fn default_optimizer_rules() -> Vec<Arc<dyn OptimizerRule + Send + Sync>> {
```

```

let rules = sail_logical_optimizer::default_optimizer_rules();
let mut custom = sail_plan_lakehouse::lakehouse_optimizer_rules();
custom.extend(
    rules
        .into_iter()
        .filter(|r| r.name() != "push_down_leaf_projections"),
);
custom
}

```

Two things are happening here:

1. Sail delegates most rule construction to `sail_logical_optimizer`.
2. Sail prepends lakehouse optimizer rules and filters out one built-in-style rule by name.

The test asserts that `expand_row_level_op` runs first. That is a Spark and lakehouse semantic requirement: row-level operations such as MERGE/DELETE/UPDATE must be expanded before generic optimizers obscure the structure needed for correct planning.

This is an important DataFusion lesson: optimizer rule order is part of engine semantics. For extensions, “add my optimizer rule” is not sufficient. The API must answer where it runs, what it can assume, and which rules it must precede or follow.

7.12 Physical Optimizers

Sail also customizes physical optimization in `crates/sail-physical-optimizer/src/lib.rs`.

The rule list includes DataFusion’s standard physical optimizer rules in the same order, then adds Sail-specific rules:

```

rules.push(Arc::new(RewriteExplicitRepartition::new()));
rules.push(Arc::new(RewriteCollectLeftHashJoin::new()));
rules.push(Arc::new(EnforceBarrierPartitioning::new()));
rules.push(Arc::new(SanityCheckPlan::new()));

```

The test compares Sail’s rule order with DataFusion’s default physical optimizer to ensure Sail does not accidentally reorder DataFusion defaults.

`RewriteExplicitRepartition` is a good example. During physical planning, Sail creates an `ExplicitRepartitionExec` placeholder. Later, the physical optimizer rewrites it:

- hash partitioning -> DataFusion `RepartitionExec`
- unknown partitioning to one partition -> `CoalescePartitionsExec`
- unknown partitioning to fewer partitions -> Sail `CoalesceExec`
- unknown partitioning to at least the input partition count -> remove the node
- round-robin -> keep Sail’s explicit repartition node

Why not decide all of that immediately in the physical planner? Because the optimizer sees the larger physical plan and can make a more context-aware choice. This keeps planning declarative and lets rewrite rules simplify the physical plan after DataFusion has done its own work.

EnforceBarrierPartitioning is another distributed-execution hint. It rewrites BarrierExec preconditions so all precondition partitions complete before the actual plan begins. That behavior matters for Spark-like commands where one operation must finish globally before another starts.

7.13 Function Mapping: Spark Names To DataFusion Expressions

Spark has a large function surface. DataFusion has its own function surface. Sail sits in between.

The function registry in `crates/sail-plan/src/function/mod.rs` collects built-in scalar, generator, aggregate, table, and window functions:

```
lazy_static! {
    pub static ref BUILT_IN_SCALAR_FUNCTIONS: HashMap<&'static str,
ScalarFunction> =
        HashMap::from_iter(scalar::list_built_in_scalar_functions());
    pub static ref BUILT_IN_GENERATOR_FUNCTIONS: HashMap<&'static str,
ScalarFunction> =
        HashMap::from_iter(generator::list_built_in_generator_functions());
    pub static ref BUILT_IN_TABLE_FUNCTIONS: HashMap<&'static str,
Arc<TableFunction>> =
        HashMap::from_iter(table::list_built_in_table_functions());
}
```

The type alias for scalar planning functions is:

```
pub(crate) type ScalarFunction =
    Arc<dyn Fn(ScalarFunctionInput) -> PlanResult<expr::Expr> + Send + Sync>;
```

This means Sail’s “function registry” is not only a map from name to UDF. It is a map from Spark function name to a small planning function that can:

- validate argument count,
- inspect Spark planning config,
- use the current schema,
- rewrite arguments,
- call a DataFusion built-in,
- call a Sail UDF,
- or construct a larger expression tree.

For simple cases, `ScalarFunctionBuilder::udf` wraps a `ScalarUDFImpl`:

```
pub fn udf<F>(f: F) -> ScalarFunction
where
    F: ScalarUDFImpl + Send + Sync + 'static,
{
    let func = ScalarUDF::from(f);
    Arc::new(move |input| Ok(func.call(input.arguments)))
}
```

For more complex Spark functions, Sail uses custom builders that produce DataFusion expressions directly. This planning-time layer is one of the main places Spark semantics are preserved.

7.14 Vectorized UDFs

The DataFusion UDF documentation explains that scalar UDFs are vectorized: they receive Arrow arrays and return Arrow arrays. Sail's function kernels follow that model.

SparkMask in crates/sail-function/src/scalar/string/spark_mask.rs implements ScalarUDFImpl:

```
impl ScalarUDFImpl for SparkMask {
    fn name(&self) -> &str {
        "spark_mask"
    }

    fn signature(&self) -> &Signature {
        &self.signature
    }

    fn return_type(&self, arg_types: &[DataType]) -> Result<DataType> {
        ...
    }

    fn invoke_with_args(&self, args: ScalarFunctionArgs) -> Result<ColumnarValue> {
        {
            ...
        }
    }
}
```

The return type function is a planning/type-checking hook. The invoke_with_args function is the execution hook. It receives ColumnarValue arguments, which may be scalar values or Arrow arrays.

The Explode function in crates/sail-function/src/scalar/explode.rs shows a different pattern:

```
fn invoke_with_args(&self, _: ScalarFunctionArgs) -> Result<ColumnarValue> {
    plan_err!(
        "{} should be rewritten during logical plan analysis",
        self.name()
    )
}
```

explode is represented like a function for analysis, but it should not execute as a normal scalar UDF. It must be rewritten into a plan shape that can expand rows. This is a good example of Spark semantics not fitting a plain expression kernel.

7.15 Table Functions And Providers

DataFusion table functions return TableProvider objects. Sail's range table function in crates/sail-plan/src/function/table/range.rs is a small complete example.

RangeTableFunction implements TableFunctionImpl:

```
impl TableFunctionImpl for RangeTableFunction {
    fn call(&self, args: &[Expr]) -> Result<Arc<dyn TableProvider>> {
        ...
        let node = RangeNode::try_new("id".to_string(), start, end, step,
num_partitions?);
        Ok(Arc::new(RangeTableProvider {
            node: Arc::new(node),
        }))
    }
}
```

The provider exposes both a logical and physical path:

```
fn get_logical_plan(&self) -> Option<Cow<'_, LogicalPlan>> {
    Some(Cow::Owned(LogicalPlan::Extension(
        logical_plan::Extension {
            node: self.node.clone(),
        },
    )))
}

async fn scan(...) -> Result<Arc<dyn ExecutionPlan>> {
    Ok(Arc::new(RangeExec::try_new(...)?))
}
```

This is a useful pattern for extension authors:

- If the planner wants a logical plan, provide a logical extension node.
- If DataFusion asks for a scan directly, provide an ExecutionPlan.
- Keep the shared semantic payload in a small node object.

Range is simple, but the shape generalizes to custom table-valued functions, specialized data sources, and extension-backed virtual tables.

7.16 DataFusion As Local Engine, Sail As Distributed Engine

DataFusion's ExecutionPlan API is partitioned and asynchronous, but DataFusion itself is not Sail's whole distributed runtime. Sail builds a distributed layer around DataFusion physical plans.

The boundary is visible in ServerSessionFactory::create_job_runner:

```
let job_runner: Box<dyn JobRunner> = match self.config.mode {
    ExecutionMode::Local => Box::new(LocalJobRunner::new()),
    ExecutionMode::LocalCluster => Box::new(ClusterJobRunner::new(...)),
    ExecutionMode::KubernetesCluster => Box::new(ClusterJobRunner::new(...)),
};
```

JobService is placed in SessionConfig as an extension. Later, Spark Connect and Flight SQL can retrieve it from the session and execute a physical plan.

This separation is one of Sail's strongest design choices:

- DataFusion plans and executes partitioned operators.

- Sail decides whether those operators run locally or as a distributed job.
- The same logical and physical plan pipeline feeds both modes.

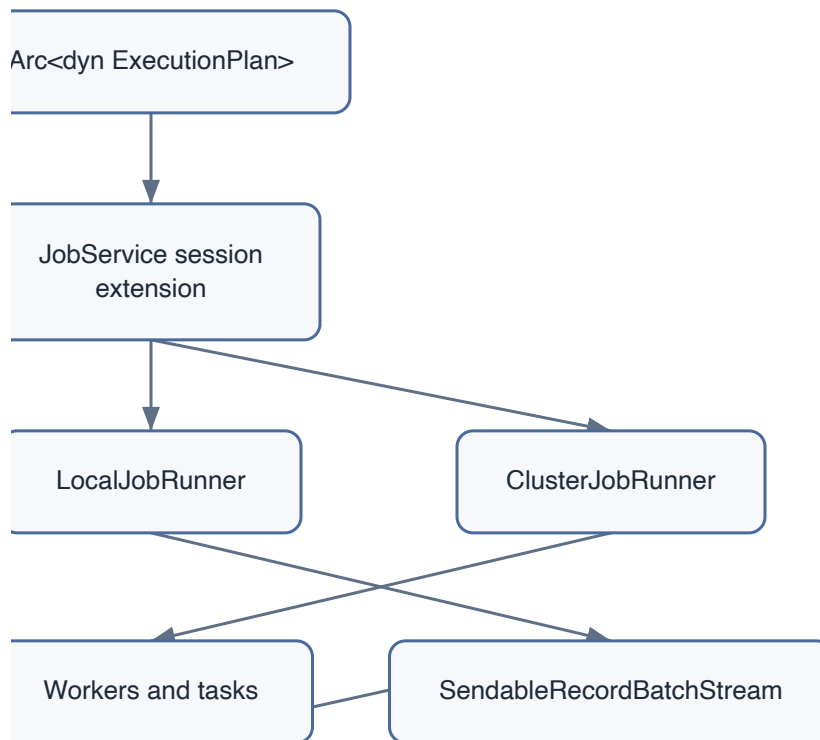


Figure 30: Flowchart 06.5

The next chapters will zoom into that distributed layer. For now, remember that DataFusion’s physical plan is the unit Sail distributes.

7.17 Spark Semantics On A DataFusion Kernel

Sail’s DataFusion integration repeatedly follows this pattern:

1. Resolve Spark/Sail input into a DataFusion-compatible plan.
2. Preserve Spark-only semantics in extension nodes or metadata.
3. Let DataFusion optimize and plan what it understands.
4. Intercept extension nodes with custom planners.
5. Implement missing execution behavior with custom ExecutionPlan nodes.

6. Return Arrow batch streams to the protocol layer.

Some examples:

Spark/Sail concept	DataFusion integration
Spark range	RangeNode -> RangeExec
Spark repartition/coalesce	ExplicitRepartitionNode -> ExplicitRepartitionExec -> physical optimizer rewrite
Sort within partitions	logical extension -> DataFusion SortExec with preserved partitioning
Spark built-in functions	planning registry -> DataFusion expressions or Sail ScalarUDFImpl
Spark generator functions	analyzed as functions, then rewritten into plan structure
Catalog commands	logical command nodes -> CatalogCommandExec
Lakehouse row-level operations	lakehouse optimizer rules before generic optimizer rules
System tables	system table extension planner and TableProvider

That table is the heart of the chapter. Sail is not asking DataFusion to become Spark. Sail is using DataFusion as a powerful substrate and adding Spark compatibility at well-defined seams.

7.18 Extension Implications

The extensions proposal in discussion #2001 is largely about opening the seams this chapter has exposed.

Today, Sail has internal extension points:

- session config extensions via `with_extension`,
- typed access via `SessionExtensionAccessor`,
- built-in function registries,
- analyzer and optimizer rule lists,
- physical optimizer rules,
- `DefaultPhysicalPlanner::with_extension_planners`,
- `UserDefinedLogicalNodeCore`,
- custom `ExecutionPlan` nodes,
- table functions and table providers,
- physical-plan codecs for distributed workers.

But those points are mostly wired inside Sail. A third-party extension API would need to make them explicit and safe.

For example, a Sedona-style spatial extension might need to register:

- scalar functions such as `ST_Area`, `ST_Intersects`, `ST_GeomFromWKB`,
- aggregate functions such as `ST_Union_Aggr`,
- logical optimizer rules for spatial predicate rewrites,
- custom physical planner nodes for spatial joins,
- Arrow extension type handling for GeoArrow metadata,
- Python package entry points for PySpark compatibility,
- distributed codecs so workers can deserialize custom physical nodes,
- and session config defaults.

DataFusion already has many of the underlying concepts. Sail's challenge is to wrap them in an API that respects Spark compatibility and distributed execution.

7.19 Reading Exercise: Trace range

Follow Spark's range from function to execution:

1. Start in `crates/sail-plan/src/function/table/mod.rs`.
2. See `range` registered as a built-in table function.
3. Open `crates/sail-plan/src/function/table/range.rs`.
4. Read `RangeTableFunction::call`.
5. Follow `RangeNode::try_new` in `crates/sail-logical-plan/src/range.rs`.
6. Jump to `ExtensionPhysicalPlanner` in `crates/sail-session/src/planner.rs`.
7. Find the `RangeNode` downcast.
8. Follow `RangeExec::try_new` and `RangeExec::execute`.
9. Observe the final `SendableRecordBatchStream`.

You have now traced a Spark-compatible table function through DataFusion's logical and physical extension APIs.

7.20 Reading Exercise: Trace repartition

Follow explicit repartitioning:

1. Start in `crates/sail-logical-plan/src/repartition.rs`.
2. Find `ExplicitRepartitionNode`.
3. Jump to `ExtensionPhysicalPlanner`.
4. Find the `ExplicitRepartitionNode` case.
5. Read `plan_explicit_partitioning`.
6. Open `crates/sail-physical-plan/src/repartition.rs`.
7. Read `ExplicitRepartitionExec`.
8. Open `crates/sail-physical-optimizer/src/explicit_repartition.rs`.
9. Observe how the placeholder is rewritten to DataFusion or Sail physical operators depending on partitioning.

This trace teaches the difference between preserving user intent and choosing the final execution shape.

7.21 Reading Exercise: Trace A Spark Function

Pick a function such as `mask`:

1. Start in `crates/sail-plan/src/function/scalar/mod.rs`.
2. Find where string functions are added.
3. Open `crates/sail-plan/src/function/scalar/string.rs`.
4. Find the mapping to `SparkMask`.
5. Open `crates/sail-function/src/scalar/string/spark_mask.rs`.
6. Read `return_type` and `invoke_with_args`.
7. Notice how planning-time Spark semantics and execution-time Arrow kernels live in different crates.

This is the pattern most third-party scalar functions will want to follow.

7.22 Takeaways

DataFusion gives Sail its query engine, but Sail decides how Spark semantics enter and leave that engine.

The central pipeline is: Sail spec -> DataFusion logical plan -> logical optimization -> Sail/DataFusion physical planning -> physical optimization -> ExecutionPlan -> Arrow RecordBatch stream.

Sail customizes DataFusion through session extensions, analyzer and optimizer rules, function planning registries, extension logical nodes, extension physical planners, custom physical operators, and physical optimizer rules. Those are the same seams the extension architecture must eventually expose to third-party packages.

The next chapter moves from “how does Sail get a physical plan?” to “how does Sail split that physical plan into a distributed job graph?”

8 Chapter 7: From Physical Plan To Job Graph

The previous chapter ended with a DataFusion physical plan:

```
Arc<dyn ExecutionPlan>
```

That is enough for local execution. DataFusion can call `execute(partition, task_context)` on the plan and return a stream of Arrow RecordBatch values.

But a distributed engine needs one more transformation. It has to decide:

- which parts of the physical plan can run together,
- where data has to be materialized,
- where shuffle boundaries exist,
- how partitions map to tasks,
- which stage outputs are consumed by which later stages,

- and which tasks run on workers versus the driver.

In Sail, that transformation is the job graph.

This chapter follows the path:

DataFusion ExecutionPlan -> Sail JobGraph -> JobTopology -> task regions -> task definitions

The job graph is where DataFusion's local, partitioned execution model becomes Sail's distributed execution model.

8.1 The Core Files

The main files for this chapter are:

Area	Files	Role
Job graph data model	<code>crates/sail-execution/src/job_graph/mod.rs</code>	Defines jobs, stages, inputs, distributions, placement, and input modes
Job graph planner	<code>crates/sail-execution/src/job_graph/planner.rs</code>	Splits a DataFusion physical plan into stages
Job runner boundary	<code>crates/sail-execution/src/job_runner.rs</code>	Chooses local execution or cluster execution
Driver job acceptance	<code>crates/sail-execution/src/driver/job_scheduler/core.rs</code>	Builds the job graph and creates job output streams
Job topology	<code>crates/sail-execution/src/driver/job_scheduler/topology.rs</code>	Groups stages into task regions and dependencies
Job state	<code>crates/sail-execution/src/driver/job_scheduler/state.rs</code>	Tracks jobs, stages, tasks, attempts, and task states
Task definition	<code>crates/sail-execution/src/task/definition.rs</code>	Defines serialized worker task inputs and outputs
Stage input placeholder	<code>crates/sail-execution/src/plan/stage_input.rs</code>	Placeholder <code>ExecutionPlan</code> node for cross-stage inputs
Shuffle plans	<code>crates/sail-execution/src/plan/shuffle_write.rs</code> , <code>shuffle_read.rs</code>	Physical data movement at stage boundaries
Job output	<code>crates/sail-execution/src/driver/output.rs</code>	Merges final task output streams into the client-facing stream

The first file to read is `job_graph/mod.rs`. It defines the vocabulary.

8.2 Local Runner Versus Cluster Runner

Sail can execute the same DataFusion physical plan locally or through the cluster runtime.

`crates/sail-execution/src/job_runner.rs` has two implementations of `JobRunner`:

- `LocalJobRunner`
- `ClusterJobRunner`

The local runner is intentionally simple:

```
let plan = trace_execution_plan(plan, options)?;  
Ok(execute_stream(plan, ctx.task_ctx())?)
```

It hands the plan directly to DataFusion's `execute_stream`.

The cluster runner sends the plan to the driver actor:

```
self.driver  
  .send(DriverEvent::ExecuteJob {  
    plan,  
    context: ctx.task_ctx(),  
    result: tx,  
  })  
  .await?;
```

That event is handled by the driver:

```
let out = self.job_scheduler.accept_job(ctx, plan, context);  
if let Ok((job_id, _)) = &out {  
  self.refresh_job(ctx, *job_id);  
  self.run_tasks(ctx);  
  self.scale_up_workers(ctx);  
}
```

So the split is:

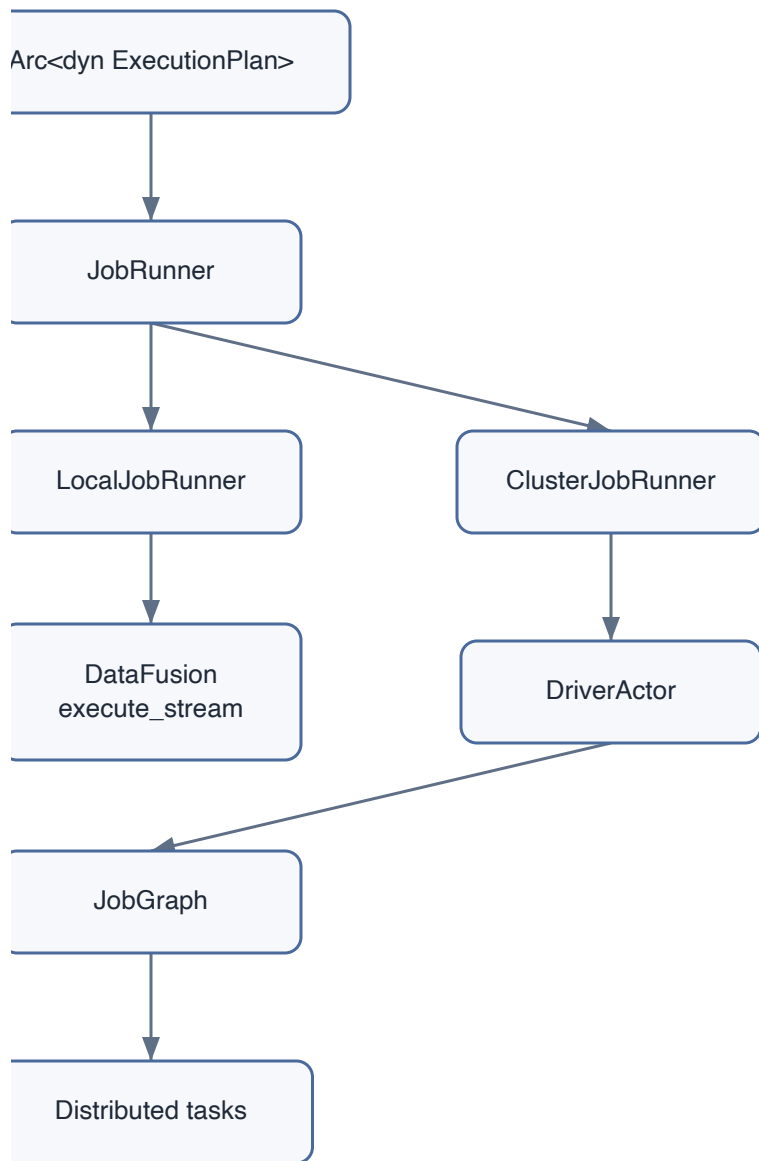


Figure 31: Flowchart 07.1

This chapter is about the cluster branch.

8.3 What A Job Graph Represents

JobGraph is defined in `crates/sail-execution/src/job_graph/mod.rs`:

```
pub struct JobGraph {  
    stages: Vec<Stage>,  
    schema: SchemaRef,  
}
```

The code comment gives the mental model:

- a job has stages,
- a stage has partitions,
- a task executes one partition of one stage,
- a task can have multiple attempts,
- each task produces output split into channels.

That last point is essential. A task does not merely produce one stream. It can produce multiple channels so downstream tasks can read:

- the same partition,
- all partitions,
- a shuffle channel from all partitions,
- all channels for broadcast,
- or a contiguous subset for rescale.

The Stage struct carries:

```
pub struct Stage {  
    pub inputs: Vec<StageInput>,  
    pub plan: Arc<dyn ExecutionPlan>,  
    pub group: String,  
    pub mode: OutputMode,  
    pub distribution: OutputDistribution,  
    pub placement: TaskPlacement,  
}
```

Each stage is still an `ExecutionPlan`. Sail does not compile to a separate mini-language for stages. Instead, it cuts a `DataFusion` physical plan into smaller `DataFusion` physical plans and links them with placeholders.

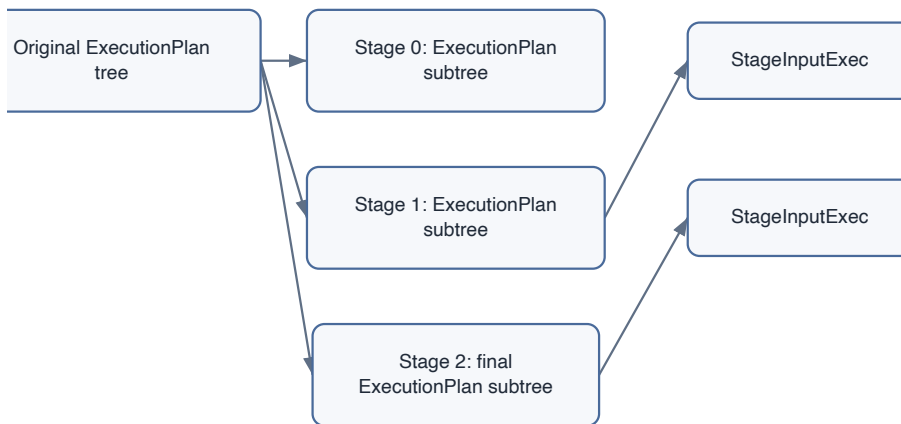


Figure 32: Flowchart 07.2

StageInputExec is the placeholder that marks “read another stage’s output here.”

8.4 StageInputExec: A Placeholder, Not An Operator

crates/sail-execution/src/plan/stage_input.rs defines StageInputExec<I>.

It implements DataFusion’s ExecutionPlan, but its execute method errors:

```

fn execute(
    &self,
    _partition: usize,
    _context: Arc<TaskContext>,
) -> Result<SendableRecordBatchStream> {
    internal_err!("{}", self.name())
}

```

That is deliberate. StageInputExec is not meant to run as-is. It is a marker inserted during job graph construction. Later, when a worker task is prepared, the task runner resolves the placeholder into a real ShuffleReadExec or stream input.

The generic parameter I changes meaning during planning:

- StageInputExec<StageInput> records a logical dependency on another stage.
- StageInputExec<usize> records an index into the current stage’s input list.

The rewrite_inputs function performs that conversion:

```

if let Some(placeholder) =
node.as_any().downcast_ref::<StageInputExec<StageInput>>() {
    let index = inputs.len();

```



```

        inputs.push(placeholder.input().clone());
        let placeholder = StageInputExec::new(index,
placeholder.properties().clone());
        Ok(Transformed::yes(Arc::new(placeholder)))
    }

```

That gives every stage:

- a plan containing numbered input placeholders,
- and a separate `inputs: Vec<StageInput>` that describes where those inputs come from.

This separation is neat. The plan stays serializable as a DataFusion physical plan, while stage dependencies stay explicit in the job graph.

8.5 Stage Inputs And Input Modes

`StageInput` has two fields:

```

pub struct StageInput {
    pub stage: usize,
    pub mode: InputMode,
}

```

`InputMode` is the distributed execution contract:

```

pub enum InputMode {
    Forward,
    Merge,
    Shuffle,
    Broadcast,
    Rescale,
}

```

The code comments are worth translating into a table:

Mode	Current partition reads	Used for
Forward	Same partition from the input stage, all channels	Pipelined one-to-one dependencies
Merge	All partitions from input stage, all channels	Sort-preserving merge and global merge-style inputs
Shuffle	Same output channel from all input partitions	Hash or round-robin repartition
Broadcast	All partitions and all channels	Shared build-side or repeated consumption
Rescale	A contiguous subset of input partitions, all channels	Coalescing from many partitions to fewer partitions

The scheduler later converts each mode into concrete `TaskInputKey` groups in `build_task_input_keys`.

For shuffle, the comment says it plainly:

```
// Enumerate channels in the outer loop and partitions in the inner loop.  
// This is the whole point of shuffle!
```

If an upstream stage has P partitions and C channels, then shuffle input groups are:

```
output partition 0 reads: (input p0, channel 0), (input p1, channel 0), ...  
output partition 1 reads: (input p0, channel 1), (input p1, channel 1), ...  
...
```

That is the exchange pattern for repartitioning.

8.6 Output Distribution

A stage's output distribution describes how each task splits its output into channels:

```
pub enum OutputDistribution {  
  Hash {  
    keys: Vec<Arc<dyn PhysicalExpr>>,  
    channels: usize,  
  },  
  RoundRobin {  
    channels: usize,  
  },  
}
```

Hash means evaluate physical expressions on each row and send the row to the corresponding channel. RoundRobin means distribute rows across channels without hash keys.

This becomes a `TaskOutputDistribution` when the scheduler creates a task definition:

```
TaskOutputDistribution::Hash {  
  keys,  
  channels,  
}
```

The hash keys are serialized physical expressions, which is one reason the extensions chapter will need to care about physical-plan codecs. If an extension introduces a custom physical expression, distributed workers must be able to deserialize it.

8.7 How `JobGraph::try_new` Starts

The entry point is:

```
pub fn try_new(plan: Arc<dyn ExecutionPlan>) -> ExecutionResult<Self>
```

The first two lines are important rewrites:

```
let plan = ensure_single_input_partition_for_global_limit(plan)?;  
let plan =  
ensure_partitioned_hash_join_if_build_side_emits_unmatched_rows(plan)?;
```

Then Sail builds an empty graph and recursively splits the plan:

```
let mut graph = Self {
    stages: vec![],
    schema: plan.schema(),
};
let last = build_job_graph(plan, PartitionUsage::Once, &mut graph)?;
let (last, inputs) = rewrite_inputs(last)?;
graph.stages.push(Stage {
    inputs,
    plan: last,
    mode: OutputMode::Pipelined,
    distribution: OutputDistribution::RoundRobin { channels: 1 },
    placement: TaskPlacement::Worker,
});
```

The final stage is always added after recursive splitting. Its output schema is the job schema. If no later stage consumes it, the driver will expose its task streams as job output.

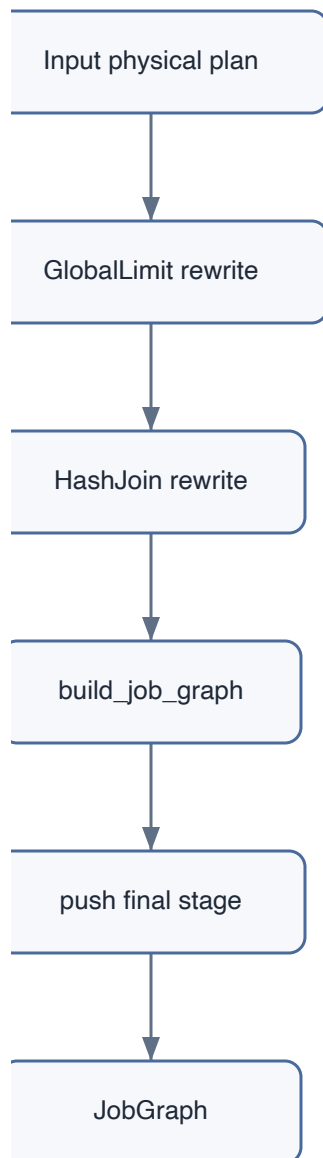


Figure 33: Flowchart 07.3

8.8 Rewrite 1: Global Limit Needs One Input Partition

`ensure_single_input_partition_for_global_limit` rewrites every `GlobalLimitExec`.

If a global limit has a real `LIMIT` or `OFFSET` and its input has more than one partition, Sail wraps the input in `CoalescePartitionsExec`:

```
let input = Arc::new(CoalescePartitionsExec::new(input.clone()));
Arc::new(GlobalLimitExec::new(input, skip, fetch))
```

Why?

A global limit is not the same as a per-partition limit. If each worker applies `LIMIT 10` locally, the whole job may return far more than 10 rows. Sail keeps any local limit optimization that `DataFusion` created, but ensures the final global limit sees a single partition.

This is an example of distributed correctness requiring a physical rewrite after `DataFusion` planning.

8.9 Rewrite 2: Collect-Left Hash Joins And Unmatched Rows

`ensure_partitioned_hash_join_if_build_side_emits_unmatched_rows` handles a more subtle distributed problem.

`DataFusion` can use `PartitionMode::CollectLeft` for hash joins. In that mode, one side is collected and reused. This is fine in local execution. In a distributed engine, it becomes tricky for join types that need to emit unmatched rows from the build side, because row-match state is not shared across all distributed partitions.

For join types such as `Left`, `LeftAnti`, `LeftSemi`, `LeftMark`, and `Full`, Sail rewrites the join to `PartitionMode::Partitioned` with explicit repartitioning on both sides:

```
HashJoinExec::try_new(
    repartition(join.left, left_exprs, partition_count)?,
    repartition(join.right, right_exprs, partition_count)?,
    join.on.clone(),
    ...
    PartitionMode::Partitioned,
    ...
)
```

This makes each output partition independently executable. It may be less clever than a local collect-left plan, but it is correct for Sail's distributed execution model.

The lesson is that physical plans produced by a single-node optimizer sometimes need a distribution-aware fixup before being split into stages.

8.10 Recursive Stage Splitting

The heart of the planner is `build_job_graph`.

It walks the physical plan tree from the leaves upward. First it recursively processes children. Then it decides whether the current node introduces a stage boundary.

These nodes introduce boundaries:

- RepartitionExec
- ExplicitRepartitionExec
- CoalescePartitionsExec
- SortPreservingMergeExec
- Sail CoalesceExec
- driver-only plans such as SystemTableExec and CatalogCommandExec

The broad shape is:

```
let children = ... build_job_graph(child, usage, graph) ...;
let plan = with_new_children_if_necessary(plan, children)?;

let plan = if let Some(repartition) =
plan.as_any().downcast_ref::<RepartitionExec>() {
    create_shuffle(child, graph, properties, consumption)?
} else if let Some(coalesce) =
plan.as_any().downcast_ref::<CoalescePartitionsExec>() {
    create_shuffle(child, graph, properties, consumption)?
} else if plan.as_any().is::<SortPreservingMergeExec>() {
    plan.with_new_children(vec![create_merge_input(child, graph)?])?
} else if let Some(coalesce) = plan.as_any().downcast_ref::<CoalesceExec>() {
    create_rescale_input(child, coalesce.output_partitions(), graph)?
} else {
    plan
};
```

The planner preserves ordinary operators inside a stage. It cuts only when the execution pattern changes across partitions.

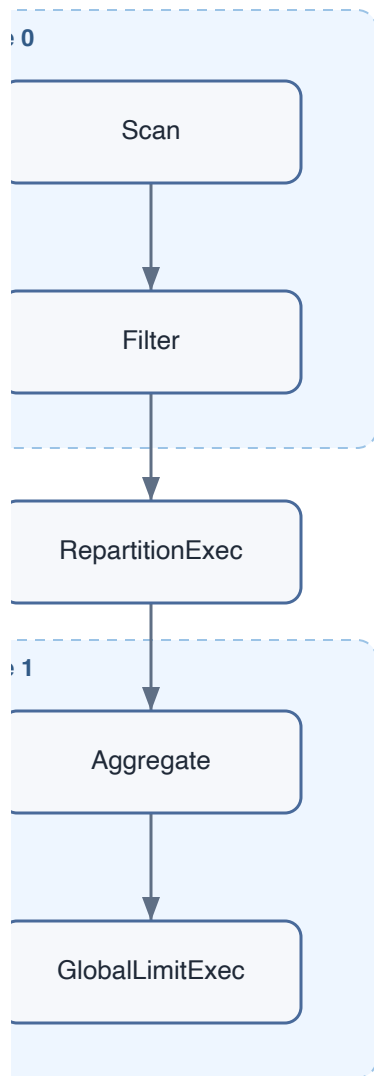


Figure 34: Flowchart 07.4

In the job graph, `RepartitionExec` itself is replaced by a stage boundary: stage 0 writes channels; stage 1 reads those channels.

8.11 Partition Usage: Single Versus Shared

The `PartitionUsage` enum has two variants:

```
enum PartitionUsage {
    Once,
```

```
    Shared,
  }
}
```

Most plan inputs are used once. But some joins reuse one side across many partitions. For example, a collect-left style join may gather the build-side data through `execute(0)` for each probe-side partition.

In local DataFusion, helper machinery can make that efficient inside one process. In distributed execution, reused data must be materialized so multiple downstream tasks can consume it safely.

Sail maps usage to shuffle consumption:

```
let consumption = match usage {
  PartitionUsage::Once => ShuffleConsumption::Single,
  PartitionUsage::Shared => ShuffleConsumption::Multiple,
};
```

Then `create_shuffle` chooses the input mode:

```
let mode = match consumption {
  ShuffleConsumption::Single => InputMode::Shuffle,
  ShuffleConsumption::Multiple => InputMode::Broadcast,
};
```

So a shared input becomes broadcast-like downstream consumption.

This is a nice example of a local execution property becoming a distributed data movement decision.

8.12 create_shuffle

`create_shuffle` is used for repartition and coalesce-style boundaries.

It converts DataFusion partitioning into Sail output distribution:

```
let distribution = match properties.partitioning.clone() {
  Partitioning::RoundRobinBatch(channels)
  | Partitioning::UnknownPartitioning(channels) => {
    OutputDistribution::RoundRobin { channels }
  }
  Partitioning::Hash(keys, channels) => OutputDistribution::Hash { keys,
channels },
};
```

Then it turns the child into a stage:

```
let (plan, inputs) = rewrite_inputs(plan.clone())?;
let stage = Stage {
  inputs,
  plan,
  mode: OutputMode::Pipelined,
  distribution,
  placement: TaskPlacement::Worker,
```



```
};
graph.stages.push(stage);
```

Finally it returns a StageInputExec placeholder for the parent plan:

```
StageInputExec::new(
    StageInput { stage: s, mode },
    properties,
)
```

The parent sees an execution plan input with the right schema and partitioning. The job graph sees a dependency on a previous stage.



Figure 35: Flowchart 07.5

8.13 Merge And Rescale Boundaries

SortPreservingMergeExec uses create_merge_input.

That creates a worker stage for the child and returns a StageInputExec with InputMode::Merge. A merge input reads all partitions from the input stage. It is how a later operator can see globally merged streams.

Sail's custom CoalesceExec uses create_rescale_input.

Rescale is different from broadcast and shuffle. It divides input partitions into contiguous ranges:

```
let start = output_partition * input_partitions / output_partitions;
let end = (output_partition + 1) * input_partitions / output_partitions;
```

Each output partition consumes only its assigned range. This is the distributed form of reducing partition count without fully merging everything into one partition.

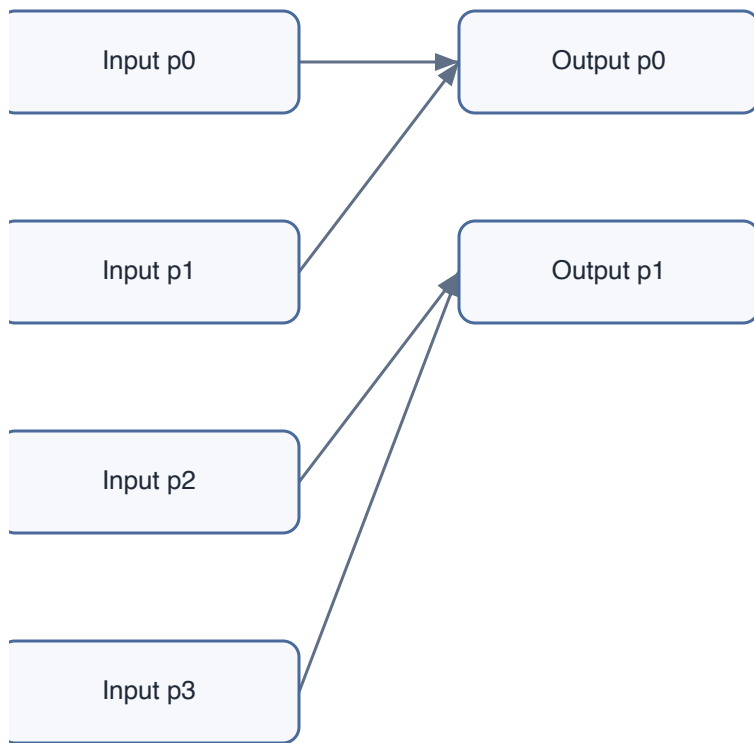


Figure 36: Flowchart 07.6

8.14 Driver Stages

Most stages run on workers:

`placement: TaskPlacement::Worker`

But some plans must run on the driver. The job graph planner recognizes:

- `SystemTableExec`
- `CatalogCommandExec`

and creates a driver stage:

```
Stage {  
  inputs: vec![],  
  plan: plan.clone(),  
  distribution: OutputDistribution::RoundRobin { channels: 1 },  
}
```

```
    placement: TaskPlacement::Driver,
  }
```

The TODO says driver stages with inputs are not supported yet. That is an important limitation for extension design. If a future extension introduces a driver-only physical operator that consumes distributed inputs, Sail would need to extend this part of the planner.

8.15 Topological Order

JobGraph stores stages in topological order:

```
/// For any stage, all its input stages are guaranteed to
/// appear before it in the list.
stages: Vec<Stage>
```

The recursive construction naturally creates earlier stages before later stages. When the final stage is pushed, all its dependencies have already been added.

This matters because stage indices become part of task stream keys:

```
TaskStreamKey {
    job_id,
    stage,
    partition,
    attempt,
    channel,
}
```

Once a stage is inserted into the graph, its index is the stable identity used by the scheduler, task runner, stream manager, and job output system.

8.16 Replicas And Repeated Consumption

JobGraph::replicas(stage) computes how many replicas of a stage's output are needed:

```
match input.mode {
    InputMode::Forward | InputMode::Shuffle | InputMode::Rescale => 1,
    InputMode::Merge | InputMode::Broadcast => {
        x.plan.output_partitioning().partition_count()
    }
}
```

The result is at least one, because final stages need output for the client even if no later stage consumes them.

Why do merge and broadcast need more replicas? Because multiple downstream partitions may read the same upstream streams. If output is pipelined and stored locally, Sail must keep enough stream replicas available for all consumers.

This is the data-plane cost of repeated consumption.

8.17 From Job Graph To Job Topology

After JobGraph::try_new, the scheduler creates a JobDescriptor:

```
let graph = JobGraph::try_new(plan)?;
let (output, stream) = build_job_output(ctx, job_id, graph.schema().clone());
let descriptor = JobDescriptor::try_new(graph, JobState::Running { output,
context })?;
```

JobDescriptor::try_new creates:

- one StageDescriptor per stage,
- one TaskDescriptor per stage partition,
- a JobTopology,
- one TaskRegionDescriptor per topology region.

The topology builder groups pipelined stages into task regions. This is the transition from “stages” to “what should be scheduled together.”

JobTopology::try_new first records stage consumers and pipelined adjacency. Then it finds connected components of pipelined stages.

If every in-component input is Forward, the component can be sliced by partition:

```
region 0: stage A partition 0, stage B partition 0
region 1: stage A partition 1, stage B partition 1
...
```

If the component has non-forward inputs, Sail creates one region containing all partitions of the component.

This captures a practical scheduling idea:

- one-to-one pipelined dependencies can run partition by partition,
- shuffle/merge/broadcast-style dependencies need broader coordination.

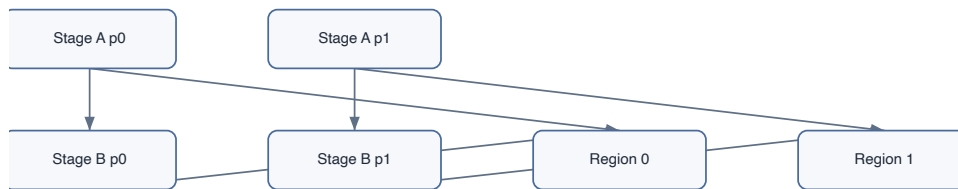


Figure 37: Flowchart 07.7

8.18 Region Dependencies

After regions are created, the topology builder adds dependencies.

For Forward input, a task depends on the corresponding partition of the input stage:

```
TaskTopology {
    stage: input.stage,
    partition: task.partition,
}
```

For all other input modes, the task depends on all partitions of the input stage:

```
for p in 0..partitions {
    TaskTopology {
        stage: input.stage,
        partition: p,
    }
}
```

This is conservative and correct. Shuffle, broadcast, merge, and rescale may need data from multiple upstream partitions, so downstream regions wait until the relevant upstream regions are complete.

8.19 Task Definitions

A worker does not receive a Rust Stage object. It receives a serialized TaskDefinition:

```
pub struct TaskDefinition {
    pub plan: Arc<[u8]>,
    pub inputs: Vec<TaskInput>,
    pub output: TaskOutput,
}
```

The plan is serialized bytes. Inputs describe where to read upstream streams. Output describes how to publish this task's result streams.

Inputs can be:

```
pub enum TaskInputLocator {
    Driver { stage, keys },
    Worker { stage, keys },
    Remote { uri, stage, keys },
}
```

Outputs can be:

```
pub enum TaskOutputLocator {
    Local { replicas },
    Remote { uri },
}
```

The current pipelined path uses local stream storage and worker/driver stream locations. Blocking remote output is present in the type system but not fully implemented in the code paths shown here.

This is another extension lesson: physical plans and expressions must be serializable if they are going to run on workers. A local-only extension is much easier than a distributed-safe extension.

8.20 Job Output

The final stage's output becomes the stream returned to Spark Connect or Flight SQL.

`build_job_output` creates:

- a `JobOutputManager`, used by the scheduler to add task streams,
- and a `SendableRecordBatchStream`, returned to the query caller.

The stream is a `RecordBatchStreamAdapter` around a receiver. Internally, `JobOutputStream` keeps a `SelectAll` of task streams.

When final stage tasks start running or succeed, `extend_job_output` adds their channels:

```
for c in 0..channels {
  let key = TaskStreamKey { job_id, stage: s, partition: p, attempt, channel:
c };
  actions.push(JobAction::ExtendJobOutput {
    handle: output.handle(),
    key,
    schema: schema.clone(),
  });
}
```

The job output stream can therefore begin returning batches while final tasks are still running. That is why the output mode is currently `Pipelined`.

The stream also handles task attempts carefully. If a later attempt supersedes an earlier attempt for the same task stream, the wrapper can mute the older stream so the client does not see duplicate output.

8.21 A Worked Example: Hash Aggregate

Imagine a query that scans a table and groups by `customer_id`:

```
SELECT customer_id, count(*)
FROM orders
GROUP BY customer_id
```

A simplified physical plan might look like:

```
AggregateExec final
  RepartitionExec Hash(customer_id, 4)
    AggregateExec partial
      TableScanExec
```

The job graph planner sees `RepartitionExec` and cuts the plan:

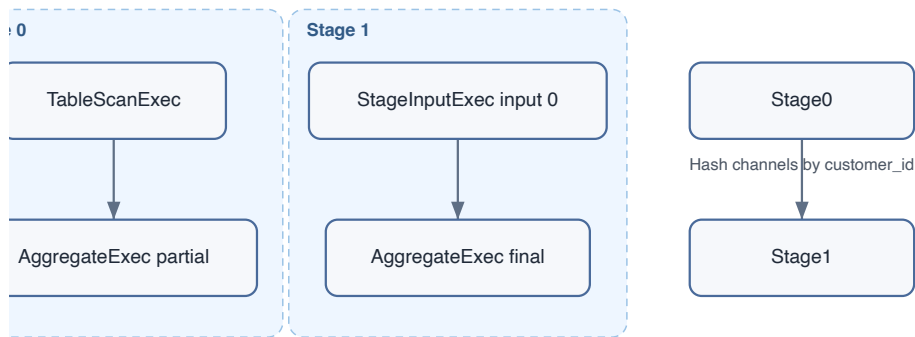


Figure 38: Flowchart 07.8

Stage 0 output distribution is Hash { keys: [customer_id], channels: 4 }. Stage 1 input mode is Shuffle. Therefore:

- Stage 0 partition 0 writes channels 0..3.
- Stage 0 partition 1 writes channels 0..3.
- Stage 1 partition 0 reads channel 0 from every Stage 0 partition.
- Stage 1 partition 1 reads channel 1 from every Stage 0 partition.
- and so on.

That is a distributed hash exchange.

8.22 A Worked Example: Global Limit

Consider:

```

SELECT *
FROM events
LIMIT 10
  
```

If the scan has many partitions, a global limit must not independently return 10 rows from every partition. Sail rewrites:

```

GlobalLimitExec
  multi-partition input
  
```

into:

```

GlobalLimitExec
  CoalescePartitionsExec
    multi-partition input
  
```

Then CoalescePartitionsExec becomes a stage boundary. The upstream stage produces partitioned output, and the final stage reads it as a coalesced input before applying the global limit.

Correctness beats parallelism here. The global decision has to be made in one place.

8.23 A Worked Example: Broadcast-Like Shared Input

Some joins reuse one side. In Sail's planner, that shows up as `PartitionUsage::Shared`. Shared usage turns into `ShuffleConsumption::Multiple`, which turns into `InputMode::Broadcast`.

The upstream stage is materialized once. Downstream partitions can all read it.

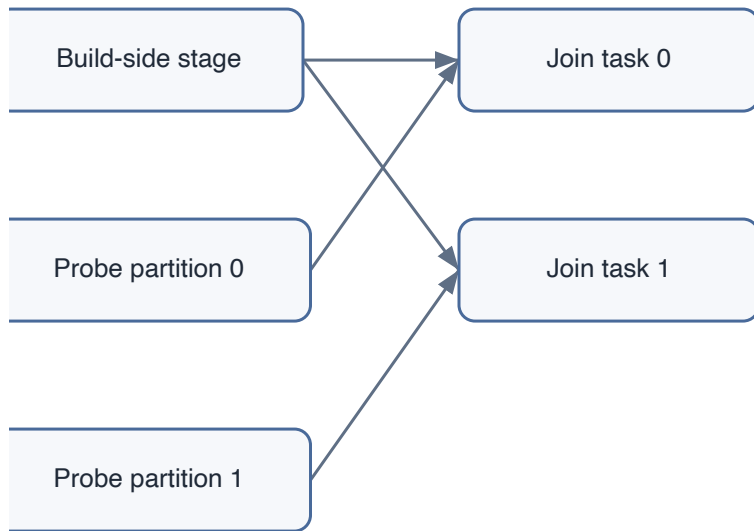


Figure 39: Flowchart 07.9

The key point is not that Sail necessarily implements every broadcast join optimization you might imagine. The key point is that the job graph has an explicit mode for repeated consumption of upstream data.

8.24 Why This Design Works

Sail does not throw away the DataFusion plan and invent a separate distributed IR. Instead, it:

1. Uses DataFusion physical plans as stage bodies.
2. Rewrites a few single-node assumptions for distributed correctness.
3. Cuts stage boundaries at exchange-like operators.
4. Replaces cross-stage edges with `StageInputExec`.
5. Tracks how each stage output is distributed into channels.
6. Lets the scheduler turn stages into task regions and attempts.

This has several benefits:

- Sail inherits DataFusion physical operators and optimizer behavior.
- Custom Sail physical nodes can participate as normal `ExecutionPlan` nodes.
- The job graph can be displayed in terms of familiar physical plans.
- Distributed correctness is concentrated in stage splitting and scheduling.
- The final output is still a normal Arrow `RecordBatch` stream.

8.25 Extension Implications

For discussion #2001, this chapter is the warning label on the box.

It is not enough for an extension to register a DataFusion function or physical planner. If the extension participates in distributed execution, it must also fit this job graph transformation.

An extension author needs to ask:

- Does my physical operator preserve partitioning?
- Does it require all input partitions at once?
- Can it run independently per partition?
- Does it need driver placement?
- Does it produce a custom physical expression used for hash distribution?
- Can its physical plan be serialized to workers?
- If it creates a new exchange-like node, how should the job graph planner cut it into stages?
- If it creates a shared input, should downstream consumption be broadcast?
- If it introduces a driver-only command, can it have inputs?

The current planner hard-codes the known Sail and DataFusion nodes. A general extension API will need a way for extensions to declare distributed planning behavior, or at least a fallback policy that rejects unsupported distributed plans clearly.

8.26 Reading Exercise: Trace A Repartition Boundary

Follow a repartition from DataFusion physical plan to job graph:

1. Open `crates/sail-execution/src/job_graph/planner.rs`.
2. Find `build_job_graph`.
3. Find the `RepartitionExec` branch.
4. Follow the call to `create_shuffle`.
5. Observe how `Partitioning::Hash` becomes `OutputDistribution::Hash`.
6. Observe how `ShuffleConsumption::Single` becomes `InputMode::Shuffle`.
7. Open `crates/sail-execution/src/driver/job_scheduler/core.rs`.
8. Find `build_task_input_keys`.
9. Read the `InputMode::Shuffle` branch.

At the end, you should be able to explain which upstream task streams a downstream shuffle partition reads.

8.27 Reading Exercise: Trace Job Acceptance

Follow a cluster query:

1. Start in `crates/sail-execution/src/job_runner.rs`.
2. Find `ClusterJobRunner::execute`.
3. Follow `DriverEvent::ExecuteJob`.
4. Open `crates/sail-execution/src/driver/actor/handler.rs`.
5. Find `handle_execute_job`.
6. Follow `job_scheduler.accept_job`.
7. Open `crates/sail-execution/src/driver/job_scheduler/core.rs`.
8. Find `JobGraph::try_new(plan)`.
9. Follow `build_job_output`.

That is the control path from a physical plan to a client-visible output stream.

8.28 Takeaways

The job graph is Sail's distributed version of a DataFusion physical plan. It keeps each stage as an `ExecutionPlan`, but replaces cross-stage edges with `StageInputExec` placeholders and records explicit input modes.

The most important enum in this chapter is `InputMode`: `Forward`, `Merge`, `Shuffle`, `Broadcast`, and `Rescale`. Those five modes describe how downstream partitions consume upstream task streams.

Before splitting the plan, Sail rewrites global limits and certain collect-left hash joins for distributed correctness. During splitting, it cuts at repartition, coalesce, merge, rescale, and driver-only nodes. After splitting, the scheduler groups stages into task regions, builds task definitions, and connects final task streams into a single Arrow output stream.

The next chapter will follow those task regions into the driver, workers, task assigner, and stream manager.

9 Chapter 8: Drivers, Workers, Tasks, And Streams

Chapter 7 explained how Sail turns a DataFusion physical plan into a distributed job graph. This chapter follows that graph into motion.

The job graph says what should run:

- stages,
- partitions,
- input modes,
- output distributions,
- driver or worker placement,
- and task stream dependencies.

The driver, workers, task assigner, task runner, and stream managers decide how that work actually happens.

If Chapter 7 was the map, Chapter 8 is the traffic system.

9.1 The Core Files

Area	Files	Role
Actor runtime	<code>crates/sail-server/src/actor.rs</code>	Small async actor system used by the driver and workers
Driver actor	<code>crates/sail-execution/src/driver/actor/*.rs</code>	Accepts jobs, workers, task updates, stream requests, and shutdown
Driver events	<code>crates/sail-execution/src/driver/event.rs</code>	Message protocol for the driver actor
Worker actor	<code>crates/sail-execution/src/worker/actor/*.rs</code>	Registers with driver, receives tasks, reports status, serves/fetches streams
Worker events	<code>crates/sail-execution/src/worker/event.rs</code>	Message protocol for worker actor
Worker pool	<code>crates/sail-execution/src/driver/worker_pool/*.rs</code>	Launches, registers, monitors, and talks to workers
Task assigner	<code>crates/sail-execution/src/driver/task_assigner/*.rs</code>	Maps task regions to driver or worker task slots
Job scheduler	<code>crates/sail-execution/src/driver/job_scheduler/*.rs</code>	Tracks job state, creates attempts, schedules regions, builds task definitions
Task runner	<code>crates/sail-execution/src/task_runner/*.rs</code>	Executes serialized DataFusion physical plans on driver or worker
Stream manager	<code>crates/sail-execution/src/stream_manager/*.rs</code>	Owens local task streams and pending stream fetches
Stream accessor	<code>crates/sail-execution/src/stream_accessor/core.rs</code>	Implements task stream reader/writer by sending actor messages
Stream service	<code>crates/sail-execution/src/stream_service/*.rs</code>	Uses Arrow Flight to fetch task streams across processes
Worker managers	<code>crates/sail-execution/src/worker_manager/*.rs</code>	Launches local or Kubernetes workers

The chapter will follow a single distributed query from the moment the cluster runner sends it to the driver until final Arrow batches are returned.

9.2 The Actor Runtime

Sail's driver and workers are actors. The actor runtime lives in `crates/sail-server/src/actor.rs`.

The trait is small:

```
pub trait Actor: Sized + Send + 'static {
    type Message: Send + SpanAssociation + 'static;
    type Options;

    fn name() -> &'static str;
    fn new(options: Self::Options) -> Self;
    async fn start(&mut self, ctx: &mut ActorContext<Self>) {}
    fn receive(&mut self, ctx: &mut ActorContext<Self>, message: Self::Message) -
> ActorAction;
    async fn stop(self, ctx: &mut ActorContext<Self>) {}
}
```

Messages are processed sequentially:

```
while let Some(MessageEnvelop { message, context }) = self.receiver.recv().await
{
    let action = self.actor.receive(&mut self.ctx, message);
    ...
    self.ctx.reap();
}
```

That gives actor state a simple programming model: the actor mutates its own fields without locks because only one message is handled at a time.

But the actor must not block. The trait comment says blocking work should be spawned through `ActorContext::spawn`. That pattern appears everywhere in driver and worker code:

- launch a worker asynchronously,
- send an RPC to a worker,
- fetch a remote stream,
- report task status,
- run a task monitor,
- merge job output streams.

The actor system gives Sail a clean split:

- state transitions happen synchronously in `receive`,
- slow IO happens in spawned futures,
- spawned futures report back by sending another actor message.

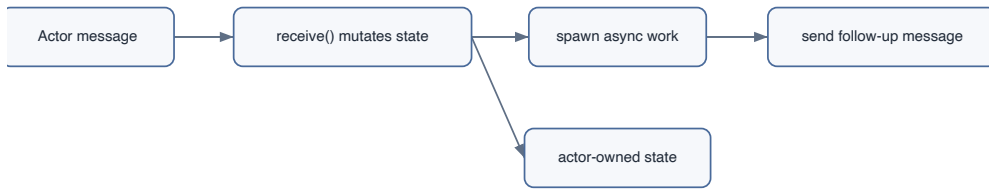


Figure 40: Flowchart 08.1

This is the control-plane style behind Sail’s distributed runtime.

9.3 Driver Actor: The Coordinator

DriverActor is defined across `crates/sail-execution/src/driver/actor`.

Its `new` method constructs the major driver subsystems:

```

let worker_pool = WorkerPool::new(...);
let job_scheduler = JobScheduler::new(...);
let task_assigner = TaskAssigner::new(...);
let stream_manager = StreamManager::new(...);

```

The driver owns:

- WorkerPool: worker lifecycle and worker RPC clients,
- JobScheduler: jobs, stages, regions, tasks, attempts,
- TaskAssigner: task slots and stream ownership,
- TaskRunner: local driver task execution,
- StreamManager: local streams owned by the driver,
- task_sequences: latest worker task status sequence numbers,
- and the driver server monitor.

The driver starts a gRPC server in `start`:

```

self.server = server
    .start(Self::serve(ctx.handle().clone(), addr).in_span(span))
    .await;

```

Once the server is ready, it starts the initial workers:

```

for _ in 0..self.options.worker_initial_count {
    self.worker_pool.start_worker(ctx);
}

```

The driver receives events such as:

- RegisterWorker
- WorkerHeartbeat

- ExecuteJob
- UpdateTask
- CreateLocalStream
- FetchWorkerStream
- CleanUpJob
- Shutdown

That event list is effectively the driver's public control-plane API.

9.4 Worker Actor: The Executor

WorkerActor has a similar shape in `crates/sail-execution/src/worker/actor`.

Its new method constructs:

- a driver client set,
- a peer tracker,
- a task runner,
- a stream manager,
- and a sequence counter for status updates.

When the worker server becomes ready, the worker registers with the driver:

```
client.register_worker(worker_id, host, port).await
```

Then it starts heartbeats:

```
loop {
    tokio::time::sleep(interval).await;
    client.report_worker_heartbeat(worker_id).await;
}
```

The worker receives events such as:

- RunTask
- StopTask
- ReportTaskStatus
- CreateLocalStream
- FetchWorkerStream
- CleanUpJob
- Shutdown

The most important handler is `handle_run_task`:

```
self.peer_tracker.track(ctx, peers);
self.task_runner
    .run_task(ctx, key, definition, self.options.session.task_ctx());
```

The worker learns about peer workers from the driver, remembers their locations, and runs the task with its own TaskContext.

9.5 Worker Launch And Registration

The worker lifecycle starts in the driver `WorkerPool`.

`start_worker`:

1. Allocates a new `WorkerId`.
2. Inserts a `WorkerDescriptor` in Pending state.
3. Schedules a pending-worker probe for timeout handling.
4. Calls the configured `WorkerManager` to launch the worker.

The launch options include:

- TLS setting,
- driver external host and port,
- heartbeat interval,
- task stream buffer,
- stream creation timeout,
- RPC retry strategy.

For local cluster mode, `LocalWorkerManager` spawns a `WorkerActor` in a local actor system:

```
let options = WorkerOptions::local(id, options, self.runtime.clone(),
self.session.clone());
let handle = state.system.spawn(options);
state.workers.insert(id, handle);
```

For Kubernetes mode, the worker manager uses the Kubernetes worker manager implementation. The driver side does not care which launch strategy is used; it just waits for a `RegisterWorker` event.

When registration arrives:

```
worker.state = WorkerState::Running {
  host,
  port,
  updated_at: Instant::now(),
  heartbeat_at: Instant::now(),
  client: None,
};
```

Then the driver schedules:

- a lost-worker probe,
- an idle-worker probe,
- and activates the worker in the task assigner.

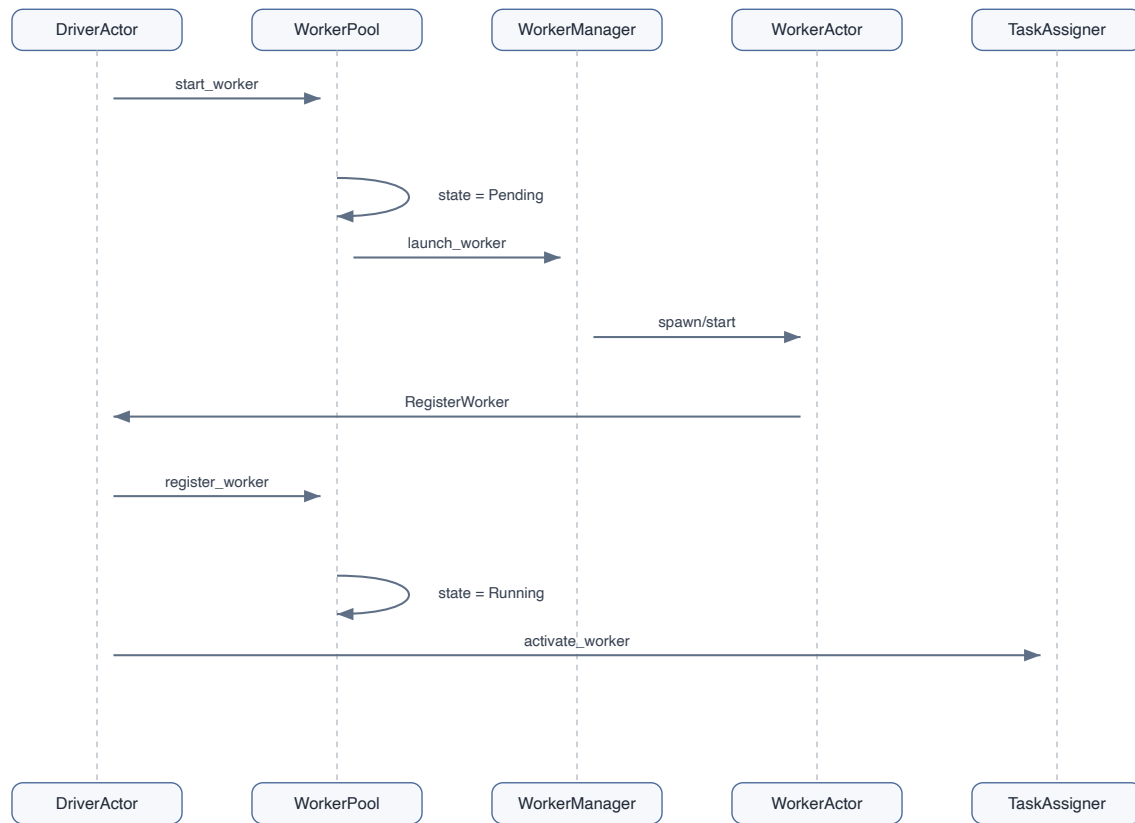


Figure 41: Sequence diagram 08.2

9.6 Worker Health

Workers send heartbeats to the driver. The driver records the latest heartbeat:

```

if let WorkerState::Running { heartbeat_at, .. } = &mut worker.state {
    *heartbeat_at = Instant::now();
    Self::schedule_lost_worker_probe(ctx, worker_id, worker, &self.options);
}

```

If the lost-worker probe fires and the heartbeat is stale, the driver:

1. Stops the worker.
2. Finds tasks assigned to that worker.
3. Marks those task attempts failed.
4. Refreshes affected jobs.
5. Tries to run tasks again.
6. Scales up workers if needed.

The handler in `driver/actor/handler.rs` makes that explicit:

```

let keys = self.task_assigner.find_worker_tasks(worker_id);
self.task_assigner.deactivate_worker(worker_id);

```



```

for key in keys.iter() {
    self.job_scheduler.update_task(
        key,
        TaskState::Failed,
        Some(message.clone()),
        Some(CommonErrorCause::Execution(message.clone())),
    );
}

```

This is the retry story at the worker level. Worker loss becomes task attempt failure. Task attempt failure becomes region rescheduling unless the maximum attempt count is exceeded.

9.7 From Job To Task Regions

The job scheduler accepts a job in `crates/sail-execution/src/driver/job_scheduler/core.rs`:

```

let graph = JobGraph::try_new(plan)?;
let (output, stream) = build_job_output(ctx, job_id, graph.schema().clone());
let descriptor = JobDescriptor::try_new(graph, JobState::Running { output,
context });?;
self.jobs.insert(job_id, descriptor);

```

After acceptance, the driver calls `refresh_job`.

`refresh_job` is the scheduler's main decision function. Its comment lists the steps:

1. Cancel task attempts in a region if any task attempt fails.
2. Add final-stage running/succeeded task streams to job output.
3. Clean up stage output streams when all consumers have succeeded.
4. Fail a job if any task exceeds max attempts.
5. Mark the job succeeded when final regions succeed.
6. Schedule regions whose dependencies have succeeded.

The important point is that the scheduler does not immediately schedule every task. It schedules task regions when dependencies allow them.

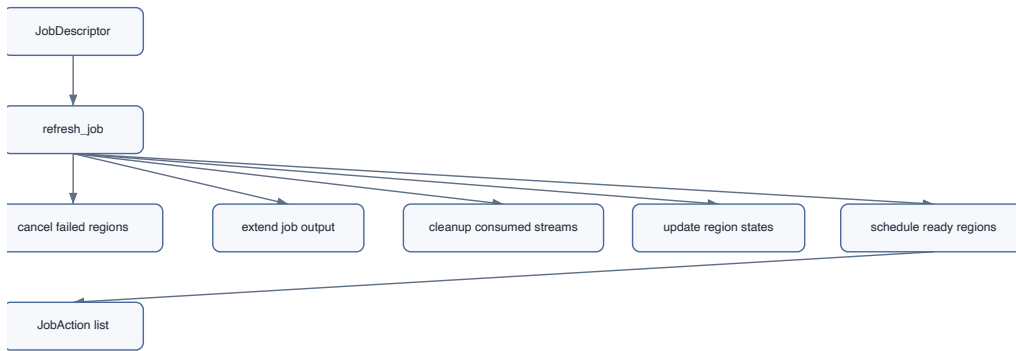


Figure 42: Flowchart 08.3

The driver then executes the returned `JobAction` values.

9.8 Task Attempts

Each stage has tasks. Each task can have multiple attempts:

```
pub struct TaskDescriptor {
    pub attempts: Vec<TaskAttemptDescriptor>,
}
```

An attempt has:

- state,
- messages,
- error cause,
- job_output_fetched,
- creation time,
- stop time.

When a task region becomes schedulable, the scheduler pushes a new attempt for each task:

```
attempts.push(TaskAttemptDescriptor {
    state: TaskState::Created,
    messages: vec![],
    cause: None,
    job_output_fetched: false,
    created_at: Utc::now(),
    stopped_at: None,
});
```

Task states are:

- Created
- Scheduled

- Running
- Succeeded
- Failed
- Canceled

The driver receives status updates from workers as `DriverEvent::UpdateTask`. Those updates include an optional sequence number. The driver ignores stale updates:

```
if sequence <= *s {
    warn!("{ } sequence {sequence} is stale", TaskKeyDisplay(&key));
    return ActorAction::Continue;
}
```

This protects the control plane from delayed or duplicate worker status messages.

9.9 Task Regions And Cascading Cancellation

Task regions are important because Sail schedules and retries them as units. If any task in a region fails, the scheduler cancels other active attempts in that region:

```
if failed {
    for t in &region.tasks {
        for (a, attempt) in task.attempts.iter_mut().enumerate() {
            if !attempt.state.is_terminal() {
                attempt.state = TaskState::Canceled;
                actions.push(JobAction::CancelTask { key });
            }
        }
    }
}
```

Why cancel the whole region?

Because a region represents a group of tasks that are pipelined or otherwise scheduled together. If one attempt fails, its peers may be producing or consuming streams that are no longer valid for that attempt set. Canceling the region keeps attempt boundaries consistent.

This is the distributed version of “do not mix outputs from different attempts unless the system has explicitly decided to do so.”

9.10 Task Assignment

The scheduler emits `JobAction::ScheduleTaskRegion`. The driver gives the region to `TaskAssigner`:

```
self.task_assigner.enqueue_tasks(region);
```

Then `run_tasks` asks for assignments:

```
let assignments = self.task_assigner.assign_tasks();
self.task_assigner.track_streams(&assignments);
```

`TaskAssigner` tracks:

- active workers,
- worker task slots,
- driver task slots,
- queued task regions,
- task assignments,
- local stream ownership,
- remote stream ownership.

Worker task slots are limited:

```
task_slots: Vec![TaskSlot::default(); self.options.worker_task_slots]
```

Driver task slots can grow:

```
/// The number of task slot can grow indefinitely.
task_slots: Vec<TaskSlot>
```

Assignment is region-aware. TaskSlotAssigner::try_assign_task_region only succeeds if the entire region can be assigned:

```
for (placement, set) in &region.tasks {
    match placement {
        TaskPlacement::Driver => ...
        TaskPlacement::Worker => {
            if let Some((worker_id, slot)) = self.next() {
                ...
            } else {
                return Err(region);
            }
        }
    }
}
```

If a region cannot fit, it goes back to the front of the queue. This can cause head-of-line blocking, but it preserves scheduling order.

9.11 Scaling Workers

The task assigner also tells the driver how many workers to request:

```
let required_slots = enqueued_slots.saturating_sub(vacant_slots);
let required_workers = required_slots
    .div_ceil(self.options.worker_task_slots)
    .min(allowed_workers);
```

The driver then starts that many workers:

```
for _ in 0..self.task_assigner.request_workers() {
    self.worker_pool.start_worker(ctx);
}
```

This is simple elastic scheduling:

- pending worker tasks imply required slots,
- active idle worker slots satisfy some of that demand,
- remaining demand becomes new workers,
- worker_max_count caps the result if configured.

9.12 Building A Task Definition

After assignment, the driver asks the scheduler for each task definition:

```
let (definition, context) =
    self.job_scheduler.get_task_definition(&entry.key, &self.task_assigner)?;
```

A TaskDefinition contains:

```
pub struct TaskDefinition {
    pub plan: Arc<[u8]>,
    pub inputs: Vec<TaskInput>,
    pub output: TaskOutput,
}
```

The plan is serialized with DataFusion’s physical plan protobuf support and Sail’s extension codec:

```
let plan =
    PhysicalPlanNode::try_from_physical_plan(stage.plan.clone(),
self.codec.as_ref())?
    .encode_to_vec();
```

Inputs come from stage.inputs, using InputMode and current task assignments to decide locations.

For pipelined worker outputs, input keys become:

```
TaskInputLocator::Worker {
    stage: input.stage,
    keys,
}
```

Each key includes:

- upstream partition,
- upstream attempt,
- channel.

The task output includes:

- distribution,
- local or remote locator,
- replica count for local pipelined output.

This object is the portable description of one task attempt.

9.13 Dispatching Tasks

Once the driver has a TaskDefinition, it dispatches by placement:

```
match assignment.assignment {
  TaskAssignment::Driver => self.task_runner.run_task(ctx, entry.key,
definition, context),
  TaskAssignment::Worker { worker_id, slot: _ } => {
    self.worker_pool.run_task(ctx, worker_id, entry.key, definition)
  }
}
```

For worker tasks, `WorkerPool::run_task`:

1. Finds or creates a worker client.
2. Tracks worker activity.
3. Sends the task definition over gRPC.
4. Includes peer worker locations the worker may need for stream fetches.
5. Reports task failure back to the driver if dispatch fails.

The peer list is optimized by remembering known peers:

```
let peers = running_workers
  .into_iter()
  .filter(|x| !worker.peers.contains(&x.worker_id))
  .collect();
```

Workers report back which peers they now know, so the driver avoids sending the same location information repeatedly.

9.14 Running A Task On A Worker

The worker receives `WorkerEvent::RunTask`, tracks peer locations, and calls `TaskRunner::run_task`.

`TaskRunner::execute_plan` performs the critical preparation:

```
let plan = PhysicalPlanNode::decode(definition.plan.as_ref())?;
let plan = plan.try_into_physical_plan(&context, self.codec.as_ref())?;
let plan = self.rewrite_parquet_adapters(plan)?;
let plan = self.rewrite_shuffle(ctx, key, &definition.inputs, &definition.output,
plan, &context)?;
let stream = plan.execute(key.partition, context)?;
```

There are two important rewrites:

1. `rewrite_parquet_adapters` adjusts Parquet scans for Delta expression adapters.
2. `rewrite_shuffle` turns stage input placeholders into reads, and wraps the task output in writes.

Then `DataFusion` executes the task partition.

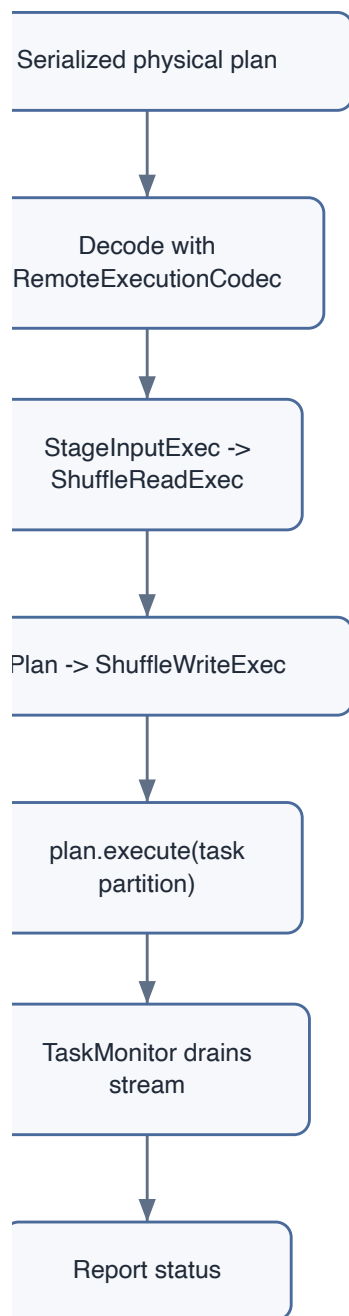


Figure 43: Flowchart 08.4

9.15 Why The Task Monitor Drains The Stream

`TaskRunner::run_task` does not simply call `execute` and report success. It spawns a `TaskMonitor`.

The monitor first reports `Running`:

```
T::Message::report_task_status(key, TaskStatus::Running, None, None)
```

Then it races execution against cancellation:

```
tokio::select! {  
    x = Self::execute(key.clone(), stream) => x,  
    x = Self::cancel(key.clone(), signal) => x,  
}
```

`execute` drains the stream:

```
while let Some(batch) = stream.next().await {  
    if let Err(e) = batch {  
        return Failed;  
    }  
}  
return Succeeded;
```

This matters because in `DataFusion`, executing a plan returns a stream. Work may not happen until the stream is polled. If `Sail` reported success immediately after obtaining the stream, it would be lying. Draining the stream ensures the task really ran and all shuffle writes were closed.

9.16 Stream Accessor: Actors As Readers And Writers

`StreamAccessor` bridges physical operators and actor messages.

It implements `TaskStreamReader`:

```
async fn open(&self, location: &TaskReadLocation, schema: SchemaRef)  
    -> Result<TaskStreamSource>
```

For each read location, it sends an actor event:

```
TaskReadLocation::Driver { key } =>  
    fetch_driver_stream(key, schema, tx)  
TaskReadLocation::Worker { worker_id, key } =>  
    fetch_worker_stream(worker_id, key, schema, tx)  
TaskReadLocation::Remote { uri, key } =>  
    fetch_remote_stream(uri, key, schema, tx)
```

It also implements `TaskStreamWriter`:

```
TaskWriteLocation::Local { key, storage } =>  
    create_local_stream(key, storage, schema, tx)  
TaskWriteLocation::Remote { uri, key } =>  
    create_remote_stream(uri, key, schema, tx)
```


This is how ShuffleReadExec and ShuffleWriteExec remain actor-agnostic. They only know about TaskStreamReader and TaskStreamWriter. The actual driver/worker communication is hidden behind StreamAccessor.

9.17 Stream Locations

Read locations are:

```
pub enum TaskReadLocation {  
    Driver { key },  
    Worker { worker_id, key },  
    Remote { uri, key },  
}
```

Write locations are:

```
pub enum TaskWriteLocation {  
    Local { storage, key },  
    Remote { uri, key },  
}
```

A TaskStreamKey identifies one stream:

job_id, stage, partition, attempt, channel

That key is the identity that ties together:

- task output,
- stream ownership,
- stream fetches,
- job output,
- cleanup,
- retry attempts.

The inclusion of attempt is especially important. If a task is retried, the new attempt writes a different stream key. Consumers can avoid accidentally mixing data from failed and replacement attempts.

9.18 Stream Manager

Both driver and worker have a StreamManager.

The stream manager owns local streams:

```
local_streams: HashMap<TaskStreamKey, LocalStreamState>
```

A local stream can be:

- pending,
- created,
- failed.

The pending state matters because a consumer may ask for a stream before the producer has created it. In that case, fetch_local_stream creates a receiver and stores its sender:

```
entry.insert(LocalStreamState::Pending { senders: vec![tx] });
ctx.send_with_delay(
    T::Message::probe_pending_local_stream(key.clone()),
    self.options.task_stream_creation_timeout,
);
```

When the producer later creates the stream, the pending senders are connected to the new stream.

If stream creation never happens, the delayed probe fails the pending stream:

```
let message = "local stream is not created within the expected time".to_string();
let cause = CommonErrorCause::Execution(message);
Self::fail_senders(senders, &cause);
*value = LocalStreamState::Failed { cause };
```

This is how Sail prevents downstream tasks from waiting forever for a missing upstream stream.

9.19 Memory Streams And Replicas

The current local stream implementation is `MemoryStream`.

Its comment explains the design:

```
/// A memory stream that can be read multiple times.
/// It maintains multiple replicas of the stream internally.
/// Since [Arc] is used inside the record batch, it is relatively cheap
/// to clone the data in multiple replicas.
```

A memory stream has one publisher and multiple receivers:

```
sender: Option<MemoryStreamReplicaSender>,
receivers: Vec<mpsc::Receiver<TaskStreamResult<RecordBatch>>>>,
```

When a batch is written, `MemoryStreamReplicaSender` tries to send it to every active replica. If a receiver is full, it uses an overflow buffer. If a receiver is closed, it drops that replica:

```
Err(mpsc::error::TrySendError::Closed(_)) => {
    dropped = true;
}
```

A closed receiver is not necessarily an error. A downstream LIMIT may stop reading early. The sink returns `Closed` only when all replicas are gone.

This replica design supports `JobGraph::replicas(stage)`: stages consumed by merge or broadcast may need multiple readers for the same output stream.

9.20 Arrow Flight For Task Streams

When a task needs a stream from another process, Sail uses Arrow Flight.

The server is `TaskStreamFlightServer` in `crates/sail-execution/src/stream_service/server.rs`. Its important method is `do_get`:

1. Decode a `TaskStreamTicket`.
2. Convert it to `TaskStreamKey`.

3. Ask a TaskStreamFetcher for the stream.
4. Encode the stream with FlightDataEncoderBuilder.
5. Return Flight data.

```
let stream = rx.await?;
let stream = stream.map_err(|e| FlightError::Tonic(Box::new(e.into())));
let stream = FlightDataEncoderBuilder::new()
    .build(stream)
    .map_err(Status::from);
```

The client is TaskStreamFlightClient:

```
let response = self.inner.get().await?.do_get(request).await?;
let stream = response.into_inner().map_err(|e| e.into());
let stream = FlightRecordBatchStream::new_from_flight_data(stream)?;
```

Again, the data plane is Arrow batches. The control plane moves task keys and locations; Flight moves the batch stream.

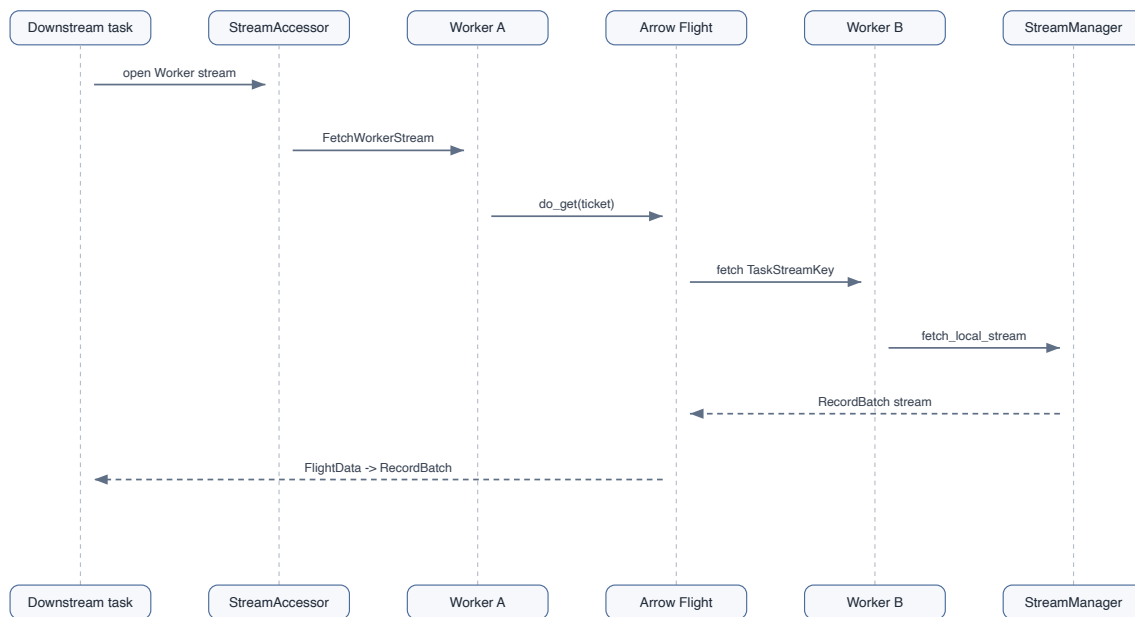


Figure 44: Sequence diagram 08.5

9.21 Peer Tracking

Workers may need to fetch streams from other workers. The driver sends peer locations along with task dispatch. The worker tracks them in PeerTracker:

```
for peer in peers {
    self.peers
        .entry(peer.worker_id)
        .or_insert_with(|| Peer::new(peer.host, peer.port));
}
```

```
}
ctx.send(WorkerEvent::ReportKnownPeers { peer_worker_ids });
```

The worker reports known peers back to the driver. The driver stores that set in the worker descriptor:

```
worker.peers.extend(peer_worker_ids);
```

The next time the driver dispatches a task to that worker, it omits peers the worker already knows.

This is an optimization, not a correctness requirement. The worker descriptor comment says the peer list may not cover all running workers, but correctness does not depend on completeness.

9.22 Cleanup And Stream Tracking

The task assigner tracks local streams because local stream ownership affects worker lifetime.

Worker resources include:

```
local_streams: IndexSet<TaskKey>
```

The comment calls this “shuffle tracking” similar to Spark. A worker may be idle from a task-slot perspective but still own active local streams needed by downstream tasks. Sail should not stop that worker until its local streams are no longer needed.

When consumers finish, the scheduler emits cleanup actions:

```
JobAction::CleanUpJob { job_id, stage: Some(s) }
```

The driver handles cleanup by untracking stream ownership and asking the relevant driver/worker stream managers to remove streams:

```
for x in self.task_assigner.untrack_local_streams(job_id, stage) {
  match x {
    TaskStreamAssignment::Driver => {
      self.stream_manager.remove_local_streams(job_id, stage);
    }
    TaskStreamAssignment::Worker { worker_id } => {
      self.worker_pool.clean_up_job(ctx, worker_id, job_id, stage)
    }
  }
}
```

This is the other half of shuffle tracking:

- keep workers alive while streams are needed,
- clean streams up when consumers have succeeded,
- then workers can become idle and eligible for stopping.

9.23 Job Output

The job output path begins when final-stage tasks are running or succeeded.

`extend_job_output` finds final stages and adds their task streams:

```
actions.push(JobAction::ExtendJobOutput {
  handle: output.handle(),
  key,
  schema: schema.clone(),
});
```

The driver resolves the stream location from task assignment:

```
Some(TaskAssignment::Driver) =>
  self.stream_manager.fetch_local_stream(ctx, &key)
Some(TaskAssignment::Worker { worker_id, .. }) =>
  self.worker_pool.fetch_task_stream(ctx, *worker_id, &key, schema.clone())
```

Then it sends the stream to the JobOutputHandle.

JobOutputStream merges all added streams using SelectAll. It stays active while new streams may arrive, then drains remaining streams once the output manager is dropped.

This is how a distributed job becomes one SendableRecordBatchStream for the caller.

9.24 One Query Lifecycle

Here is the complete lifecycle in one diagram:

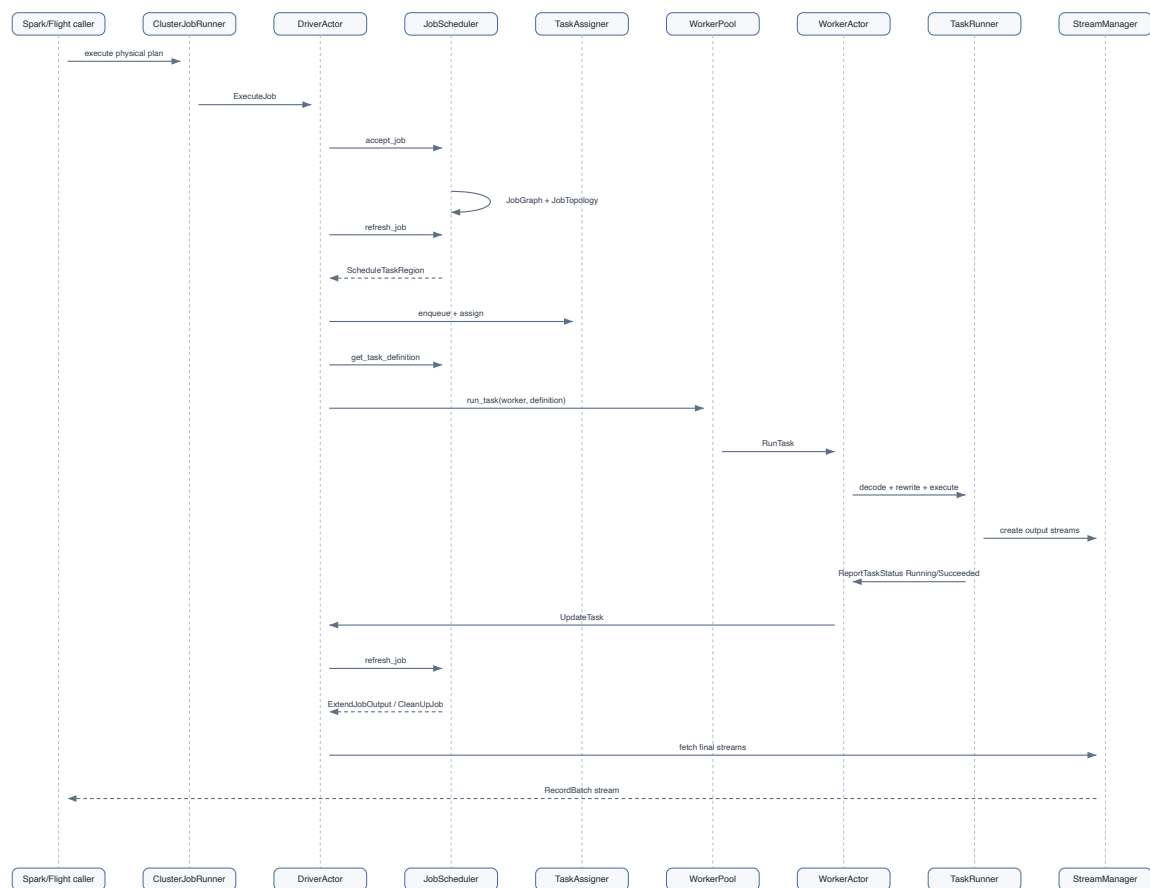


Figure 45: Sequence diagram 08.6

It is a lot of machinery, but each piece has a bounded job.

9.25 Failure And Retry Story

Sail's retry model is attempt-based:

- A task attempt fails if its monitor sees a stream error.
- A task attempt can be canceled explicitly.
- A lost worker causes all assigned task attempts to fail.
- A failed attempt causes the whole task region to cancel active peers.
- A region can be rescheduled by creating new attempts.
- If attempts exceed the configured max, the region and job fail.

The job output stream standardizes data-plane and control-plane errors through `CommonErrorCause`, so the client sees coherent failures whether the error comes from:

- a task stream,
- a task status update,
- job output failure,
- or cleanup/shutdown.

The architecture is intentionally conservative. It avoids mixing task attempts and treats region failure as a reason to restart the region.

9.26 Why This Design Fits Rust

This part of Sail shows several Rust strengths:

- actor-owned mutable state avoids large shared locks,
- Arc shares immutable plans, schemas, and clients,
- trait objects abstract workers, streams, actors, and job runners,
- async tasks isolate slow IO from actor message handling,
- enums make control-plane states explicit,
- typed IDs prevent accidental confusion between jobs, tasks, streams, and workers,
- oneshot channels turn actor messages into request/response APIs.

The design is not “just async Rust.” It is a careful layering:

Actor messages -> scheduler state -> task definitions -> physical plan execution
-> stream IO

Each layer is explicit enough to inspect and test.

9.27 Extension Implications

For the final extension chapter, this control plane raises several requirements.

A distributed-safe extension must consider:

- Can its physical plan be serialized into a TaskDefinition?
- Does RemoteExecutionCodec know how to decode it on workers?
- Does it require worker-local state?
- Does it require driver-only coordination?
- Does it produce task streams that can be retried safely?
- Does it rely on external resources that must be available in worker pods?
- Does it need peer-to-peer stream access?
- Does it need cleanup hooks when a job or stage finishes?
- Does it need custom task placement?

This is where a simple plugin API becomes a distributed systems API. A scalar UDF that uses Arrow arrays is easy. A custom physical operator with new stream semantics is much more serious.

Sail’s existing control plane gives us the vocabulary to design those capabilities precisely.

9.28 Reading Exercise: Follow A Task To A Worker

Trace a task from scheduling to worker execution:

1. Open `crates/sail-execution/src/driver/actor/handler.rs`.
2. Find `run_tasks`.
3. Follow `task_assigner.assign_tasks`.
4. Follow `job_scheduler.get_task_definition`.

5. Follow `worker_pool.run_task`.
6. Open `crates/sail-execution/src/worker/actor/handler.rs`.
7. Find `handle_run_task`.
8. Follow `task_runner.run_task`.
9. Open `crates/sail-execution/src/task_runner/core.rs`.
10. Read `execute_plan`.

At the end, you should be able to say how a stage partition becomes `plan.execute(key.partition, context)`.

9.29 Reading Exercise: Follow A Stream Fetch

Trace a downstream task reading an upstream worker stream:

1. Start in `TaskRunner::rewrite_shuffle`.
2. Find where `StageInputExec<usize>` becomes `ShuffleReadExec`.
3. Follow `StreamAccessor::new(handle.clone())`.
4. Open `crates/sail-execution/src/stream_accessor/core.rs`.
5. Read `TaskStreamReader::open`.
6. Follow `fetch_worker_stream`.
7. On the worker, open `worker/actor/handler.rs`.
8. Find `handle_fetch_worker_stream`.
9. If the stream is remote, follow `TaskStreamFlightClient`.
10. If the stream is local, follow `StreamManager::fetch_local_stream`.

This trace connects the control-plane location lookup to the Arrow Flight data plane.

9.30 Reading Exercise: Follow Worker Loss

Trace worker failure handling:

1. Open `crates/sail-execution/src/driver/actor/handler.rs`.
2. Find `handle_probe_lost_worker`.
3. Follow `worker_pool.stop_worker`.
4. Follow `task_assigner.find_worker_tasks`.
5. Follow `job_scheduler.update_task(... Failed ...)`.
6. Follow `refresh_job`.
7. Find `cascade_cancel_task_attempts`.
8. Find `schedule_task_regions`.

This shows how infrastructure failure becomes task attempt retry.

9.31 Takeaways

Sail's distributed runtime is actor-driven. The driver actor coordinates jobs, workers, task assignment, stream ownership, and cleanup. Worker actors register with the driver, heartbeat, run serialized task definitions, serve local streams, and report task status.

The task runner turns a serialized DataFusion physical plan back into an executable plan, rewrites stage inputs into `ShuffleReadExec`, wraps outputs in `ShuffleWriteExec`, and drains the resulting stream through a task monitor.

Streams are identified by (job, stage, partition, attempt, channel). Stream managers handle pending, created, failed, replicated, and cleaned-up local streams. Arrow Flight carries streams between processes.

The next chapter zooms in on shuffle and data movement, using the stream and task machinery from this chapter as the foundation.

10 Chapter 9: Shuffle And Data Movement

Shuffle is where a distributed query engine proves that it is actually distributed.

Up to this point, the book has followed Sail from the front door through logical plans, DataFusion physical plans, stage graphs, drivers, workers, and task execution. This chapter zooms in on the data plane: how a `RecordBatch` produced by one task becomes input to another task, possibly on another worker, under a distribution chosen by the query plan.

In Sail, this movement is expressed in a compact set of ideas:

- Data moves as Arrow `RecordBatch` streams.
- A stage boundary is represented in the physical plan by `StageInputExec`.
- At task runtime, `StageInputExec` is rewritten into `ShuffleReadExec`.
- The task's final physical plan is wrapped in `ShuffleWriteExec`.
- The driver assigns stream keys, channel numbers, and read/write locations.
- The stream subsystem moves batches through local memory today, with Arrow Flight as the remote transport shape.

That last phrase is important: Sail already has the architecture of a networked shuffle service, but some remote and blocking pieces are intentionally not finished. This makes the codebase unusually good for learning. You can see the shape of a distributed engine without getting lost in years of accumulated production machinery.

10.1 Code Map

The core shuffle code lives in these files:

Concern	File
Physical shuffle writer	<code>crates/sail-execution/src/ plan/shuffle_write.rs</code>
Physical shuffle reader	<code>crates/sail-execution/src/ plan/shuffle_read.rs</code>
Merging task streams	<code>crates/sail-execution/src/stream/ merge.rs</code>
Round-robin partitioning	<code>crates/sail-physical-plan/ src/repartition.rs</code>
Task input/output definitions	<code>crates/sail-execution/src/ task/definition.rs</code>
Scheduler input/output placement	<code>crates/sail-execution/src/driver/ job_scheduler/core.rs</code>
Runtime shuffle rewrite	<code>crates/sail-execution/src/task_runner/ core.rs</code>
Stream reader and writer traits	<code>crates/sail-execution/src/stream/ reader.rs, crates/sail-execution/src/ stream/writer.rs</code>
Actor bridge to stream manager	<code>crates/sail-execution/src/ stream_accessor/core.rs</code>
Local stream manager	<code>crates/sail-execution/src/ stream_manager/core.rs</code>
In-memory stream replicas	<code>crates/sail-execution/src/ stream_manager/local.rs</code>
Arrow Flight stream service	<code>crates/sail-execution/src/ stream_service/server.rs, crates/sail- execution/src/stream_service/client.rs</code>

If Chapter 8 was about who runs the work, this chapter is about how the work's bytes find the next consumer.

10.2 The Vocabulary Of Movement

Sail's shuffle layer uses a small vocabulary. Once these terms are clear, the rest of the code becomes much easier to read.

Term	Meaning
Stage	A group of physical operators that can run without needing data from a later exchange.
Task	One partition of a stage, executed as one attempt.
Partition	A task-level unit of parallelism within a stage.
Channel	A logical output lane from a producer stage to a consumer stage.
Attempt	A retry number for a task. Attempts are part of stream keys.
Task input	A set of stream locations that a task should read.
Task output	A distribution and locator that describe where a task should write.
Stream key	The stable identity of a task stream: job, stage, partition, attempt, channel.
Location	Where the stream lives: driver, worker, or remote URI for reads; local or remote for writes.

The central identity is TaskStreamKey. Conceptually, it looks like this:

```
TaskStreamKey {
    job_id,
    stage,
    partition,
    attempt,
    channel,
}
```

The channel field is the part that turns one task output into many possible inputs. If a producer stage has four upstream partitions and eight shuffle channels, each producer task may write eight streams. A downstream task then reads the channel or channels assigned to it from all relevant producer partitions.

That gives Sail the basic distributed exchange shape:

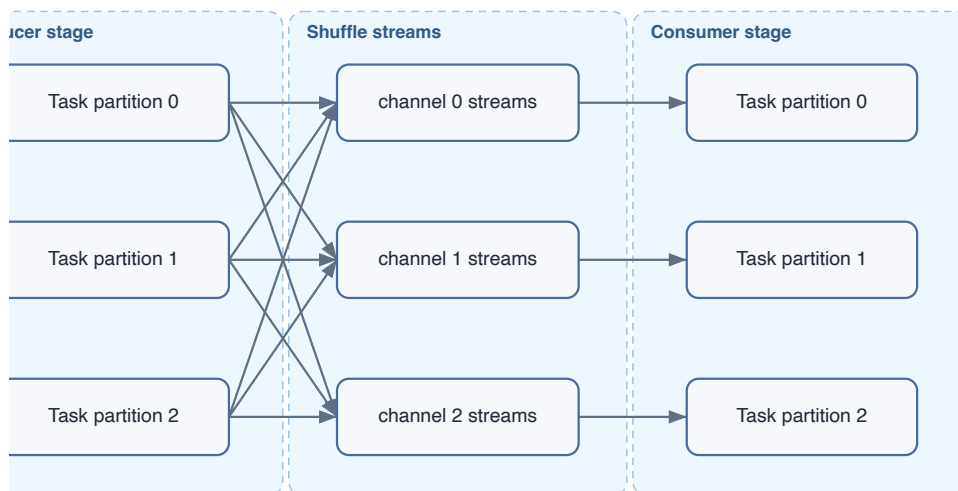


Figure 46: Flowchart 09.1

For a hash shuffle, “channel 1” means “rows whose hash maps to bucket 1.” For a round-robin shuffle, it means “the next row assigned to lane 1.” For a broadcast-like movement, the scheduler may arrange the input keys so multiple consumers can read the same producer output.

10.3 From Job Graph To Task Definition

The job graph knows that one stage depends on another. It does not directly contain open streams. Before a worker can execute a task, the driver must turn graph edges into concrete task inputs and outputs.

That happens in `JobScheduler::get_task_input()` and `JobScheduler::get_task_output()` in `crates/sail-execution/src/driver/job_scheduler/core.rs`.

For task inputs, the scheduler:

1. Finds the producer stage for the input.
2. Determines the producer partition count and channel count.
3. Computes the `TaskInputKey` values that this consumer task should read.
4. Finds the latest successful or assigned attempt for each producer partition.
5. Turns those keys into a `TaskInputLocator`.

The input locator records where the consumer should fetch streams from:

```
TaskInputLocator::Driver { keys }
TaskInputLocator::Worker { worker_id, keys }
TaskInputLocator::Remote { uri, keys }
```

For task outputs, the scheduler:

1. Reads the output distribution from the job graph.
2. Serializes hash expressions if the output is hash-partitioned.
3. Chooses the output locator.
4. Returns a `TaskOutput`.

Today, pipelined output is local:

```
TaskOutputLocator::Local { replicas }
```

The remote and blocking-output branches are present as design points, but blocking output placement still has `todo!()` markers. That is one of the places where the extensions proposal can hook into the architecture later.

The result is not an open socket or a live stream. It is a serializable task definition: inputs, outputs, partition numbers, attempts, and encoded expressions. That definition can be sent to a worker.

10.4 The Runtime Rewrite

The most important shuffle transition happens inside `TaskRunner::rewrite_shuffle()` in `crates/sail-execution/src/task_runner/core.rs`.

The stage-level physical plan contains placeholders:

```
StageInputExec<usize>
```

Those placeholders are not executable by themselves. They say, “this is where this stage reads from an upstream stage.” When the task runner receives the concrete `TaskInput` values from the driver, it rewrites each placeholder into a real reader:

```
StageInputExec<usize> -> ShuffleReadExec
```

Then the task runner wraps the whole plan with a writer:

```
plan -> ShuffleWriteExec(plan)
```

The shape is:

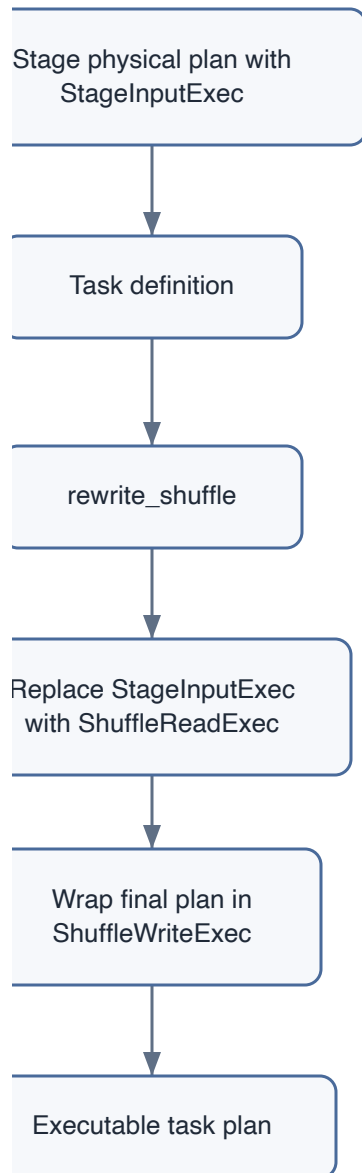


Figure 47: Flowchart 09.2

That rewrite is the bridge between planning and execution:

- Planning says which stages depend on which other stages.
- Scheduling says where the task’s input and output streams live.
- Runtime rewriting turns those locations into physical execution nodes.

This is a powerful Rust pattern in miniature. The stage plan is generic and reusable. The task plan is concrete and contextual. Sail uses a tree transform to keep those concerns separated until the last responsible moment.

10.5 Writing Shuffle Output

ShuffleWriteExec is a DataFusion ExecutionPlan implementation, but it behaves a little differently from ordinary relational operators. It does not produce meaningful rows downstream. Its job is to consume its child plan, partition the child batches, and write the resulting batches into task streams.

Its main fields are:

```
pub struct ShuffleWriteExec {
    plan: Arc<dyn ExecutionPlan>,
    shuffle_partitioning: Partitioning,
    locations: Vec<Vec<TaskWriteLocation>>,
    properties: Arc<PlanProperties>,
    writer: Arc<dyn TaskStreamWriter>,
}
```

Read those fields as a sentence:

“Run this child plan, partition its output according to shuffle_partitioning, and write this task partition’s output to the given locations using a TaskStreamWriter.”

The locations field is a two-dimensional vector:

```
locations[input_partition][channel]
```

During rewrite_shuffle(), Sail creates a vector with one outer entry per output partition and fills only the current task partition:

```
locations[key.partition].extend(output.locations(key))
```

That means ShuffleWriteExec::execute(partition, context) can look up the exact write locations for the partition DataFusion is asking it to execute.

10.5.1 Partitioning

Sail supports two writer-side partitioning modes here:

```
Partitioning::Hash(keys, channels)
Partitioning::RoundRobinBatch(channels)
```

There is also UnknownPartitioning, which Sail treats like round-robin for write purposes.

Hash partitioning uses DataFusion’s BatchPartitioner. This is the natural choice because DataFusion already knows how to evaluate physical expressions against Arrow batches and assign rows to hash buckets.

Round-robin partitioning uses Sail's own `RowRoundRobinPartitioner` in `crates/sail-physical-plan/src/repartition.rs`. It is intentionally Arrow-native:

1. Build row-index arrays for each destination partition.
2. Use Arrow's `take_arrays()` compute kernel to select the rows for each partition.
3. Construct new `RecordBatch` values with the same schema.

That avoids converting rows into Rust structs or ad hoc values. The shuffle layer stays columnar.

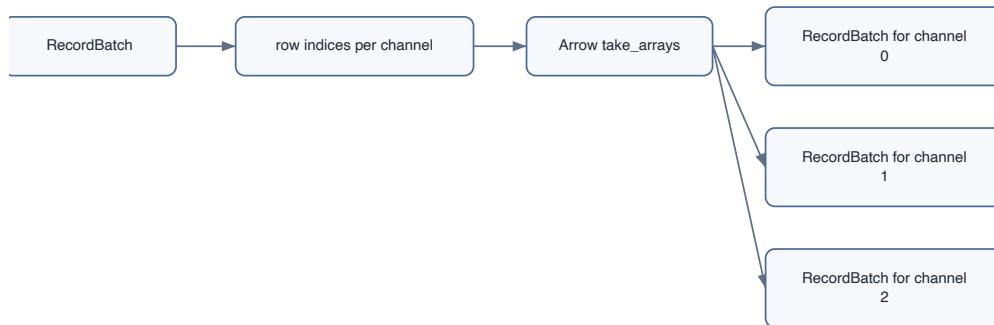


Figure 48: Flowchart 09.3

The start index for round-robin is derived from the input partition:

```
start = (input_partition * num_partitions) / num_input_partitions
```

That small detail helps distribute initial rows across output channels when multiple input partitions are writing at once.

10.5.2 Sinks And Side Effects

The heart of shuffle writing is the `shuffle_write()` helper:

1. Open one sink per write location.
2. Pull batches from the child plan stream.
3. Partition each batch into per-channel batches.
4. Write each per-channel batch to its sink.
5. Close remaining sinks when input is exhausted.

A simplified sketch:

```

let mut sinks = locations
    .into_iter()
    .map(|location| writer.open(location, schema.clone()))
    .collect::<FuturesOrdered<_>>();

while let Some(batch) = stream.next().await.transpose()? {
    let partitions = partitioner.partition(&batch)?;
  
```



```

        for (sink, maybe_batch) in sinks.iter_mut().zip(partitions) {
            if let Some(batch) = maybe_batch {
                sink.write(batch).await?;
            }
        }
    }

    for sink in sinks {
        sink.close().await?;
    }
}

```

The actual code tracks sink state:

```

TaskStreamSinkState::Ok
TaskStreamSinkState::Error
TaskStreamSinkState::Closed

```

This matters because a downstream consumer may stop early. A LIMIT query is the classic example: once the driver has enough rows, some readers may close. Sail treats closed sinks differently from failed sinks so that early termination does not necessarily become a query failure.

`ShuffleWriteExec::execute()` returns a stream, because DataFusion expects every physical operator to return a `SendableRecordBatchStream`. But the useful work happens as a side effect: writing to task streams. After writing, the operator emits an empty `RecordBatch`.

That makes `ShuffleWriteExec` a boundary operator. It turns a normal DataFusion stream into Sail task output.

10.6 Reading Shuffle Input

`ShuffleReadExec` is the mirror image. Its fields are:

```

pub struct ShuffleReadExec {
    locations: Vec<Vec<TaskReadLocation>>,
    properties: Arc<PlanProperties>,
    reader: Arc<dyn TaskStreamReader>,
}

```

Again, read the fields as a sentence:

“For this output partition, open these task stream locations using this reader, then merge the resulting Arrow streams.”

`execute(partition, context):`

1. Looks up `locations[partition]`.
2. Opens each location with `reader.open(location, schema.clone())`.
3. Merges all opened streams into one `RecordBatchStream`.

The merge is handled by `MergedRecordBatchStream` in `crates/sail-execution/src/stream/merge.rs`. Internally, it uses a `SelectAll` over the task streams. That means batches are yielded as upstream streams become ready, not by fully draining one producer before reading the next.

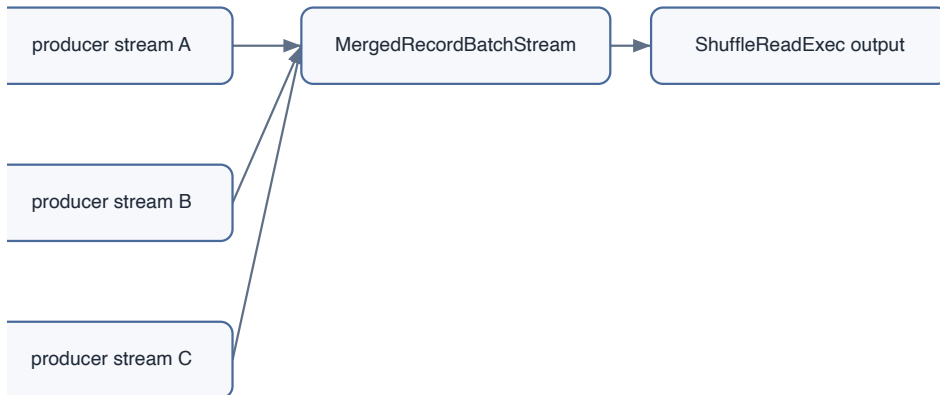


Figure 49: Flowchart 09.4

This is why a consumer task can start processing a pipelined shuffle before every producer has finished, as long as its input streams are available.

10.7 Locations Become Streams

ShuffleReadExec and ShuffleWriteExec do not know whether a stream is in process, on another worker, or behind an Arrow Flight endpoint. They depend on two traits:

```

pub trait TaskStreamReader {
  async fn open(
    &self,
    location: TaskReadLocation,
    schema: SchemaRef,
  ) -> Result<TaskStreamSource>;
}

pub trait TaskStreamWriter {
  async fn open(
    &self,
    location: TaskWriteLocation,
    schema: SchemaRef,
  ) -> Result<Box<dyn TaskStreamSink>>;
}

```

The concrete implementation used by tasks is StreamAccessor. It sends actor messages to the stream manager:

```

ShuffleReadExec
  -> TaskStreamReader::open

```

```
-> StreamAccessor  
-> actor message  
-> StreamManager
```

```
ShuffleWriteExec  
-> TaskStreamWriter::open  
-> StreamAccessor  
-> actor message  
-> StreamManager
```

This is a nice example of Rust interface design in Sail:

- The execution plans depend on small async traits.
- The actor system stays outside the DataFusion operator implementation.
- Local and remote stream mechanisms can evolve behind the accessor boundary.

10.8 Local Memory Streams

The local stream manager has three visible states for local streams:

```
Pending  
Created  
Failed
```

This solves a real scheduling race. A consumer may ask for a stream before the producer has created it. Rather than fail immediately, the manager can register that the stream is pending and wake the reader when the producer creates it. If creation never happens, the pending stream eventually times out.

For in-memory streams, Sail uses replicas. A local output location includes:

```
LocalStreamStorage::Memory { replicas }
```

The producer writes each batch to the active replica senders. This supports multiple readers for the same produced stream, which is useful for broadcast-like movement and for cases where more than one consumer needs the same task output.

The memory stream implementation also handles closed receivers. If a receiver is closed, the producer can keep writing to remaining active replicas. Once no active replicas remain, the sink can report `Closed`.

That behavior is one of the quiet but important pieces of distributed query execution: the data plane must distinguish “nobody needs this anymore” from “the query is broken.”

10.9 Arrow Flight As The Remote Shape

Local streams are the implemented fast path, but Sail’s stream service shows the remote transport shape: Arrow Flight.

On the server side, `do_get`:

1. Decodes a `TaskStreamTicket`.
2. Fetches the requested task stream.

3. Encodes RecordBatch values as Flight data.
4. Returns a Flight stream.

On the client side, `TaskStreamFlightClient::fetch_task_stream()`:

1. Builds a ticket for the requested task stream.
2. Calls Flight `do_get`.
3. Wraps the returned Flight data as a RecordBatch stream.

The important architecture point is that Flight does not replace Arrow batches. It transports them. Sail's task operators still speak in RecordBatch streams on both sides of the network boundary.

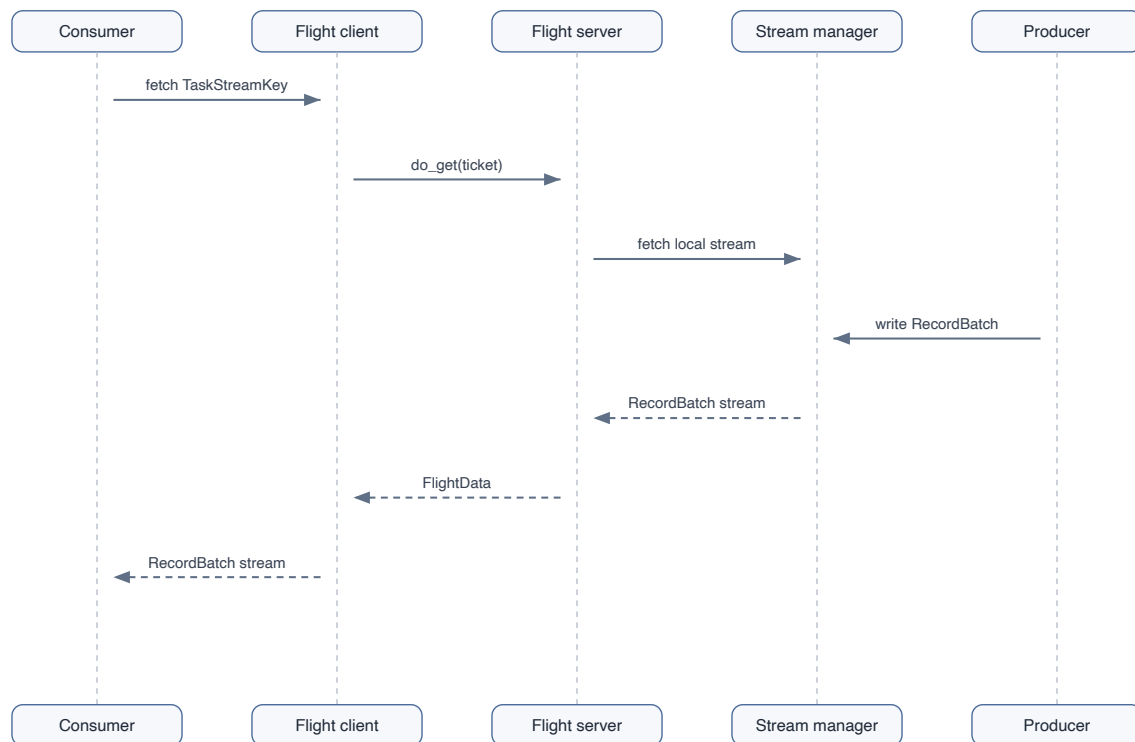


Figure 50: Sequence diagram 09.5

This is the main reason Arrow Flight fits a system like Sail so well. It lets the engine preserve its columnar execution model while crossing process and machine boundaries.

10.10 A Hash Shuffle Walkthrough

Imagine a query like:

```

SELECT customer_id, COUNT(*)
FROM orders
GROUP BY customer_id
  
```

At scale, each worker can scan a subset of orders, but final groups must be brought together by `customer_id`. Rows with the same `customer_id` need to land in the same downstream partition.

The high-level plan is:

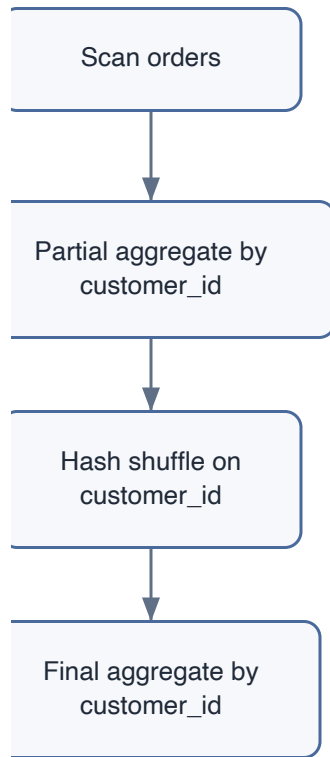


Figure 51: Flowchart 09.6

In Sail terms:

1. The planner inserts a stage boundary at the exchange.
2. The producer stage output distribution is Hash.
3. The scheduler serializes the hash expression into `TaskOutputDistribution::Hash`.
4. The task runner decodes that expression back into DataFusion physical expressions.
5. `ShuffleWriteExec` uses DataFusion's `BatchPartitioner`.
6. Each producer task writes one stream per hash channel.
7. Each consumer task opens the producer streams for its assigned channel.
8. `ShuffleReadExec` merges those streams.
9. The final aggregate sees a normal input stream of Arrow batches.

The row movement looks like this:

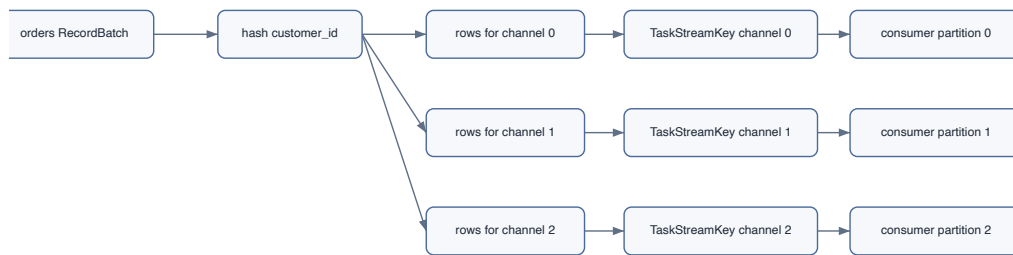


Figure 52: Flowchart 09.7

Notice what does not happen:

- Sail does not serialize rows into a custom row format at the shuffle boundary.
- Sail does not make `ShuffleReadExec` understand hash expressions.
- Sail does not make the scheduler evaluate data.

Each layer keeps a narrow job.

10.11 Other Movement Patterns

Hash shuffle is the easiest to visualize, but Sail's input-key construction can represent several movement patterns.

10.11.1 One-To-One

A downstream partition reads the corresponding upstream partition. This is the cheapest movement pattern and is useful when partitioning is already compatible.

```
producer partition 0 -> consumer partition 0
producer partition 1 -> consumer partition 1
```

10.11.2 Hash

Every producer may write every channel. Each consumer reads the channel or channels assigned to it.

```
producer partition N, channel C -> consumer partition C
```

10.11.3 Round Robin

Rows are spread across output channels without using data values as keys. Sail's round-robin partitioner uses Arrow take kernels to construct the destination batches.

10.11.4 Broadcast

The same upstream output is made readable by multiple downstream consumers. Local memory stream replicas are the stream-level mechanism that makes this possible.

10.11.5 Merge

Many upstream streams are merged into one downstream stream. `MergedRecordBatchStream` is the simple core abstraction here.

10.11.6 Rescale

Producer and consumer partition counts may differ. The scheduler's input-key builder can assign groups of producer streams to consumer partitions.

The exact key-building logic belongs to the scheduler, not the stream operators. This is another example of Sail's separation between control plane and data plane.

10.12 Failure, Attempts, And Early Termination

Distributed data movement needs identity. Without identity, retries are dangerous: a consumer might accidentally read data from an old failed attempt.

Sail includes attempt in every task stream key:

```
job_id / stage / partition / attempt / channel
```

The scheduler chooses the latest attempt when building task input keys. That lets the system distinguish replacement work from old work.

There are also several important runtime behaviors:

- Pending streams allow consumers and producers to start in either order.
- Pending stream timeouts prevent consumers from waiting forever.
- Closed sinks can be normal if downstream no longer needs the data.
- Failed streams are distinct from closed streams.
- Stream errors are mapped back into `DataFusion` errors at the merge boundary.

These are small details, but they are the difference between a toy exchange and an engine that can tolerate real distributed timing.

10.13 What Is Still Open

Sail's shuffle architecture is intentionally extensible, but several pieces are still not complete:

- Blocking shuffle output placement is not implemented.
- Remote stream creation and fetch paths are design points rather than complete production paths.
- Disk-backed local shuffle storage exists in the type model, but memory is the main implemented path.
- More sophisticated backpressure, spill, and shuffle cleanup policies would be needed for a large production deployment.

For the purposes of this book, that is a feature. The code shows the essential shape: plans, task definitions, stream keys, local memory streams, and Arrow Flight transport. The missing pieces are exactly where extension proposals can become concrete.

10.14 Extension Hooks

Shuffle is one of the most important places for extensions because it sits between query semantics and physical deployment.

A Sail extension that wants to influence data movement could attach at several levels:

Extension goal	Likely hook
New partitioning strategy	Job graph output distribution and <code>TaskOutput::partitioning()</code>
Custom hash expression support	Physical expression serialization and parsing
Alternative shuffle transport	<code>TaskStreamReader</code> , <code>TaskStreamWriter</code> , and <code>StreamAccessor</code>
External shuffle service	<code>TaskReadLocation::Remote</code> , <code>TaskWriteLocation::Remote</code> , Arrow Flight service
Disk or object-store shuffle	<code>LocalStreamStorage</code> , remote locators, blocking output placement
Adaptive repartitioning	Scheduler key construction and stage output metadata
Broadcast optimization	Replica planning and input-key construction

This gives us a preview of the final chapter. The extensions proposal should not be treated as a plugin system floating above the engine. For distributed query processing, extensions need to meet Sail at the same boundaries Sail already uses internally:

- plan nodes,
- physical expressions,
- task definitions,
- stream locations,
- shuffle distributions,
- catalog and function registries,
- and execution services.

The cleanest extension architecture will preserve those boundaries rather than bypass them.

10.15 Reading Exercise

Trace one hash-shuffled row through the code:

1. Start in `TaskRunner::rewrite_shuffle()`.
2. Find where `TaskOutput::partitioning()` converts task output metadata into `Partitioning::Hash`.
3. Open `ShuffleWriteExec::execute()` and follow the creation of the partitioner.
4. Follow `shuffle_write()` until it calls `sink.write(batch)`.

5. Open `StreamAccessor` and see how the write location becomes a stream-manager message.
6. Then reverse direction through `ShuffleReadExec::execute()`.
7. End in `MergedRecordBatchStream::poll_next()`.

The important question to ask at every step is: “Is this layer deciding where data should go, or only carrying out a decision made earlier?”

That question is the key to reading distributed query engines.

10.16 Takeaways

Sail’s shuffle layer is small enough to study and rich enough to teach the real ideas:

- Shuffle is expressed as Arrow `RecordBatch` streams, not row objects.
- The scheduler chooses keys, attempts, channels, and locations.
- The task runner rewrites stage placeholders into concrete shuffle operators.
- `ShuffleWriteExec` partitions and writes batches as a side effect.
- `ShuffleReadExec` opens task streams and merges them.
- Local memory streams support pending readers and replicas.
- Arrow Flight provides the natural shape for remote batch transport.
- The open areas around remote, blocking, disk, and adaptive shuffle are prime extension points.

The next chapter moves from movement to memory and execution behavior: how Arrow batch size, streaming, boundedness, and operator properties influence distributed execution.

11 Chapter 10: The Sail Spec And Plan Resolver

Spark Connect sends unresolved intent. DataFusion executes resolved logical and physical plans. Sail’s plan layer is the translation space between those two worlds.

This chapter is about that translation space.

In previous chapters, we followed query execution once DataFusion had a physical plan. Now we step earlier in the life of a query, into the code that decides what a Spark Connect relation, SQL statement, unresolved column, function call, catalog command, or UDF registration actually means inside Sail.

The central idea is simple but powerful:

Spark Connect proto or SQL

- > Sail spec
- > DataFusion LogicalPlan
- > DataFusion optimizer
- > Sail/DataFusion physical planning
- > distributed execution

The Sail spec is not DataFusion’s logical plan. It is also not exactly Spark Connect’s protobuf model. It is Sail’s own unresolved intermediate representation, designed to be easy to parse, serialize, inspect, and resolve.

That makes it one of the most important extension points in the whole architecture.

11.1 Code Map

The main files for this chapter are:

Concern	File
Plan execution entry point	<code>crates/sail-plan/src/lib.rs</code>
Resolver entry point	<code>crates/sail-plan/src/resolver/plan.rs</code>
Resolver state	<code>crates/sail-plan/src/resolver/state.rs</code>
Query resolver dispatch	<code>crates/sail-plan/src/resolver/query/mod.rs</code>
Expression resolver dispatch	<code>crates/sail-plan/src/resolver/expression/mod.rs</code>
Command resolver dispatch	<code>crates/sail-plan/src/resolver/command/mod.rs</code>
Attribute resolution	<code>crates/sail-plan/src/resolver/expression/attribute.rs</code>
Function resolution	<code>crates/sail-plan/src/resolver/expression/function.rs</code>
Table and data source reads	<code>crates/sail-plan/src/resolver/query/read.rs</code>
Repartition nodes	<code>crates/sail-plan/src/resolver/query/repartition.rs</code>
WithRelations support	<code>crates/sail-plan/src/resolver/query/with_relations.rs</code>
Sail spec plan model	<code>crates/sail-common/src/spec/plan.rs</code>
Sail spec expression model	<code>crates/sail-common/src/spec/expression.rs</code>
Sail spec data types	<code>crates/sail-common/src/spec/data_type.rs</code>
Spark Connect relation conversion	<code>crates/sail-spark-connect/src/proto/plan.rs</code>
Spark Connect expression conversion	<code>crates/sail-spark-connect/src/proto/expression.rs</code>
Spark Connect execution handler	<code>crates/sail-spark-connect/src/service/plan_executor.rs</code>
DataFusion extension physical planner	<code>crates/sail-session/src/planner.rs</code>

Useful external references:

- [Spark Connect Overview](#)

- [Apache Spark Connect architecture page](#)
- [Application Development with Spark Connect](#)
- [Spark Connect base.proto](#)
- [Spark Connect relations.proto](#)
- [Spark Connect expressions.proto](#)

Spark’s own documentation describes Spark Connect as a client-server architecture where clients send unresolved logical plans over gRPC and receive Arrow-encoded batches back. Sail follows that same shape, but swaps Spark’s analyzer and engine for Sail’s resolver, DataFusion’s optimizer, and Sail’s distributed runtime.

11.2 Why Sail Has A Spec Layer

The spec layer lives in `sail_common::spec`.

At first glance, it may look like a reimplementation of Spark Connect relations and expressions. That is only partly true. The comment above `spec::Plan` explains the design: the starting point is Spark Connect’s Relation model, but Sail makes intentional changes.

The spec layer:

- separates query plans from command plans,
- avoids raw SQL strings as unresolved plan nodes,
- uses parsed schemas rather than schema strings,
- prefers Rust naming and serde-friendly enum shapes,
- adds nodes needed by SQL and Sail that are not direct Spark Connect relation nodes,
- stores data types, literals, expressions, and plans in one common model.

That gives Sail a stable internal contract:

```
pub enum Plan {
    Query(QueryPlan),
    Command(CommandPlan),
}
```

Both query and command plans carry optional `plan_id` values:

```
pub struct QueryPlan {
    pub node: QueryNode,
    pub plan_id: Option<i64>,
}

pub struct CommandPlan {
    pub node: CommandNode,
    pub plan_id: Option<i64>,
}
```

Those plan IDs matter because Spark Connect clients often reference subplans and attributes by ID. Sail preserves that identity through conversion and uses it during resolution.

11.3 The Full Planning Pipeline

The public planning entry point is `resolve_and_execute_plan()` in `crates/sail-plan/src/lib.rs`.

Despite the name, it does more than execute. It performs the whole plan path up to an executable physical plan:

```
pub async fn resolve_and_execute_plan(
    ctx: &SessionContext,
    config: Arc<PlanConfig>,
    plan: spec::Plan,
) -> PlanResult<(Arc<dyn ExecutionPlan>, Vec<StringifiedPlan>)>
```

The steps are:

1. Build a `PlanResolver`.
2. Resolve a `spec::Plan` into a `DataFusion LogicalPlan`.
3. Record the initial logical plan for explain output.
4. Ask `DataFusion` to create a `DataFrame` from the logical plan.
5. Optimize the logical plan with `DataFusion`'s session state.
6. Rewrite streaming plans if needed.
7. Ask the session query planner to create a physical plan.
8. Rename physical output fields back to user-facing names when needed.
9. Return the physical plan and plan strings.

In diagram form:

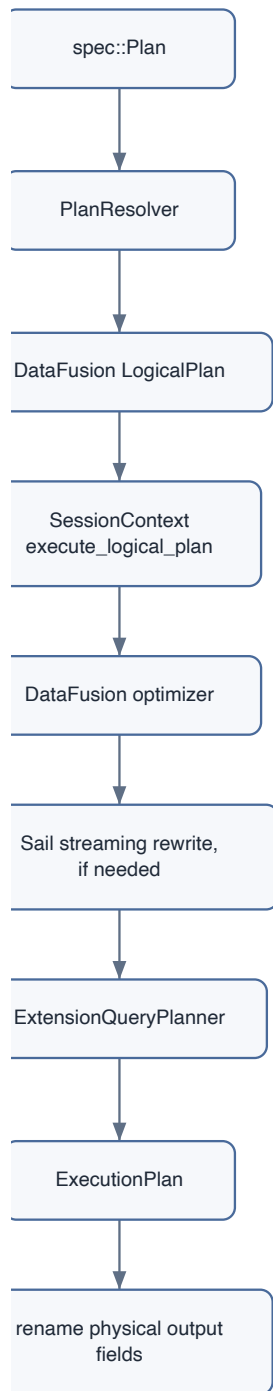


Figure 53: Flowchart 10.1

This is the same boundary we saw from the other side in earlier chapters:

- Chapter 6 focused on DataFusion plans.
- Chapter 7 split physical plans into a job graph.
- This chapter explains how Sail gets to the logical plan in the first place.

11.4 Spark Connect To Sail Spec

Spark Connect requests arrive as generated protobuf Rust types in `crates/sail-spark-connect`. The conversion layer turns those types into Sail spec values.

The important conversion file is:

```
crates/sail-spark-connect/src/proto/plan.rs
```

For relations:

```
impl TryFrom<Relation> for spec::Plan
impl TryFrom<Relation> for spec::QueryPlan
impl TryFrom<Relation> for spec::CommandPlan
impl TryFrom<RelType> for RelationNode
```

This conversion has to make a decision: is a Spark Connect relation a query node or a command node?

Sail uses an internal helper enum:

```
enum RelationNode {
    Query(spec::QueryNode),
    Command(spec::CommandNode),
}
```

That matters because Spark Connect has several operation shapes. Some produce relations; others execute side effects or return command results. Sail normalizes them into `spec::Plan::Query` or `spec::Plan::Command`.

For expressions, the conversion file is:

```
crates/sail-spark-connect/src/proto/expression.rs
```

There, Spark Connect expressions become `spec::Expr`:

```
impl TryFrom<Expression> for spec::Expr
```

Examples:

Spark Connect expression	Sail spec expression
Literal	spec::Expr::Literal
UnresolvedAttribute	spec::Expr::UnresolvedAttribute
UnresolvedFunction	spec::Expr::UnresolvedFunction
ExpressionString	parsed SQL expression, then Sail spec
UnresolvedStar	spec::Expr::UnresolvedStar
Alias	spec::Expr::Alias
Cast	spec::Expr::Cast
Window	spec::Expr::Window
CommonInlineUserDefinedFunction	spec::Expr::CommonInlineUserDefinedFunction
SubqueryExpression	spec::Expr::Subquery

This is the first important lesson: Sail does not resolve columns or functions while converting protobuf messages. It only parses the client's intent into Sail's own unresolved representation.

That keeps Spark Connect compatibility code separate from analysis.

11.5 SQL To Sail Spec

Spark Connect is not the only source of spec plans. SQL also enters the same model.

The conversion layer uses `sail_sql_analyzer` helpers such as:

```

parse_one_statement
from_ast_statement
parse_expression
from_ast_expression
parse_object_name
from_ast_object_name

```

For example, Spark Connect may send an `ExpressionString`. Sail parses that expression string into an AST, then converts the AST into `spec::Expr`.

The same pattern appears for SQL commands in `handle_execute_sql_command()`:

```

SQL string
-> Spark Connect relation shape
-> Sail spec plan
-> resolver
-> DataFusion logical plan

```

This unification is important. It means that SQL and DataFrame APIs meet before DataFusion planning, not after.

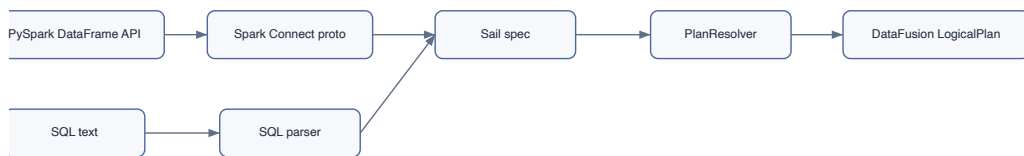


Figure 54: Flowchart 10.2

For extension authors, this is a clue: if an extension is only available through SQL, it is not really integrated with the Spark Connect-style architecture. A good extension should be expressible in the spec layer or reachable through a spec-producing parser.

11.6 The Resolver Entry Point

The resolver itself is tiny at the top:

```
pub struct PlanResolver<'a> {
    ctx: &'a SessionContext,
    config: Arc<PlanConfig>,
}
```

It holds:

- a DataFusion SessionContext,
- a PlanConfig.

The main entry point is `resolve_named_plan()` in `crates/sail-plan/src/resolver/plan.rs`.

```
pub async fn resolve_named_plan(&self, plan: spec::Plan) -> PlanResult<NamedPlan>
```

It returns a `NamedPlan`:

```
pub struct NamedPlan {
    pub plan: LogicalPlan,
    pub fields: Option<Vec<String>>,
}
```

The `fields` value is subtle. Sail often gives internal columns opaque field IDs during analysis to avoid name collisions. But the user still expects Spark-like output names. For query plans, `resolve_named_plan()` captures the user-facing field names so the physical plan can later be renamed back.

Commands do not get output field renaming in the same way:

```
spec::Plan::Query    -> LogicalPlan plus user-facing fields
spec::Plan::Command -> LogicalPlan with fields = None
```

11.7 Resolver State

Most of the interesting resolver behavior is not in `PlanResolver` itself. It is in `PlanResolverState`.

The state tracks:

- generated internal field IDs,
- user-facing field names,
- hidden fields,
- plan IDs attached to fields,
- the outer query schema for correlated subqueries,
- aggregate resolution mode,
- CTEs,
- WithRelations subquery references,
- temporary config flags,
- positional and named parameter values.

The most important field-resolution trick is this:

```
user name: "customer_id"  
internal field ID: "#7"
```

The DataFusion logical plan may carry #7, but Sail remembers that the user-facing name is `customer_id`.

Why do this? Because DataFrame plans can easily contain duplicate column names:

```
SELECT left.id, right.id  
FROM left  
JOIN right ON left.id = right.id
```

If both columns were simply named `id`, subsequent resolution would become ambiguous too early or in the wrong way. Sail uses opaque internal names to keep the logical plan well-formed while preserving Spark-facing names for display and final output.



Figure 55: Flowchart 10.3

The state also records hidden fields. Hidden fields are temporary columns needed for analysis or execution but not intended to appear in the final output. For example, a join or sort rewrite may need to carry a helper field through a subplan.

After resolving a query, `resolve_query_plan()` calls `remove_hidden_fields()` so the public logical plan does not expose those helper fields.

11.8 Query Resolution

The query resolver dispatch lives in:

`crates/sail-plan/src/resolver/query/mod.rs`

It is an async recursive dispatcher over `spec::QueryNode`.

The main structure is:

```
match plan.node {
  QueryNode::Read { .. } => ...
  QueryNode::Project { .. } => ...
  QueryNode::Filter { .. } => ...
  QueryNode::Join(join) => ...
  QueryNode::Aggregate(aggregate) => ...
  QueryNode::Repartition { .. } => ...
  QueryNode::WithRelations { .. } => ...
  ...
}
```

Each variant delegates to a focused resolver file:

Query shape	Resolver file
Read	query/read.rs
Project	query/project.rs
Filter	query/filter.rs
Join	query/join.rs
Aggregate	query/aggregate.rs
Sort	query/sort.rs
Limit	query/limit.rs
Repartition	query/repartition.rs
WithRelations	query/with_relations.rs
UDF and UDTF plan forms	query/udf.rs, query/udtf.rs

After each query node is resolved, the dispatcher does two things:

```
verify_query_plan(plan, state)
register_schema_with_plan_id(plan, plan_id, state)
```

That means every resolved plan must use fields known to the resolver state. If a new resolver creates a field and forgets to register it, Sail fails with an internal resolver error.

That is a useful invariant. It catches bugs at the boundary where Spark-compatible names become DataFusion columns.

11.9 Reading Tables And Data Sources

query/read.rs is a rich file because reading is where catalogs, file formats, temporary views, CTEs, and dynamic table names meet.

For a named table, Sail handles several cases:

1. If the name looks like <format>.<path> and the format is registered, treat it as a direct data source read.
2. If the name matches a CTE, use the CTE plan.
3. Otherwise, ask the CatalogManager for a table or view.
4. For a table, build SourceInfo and ask TableFormatRegistry to create a source.
5. For a persistent view, parse the stored SQL definition and resolve it.
6. For a temporary view, clone and rename the stored logical plan.

The rough flow:

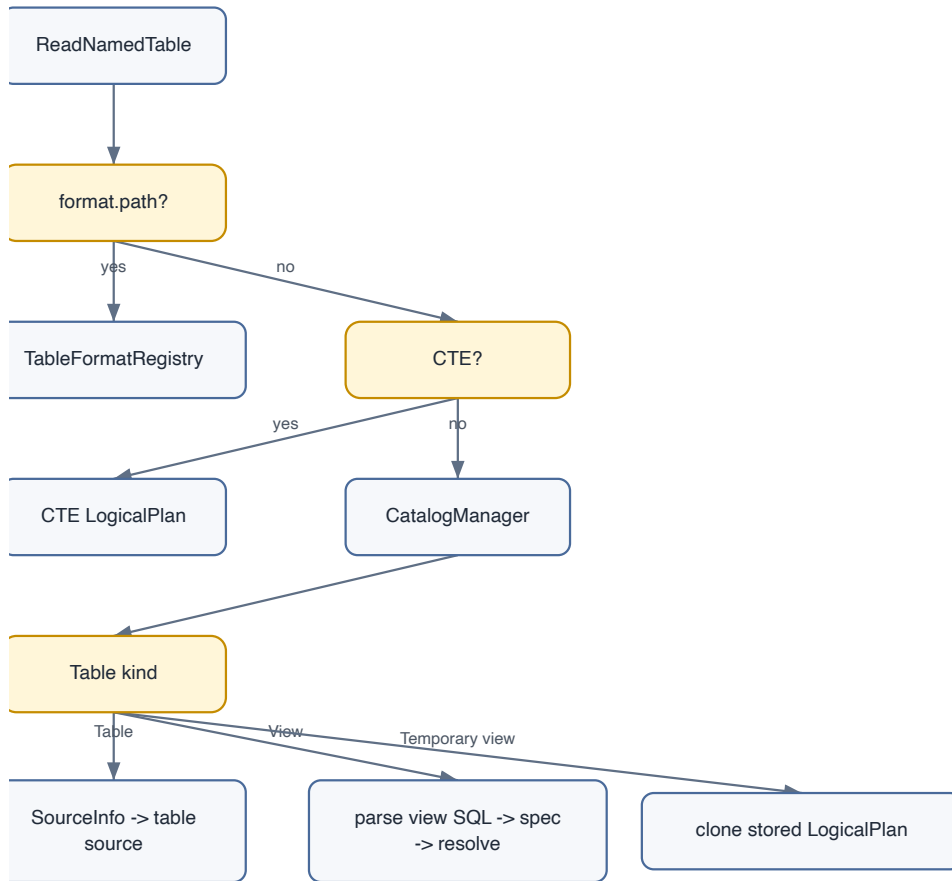


Figure 56: Flowchart 10.4

This is where Sail's session extensions begin to matter:

- CatalogManager supplies table and view metadata.
- TableFormatRegistry turns format-specific metadata into table sources.
- PlanService provides display and formatting helpers elsewhere in resolution.

Those are not global singletons. They are DataFusion session extensions. That design lets Sail attach Spark-compatible services to a normal DataFusion SessionContext.

11.10 Project Resolution And Rewriters

Projection is deceptively complex. The Project node has to handle ordinary expressions, wildcard expansion, aliases, generators, windows, aggregate shortcuts, and Spark-specific functions like `spark_partition_id()`.

`resolve_query_project()` follows this pattern:

1. Resolve the input plan, or create a one-row empty input when the project has no input.
2. Resolve each spec expression into a `NamedExpr`.
3. Expand wildcards.
4. Run projection rewriters.
5. Rewrite multi-expression functions.
6. If aggregate functions are present, rewrite the projection as an aggregate.
7. Otherwise build a `DataFusion Projection`.

The projection rewriters are especially instructive:

`MonotonicIdRewriter`
`SparkPartitionIdRewriter`
`ExplodeRewriter`
`WindowRewriter`

They transform expressions that cannot remain as plain scalar expressions into plan shapes that `DataFusion` can execute.

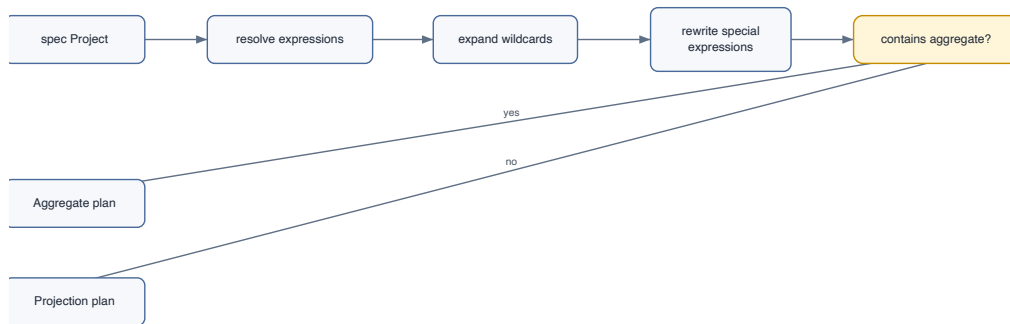


Figure 57: Flowchart 10.5

This pattern appears across `Sail`: the spec layer preserves Spark intent, then the resolver reshapes that intent into `DataFusion`-compatible logical plans.

11.11 Attribute Resolution

Attributes are where users feel analysis quality most sharply.

The resolver handles:

- case-insensitive matching,
- qualified names,
- nested struct fields,
- plan ID filtering,
- aggregate aliases,

- hidden fields,
- outer references for correlated subqueries.

The central method is:

```
resolve_expression_attribute(...)
```

It tries candidates in a careful order:

1. Aggregate fields visible in HAVING.
2. Normal fields and nested fields.
3. Aggregate grouping fields.
4. Hidden fields.
5. Outer query fields.

Nested-field resolution is Arrow-aware. For a struct field, Sail builds a DataFusion `get_field` scalar function expression rather than inventing a custom row accessor.

For example:

```
SELECT address.city FROM customers
```

becomes conceptually:

```
column(address)
  -> get_field("city")
```

Qualified matching supports forms like:

```
column
table.column
schema.table.column
catalog.schema.table.column
```

The helper `qualifier_matches()` performs case-insensitive comparison against DataFusion `TableReference` values.

This is one of the places where the resolver has to act more like Spark than vanilla DataFusion. The user's unresolved expression is not just a name; it carries Spark resolution expectations.

11.12 Function Resolution

Function resolution lives mainly in:

```
crates/sail-plan/src/resolver/expression/function.rs
crates/sail-plan/src/function/mod.rs
```

The function resolver follows a layered lookup:

1. Normalize the function name.
2. Extract named arguments.
3. Check the catalog for registered functions, including PySpark UDFs.
4. Check Sail built-in scalar and generator functions.
5. Check Sail built-in aggregate functions.

6. Build a DataFusion expression.
7. Format a Spark-like display name.

The catalog lookup comes before built-ins because Spark Connect does not reliably mark all UDF calls in a way Sail can trust. The code even notes this:

`is_user_defined_function` is always false, so we need to check UDFs before built-in functions.

For built-ins, Sail has registries:

```
BUILT_IN_SCALAR_FUNCTIONS
BUILT_IN_GENERATOR_FUNCTIONS
BUILT_IN_TABLE_FUNCTIONS
```

For aggregate functions, the resolver also handles clauses like:

- DISTINCT,
- FILTER,
- ORDER BY,
- IGNORE NULLS.

For PySpark UDFs, Sail carries enough information to later execute Python code:

```
pub(super) struct PythonUdf {
    pub python_version: String,
    pub eval_type: spec::PySparkUdfType,
    pub command: Vec<u8>,
    pub output_type: DataType,
}
```

This is the same pattern we saw in the PySpark chapter: the Python function is not run in the resolver. It is represented as a plan expression that later physical execution can evaluate.

11.13 Commands Become Logical Plans Too

Commands enter through `spec::CommandPlan` and are resolved in:

```
crates/sail-plan/src/resolver/command/mod.rs
```

The command resolver handles catalog operations, writes, streaming writes, explains, inserts, merge, deletes, variables, and view/table/database DDL.

Many catalog commands become DataFusion extension logical plans:

```
LogicalPlan::Extension(Extension {
    node: Arc::new(CatalogCommandNode::try_new(self.ctx, command)?),
})
```

Later, the physical planner recognizes `CatalogCommandNode` and turns it into:

```
CatalogCommandExec
```

This is the key pattern for command execution:

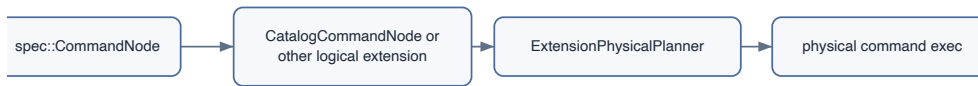


Figure 58: Flowchart 10.6

This lets Sail preserve DataFusion’s plan pipeline even for operations that are not ordinary relational queries.

11.14 Logical Extension Nodes

Sail uses DataFusion logical extension nodes for Spark-specific plan concepts that DataFusion does not natively model.

Examples include:

- RangeNode,
- ShowStringNode,
- MapPartitionsNode,
- MonotonicIdNode,
- SparkPartitionIdNode,
- SortWithinPartitionsNode,
- SchemaPivotNode,
- FileWriteNode,
- FileDeleteNode,
- MergeIntoNode,
- ExplicitRepartitionNode,
- streaming source/filter/limit/collector nodes,
- CatalogCommandNode,
- BarrierNode.

These nodes are planned in `crates/sail-session/src/planner.rs` by `ExtensionPhysicalPlanner`.

That planner is installed through `ExtensionQueryPlanner`, which builds a DataFusion `DefaultPhysicalPlanner` with extension planners:

```

lakehouse extension planners
system table physical planner
Sail extension physical planner
  
```

This is a crucial architectural point: Sail does not fork DataFusion’s planner. It uses DataFusion’s extension hooks.

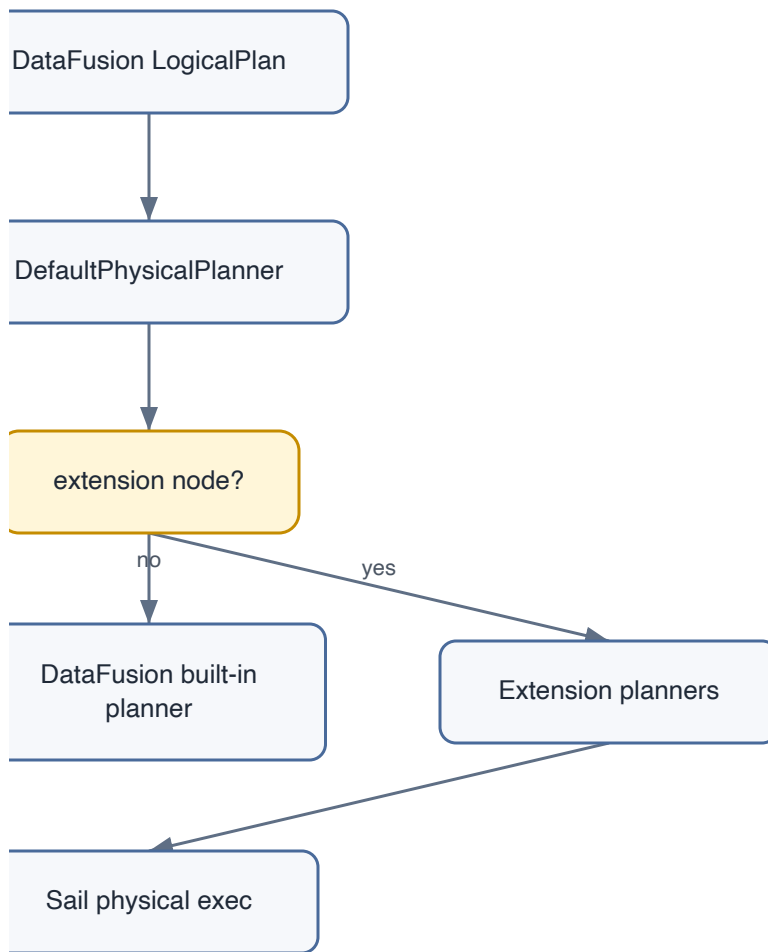


Figure 59: Flowchart 10.7

For discussion #2001, this pattern is already half of the answer. Third-party integrations need a disciplined way to register logical and physical extension behavior without hard-coding every integration into `sail-session/src/planner.rs`.

11.15 Repartition As A Bridge To Distributed Execution

`query/repartition.rs` is a compact example that connects this chapter to the shuffle chapters.

Spark-facing repartition intent becomes a Sail logical extension node:

```
ExplicitRepartitionNode::new(
  Arc::new(input),
  Some(num_partitions),
  ExplicitRepartitionKind::RoundRobin,
  vec![],
)
```

For `repartitionByExpression`, Sail resolves the partition expressions and creates:

```
ExplicitRepartitionKind::Hash
```

Later, `ExtensionPhysicalPlanner` turns `ExplicitRepartitionNode` into `ExplicitRepartitionExec`, using `DataFusion` physical expressions and partitioning:

```
RoundRobin -> Partitioning::RoundRobinBatch
Hash       -> Partitioning::Hash
Coalesce   -> UnknownPartitioning with fewer partitions
```

Then Chapter 7's job graph planner and Chapter 9's shuffle operators take over.

This path is worth memorizing:

```
Spark repartition call
-> spec::QueryNode::Repartition
-> ExplicitRepartitionNode
-> ExplicitRepartitionExec
-> distributed stage boundary
-> ShuffleWriteExec / ShuffleReadExec
```

That is how a user-level API becomes data movement.

11.16 WithRelations And Plan IDs

Spark Connect can send a root relation plus referenced relations. Sail models this as:

```
QueryNode::WithRelations { root, references }
```

The resolver stores the references in `PlanResolverState` by `plan_id`.

It also handles a useful PySpark pattern: SQL strings can refer to DataFrames passed as arguments. The conversion may wrap those DataFrames in `SubqueryAlias` nodes inside `WithRelations`. Sail resolves those references and registers them as CTE-like table names for the root query.

The flow is:

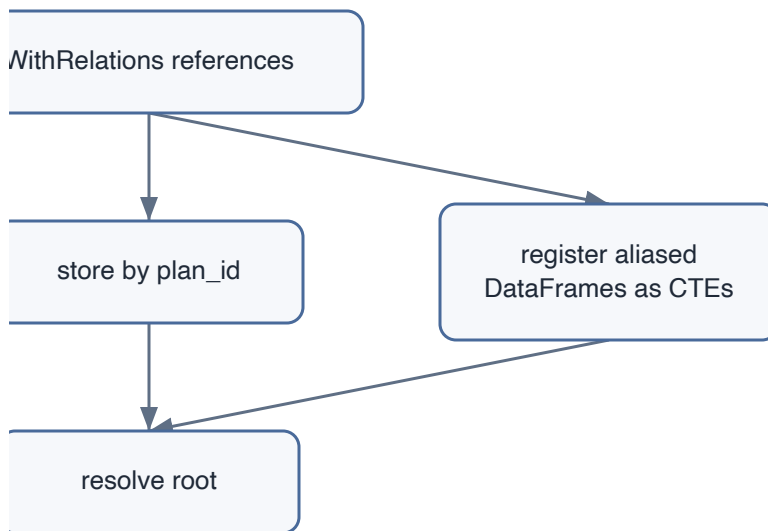


Figure 60: Flowchart 10.8

The scoping helpers in `PlanResolverState` make this safe:

- `enter_with_relations_scope()`,
- `enter_cte_scope()`.

Each helper restores the previous state on drop. This is a very Rust-flavored design: scope cleanup is tied to ownership and `Drop`, so temporary resolver state does not leak into the outer query.

11.17 Commands In Spark Connect Execution

The Spark Connect plan executor in:

`crates/sail-spark-connect/src/service/plan_executor.rs`

uses different modes for different operations.

Normal relations are lazy:

```

handle_execute_relation
-> relation.try_into()
-> handle_execute_plan(..., Lazy)
  
```

Commands such as UDF registration and writes are eager and silent:

```

handle_execute_register_function
handle_execute_write_operation
handle_execute_create_dataframe_view
handle_execute_write_operation_v2

```

They build a `spec::Plan::Command`, resolve it, execute it, drain the stream, and return completion metadata rather than a normal relation stream.

SQL command handling has one extra twist. If a SQL string resolves to a command, Sail executes the command and returns a local relation containing the command result. That matches Spark Connect's expectation that a SQL command can return an opaque relation for the client to use.

11.18 A Worked Example: DataFrame Filter And Project

Consider a PySpark call:

```
df = spark.table("orders").where("amount > 100").select("customer_id", "amount")
```

Spark Connect sends an unresolved relation tree roughly like:

```

Project(customer_id, amount)
  Filter(amount > 100)
    Read(NamedTable orders)

```

Sail first converts that into spec:

```

spec::QueryNode::Project
  spec::QueryNode::Filter
    spec::QueryNode::Read

```

Then the resolver walks bottom-up:

1. ReadNamedTable asks the catalog for orders.
2. The table source is created and fields are registered with internal IDs.
3. Filter resolves amount against the input schema.
4. Project resolves customer_id and amount.
5. The final schema is verified.
6. User-facing field names are captured for later physical renaming.

The interesting part is the field mapping:

```

customer_id -> #0
amount      -> #1

```

DataFusion sees stable internal columns. The Spark client eventually sees the expected names.

11.19 A Worked Example: Repartition By Customer

Now consider:

```
df.repartition(16, "customer_id")
```

At the spec level:

```

QueryNode::RepartitionByExpression {
  partition_expressions: [UnresolvedAttribute(customer_id)],
  num_partitions: Some(16),
}

```

The resolver:

1. Resolves the input plan.
2. Resolves `customer_id` into a DataFusion column expression.
3. Builds an `ExplicitRepartitionNode` with `ExplicitRepartitionKind::Hash`.

The physical planner:

1. Converts the logical expression into a physical expression.
2. Builds `Partitioning::Hash(expressions, 16)`.
3. Creates `ExplicitRepartitionExec`.

The distributed planner:

1. Sees the repartition boundary.
2. Creates producer and consumer stages.
3. Schedules shuffle channels.
4. Uses the runtime shuffle read/write path from Chapter 9.

One user-level method call has passed through four layers:

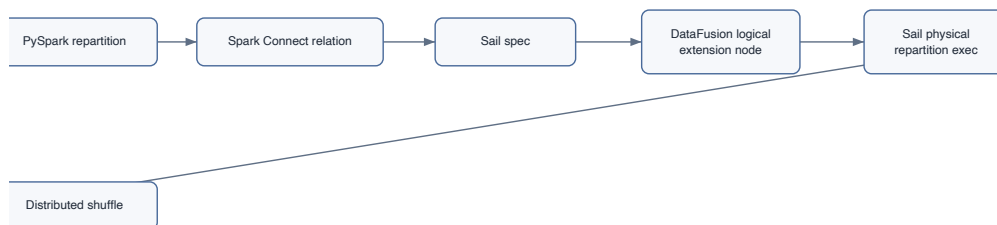


Figure 61: Flowchart 10.9

11.20 A Worked Example: Registered Python UDF

A registered Python UDF follows a different path.

First, Spark Connect sends a register-function command. Sail builds:

```

spec::Plan::Command(
  spec::CommandNode::RegisterFunction(...)
)

```

The command resolver stores the function in the catalog.

Later, a query calls the function:

```
spark.sql("SELECT my_udf(x) FROM t")
```

The expression resolver sees:

```
spec::Expr::UnresolvedFunction("my_udf", [x])
```

It checks the catalog before built-ins, finds the PySpark unresolved UDF, resolves the argument expressions, and builds a Python UDF expression that can be planned and executed later.

The UDF command bytes stay as bytes. The resolver does not deserialize Python logic or execute Python code. It only creates a typed plan representation.

That distinction is vital for distributed execution. Workers need a serializable plan and enough metadata to run the UDF in the right execution context.

11.21 Extension Implications

Discussion #2001 asks for an extension API for third-party DataFusion integrations:

- UDFs,
- optimizer rules,
- planner extensions,
- probably catalog/session configuration hooks,
- and Python-discoverable packages such as a hypothetical `pysail-sedona`.

This chapter reveals why the extension story cannot be only a function registry.

Extensions may need to participate in several phases:

Phase	Why extensions need it
Spark Connect conversion	To accept custom relation, expression, or command messages.
Sail spec	To represent extension intent in a language-neutral, serializable form.
SQL analysis	To parse extension SQL syntax or functions.
Logical resolution	To bind names, tables, functions, and types.
Logical optimization	To rewrite extension plans before physical planning.
Physical planning	To turn extension logical nodes into execution plans.
Plan encoding	To send physical expressions or nodes to workers.
Worker registration	To ensure workers can execute extension functions and operators.

The current architecture has useful internal patterns, but most of them are wired into Sail itself:

- built-in function registries are static maps,
- logical extension nodes are known to Sail crates,
- ExtensionPhysicalPlanner has hard-coded downcasts,
- lakehouse planners are installed through a dedicated helper,
- PySpark UDFs are special-cased in resolver paths,
- Spark Connect custom extension handling is not a general plugin registry.

A mature extension design would turn those internal patterns into explicit contracts.

11.22 A Proposed Resolver-Side Extension Shape

One possible architecture is a staged extension trait family rather than one giant trait.

For example:

```
pub trait SailPlanExtension: Send + Sync {
    fn name(&self) -> &'static str;
    fn register_functions(&self, registry: &mut FunctionRegistry) ->
PlanResult<()>;
    fn register_table_functions(&self, registry: &mut TableFunctionRegistry) ->
PlanResult<()>;
    fn logical_resolvers(&self) -> Vec<Arc<dyn ExtensionLogicalResolver>>;
    fn logical_optimizer_rules(&self) -> Vec<Arc<dyn LogicalRewriter>>;
    fn physical_planners(&self) -> Vec<Arc<dyn ExtensionPlanner + Send + Sync>>;
    fn codecs(&self) -> Vec<Arc<dyn ExtensionCodec>>;
}
```

The goal would be to let an extension say:

```
I know how to parse or receive this intent.
I know how to resolve it into a logical node.
I know how to optimize it.
I know how to plan it physically.
I know how to encode it for workers.
```

For Spark Connect specifically, extensions also need a protocol story. Spark Connect’s own extension guidance defines Relation.extension, Command.extension, and Expression.extension, each typed as google.protobuf.Any. Sail’s spec layer can mirror that by introducing a type_url-indexed dispatcher in the resolver:

```
Connect Relation/Expression/Command .extension
-> SparkConnectExtensionDispatcher::dispatch(type_url, payload)
-> extension handler resolves payload
-> either:
    spec::QueryNode built from existing operators (pattern A, plan-time only),
    or:
    spec::QueryNode::Extension { ... } for a logical extension node (pattern
B)
-> normal Sail planning and DataFusion execution
```

Pattern A extensions never need an execution-time integration. Pattern B extensions hand off to a logical extension node and the rest of the chapter 13 extension stack. This makes the resolver

the dispatch point for what chapter 13 calls the *plan-time extension boundary*: a stable, protobuf-versioned, language-neutral channel that is independent of the Rust/DataFusion-FFI work needed for custom physical operators. Chapter 13 develops the full dispatcher design.

11.23 Design Rules For Future Extensions

The resolver code suggests several design rules.

First, preserve unresolved intent until enough context exists. The protobuf conversion layer should parse and normalize, but not bind names too early.

Second, keep Spark-facing names separate from engine-facing names. Any extension that creates fields should register them through resolver state or an equivalent API.

Third, distinguish query nodes from command nodes. Side-effecting extensions should not pretend to be ordinary projections.

Fourth, make worker compatibility explicit. If an extension creates physical operators, workers must have the same extension and codec registrations.

Fifth, use DataFusion extension hooks where possible. Sail's strength is that it extends DataFusion rather than replacing it.

Sixth, expose ordering and collision rules. Function names, optimizer rules, and physical planners all need deterministic registration behavior.

11.24 Reading Exercise

Trace a simple query:

```
SELECT customer_id, count(*)  
FROM orders  
GROUP BY customer_id
```

Suggested path:

1. Start in `crates/sail-spark-connect/src/proto/plan.rs` if the query arrives through Spark Connect, or in the SQL analyzer path if it starts as SQL.
2. Find the resulting `spec::QueryNode` and `spec::Expr` values.
3. Open `crates/sail-plan/src/resolver/plan.rs`.
4. Follow `resolve_named_plan()` into `resolve_query_plan()`.
5. In `query/read.rs`, follow table resolution for orders.
6. In `expression/attribute.rs`, follow `customer_id`.
7. In `expression/function.rs`, follow `count`.
8. In `query/aggregate.rs`, follow group-by planning.
9. Return to `resolve_and_execute_plan()` and see how DataFusion optimization begins.

The core question is:

At this line, are we still describing user intent, or have we bound that intent to a DataFusion plan object?

Once you can answer that, the resolver stops feeling like a forest and starts feeling like a set of well-marked trails.

11.25 Takeaways

The Sail spec and resolver form the semantic center of the engine:

- Spark Connect protobufs and SQL both become Sail spec plans.
- The Sail spec is an unresolved, serializable representation of Spark-compatible query and command intent.
- `PlanResolver` turns spec plans into DataFusion logical plans.
- `PlanResolverState` tracks internal field IDs, user-facing names, hidden fields, plan IDs, CTEs, subqueries, and temporary scopes.
- Query, expression, and command resolution are split into focused modules.
- Sail uses DataFusion logical extension nodes for Spark-specific behavior.
- The session physical planner turns those extension nodes into Sail physical execution plans.
- Extension proposal #2001 should build on these existing boundaries rather than bypassing them.

The next chapter turns from plans to callable behavior: functions, UDFs, UDAFs, UDTFs, codecs, and why distributed execution makes serialization and worker-side registration non-negotiable.

12 Chapter 11: Functions, UDFs, And Codecs

A function call looks small in a query:

```
SELECT lower(name), my_python_udf(amount)
FROM orders
```

Inside a distributed engine, that little expression is a contract.

The driver must resolve the function name. The logical plan must carry the right DataFusion expression. The physical plan must know how to execute it. If the query runs on workers, the function implementation and all of its parameters must survive serialization. If the function is a Python UDF, the worker must also reconstruct a PySpark-compatible payload and call Python with Arrow data in the right shape.

This chapter is about that contract.

Sail's function architecture spans four layers:

```
spec::Expr / spec::CommandNode
  -> PlanResolver function logic
  -> DataFusion UDF/UDAF/window/stream objects
  -> RemoteExecutionCodec for worker execution
```

That makes functions one of the best places to understand why extension proposal #2001 is not just about registering names. Distributed extensions must be resolvable, plannable, serializable, decodable, and executable everywhere the query can run.

12.1 Code Map

The main files for this chapter are:

Concern	File
Spec representation of UDFs and UDTFs	crates/sail-common/src/spec/expression.rs
Built-in function registry	crates/sail-plan/src/function/mod.rs
Built-in scalar groups	crates/sail-plan/src/function/scalar/mod.rs
Built-in aggregate functions	crates/sail-plan/src/function/aggregate.rs
Built-in table functions	crates/sail-plan/src/function/table/mod.rs
Function expression resolution	crates/sail-plan/src/resolver/expression/function.rs
Inline Python UDF resolution	crates/sail-plan/src/resolver/expression/udf.rs
Function registration commands	crates/sail-plan/src/resolver/command/function.rs
UDTF resolution	crates/sail-plan/src/resolver/query/udtf.rs
Catalog function storage	crates/sail-catalog/src/manager/function.rs
Stream UDF trait	crates/sail-common-datafusion/src/udf.rs
Map partitions physical operator	crates/sail-physical-plan/src/map_partitions.rs
PySpark scalar UDF implementation	crates/sail-python-udf/src/udf/pyspark_udf.rs
PySpark aggregate UDF implementation	crates/sail-python-udf/src/udf/pyspark_udaf.rs
PySpark UDTF implementation	crates/sail-python-udf/src/udf/pyspark_udtf.rs
PySpark payload building	crates/sail-python-udf/src/cereal/pyspark_udf.rs
PySpark stream bridge	crates/sail-python-udf/src/stream.rs
Remote execution codec	crates/sail-execution/src/codec.rs
Codec protobuf schema	crates/sail-execution/proto/sail/plan/physical.proto
Server session setup	crates/sail-session/src/session_factory/server.rs
Worker session setup	crates/sail-session/src/session_factory/worker.rs

12.2 The Function Lifecycle

A function in Sail can enter from several front doors:

- a Spark Connect UnresolvedFunction,
- a SQL function call,
- a Spark Connect inline Python UDF expression,
- a function registration command,
- a table-valued function relation,
- an internal rewrite that inserts a helper UDF.

All of those eventually need to become DataFusion expressions or Sail physical operators.

The lifecycle looks like this:

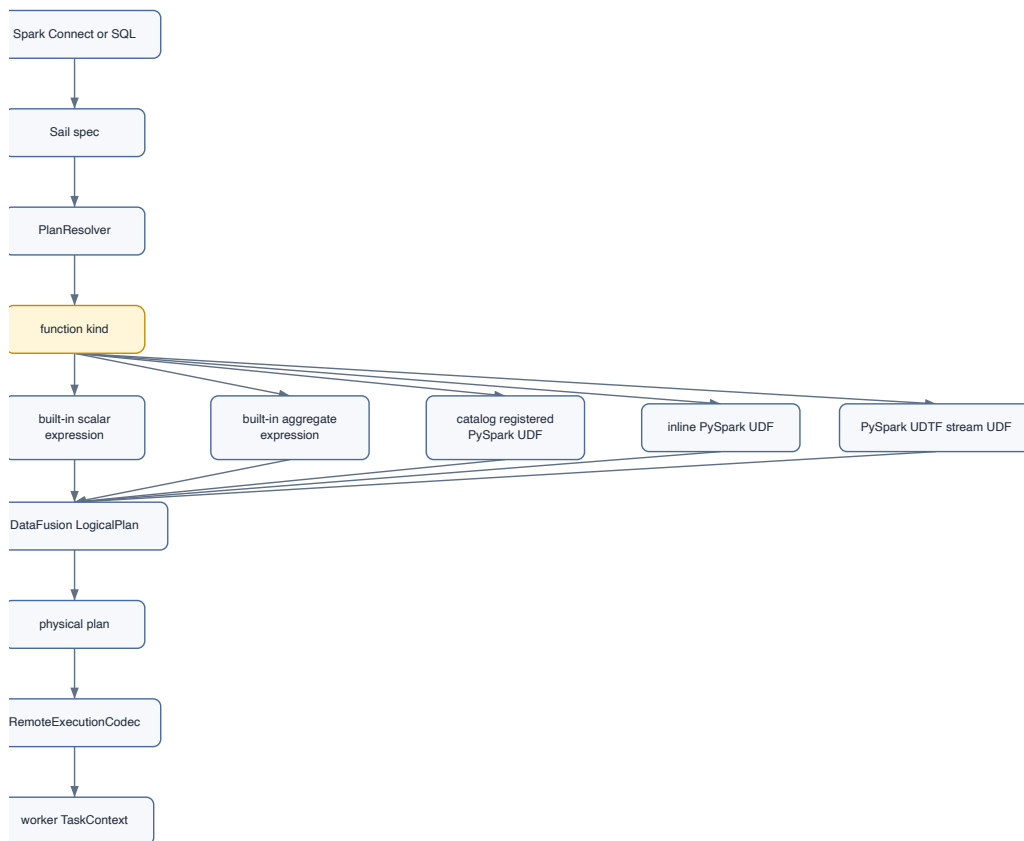


Figure 62: Flowchart 11.1

The key point is that the resolver does not execute functions. It builds objects that DataFusion and Sail can execute later.

12.3 Spec Representation

The spec layer models inline user-defined functions in `crates/sail-common/src/spec/expression.rs`.

Scalar UDFs use:

```
pub struct CommonInlineUserDefinedFunction {
    pub function_name: Identifier,
    pub deterministic: bool,
    pub is_distinct: bool,
    pub arguments: Vec<Expr>,
    pub function: FunctionDefinition,
}
```

The function definition can be:

```
pub enum FunctionDefinition {
    PythonUdf {
        output_type: DataType,
        eval_type: PySparkUdfType,
        command: Vec<u8>,
        python_version: String,
        additional_includes: Vec<String>,
    },
    ScalarScalaUdf { ... },
    JavaUdf { ... },
}
```

Table functions use:

```
pub struct CommonInlineUserDefinedTableFunction {
    pub function_name: Identifier,
    pub deterministic: bool,
    pub arguments: Vec<Expr>,
    pub function: TableFunctionDefinition,
}
```

Today, the table function definition is Python-specific:

```
pub enum TableFunctionDefinition {
    PythonUdtf {
        return_type: Option<DataType>,
        eval_type: PySparkUdfType,
        command: Vec<u8>,
        python_version: String,
    },
}
```

The `command: Vec<u8>` field is the serialized Python payload from PySpark. Sail treats it as opaque bytes until it is time to construct a PySpark worker-compatible payload.

The `PySparkUdfType` enum mirrors PySpark evaluation modes:

```

Batched
ArrowBatched
ScalarPandas
GroupedAggPandas
ScalarPandasIter
ScalarArrow
ScalarArrowIter
GroupedAggArrow
Table
ArrowTable
ArrowUdtf
...

```

Different evaluation types imply different execution shapes:

Eval type family	Sail execution shape
Scalar batch UDF	DataFusion ScalarUDFImpl
Scalar Pandas or Arrow UDF	DataFusion ScalarUDFImpl with Python bridge
Grouped aggregate UDF	DataFusion AggregateUDFImpl
Map iterator UDF	Sail StreamUDF through MapPartitionsExec
UDTF	Sail StreamUDF through MapPartitionsExec

This is why the function type must be explicit in the spec. The same surface concept, “a Python function,” can require very different execution machinery.

12.4 Built-In Function Registry

Sail’s built-in Spark-compatible functions are registered in static maps in `crates/sail-plan/src/function/mod.rs`.

```

lazy_static! {
    pub static ref BUILT_IN_SCALAR_FUNCTIONS: HashMap<&'static str,
ScalarFunction> =
        HashMap::from_iter(scalar::list_built_in_scalar_functions());

    pub static ref BUILT_IN_GENERATOR_FUNCTIONS: HashMap<&'static str,
ScalarFunction> =
        HashMap::from_iter(generator::list_built_in_generator_functions());

    pub static ref BUILT_IN_TABLE_FUNCTIONS: HashMap<&'static str,
Arc<TableFunction>> =
        HashMap::from_iter(table::list_built_in_table_functions());
}

```

The scalar registry is assembled from many focused modules:

array
bitwise
collection
conditional
conversion
csv
datetime
geo
hash
json
lambda
map
math
misc
predicate
string
struct
url
variant
xml

This layout is worth copying in your mental model. There is no single monstrous function resolver where every implementation lives. Instead:

- `sail-plan` maps Spark-compatible names to expression builders.
- `sail-function` implements many custom DataFusion UDFs and UDAFs.
- `datafusion-spark` and DataFusion built-ins provide additional behavior.
- `sail-python-udf` implements Python-backed functions.

The aggregate registry works similarly, but aggregate functions often have to adapt Spark syntax to DataFusion's expected function shape.

For example, Spark's ordered-set syntax:

```
percentile_cont(0.5) WITHIN GROUP (ORDER BY col)
```

does not arrive in the same argument order DataFusion expects. Sail's aggregate resolver extracts the ordered column and percentile argument, then builds the DataFusion aggregate expression.

That pattern is common: Sail preserves Spark semantics while targeting DataFusion's execution model.

12.5 Function Resolution Order

Function calls are resolved in:

```
crates/sail-plan/src/resolver/expression/function.rs
```

The core method is:

```
resolve_expression_function(...)
```

The resolution order is deliberate:

1. Normalize the function name.
2. Extract keyword arguments from `NamedArgument` expressions.
3. Merge named arguments from SQL analyzer paths.
4. Reject duplicate keyword arguments.
5. Check the `CatalogManager` for registered functions.
6. Resolve and type-check arguments.
7. If a registered PySpark UDF exists, build a Python UDF expression.
8. Otherwise try Sail built-in scalar or generator functions.
9. Otherwise try Sail built-in aggregate functions.
10. Format the output expression name with `PlanService`.

The catalog check happens before built-ins because Spark Connect does not reliably set the `is_user_defined_function` flag. Sail chooses the behavior users expect: if a function was registered by name, it should be found.

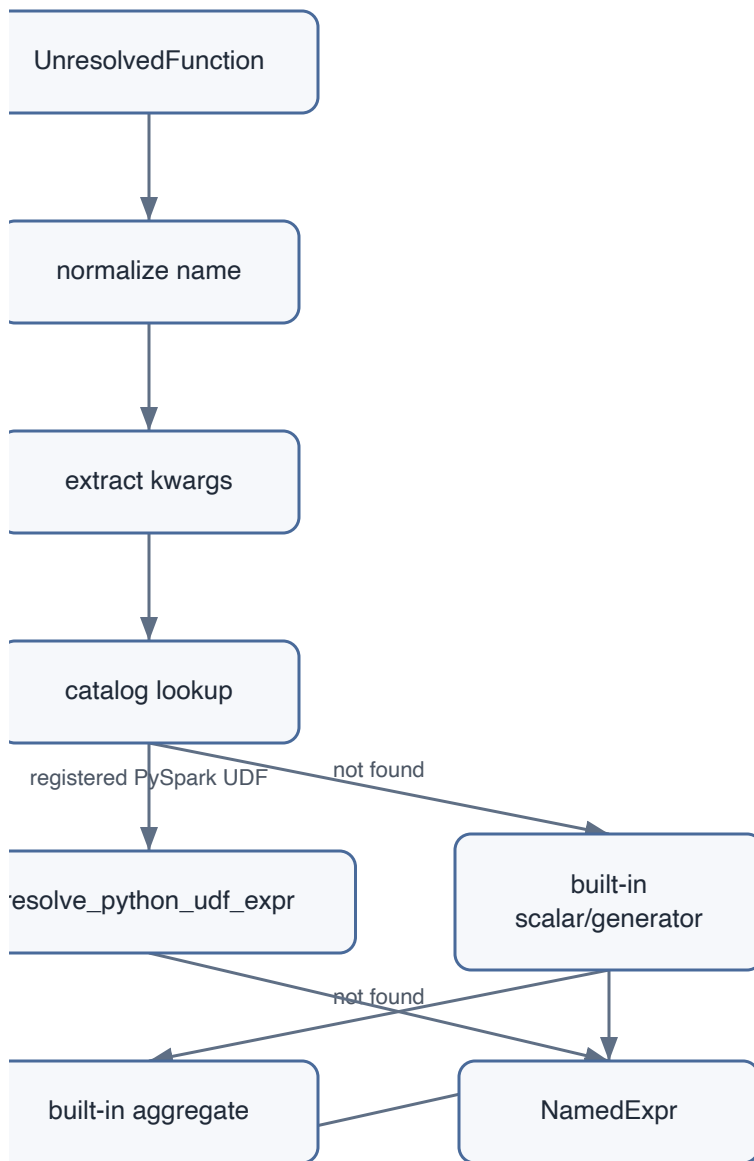


Figure 63: Flowchart 11.2

This is also where Sail handles Spark-specific display names. The expression object is DataFusion-compatible, but the output name should still look like Spark.

12.6 Registered PySpark UDFs

Registration commands are handled in:

```
crates/sail-plan/src/resolver/command/function.rs
```

For a scalar Python UDF registration, Sail:

1. Enters a resolver config scope.
2. Enables large Arrow variable types for UDF resolution.
3. Resolves the Python UDF definition and output type.
4. Wraps the unresolved Python function in `PySparkUnresolvedUDF`.
5. Tracks the function in `CatalogManager`.
6. Creates a catalog command logical plan.

Conceptually:

```
RegisterFunction command
-> PySparkUnresolvedUDF
-> CatalogManager::track_function
-> CatalogCommand::RegisterFunction
```

`CatalogManager` stores functions case-insensitively:

```
fn canonical_function_name(name: &str) -> Arc<str> {
    name.to_ascii_lowercase().into()
}
```

When the query later calls that name, the expression resolver finds it in the catalog and creates the real executable UDF expression with the correct input types.

That two-step model matters:

- Registration knows the Python command bytes and declared return type.
- Call-site resolution knows the actual argument expressions and input types.

You need both to build the runtime payload.

12.7 Inline PySpark UDFs

Spark Connect can also carry a UDF inline inside an expression:

```
spec::Expr::CommonInlineUserDefinedFunction
```

That path is handled by:

```
crates/sail-plan/src/resolver/expression/udf.rs
```

The resolver:

1. Extracts positional and keyword arguments.
2. Rejects duplicate kwargs.
3. Rejects positional arguments after keyword arguments.
4. Resolves argument expressions and display names.
5. Resolves the Python UDF definition.

6. Computes input Arrow data types from the resolved arguments.
7. Builds a PySpark worker-compatible payload.
8. Creates a DataFusion scalar or aggregate UDF expression.

The payload build step is the heart of the integration:

```
let payload = PySparkUdfPayload::build(
  &function.python_version,
  &function.command,
  function.eval_type,
  &arg_offsets,
  &input_types,
  kwarg_names,
  &self.config.pyspark_udf_config,
)?;
```

The output depends on the PySpark eval type.

PySpark eval type	Sail object
Batched	PySparkUDF with PySparkUdfKind::Batch
ArrowBatched	PySparkUDF with PySparkUdfKind::ArrowBatch
ScalarPandas	PySparkUDF with PySparkUdfKind::ScalarPandas
ScalarPandasIter	PySparkUDF with PySparkUdfKind::ScalarPandasIter
ScalarArrow	PySparkUDF with PySparkUdfKind::ScalarArrow
ScalarArrowIter	PySparkUDF with PySparkUdfKind::ScalarArrowIter
GroupedAggPandas	PySparkGroupAggregateUDF with Pandas mode
GroupedAggArrow	PySparkGroupAggregateUDF with Arrow mode

Unsupported eval types are rejected early if they do not make sense for a scalar inline UDF.

12.8 Scalar Python Execution

The executable scalar UDF object is PySparkUDF in:

crates/sail-python-udf/src/udf/pyspark_udf.rs

It implements DataFusion's ScalarUDFImpl.

Its fields include:

```

kind: PySparkUdfKind,
name: String,
payload: Vec<u8>,
deterministic: bool,
input_types: Vec<DataType>,
output_type: DataType,
config: Arc<PySparkUdfConfig>,
udf: LazyPyObject,

```

The LazyPyObject is important. The Python callable is not eagerly loaded when the plan is built. It is loaded when the UDF is invoked:

payload bytes

```

-> PySparkUdfPayload::load
-> PySpark read_udfs
-> PySpark wrapper function

```

At execution time, invoke_with_args():

1. Converts DataFusion ColumnarValue inputs into Arrow arrays.
2. Converts those arrays into Python/PyArrow values.
3. Calls the Python wrapper.
4. Converts returned Python data back into Arrow ArrayData.
5. Casts the result to the declared output type.
6. Returns a DataFusion ColumnarValue::Array.

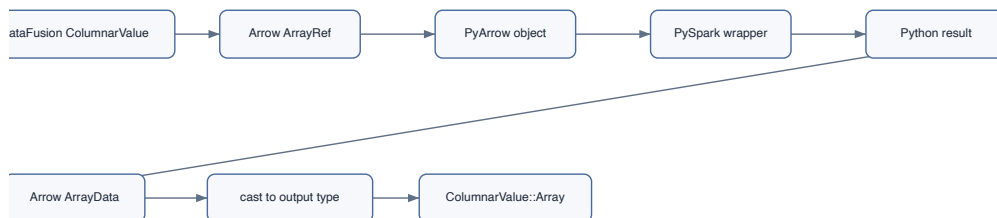


Figure 64: Flowchart 11.3

This is the practical meaning of “Arrow UDF” in Sail. Arrow is the boundary format between Rust execution and Python execution.

12.9 Grouped Aggregate Python UDFs

Grouped aggregate UDFs use:

```
crates/sail-python-udf/src/udf/pyspark_udaf.rs
```

The central object is PySparkGroupAggregateUDF, which implements AggregateUDFImpl.

It supports two modes:

```
pub enum PySparkGroupAggKind {
    Pandas,
    Arrow,
}
```

The accumulator path is important. DataFusion aggregate functions do not call a scalar function once per input batch. They create accumulators. Sail uses BatchAggregateAccumulator with a BatchAggregator implementation that calls Python over collected Arrow arrays.

The resolver also enforces a Spark analysis rule: aggregate UDF arguments cannot contain nested aggregate functions.

There is another small but revealing workaround: DataFusion requires at least one input to an aggregate function. For a zero-argument Python aggregate UDF, Sail injects a dummy Int64 literal and records the actual argument count separately so the Python function still receives the right argument list.

That is the kind of adapter code a compatibility engine accumulates. DataFusion and Spark are close enough to compose, but not identical.

12.10 UDTFs And Stream UDFs

Python table functions are not scalar expressions. They take input rows or batches and emit zero or more output rows. Sail models them as stream transformations.

The common trait is:

```
pub trait StreamUDF: DynObject + Debug + Send + Sync {
    fn name(&self) -> &str;
    fn output_schema(&self) -> SchemaRef;
    fn invoke(&self, input: SendableRecordBatchStream) ->
    Result<SendableRecordBatchStream>;
}
```

That trait lives in:

crates/sail-common-datafusion/src/udf.rs

PySparkUDTF implements StreamUDF in:

crates/sail-python-udf/src/udf/pyspark_udtf.rs

The resolver creates a MapPartitionsNode, which the physical planner turns into MapPartitionsExec.

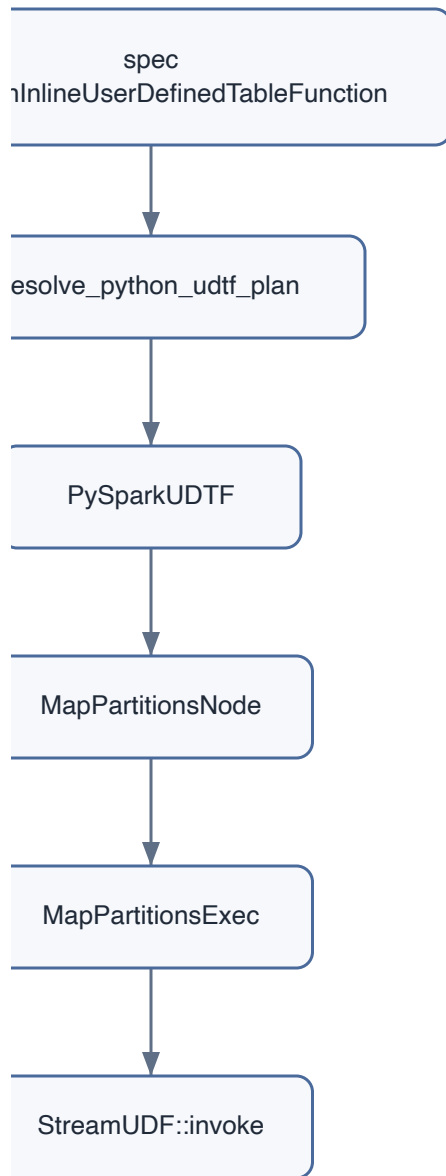


Figure 65: Flowchart 11.4

MapPartitionsExec is simple:

1. Execute the input physical plan for one partition.
2. Pass the resulting RecordBatch stream to the StreamUDF.
3. Wrap the output stream with the expected schema.

That is exactly the right abstraction for UDTFs. A UDTF is not “one input value -> one output value.” It is “one stream partition -> another stream partition.”

12.11 The Python Stream Bridge

The stream bridge lives in:

```
crates/sail-python-udf/src/stream.rs
```

PyMapStream converts a Rust RecordBatch stream into a Python iterator of PyArrow batches, then converts the Python output iterator back into a Rust RecordBatch stream.

The bridge uses a separate thread:

```
Rust input stream
-> PyInputStream.__next__
-> Python function iterator
-> output channel
-> Rust RecordBatchStream
```

The separate thread exists because the Python iterator performs blocking calls into a Tokio stream. The bridge uses a stop signal so the Rust side can tell the Python input iterator to stop.

The output path:

1. Calls the Python function with the input iterator.
2. Iterates over Python output batches.
3. Ignores empty batches in compatibility-sensitive cases.
4. Converts PyArrow batches to Arrow RecordBatch.
5. Casts batches positionally to the declared output schema.
6. Sends them through a channel to the Rust stream.

This is a useful pattern for any extension that needs to cross a runtime boundary: make the boundary stream-shaped, schema-aware, and cancellation-aware.

12.12 The Remote Execution Codec

So far, we have talked about resolving and executing functions in one process. But Sail has a distributed runtime. The driver builds a physical plan. Workers execute tasks. Workers must reconstruct every custom physical plan node and every custom function that appears inside physical expressions.

That is the job of RemoteExecutionCodec:

```
crates/sail-execution/src/codec.rs
```

It implements DataFusion’s PhysicalExtensionCodec.

The codec handles:

- extended physical plan nodes,
- extended physical expressions,
- scalar UDFs,
- aggregate UDFs,
- window UDFs,
- stream UDFs,
- schemas,
- data types,
- scalar values,
- partitioning,
- statistics,
- file scan configs,
- Sail lakehouse execution nodes,
- Python UDF configuration and payloads.

The protobuf definitions live in:

```
crates/sail-execution/proto/sail/plan/physical.proto
```

The file starts with a telling comment: DataFusion data structures are often stored as opaque bytes because DataFusion’s protobuf definitions can change. Sail uses its own extended protobuf messages for Sail-specific nodes and wraps DataFusion’s protobuf encoding where needed.

12.13 Encoding Scalar UDFs

Scalar UDF encoding happens in `try_encode_udf()`.

The current design has two broad paths:

1. If the UDF is a known Sail/DataFusion built-in that the worker can reconstruct by name, encode it as `StandardUdf`.
2. If the UDF carries state or custom configuration, encode the required fields into a specific protobuf variant.

Examples of stateful scalar UDF variants:

```
PySparkUdf
PySparkCoGroupMapUdf
DropStructFieldUdf
ExplodeUdf
SparkUnixTimestampUdf
StructFunctionUdf
ArraysZipUdf
UpdateStructFieldUdf
TimestampNowUdf
SparkTimestampUdf
SparkDateUdf
SparkFromCsvUdf
SparkFromJsonUdf
...
```

For PySparkUDF, the codec stores:

```
kind
name
payload
deterministic
input_types
output_type
config
```

That is enough for a worker to reconstruct:

```
PySparkUDF::new(
  kind,
  name,
  payload,
  deterministic,
  input_types,
  output_type,
  Arc::new(config),
)
```

For a UDF like StructFunction, the codec only needs the field names. For a UDF like SparkTimestamp, it needs timezone and is_try. The encoding shape follows the state needed to reconstruct the function.

12.14 Decoding Scalar UDFs

Decoding happens in `try_decode_udf()`.

The codec receives a function name and an extension buffer. If the extension buffer is `StandardUdf`, it reconstructs the function by matching on the name:

```
"spark_array" -> SparkArray::new()
"spark_split" -> SparkSplit::new()
"spark_xxhash64" -> SparkXxhash64::new()
...
```

If the buffer contains a richer variant, the codec decodes the fields and constructs the object directly.

That distinction is a bit manual today. The code even has a TODO:

Implement custom registry to avoid codec for built-in functions

This is another bright signpost for discussion #2001. A third-party extension should not need to patch a giant match statement in `RemoteExecutionCodec` just to make a custom function work on workers.

12.15 Encoding Aggregate And Window UDFs

Aggregate UDFs follow the same idea.

Known standard aggregate UDFs encode as:

StandardUdaf

Then the worker decodes by name:

```
bitmap_and_agg
histogram_numeric
kurtosis
max_by
mode
percentile
product
try_avg
try_sum
...
```

Python grouped aggregate UDFs carry their payload, input names, input types, output type, deterministic flag, kind, config, and actual argument count.

Window UDF support is narrower. The codec has `ExtendedWindowUdf`, and currently the standard custom path includes `ntile` through `SparkNtile`.

The broader lesson is the same: every non-standard callable object needs a worker-side reconstruction story.

12.16 Encoding Stream UDFs

Stream UDFs are not DataFusion scalar expressions, so they have their own codec path:

```
fn try_encode_stream_udf(&self, udf: &dyn StreamUDF) -> Result<ExtendedStreamUdf>
fn try_decode_stream_udf(&self, udf: ExtendedStreamUdf) -> Result<Arc<dyn
StreamUDF>>
```

The current variants include:

```
PySparkMapIterUdf
PySparkUdtf
```

For `PySparkUDTF`, the codec stores:

```
kind
name
payload
input_names
input_types
passthrough_columns
function_return_type
function_output_names
deterministic
config
```

That mirrors the constructor for `PySparkUDTF::try_new()`.

This design is clean in one important way: stream UDFs are encoded with the physical operator that uses them. `MapPartitionsExec` does not need to know how a Python UDTF works. It only needs a `StreamUDF`.

12.17 Worker Sessions And Built-Ins

The server session factory deliberately does not add all DataFusion default features:

We do not add default features to the session state, since we manage table formats and functions ourselves.

But the worker session factory does add default features:

We still add default features for the worker session since we need built-in functions to be available for the codec when decoding the execution plan.

That comment is small but important. It tells us that decoding a physical plan is not only a bytes-to-struct operation. It may depend on what functions and features are registered in the worker's `SessionState`.

This is one of the hard requirements for extensions:

The driver and every worker must agree on the callable universe.

If the driver can plan a function the worker cannot decode, the query fails at task startup. If the worker can decode but not execute the function, the query fails during batch execution.

12.18 Distributed Example: Built-In Function

Consider:

```
SELECT xxhash64(customer_id)
FROM orders
```

The path is:

1. Spark Connect or SQL creates `spec::Expr::UnresolvedFunction`.
2. The resolver normalizes the function name.
3. The built-in scalar function registry maps it to a Spark-compatible expression.
4. DataFusion physical planning creates a physical expression containing a UDF.
5. The job graph planner splits the plan into distributed stages if needed.
6. The task definition serializes the physical plan with `RemoteExecutionCodec`.
7. The worker decodes the UDF as a known standard UDF by name.
8. The worker executes the function against Arrow arrays.

The function itself may feel local, but in cluster mode it has to survive this trip:

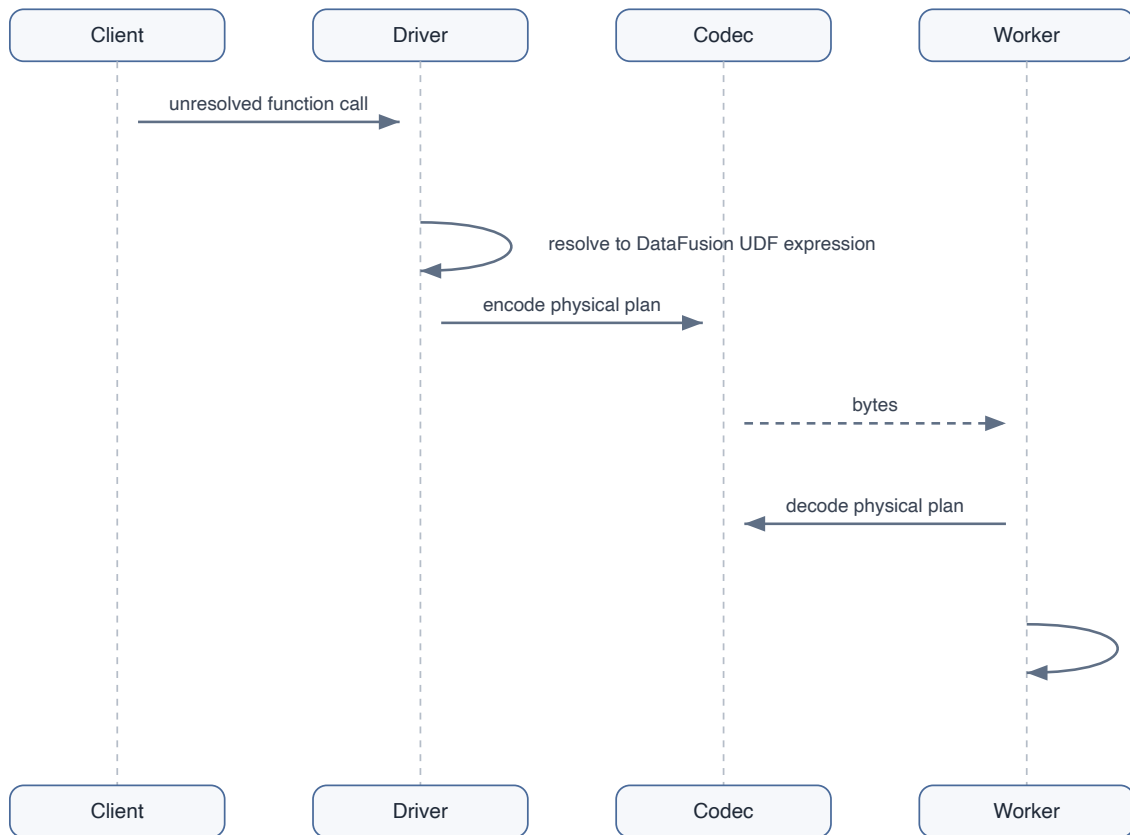


Figure 66: Sequence diagram 11.5

12.19 Distributed Example: Python Scalar UDF

Now consider:

```

@udf("long")
def plus_one(x):
    return x + 1

df.select(plus_one("amount"))
  
```

The path is richer:

1. PySpark serializes the Python function command.
2. Spark Connect sends the UDF information to Sail.
3. Sail stores or resolves the UDF as `CommonInlineUserDefinedFunction`.
4. The resolver computes input Arrow types from the call site.
5. `PySparkUdfPayload::build()` writes a PySpark-compatible payload:
 - eval type,
 - config,
 - optional input type JSON for certain PySpark versions,

- profiling flag for PySpark 4,
 - argument offsets,
 - keyword argument names,
 - serialized command bytes.
6. Sail creates PySparkUDF.
 7. The physical plan is encoded.
 8. The codec encodes kind, payload, input types, output type, and config.
 9. The worker decodes PySparkUDF.
 10. At execution time, the UDF loads the Python callable lazily.
 11. Arrow arrays cross into Python.
 12. Python returns Arrow-compatible data.
 13. Sail casts the result to the declared output type.

That is a lot of machinery, but each part has a job:

```
spec      -> preserve user intent and Python command
resolver  -> bind arguments and types
UDF object -> implement DataFusion execution
codec     -> move the object to workers
Python    -> run user code over Arrow-shaped data
```

12.20 Distributed Example: Python UDTF

A UDTF is stream-shaped:

```
class SplitWords:
    def eval(self, text):
        for word in text.split():
            yield (word,)
```

The Sail path is:

1. Resolve the UDTF definition and arguments.
2. Determine the return type. If no static return type exists, call the Python `analyze` method during query analysis.
3. Build a `PySparkUdtfPayload`.
4. Create a `PySparkUDTF` stream UDF.
5. Build a `MapPartitionsNode`.
6. Physical planning creates `MapPartitionsExec`.
7. The codec encodes both the physical operator and its stream UDF.
8. The worker decodes `MapPartitionsExec` and `PySparkUDTF`.
9. At runtime, the input partition stream is passed into Python.
10. Python yields output batches, and Rust turns them back into `RecordBatch` streams.

Diagram:

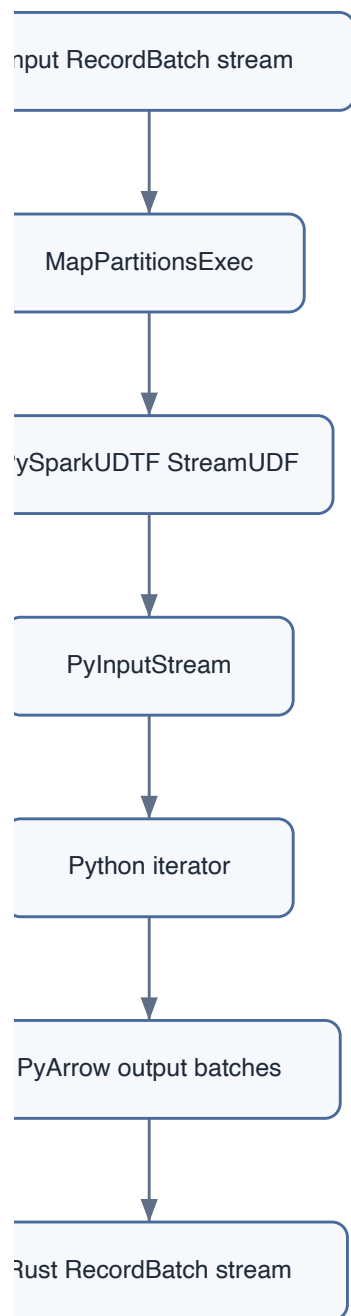


Figure 67: Flowchart 11.6

This is the same distributed principle again: the operator and the callable object must both be serializable.

12.21 Codec Design Lessons

`RemoteExecutionCodec` is not glamorous code, but it is the backbone of cluster execution.

It teaches several lessons:

First, function identity is not enough. Some functions need state:

- timezone,
- field names,
- ANSI mode,
- safe/try flags,
- input types,
- output types,
- Python payload bytes,
- PySpark config,
- passthrough column counts.

Second, the worker must reconstruct the same behavior, not merely a function with the same name.

Third, built-ins and extensions need different handling. Built-ins can sometimes be reconstructed by name. Extension functions need explicit registration and encoding.

Fourth, codecs are versioned contracts even when no version field is visible. If you change a function's fields, you have changed the bytes a worker expects.

Fifth, `DataFusion`'s extension codec hooks are exactly the right place to integrate custom behavior, but `Sail` needs a registry around them to avoid central matches.

12.22 Extension Implications

For discussion #2001, functions and codecs expose the sharpest edge of the design.

A third-party extension may want to add:

- a scalar UDF,
- an aggregate UDF,
- a window UDF,
- a table function,
- a stream UDF,
- a physical expression,
- a physical operator,
- a logical optimizer rewrite,
- a Spark Connect custom expression,
- a Python package that registers all of the above.

To work in local mode, registering a `DataFusion` UDF may be enough.

To work in cluster mode, that is not enough.

A distributed extension needs:

Need	Why
Logical name registration	So the resolver can bind function calls.
Type inference	So the logical plan can be checked and named.
Execution implementation	So DataFusion can evaluate the function.
Physical encoding	So the driver can serialize plans.
Physical decoding	So workers can reconstruct plans.
Worker installation	So the implementation exists in worker processes.
Version compatibility	So encoded bytes match decoder expectations.
Collision policy	So two extensions cannot silently claim the same name.
Ordering policy	So optimizer and planner hooks run deterministically.

This is the difference between a plugin that works in a notebook and an extension that works in a distributed query engine.

Everything in this section concerns what chapter 13 calls the *execution-time boundary*: the work that happens once per batch on a worker. A separate *plan-time boundary* - how user intent enters Sail in the first place - has its own ABI story. Chapter 13 routes plan-time intent through Spark Connect's `Relation.extension`, `Command.extension`, and `Expression.extension` messages and uses the codec mechanism below only for execution-time concerns. The two boundaries can ship independently, and a Pattern A extension (one that decomposes to existing DataFusion operators) skips the codec work entirely.

12.23 A Proposed Extension Codec Registry

The current codec knows about Sail's built-ins through downcasts and name matches. That is fine for core code, but third-party extensions need a more open shape.

One possible design:

```
pub trait FunctionCodec: Send + Sync {
    fn type_url(&self) -> &'static str;

    fn encode_scalar_udf(&self, udf: &ScalarUDF) -> Option<PlanResult<Vec<u8>>>;
    fn decode_scalar_udf(&self, name: &str, bytes: &[u8]) ->
Option<PlanResult<Arc<ScalarUDF>>>;

    fn encode_aggregate_udf(&self, udf: &AggregateUDF) ->
Option<PlanResult<Vec<u8>>>;
```

```

    fn decode_aggregate_udf(&self, name: &str, bytes: &[u8]) ->
Option<PlanResult<Arc<AggregateUDF>>>>;

    fn encode_stream_udf(&self, udf: &dyn StreamUDF) ->
Option<PlanResult<Vec<u8>>>>;
    fn decode_stream_udf(&self, bytes: &[u8]) -> Option<PlanResult<Arc<dyn
StreamUDF>>>>;
}

```

The actual API could be different, but the design goal is clear:

Core codec dispatches to registered extension codecs.
Extension codecs own their wire format.
Workers and drivers register the same codecs.

The protobuf could use a generic extension envelope:

```

message ExtensionFunction {
    string provider = 1;
    string name = 2;
    string version = 3;
    bytes payload = 4;
}

```

Then an extension like a geospatial package could encode its own UDFs without editing Sail's central codec every time.

12.24 A Proposed Function Registration Model

The function side also wants a registry that separates names from implementations:

```

pub trait SailFunctionExtension: Send + Sync {
    fn name(&self) -> &'static str;

    fn register_scalar_functions(&self, registry: &mut ScalarFunctionRegistry) ->
PlanResult<()>;
    fn register_aggregate_functions(&self, registry: &mut
AggregateFunctionRegistry) -> PlanResult<()>;
    fn register_table_functions(&self, registry: &mut TableFunctionRegistry) ->
PlanResult<()>;
    fn register_stream_functions(&self, registry: &mut StreamFunctionRegistry) ->
PlanResult<()>;
    fn register_codecs(&self, registry: &mut CodecRegistry) -> PlanResult<()>;
}

```

This lets Sail enforce:

- duplicate-name errors,
- deterministic registration order,
- per-session enablement,
- worker compatibility checks,
- explain output that names which extension supplied a function.

The key design rule is that registration must happen on both driver and worker sessions. Otherwise distributed execution becomes a coin toss.

12.25 Reading Exercise

Trace this query:

```
df.select(my_udf("x"))
```

Suggested path:

1. Start with `CommonInlineUserDefinedFunction` in `crates/sail-common/src/spec/expression.rs`.
2. Follow `resolve_expression_common_inline_udf()` in `crates/sail-plan/src/resolver/expression/udf.rs`.
3. Watch how input types are computed from resolved arguments.
4. Open `PySparkUdfPayload::build()` in `crates/sail-python-udf/src/cereal/pyspark_udf.rs`.
5. Follow the creation of `PySparkUDF`.
6. Open `PySparkUDF::invoke_with_args()`.
7. Then jump to `RemoteExecutionCodec::try_encode_udf()`.
8. Follow `UdfKind::PySpark`.
9. Read `RemoteExecutionCodec::try_decode_udf()`.
10. Confirm that the worker reconstructs the same `PySparkUDF`.

The key question:

What data must cross the driver-worker boundary for this function to behave the same on the worker as it did in the driver's plan?

That question is the whole chapter in miniature.

12.26 Takeaways

Functions in Sail are distributed execution contracts:

- Built-ins are resolved through Sail's Spark-compatible function maps.
- Registered PySpark UDFs are stored in the catalog and materialized at call sites.
- Inline PySpark UDFs carry command bytes directly in the spec expression.
- Scalar Python UDFs implement `DataFusion ScalarUDFImpl`.
- Grouped Python aggregate UDFs implement `DataFusion AggregateUDFImpl`.
- Python UDTFs use Sail's `StreamUDF` abstraction and `MapPartitionsExec`.
- Arrow arrays and record batches are the runtime boundary between Rust and Python.
- `RemoteExecutionCodec` makes custom plans and functions executable on workers.
- Extension proposal #2001 must include codec, registration, and worker compatibility stories, not only a way to add names to a function map.

The next chapter moves from callable behavior to tables: catalogs, table formats, lakehouse scans and writes, and how file and table providers cross the Sail/DataFusion boundary.

13 Chapter 12: Catalogs, Lakehouse Tables, And File Formats

So far, the book has followed queries from Spark Connect through Sail specs, DataFusion logical plans, distributed physical plans, tasks, streams, shuffles, and functions. This chapter turns toward storage.

Storage in Sail is not a single subsystem. It is a set of contracts:

- catalogs answer “what is this name?”
- table metadata answers “what schema, location, format, partitioning, and properties does it have?”
- table formats answer “how do I read or write this storage layout?”
- physical planners answer “what executable DataFusion plan should do the work?”

That separation is one of the most important architectural lessons in Sail. It lets Spark-compatible commands talk to Hive Metastore, Glue, Unity, Iceberg REST, OneLake, memory catalogs, ordinary files, Delta Lake, Iceberg tables, and Python data sources without forcing all of those concepts into one giant table abstraction.

The short version is:

```
Spark table name or data source
-> CatalogManager or direct format lookup
-> TableStatus / SourceInfo / SinkInfo
-> TableFormatRegistry
-> DataFusion TableSource or ExecutionPlan
```

The long version is this chapter.

13.1 Code Map

The main files for this chapter are:

Concern	File
Catalog manager	<code>crates/sail-catalog/src/manager/mod.rs</code>
Catalog table/view status	<code>crates/sail-common-datafusion/src/catalog/status.rs</code>
Catalog command enum	<code>crates/sail-catalog/src/command.rs</code>
Catalog command physical exec	<code>crates/sail-physical-plan/src/catalog_command.rs</code>
Session catalog construction	<code>crates/sail-session/src/catalog.rs</code>
Table format trait and registry	<code>crates/sail-common-datafusion/src/datasource.rs</code>
Session table format registration	<code>crates/sail-session/src/formats.rs</code>
Named table and data source reads	<code>crates/sail-plan/src/resolver/query/read.rs</code>
Write command resolution	<code>crates/sail-plan/src/resolver/command/write.rs</code>
Logical file write node	<code>crates/sail-logical-plan/src/file_write.rs</code>
Physical file write planning	<code>crates/sail-physical-plan/src/file_write.rs</code>
Logical/physical delete planning	<code>crates/sail-logical-plan/src/file_delete.rs</code> , <code>crates/sail-physical-plan/src/file_delete.rs</code>
Generic listing table formats	<code>crates/sail-data-source/src/listing/source.rs</code>
Parquet format example	<code>crates/sail-data-source/src/formats/parquet/mod.rs</code>
Delta table format	<code>crates/sail-delta-lake/src/table_format.rs</code>
Iceberg table format	<code>crates/sail-iceberg/src/table_format.rs</code>
Lakehouse extension planners	<code>crates/sail-plan-lakehouse/src/lib.rs</code>
Python data source table format	<code>crates/sail-data-source/src/formats/python/table_format.rs</code>

13.2 The Storage Boundary

Sail has to preserve Spark behavior while using DataFusion as the execution kernel. That means it cannot simply expose DataFusion’s catalog model directly to clients. Spark has its own rules for:

- one-part, two-part, and three-part names,

- current catalog and current database,
- temporary and global temporary views,
- `USING` `parquet`, `USING` `delta`, and `spark.read.format(...)`,
- save modes such as `append`, `overwrite`, `ignore`, and `error-if-exists`,
- table properties and data source options,
- time travel syntax,
- row-level commands such as `DELETE` and `MERGE`.

Sail translates that world into a smaller set of internal contracts.

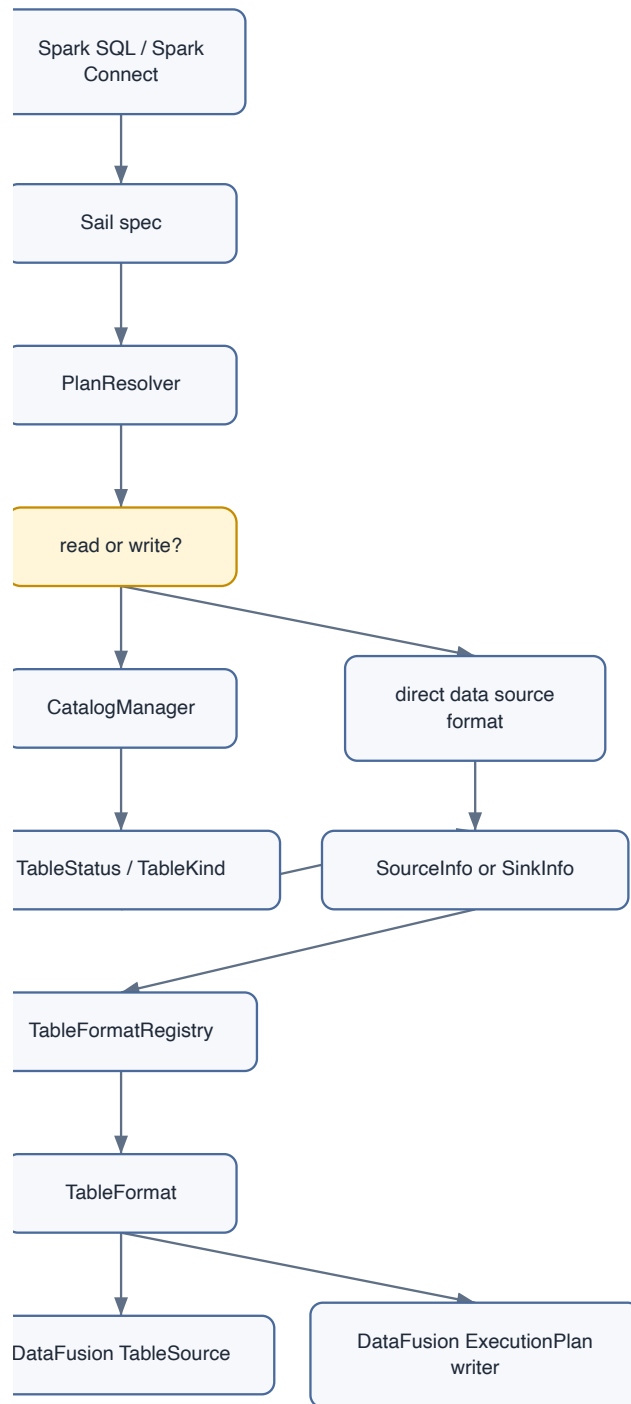


Figure 68: Flowchart 12.1

The table format layer is where Arrow and DataFusion become visible again. Reads produce a `TableSource`, which DataFusion can scan. Writes produce an `ExecutionPlan`, which DataFusion can run.

13.3 Catalogs: Names Before Data

The `CatalogManager` in `crates/sail-catalog/src/manager/mod.rs` is a session extension. It owns:

- the configured catalog providers,
- the default catalog,
- the default database,
- the global temporary database,
- temporary views,
- registered functions,
- tracked logical plans and function objects.

Its most important job is name resolution. A query like this:

```
SELECT * FROM sales.orders
```

does not say whether `sales` is a catalog or a database. Sail follows Spark-style resolution:

```
[name]
  -> default catalog + default database + table

[prefix..., table]
  -> if prefix starts with a known catalog, use that catalog
  -> otherwise use default catalog and treat prefix as database
```

The result of resolution is not data. It is metadata: a `TableStatus`. The table can be a physical table, a view, a temporary view, or a global temporary view.

```
pub enum TableKind {
    Table { ... },
    View { ... },
    TemporaryView { ... },
    GlobalTemporaryView { ... },
}
```

That enum is small, but it carries a lot of Spark compatibility:

Kind	What Sail does with it
Table	Builds a table scan using the table's format, location, schema, properties, and partitioning.
View	Parses the stored SQL definition and resolves it again into a logical plan.
TemporaryView	Reuses a stored logical plan from the session.
GlobalTemporaryView	Reuses a stored logical plan from the configured global temporary database.

This is why catalogs come before file formats. Sail cannot decide whether to invoke Parquet, Delta, Iceberg, or Python data source code until it knows what the name refers to.

13.4 Catalog Providers

Session startup builds the catalog manager in `crates/sail-session/src/catalog.rs`. The configured catalog list can include:

- memory catalog,
- Iceberg REST catalog,
- Unity catalog,
- OneLake catalog,
- Glue catalog,
- Hive Metastore catalog,
- the built-in system catalog.

Some catalog providers are wrapped in `RuntimeAwareCatalogProvider`, which lets them perform blocking or IO-heavy setup on the runtime intended for IO. Some are wrapped in `CachingCatalogProvider`, depending on whether the configuration asks for global or session cache behavior.

The key Rust idea here is trait objects:

```
HashMap<Arc<str>, Arc<dyn CatalogProvider>>
```

Sail does not need every catalog backend to have the same concrete Rust type. It needs each backend to implement the `CatalogProvider` contract.

That same shape reappears for table formats:

```
HashMap<String, Arc<dyn TableFormat>>
```

This is a pattern worth remembering for extension design: Sail favors small traits plus session registries over large enums that must know every implementation.

13.5 Table Formats

The central storage interface is `TableFormat` in `crates/sail-common-datafusion/src/datasource.rs`.

Its core methods are:

```
#[async_trait]
pub trait TableFormat: Debug + Send + Sync {
    fn name(&self) -> &str;

    async fn create_source(
        &self,
        ctx: &dyn Session,
        info: SourceInfo,
    ) -> Result<Arc<dyn TableSource>>;

    async fn infer_schema(
        &self,
        ctx: &dyn Session,
        info: SourceInfo,
    ) -> Result<SchemaRef>;

    async fn create_writer(
        &self,
        ctx: &dyn Session,
        info: SinkInfo,
    ) -> Result<Arc<dyn ExecutionPlan>>;
}
```

There are extra hooks for row-level writes and table alteration:

```
async fn create_row_level_writer(...)
async fn alter_table_properties(...)
async fn alter_table_column_type(...)
fn merge_strategy(&self) -> MergeStrategy
```

This interface tells us exactly where DataFusion sits in the storage architecture:

- reads become Arc<dyn TableSource>;
- writes become Arc<dyn ExecutionPlan>;
- schema inference returns Arrow SchemaRef;
- format-specific behavior stays behind the trait.

The registry is intentionally simple:

```
pub struct TableFormatRegistry {
    formats: RwLock<HashMap<String, Arc<dyn TableFormat>>>,
}
```

Names are lowercased at registration and lookup time. That makes format("parquet"), USING PARQUET, and mixed-case user input converge on the same format implementation.

13.6 Session Format Registration

crates/sail-session/src/formats.rs builds the table format registry for each server session.

Built-in formats include:


```
arrow
avro
binary
csv
json
parquet
text
socket
rate
console
noop
```

External formats are registered after the built-ins:

```
delta
iceberg
discovered Python data sources
```

This matters for discussion #2001. Sail already has a working registry pattern for one important category of extension. The last chapter will generalize that lesson: a third-party extension should be able to contribute functions, optimizer rules, physical planners, codecs, table formats, and perhaps catalog providers through a unified registration story.

13.7 SourceInfo: A Read Request In One Struct

Reads pass through SourceInfo:

```
pub struct SourceInfo {
    pub paths: Vec<String>,
    pub schema: Option<Schema>,
    pub constraints: Constraints,
    pub partition_by: Vec<String>,
    pub bucket_by: Option<BucketBy>,
    pub sort_order: Vec<Vec<Sort>>,
    pub options: Vec<OptionLayer>,
}
```

This struct is the bridge from Spark concepts to a format-specific reader. It can represent:

- a direct `spark.read.format("parquet").load(path)`,
- a SQL table with catalog metadata,
- a path-based table reference such as `delta./tmp/events`,
- time-travel options,
- partition information,
- table properties and user options.

The interesting field is `options: Vec<OptionLayer>`.

```
pub enum OptionLayer {
    TablePropertyList { items: Vec<(String, String)> },
    OptionList { items: Vec<(String, String)> },
    TableLocation { value: String },
}
```

```
AsOfTimestamp { value: String },
AsOfIntegerVersion { value: i64 },
AsOfStringVersion { value: String },
}
```

An option is not just a string map because not all options have the same meaning. Sail needs to preserve the difference between:

- catalog table properties,
- user-provided data source options,
- a table location,
- timestamp or version time travel.

Older or simpler data sources can collapse layers into opaque options with `into_opaque_options()`. Lakehouse sources can interpret the layers more precisely.

13.8 Reading A Named Table

The main read path is `resolve_query_read_named_table()` in `crates/sail-plan/src/resolver/query/read.rs`.

It has several branches:

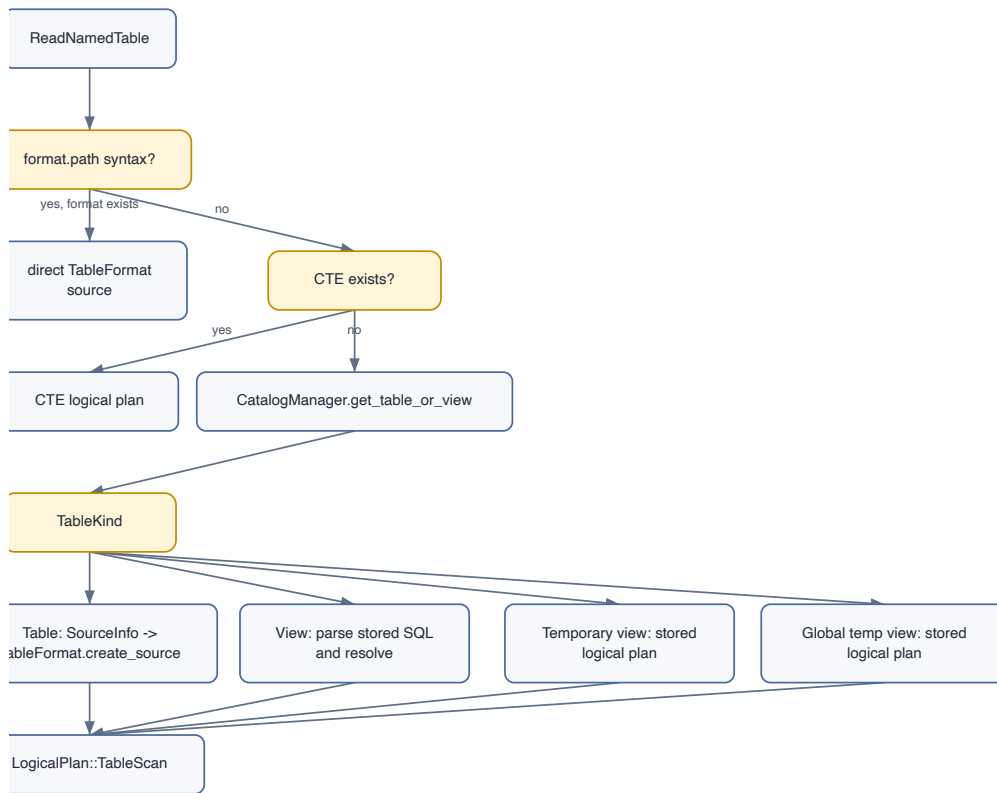


Figure 69: Flowchart 12.2

The direct format branch supports Spark-style path tables:

```

SELECT * FROM parquet.`/tmp/orders`
SELECT * FROM delta.`/lake/events` VERSION AS OF 12

```

If the prefix is a registered format, Sail does not ask the catalog. It builds SourceInfo directly from the path and options, then calls:

```
registry.get(format)?.create_source(&ctx.state(), info).await
```

For catalog tables, Sail reads TableKind::Table metadata:

- columns become an Arrow schema,
- constraints become DataFusion constraints,
- location becomes a path,
- format selects the table format,
- partitioning, bucketing, and sorting are preserved,
- table properties and user options become layered options.

The output is a `DataFusion LogicalPlan::TableScan`.

There is a subtle Spark-compatibility detail after source creation: `resolve_table_source_with_rename()` handles duplicate column names and stored column names. DataFusion's normal schema assumptions do not always match Spark's tolerance for duplicate or case-insensitive field names, so Sail wraps or renames where needed.

13.9 Reading A Data Source

The data source read path is simpler. `resolve_query_read_data_source()` handles queries that already name a format explicitly:

```
df = spark.read.format("json").schema(schema).option("multiLine",
"true").load(path)
```

The resolver:

1. requires a format name,
2. resolves the optional schema,
3. builds `SourceInfo` from paths, schema, and options,
4. looks up the table format,
5. asks it for a `TableSource`,
6. turns that into an unnamed table scan.

No catalog lookup is necessary.

13.10 Listing Formats: Parquet As The Normal Case

Most ordinary file formats use `ListingTableFormat<T>` in `crates/sail-data-source/src/listing/source.rs`.

Parquet is a good example:

```
pub type ParquetTableFormat = ListingTableFormat<ParquetFormatFactory>;
```

The factory creates a read format and write format:

```
impl FormatFactory for ParquetFormatFactory {
    type Read = ParquetReadFormat;
    type Write = ParquetWriteFormat;

    fn name() -> &'static str { "parquet" }
    fn read(...) -> Result<Self::Read> { ... }
    fn write(...) -> Result<Self::Write> { ... }
}
```

For reads, `ListingTableFormat`:

- resolves paths into `ListingTableUrls`,
- creates a `DataFusion FileFormat`,
- infers schema if the caller did not provide one,
- discovers partition columns from `key=value` path segments,
- builds `ListingOptions`,

- creates a DataFusion ListingTable,
- wraps it as a TableSource.

For writes, it:

- finds the output path in options,
- rejects unsupported bucketing and partition transforms,
- creates the format-specific writer,
- builds a DataFusion FileSinkConfig,
- calls `create_writer_physical_plan()`.

So ordinary file formats are thin adapters around DataFusion’s listing table and file writer machinery. Sail adds Spark option handling, path behavior, partition discovery, and compatibility checks.

13.11 SinkInfo: A Write Request In One Struct

Writes pass through SinkInfo:

```
pub struct SinkInfo {
    pub input: Arc<dyn ExecutionPlan>,
    pub mode: PhysicalSinkMode,
    pub partition_by: Vec<CatalogPartitionField>,
    pub bucket_by: Option<BucketBy>,
    pub sort_order: Option<LexRequirement>,
    pub options: Vec<OptionLayer>,
    pub logical_schema: Option<DFSchemaRef>,
}
```

The split between SinkMode and PhysicalSinkMode is important.

At logical planning time, overwrite-by-condition can still carry a logical DataFusion expression:

```
SinkMode::OverwriteIf { condition }
```

At physical planning time, Sail preserves both the expression and the original SQL source string:

```
PhysicalSinkMode::OverwriteIf {
    condition: Some(condition),
    source,
}
```

The `logical_schema` field is also important. Physical planning can lose Arrow field metadata. Delta generated columns need that metadata, so Sail carries the logical schema down to the writer.

13.12 Write Resolution

`crates/sail-plan/src/resolver/command/write.rs` is the central write resolver. It uses `WritePlanBuilder` to collect:

- target: data source or catalog table,
- mode: error, ignore, append, replace, truncate, conditional truncate, partition truncate,
- format,

- partitioning,
- bucketing,
- sorting,
- options,
- table properties,
- external-table flag.

The output is usually:

```
BarrierNode(  
  preconditions = catalog commands, if needed  
  plan = FileWriteNode(input, FileWriteOptions)  
)
```

The barrier is how Sail sequences catalog-side effects before the data write. For example, a `CREATE TABLE AS SELECT` may need to create or replace catalog metadata before the file writer runs.

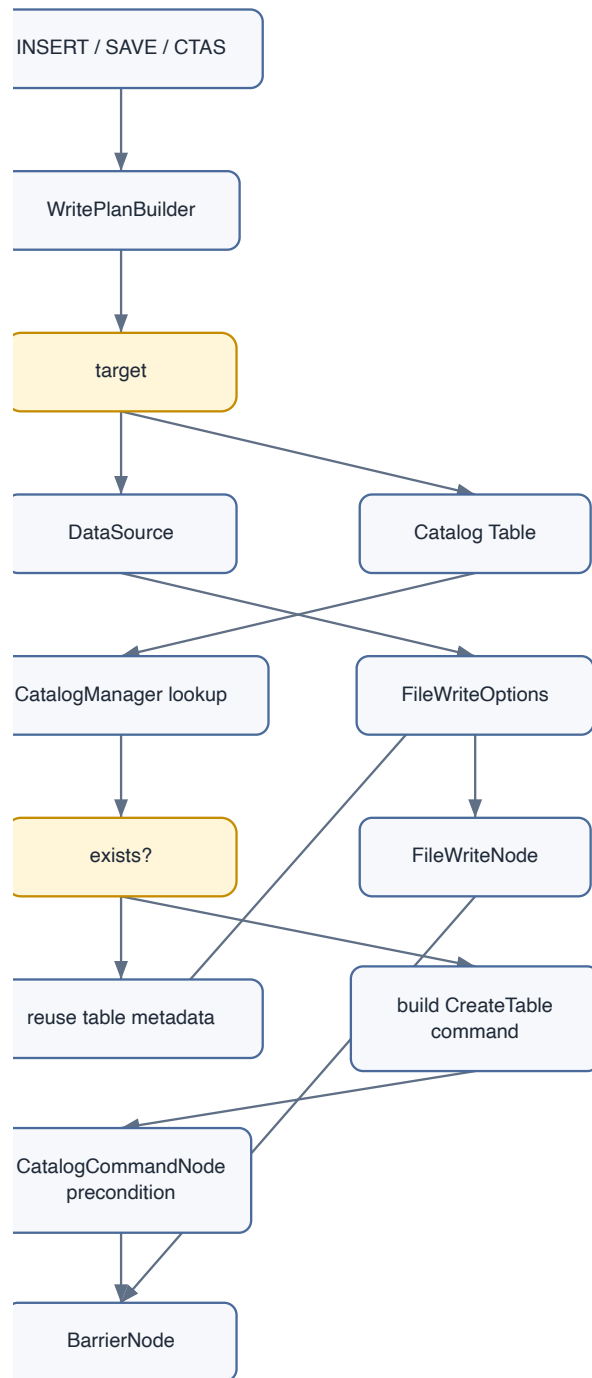


Figure 70: Flowchart 12.3

For existing catalog tables, Sail inherits stored metadata:

- location,
- format,
- partition fields,
- sort order,
- bucket spec,
- table properties.

For new tables, Sail constructs a `CatalogCommand::CreateTable` with the input schema and desired metadata.

13.13 Column Matching And Generated Columns

Spark has multiple write column matching modes:

- by position,
- by name,
- by an explicit column list.

Sail rewrites the input projection accordingly before building the file write node. That means the storage writer receives batches in table order, with casts and aliases already inserted.

Generated columns make this more interesting. Delta can store generation expression metadata on Arrow fields. Sail's write resolver:

1. detects generated columns from field metadata,
2. allows missing generated columns in user input,
3. computes generated expressions from the provided columns,
4. checks user-provided generated values when present,
5. attaches generation metadata to output aliases.

That logic lives before physical writing because it is relational expression work. The Delta writer should receive a plan whose output already satisfies generated column semantics.

13.14 FileWriteNode And Physical Planning

`FileWriteNode` in `crates/sail-logical-plan/src/file_write.rs` is a custom `DataFusion` logical extension node. It carries:

```
pub struct FileWriteOptions {
    pub format: String,
    pub mode: SinkMode,
    pub partition_by: Vec<CatalogPartitionField>,
    pub sort_by: Vec<Sort>,
    pub bucket_by: Option<BucketBy>,
    pub options: Vec<OptionLayer>,
}
```

It has one logical input: the query whose rows should be written.

The physical planner handles it in two places:

- the lakehouse planner intercepts Delta/Iceberg writes;
- the general session extension planner handles ordinary writes.

Both paths eventually call `create_file_write_physical_plan()` in `crates/sail-physical-plan/src/file_write.rs`.

That function:

1. maps `SinkMode` to `PhysicalSinkMode`,
2. creates physical sort requirements,
3. builds `SinkInfo`,
4. looks up the `TableFormat`,
5. calls `create_writer()`.

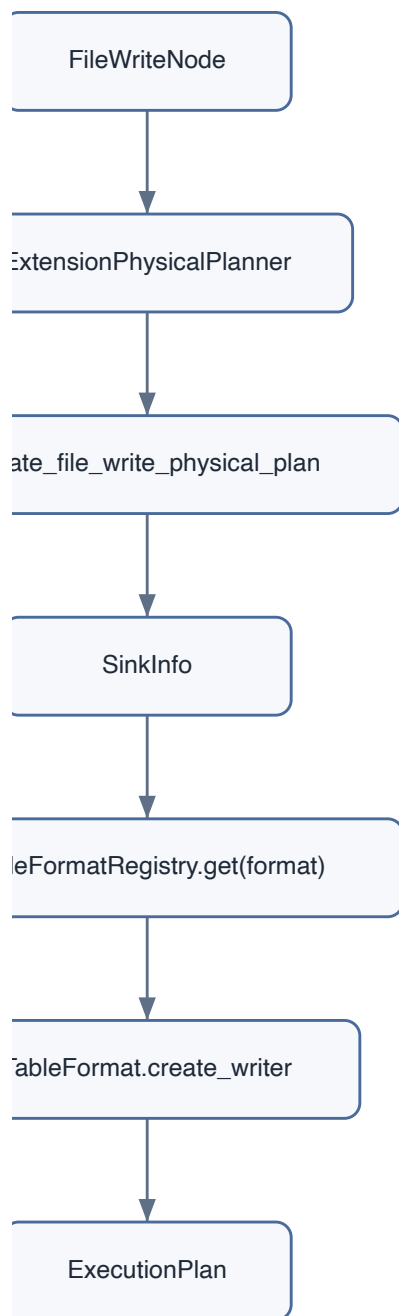


Figure 71: Flowchart 12.4

The storage writer is just another DataFusion physical plan node. In distributed execution, it can become part of the job graph like other physical operators.

13.15 Catalog Commands As Physical Plans

Catalog commands also become DataFusion plans.

The resolver wraps commands in `CatalogCommandNode`. The session planner converts that node into `CatalogCommandExec` in `crates/sail-physical-plan/src/catalog_command.rs`.

At execution time, `CatalogCommandExec`:

1. retrieves `CatalogManager` from the task context extension,
2. executes the command,
3. returns a single Arrow `RecordBatch`.

That design keeps commands inside the same query execution interface as scans and writes. A command can produce Spark-compatible tabular output, such as `SHOW TABLES` or `DESCRIBE TABLE`, without inventing a separate result transport.

13.16 Delta Lake

Delta implements `TableFormat` in `crates/sail-delta-lake/src/table_format.rs`.

For reads, `DeltaTableFormat`:

- parses the table path into a URL,
- resolves Delta read options,
- opens the Delta table through the object store registry,
- creates a Delta table source.

For writes, it:

- requires a path,
- rejects unsupported streaming writes,
- rejects unsupported bucketing,
- handles partition column validation,
- opens an existing table snapshot when present,
- resolves Delta write options and table properties,
- preserves generated column expressions from the logical schema,
- builds a `DeltaPhysicalPlanner`,
- returns the writer execution plan.

Delta also implements row-level writing:

```
async fn create_row_level_writer(  
    &self,  
    ctx: &dyn Session,  
    info: RowLevelWriteInfo,  
) -> Result<Arc<dyn ExecutionPlan>>
```

The row-level implementation chooses between eager copy-on-write and merge-on-read based on the requested command and detected table properties.

For example:

Command	Strategy	Delta planner path
DELETE	eager	plan_delete
DELETE	merge-on-read	plan_delete_mor
MERGE	eager	plan_merge
MERGE	merge-on-read	plan_merge_mor
UPDATE	not implemented yet	returns not implemented

This is a good example of why `TableFormat` cannot stop at “read files” and “write files.” Lakehouse formats own transaction logs, table protocols, deletion vectors, metadata actions, and row-level rewrite strategies.

13.17 Iceberg

Iceberg implements `TableFormat` in `crates/sail-iceberg/src/table_format.rs`.

For reads, it:

- parses the table URL,
- resolves Iceberg read options,
- loads table metadata,
- creates an `IcebergTableProvider`,
- wraps it in `IcebergTableSource`.

For writes, it:

- requires a path,
- rejects unsupported bucketing,
- resolves write options,
- checks whether metadata files already exist,
- validates partition spec compatibility,
- builds an `IcebergTableConfig`,
- uses `IcebergPlanBuilder` to create the execution plan.

Iceberg also has format-specific table alteration support. For example, `alter_table_properties()` updates Iceberg metadata files with conflict retry logic.

The file contains an explicit TODO for row-level DELETE/UPDATE/MERGE. That makes Iceberg a useful contrast with Delta: both are table formats, but their current row-level capabilities differ.

13.18 Lakehouse Extension Planners

Lakehouse tables need special physical planning. `crates/sail-plan-lakehouse/src/lib.rs` adds extension planners:

```
pub fn new_lakehouse_extension_planners() -> Vec<Arc<dyn ExtensionPlanner + Send
+ Sync>> {
    vec![
        Arc::new(sail_delta_lake::planner::DeltaTablePhysicalPlanner),
        Arc::new(sail_iceberg::IcebergTablePhysicalPlanner),
        Arc::new(DeltaExtensionPlanner),
    ]
}
```

The session query planner installs these before the general Sail extension planner. That gives lakehouse planners first chance to handle lakehouse-specific nodes.

DeltaExtensionPlanner handles:

- FileWriteNode for lakehouse formats,
- FileDeleteNode for lakehouse deletes that were not expanded,
- RowLevelWriteNode,
- MergeCardinalityCheckNode.

The row-level path looks like this:

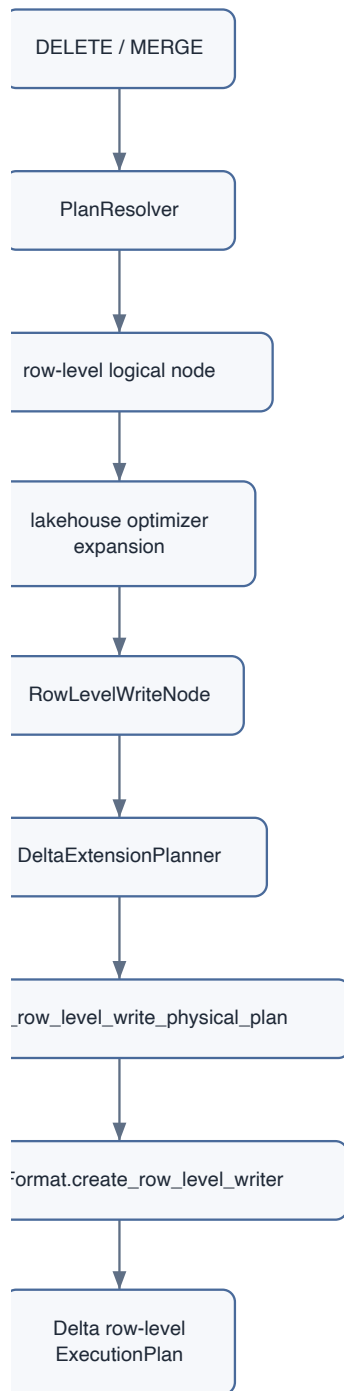


Figure 72: Flowchart 12.5

This is one of the places where distributed query processing and storage semantics meet. A MERGE is not just a local table operation. Sail must identify target files, join target rows with source rows, check cardinality, decide row operations, and then commit the result in the table format's transaction protocol.

13.19 Row-Level Metadata Columns

Sail reserves internal column names for row-level operations:

```
pub const MERGE_FILE_COLUMN: &str = "__sail_file_path";
pub const MERGE_ROW_INDEX_COLUMN: &str = "__sail_file_row_index";
pub const OPERATION_COLUMN: &str = "__sail_operation_type";
pub const MERGE_SOURCE_METRIC_COLUMN: &str = "__sail_merge_source_metric";
```

These columns are not user data. They are execution metadata used to track:

- which file a target row came from,
- which row inside that file was touched,
- what operation should happen to the row,
- source-side merge metrics.

MergeCapableSource exposes hooks for sources that can add file and row-index columns:

```
pub trait MergeCapableSource {
    fn file_column_name(&self) -> Option<&str>;
    fn with_file_column(self, name: Option<String>) -> Self;
    fn row_index_column_name(&self) -> Option<&str>;
    fn with_row_index_column(self, name: Option<String>) -> Self;
}
```

This is a tiny interface with large implications. A row-level command can only be planned safely if the scan can identify the physical rows or files that must be rewritten or deleted.

13.20 Python Data Sources

Python data sources are registered into the same TableFormatRegistry as Parquet, Delta, and Iceberg.

PythonTableFormat in `crates/sail-data-source/src/formats/python/table_format.rs` can represent:

- an entry-point discovered data source,
- a session-registered data source with embedded pickled class bytes.

For reads, it:

1. merges option layers into opaque Python options,
2. unpickles and instantiates the Python data source class,
3. obtains or discovers an Arrow schema,
4. builds a PythonTableProvider,
5. returns it as a DataFusion TableSource.

For writes, it:

1. maps Spark save modes to a Python overwrite boolean and a mode option,
2. asks the Python executor for a writer,
3. builds PythonDataSourceWriteExec,
4. wraps it in PythonDataSourceWriteCommitExec.

This is one of Sail's most concrete extension prototypes. A Python package can provide a data source, and Sail can expose it through ordinary Spark syntax:

```
df = spark.read.format("my_source").option("k", "v").load()
df.write.format("my_source").mode("overwrite").save()
```

The extension challenge is that Python data sources currently plug into one registry. Discussion #2001 asks for a broader version of that idea across DataFusion integrations.

13.21 Example: Parquet Read

A PySpark user writes:

```
df = spark.read.format("parquet").load("/tmp/orders")
df.select("order_id", "total").show()
```

Conceptually, Sail does this:

```
ReadDataSource(format = "parquet", paths = ["/tmp/orders"])
  -> SourceInfo
  -> TableFormatRegistry["parquet"]
  -> ListingTableFormat<ParquetFormatFactory>::create_source
  -> DataFusion ListingTable
  -> LogicalPlan::TableScan
```

The Arrow schema comes either from the user's explicit schema or from DataFusion's Parquet schema inference.

13.22 Example: Delta CTAS

A user writes:

```
CREATE TABLE lake.events
USING delta
LOCATION '/lake/events'
AS
SELECT * FROM raw_events
```

Sail needs two effects:

1. create catalog metadata for lake.events;
2. write the selected rows to a Delta table.

The logical shape is:

```
BarrierNode
  precondition: CatalogCommandNode(CreateTable)
  plan: FileWriteNode(format = "delta", mode = ErrorIfExists, path = "/lake/
events")
```


At physical planning time:

```
CatalogCommandNode -> CatalogCommandExec
FileWriteNode       -> DeltaTableFormat.create_writer(...)
BarrierNode         -> BarrierExec(preconditions, write_plan)
```

The barrier keeps the command sequencing explicit.

13.23 Example: Delta MERGE

A MERGE starts as a Spark command:

```
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET value = s.value
WHEN NOT MATCHED THEN INSERT *
```

For a lakehouse table, Sail's planner must do more than create a writer:

- resolve the target table,
- scan target rows with file path and row index metadata,
- join source and target rows,
- classify each row into an operation,
- check merge cardinality,
- hand row-level write info to the table format,
- let Delta commit the final file and log changes.

That path is why RowLevelWriteInfo carries so much context:

```
pub struct RowLevelWriteInfo {
  pub command: RowLevelCommand,
  pub target: RowLevelTargetInfo,
  pub condition: Option<ExprWithSource>,
  pub expanded_input: Option<Arc<dyn ExecutionPlan>>,
  pub touched_file_plan: Option<Arc<dyn ExecutionPlan>>,
  pub deletion_vector_plan: Option<Arc<dyn ExecutionPlan>>,
  pub with_schema_evolution: bool,
  pub operation_override: Option<RowLevelOperationType>,
  pub merge_strategy: MergeStrategy,
}
```

This is storage-aware distributed query processing. The query engine supplies the relational work; the table format supplies the commit protocol.

13.24 Extension Lessons

The catalog and table format code already demonstrates several extension principles that matter for the final chapter:

Principle	Storage example
Use session registries	TableFormatRegistry is installed as a session extension.
Keep traits small	TableFormat has focused read/write/row-level hooks.
Preserve layered semantics	OptionLayer avoids flattening every option too early.
Separate metadata from execution	CatalogManager resolves names; table formats build execution plans.
Let specialized planners intercept early	Lakehouse planners run before the general extension planner.
Carry distributed requirements explicitly	row-level columns and RowLevelWriteInfo encode what workers need.
Return DataFusion objects	sources and writers integrate with DataFusion rather than bypassing it.

For discussion #2001, this suggests a useful design direction:

```
pub trait SailExtension {
    fn register_table_formats(&self, registry: &TableFormatRegistry) ->
    Result<()> { ... }
    fn register_catalogs(&self, builder: &mut CatalogRegistryBuilder) ->
    Result<()> { ... }
    fn register_functions(&self, registry: &mut FunctionRegistry) -> Result<()>
    { ... }
    fn optimizer_rules(&self) -> Vec<Arc<dyn OptimizerRule>> { ... }
    fn physical_planners(&self) -> Vec<Arc<dyn ExtensionPlanner + Send + Sync>>
    { ... }
    fn codecs(&self) -> Vec<Arc<dyn PhysicalPlanCodecExtension>> { ... }
}
```

The exact API may differ, but the storage layer gives us the template: register capabilities into session-owned registries, keep DataFusion as the execution kernel, and make distributed serialization a first-class part of the contract.

13.25 Reading Exercises

- Trace `spark.read.format("parquet").load(path)` through `resolve_query_read_data_source()` and `ListingTableFormat::create_source()`.
- Trace a catalog table read through `CatalogManager.get_table_or_view()` and `resolve_query_read_named_table()`.
- Compare `DeltaTableFormat::create_writer()` and `IcebergTableFormat::create_writer()`. Note which checks are generic and which are table-format-specific.

4. Find where `FileWriteNode` becomes an `ExecutionPlan`.
5. Read `PythonTableFormat` as a miniature extension system.

13.26 Key Takeaways

Catalogs and table formats are the storage-facing half of Sail’s architecture. The catalog answers what a name means. The table format answers how to read or write the underlying data. The planner glues both to DataFusion.

Ordinary files mostly adapt DataFusion listing tables. Lakehouse formats add transaction logs, table protocols, schema evolution, generated columns, row-level metadata, and commit strategies. Python data sources prove that Sail can already discover third-party data providers and expose them through Spark syntax.

The final chapter will connect all of this back to the extension proposal: how to turn these local patterns into a coherent extension architecture for Sail.

14 Chapter 13: Extension Architecture: From Proposal To Design

The first twelve chapters treated Sail as a system to read. This final chapter treats it as a system to extend.

The extension proposal in discussion #2001 is titled “Extension API for third-party DataFusion integrations (UDFs, optimizer rules, planner extensions).” It starts from a practical problem: integrating a real DataFusion extension, such as Apache SedonaDB, currently requires editing Sail internals across multiple crates. A useful extension does not only add one function. It may add scalar functions, aggregate functions, table functions, session configuration, logical optimizer rules, physical planner extensions, physical operators, and distributed codec behavior.

That is the key lesson of the whole book. In Sail, an extension is not a plugin point. It is a path through the query engine.

```
client API
-> Spark Connect request
-> Sail spec
-> plan resolver
-> DataFusion logical plan
-> analyzer and optimizer rules
-> physical planner extension
-> physical optimizer rules
-> distributed codec
-> worker session
-> Arrow batch execution
```

If an extension works only on the driver, it is not a distributed extension. If it works only during planning, it is not an executable extension. If it works only in a custom Rust binary, it does not solve the `pip install pysail pysail-sedona` user experience that the proposal calls out.

This chapter proposes an architecture for Sail extensions by assembling the patterns we have already seen.

14.1 Code Map

The main files for this chapter are:

Concern	File
Proposal touch point: session mutator	<code>crates/sail-session/src/session_factory/server.rs</code>
Proposal touch point: query planner chain	<code>crates/sail-session/src/planner.rs</code>
Proposal touch point: logical optimizer list	<code>crates/sail-session/src/optimizer.rs</code>
Proposal touch point: scalar and table function maps	<code>crates/sail-plan/src/function/mod.rs</code>
Proposal touch point: aggregate function map	<code>crates/sail-plan/src/function/aggregate.rs</code>
Proposal touch point: window function map	<code>crates/sail-plan/src/function/window.rs</code>
Proposal touch point: distributed physical codec	<code>crates/sail-execution/src/codec.rs</code>
Existing extension pattern: session extensions	<code>crates/sail-common-datafusion/src/extension.rs</code>
Existing extension pattern: table format registry	<code>crates/sail-common-datafusion/src/datasource.rs</code>
Existing extension pattern: format registration	<code>crates/sail-session/src/formats.rs</code>
Existing extension pattern: Python data source discovery	<code>crates/sail-data-source/src/formats/python/discovery.rs</code>
Existing extension pattern: Python table format	<code>crates/sail-data-source/src/formats/python/table_format.rs</code>
Existing extension pattern: lakehouse planners	<code>crates/sail-plan-lakehouse/src/lib.rs</code>
Existing extension pattern: physical plan nodes	<code>crates/sail-physical-plan/src/</code>

14.2 What The Proposal Is Really Asking For

Discussion #2001 describes a third-party extension that needs all of these dimensions:

- scalar UDFs at plan time,
- aggregate UDAFs at plan time,

- window UDFs at plan time,
- generator and table functions,
- session configuration extensions,
- logical optimizer rules,
- physical optimizer rules,
- physical extension planners,
- distributed worker re-resolution of UDFs and UDAFs,
- Python entry-point discovery for pysail users.

SedonaDB is the motivating example. A spatial query may start as normal SQL:

```
SELECT *
FROM points p, polygons g
WHERE ST_Intersects(p.geom, g.geom)
```

Without an extension-aware planner, this can look like a cross join plus a filter. With the right functions and optimizer rules, it can become a spatial join:

```
CrossJoin + ST_Intersects filter
-> logical optimizer rule
-> SpatialJoinPlanNode
-> extension physical planner
-> SpatialJoinExec
```

That single improvement needs more than one hook. The function resolver must know `ST_Intersects`. The optimizer must know the function is spatial and join-like. The physical planner must know how to create `SpatialJoinExec`. The distributed codec must know how to serialize and deserialize any extension physical plan that reaches a worker.

The proposal's important insight is that these hooks should be registered together. The extension author should not need to chase every hardcoded map and planner list in the Sail workspace.

14.3 Two Boundaries, Not One

The list above looks like one extension surface. It is actually two.

plan-time boundary	execution-time boundary
client expresses intent	operators run on Arrow batches
Sail resolves it	workers re-resolve UDFs and plans
-> once per query	-> once per batch
-> stability matters	-> performance matters
-> performance does not	-> version coupling tolerable

These two boundaries pull a stable plugin ABI in opposite directions.

The plan-time boundary wants forward and backward wire compatibility, language neutrality, and a format that survives across years of Sail releases. A user writing `SELECT ST_Intersects(p.geom, g.geom)` should not care which DataFusion version the server was built against, or whether their extension was compiled with a different Rust toolchain than Sail itself.

The execution-time boundary wants zero-copy access to Arrow buffers, direct DataFusion ExecutionPlan integration, and native function dispatch. It cannot afford a protobuf round trip per record batch, and it has no realistic way to remain ABI-stable across major DataFusion upgrades without recompilation.

Discussion #2001 implicitly conflates these. A unified SailExtension trait is one way to register both, but the mechanism for *crossing* each boundary can be different. The recommended architecture in this chapter uses:

- a plan-time extension surface built on Spark Connect’s existing extension messages,
- an execution-time extension surface built on Sail traits that resolve to DataFusion FFI when packaged for distribution,
- one in-memory SailExtension object that registers contributions to both.

Some extensions only use one boundary. A library of ST_* scalar functions that decomposes into existing DataFusion expressions never needs execution-time integration. A custom physical operator like SpatialJoinExec needs both, because workers must reconstruct it from a wire format the codec understands.

The rest of the chapter develops each side. Sections from “The Core Trait” through “Physical Codec Extensions” cover the execution-time half. The section on “Spark Connect As The Plan-Time Extension Surface” covers the plan-time half. The “Versioning And ABI” section then explains why the two halves should carry different version stories.

14.4 Existing Patterns Worth Keeping

Sail already has several good extension shapes.

14.4.1 Session Extensions

Sail stores session-scoped services in DataFusion SessionConfig extensions. We saw this repeatedly:

```
.with_extension(create_table_format_registry()?)
.with_extension(Arc::new(create_catalog_manager(...)?))
.with_extension(Arc::new(ActivityTracker::new()))
.with_extension(Arc::new(JobService::new(job_runner)))
.with_extension(Arc::new(RepartitionBufferConfig::new(...)))
.with_extension(Arc::new(SystemTableService::new(...)))
.with_extension(Arc::new(DeltaTableCache::default()))
```

This is a strong pattern because downstream code can ask for typed services:

```
let registry = ctx.extension::<TableFormatRegistry>()?;
```

Extension APIs should lean into this rather than inventing a separate global plugin container.

14.4.2 Table Format Registry

TableFormatRegistry is a compact example of a capability registry:

```
registry.register(Arc::new(ParquetTableFormat::default()))?;
DeltaTableFormat::register(registry)?;
```

```
IcebergTableFormat::register(registry)?;
PythonTableFormat::register_all(registry)?;
```

The registry owns a map from name to `Arc<dyn TableFormat>`. That is the right shape for plugin-contributed capabilities:

- names are explicit,
- implementations are trait objects,
- registration happens during session construction,
- use happens through a typed lookup.

14.4.3 Python Entry-Point Discovery

Python data sources already demonstrate runtime discovery:

```
entry point group: pysail.datasources
-> importlib.metadata.entry_points(...)
-> load Python class
-> validate class
-> pickle class
-> register PythonTableFormat
```

This is not the same as native Rust extension discovery, but it proves an important user experience: a package can be installed into the Python environment and become available to Sail without editing Sail's source.

The proposed `pysail.extensions` group is a broader version of the same idea.

14.4.4 Lakehouse Planner Chain

Lakehouse planning already contributes physical planners:

```
vec![
  Arc::new(sail_delta_lake::planner::DeltaTablePhysicalPlanner),
  Arc::new(sail_iceberg::IcebergTablePhysicalPlanner),
  Arc::new(DeltaExtensionPlanner),
]
```

The session query planner combines these with the system table planner and the general Sail extension planner.

This tells us that extension planner ordering is not theoretical. It already matters. Lakehouse planners need a chance to handle lakehouse nodes before the fallback Sail planner handles ordinary extension nodes.

14.4.5 Spark Connect Extension Messages

Spark Connect's protobuf already has the hooks we need on the plan-time side. Three messages carry opaque `google.protobuf.Any` payloads:

<code>Relation.extension</code>	custom logical relation
<code>Command.extension</code>	custom session or catalog command
<code>Expression.extension</code>	custom expression or function call

These exist for the same reason Sail’s logical extension nodes exist: to let new operations enter the planner without changing the planner’s core types. Chapter 10 notes this in passing - “Spark Connect’s own extension guidance talks about extending the protocol through relation, expression, and command operation types” - and proposes that Sail’s spec layer could mirror that shape. This chapter takes that proposal as a first-class architecture decision.

Sail’s resolver is the natural dispatch point. Today the resolver converts every well-known Spark Connect relation into a `spec::QueryNode`. An Any payload could be dispatched to a registered handler keyed by the message’s `type_url`. The handler returns either a Sail spec node, a DataFusion logical plan, or a logical extension node that the rest of the pipeline already knows how to carry.

This pattern has properties the in-process trait does not have:

- the wire format is protobuf, so forward and backward compatibility follow standard proto rules,
- extension authors do not need to link any Sail crate, so a Python-only extension can emit Spark Connect messages and dispatch entirely on Sail’s side,
- the same extension proto can target any Spark-Connect-compatible engine, not only Sail,
- worker compatibility is reduced to “the worker can decode whatever the driver shipped”, which is the codec problem chapter 11 already covers.

What it does not solve is execution. Once the resolver has dispatched an Any payload into a logical extension node, the rest of the pipeline still needs the node, its physical equivalent, and its codec. Spark Connect is a plan-time channel, not an execution-time one.

The “Spark Connect As The Plan-Time Extension Surface” section later in this chapter develops the dispatcher design.

14.5 Current Gaps

The proposal identifies several hardcoded areas. Reading the current code confirms the shape of the gap.

Function registration is static:

```
pub static ref BUILT_IN_SCALAR_FUNCTIONS: HashMap<&'static str, ScalarFunction> =  
    HashMap::from_iter(scalar::list_built_in_scalar_functions());
```

Aggregate and window functions have similar built-in maps. That is fine for Sail’s own compatibility functions, but awkward for third-party functions.

Session mutation exists:

```
pub trait ServerSessionMutator: Send {  
    fn mutate_config(...)  
    fn mutate_state(...)  
    fn mutate_runtime_env(...)  
}
```

That is useful for embedders that build Sail as a library, but it does not provide a complete plugin system:

- it does not expose plan-time function registries,
- it does not expose codec fallback registries,
- it does not help `sail` CLI or `pysail` users discover installed extensions,
- it does not bundle all extension dimensions under one name.

The physical codec is also hardcoded. `RemoteExecutionCodec` knows how to encode and decode Sail's physical plan nodes and UDFs. That is necessary, but a third-party operator needs some way to participate in the same serialization path.

The planner chain has one more issue from the proposal: an extension planner that does not recognize a node should return `Ok(None)` so `DataFusion` can try the next planner. If Sail's fallback planner returns an internal error for every unknown node, then third-party planners must always be ordered before it or planning short-circuits with a confusing error.

That is a small mechanical fix, but it is also a design principle: extension chains must compose by declining work, not by failing on unfamiliar work.

Finally, Spark Connect's `Relation.extension`, `Command.extension`, and `Expression.extension` fields exist in the protocol but have no general dispatcher on Sail's side. Today an extension that wants to express a custom relation has to contribute Rust code in a Sail crate, because there is no other way for an opaque Any payload to reach a handler. That gap is what makes the plan-time boundary feel like the same problem as the execution-time boundary - it is not, but Sail does not yet have the dispatcher that would let them be solved separately.

14.6 Design Goal

The goal is not “allow plugins.” That phrase is too vague.

The goal is:

Let a third-party crate or Python package contribute a named, version-compatible set of query capabilities to Sail, and make those capabilities available consistently during planning, optimization, physical planning, distributed serialization, worker execution, and user-facing discovery.

That implies five requirements:

1. One extension object should describe all of its contributions.
2. Contributions should register into existing Sail and `DataFusion` registries.
3. Driver and worker sessions should load compatible extension sets.
4. Distributed plans should encode enough information for workers to rebuild extension functions and physical nodes.
5. Conflicts and ordering should be explicit.

14.7 The Core Trait

`SailExtension` is the in-process object that registers an extension's contributions. It is not the stable plugin ABI by itself - the plan-time boundary uses Spark Connect protobuf and the execu-

tion-time boundary uses DataFusion FFI when packaged across processes - but inside a single Sail server it is the one place an extension declares what it provides.

A reasonable first draft is:

```
pub trait SailExtension: Send + Sync {
    fn name(&self) -> &'static str;
    fn version(&self) -> Option<&'static str> { None }

    fn configure_session(&self, config: SessionConfig) -> Result<SessionConfig> {
        Ok(config)
    }

    // Plan-time boundary: Spark Connect protobuf dispatch.
    fn spark_connect_relations(&self)
        -> Vec<Arc<dyn SparkConnectRelationExtension>> { vec![] }
    fn spark_connect_commands(&self)
        -> Vec<Arc<dyn SparkConnectCommandExtension>> { vec![] }
    fn spark_connect_expressions(&self)
        -> Vec<Arc<dyn SparkConnectExpressionExtension>> { vec![] }

    // Execution-time boundary: DataFusion-shaped contributions.
    fn register_functions(&self, registry: &mut FunctionExtensionRegistry) ->
Result<()> {
        Ok(())
    }

    fn register_table_formats(&self, registry: &TableFormatRegistry) ->
Result<()> {
        Ok(())
    }

    fn register_catalogs(&self, registry: &mut CatalogExtensionRegistry) ->
Result<()> {
        Ok(())
    }

    fn analyzer_rules(&self) -> Vec<Arc<dyn AnalyzerRule + Send + Sync>> {
        vec![]
    }

    fn logical_optimizer_rules(&self) -> Vec<Arc<dyn OptimizerRule + Send +
Sync>> {
        vec![]
    }

    fn physical_optimizer_rules(&self) -> Vec<Arc<dyn PhysicalOptimizerRule +
Send + Sync>> {
        vec![]
    }
}
```

```

    fn extension_planners(&self) -> Vec<Arc<dyn ExtensionPlanner + Send + Sync>>
{
    vec![]
}

    fn physical_codecs(&self) -> Vec<Arc<dyn PhysicalPlanCodecExtension>> {
        vec![]
    }
}

```

This trait deliberately does not make every extension implement every concern. Most methods return empty contributions. A pure plan-time extension might only implement `spark_connect_relations` and resolve into existing DataFusion operators. A data source extension might only register table formats. A spatial extension might use both halves: Spark Connect dispatch for the user-visible DataFrame syntax, plus functions, optimizer rules, a planner, and a codec for execution. A catalog extension might register a catalog provider factory and nothing else.

14.8 Extension Registry

The session factory should not pass around loose vectors of everything. It should own one extension registry:

```

pub struct SailExtensionRegistry {
    extensions: Vec<Arc<dyn SailExtension>>,
}

```

It can expose typed collection methods:

```

impl SailExtensionRegistry {
    pub fn configure_session(&self, config: SessionConfig) ->
Result<SessionConfig>;
    pub fn spark_connect_dispatcher(&self) ->
Arc<SparkConnectExtensionDispatcher>;
    pub fn register_functions(&self, registry: &mut FunctionExtensionRegistry) ->
Result<()>;
    pub fn register_table_formats(&self, registry: &TableFormatRegistry) ->
Result<()>;
    pub fn analyzer_rules(&self) -> Vec<Arc<dyn AnalyzerRule + Send + Sync>>;
    pub fn logical_optimizer_rules(&self) -> Vec<Arc<dyn OptimizerRule + Send +
Sync>>;
    pub fn physical_optimizer_rules(&self) -> Vec<Arc<dyn PhysicalOptimizerRule +
Send + Sync>>;
    pub fn extension_planners(&self) -> Vec<Arc<dyn ExtensionPlanner + Send +
Sync>>;
    pub fn physical_codecs(&self) -> Vec<Arc<dyn PhysicalPlanCodecExtension>>;
}

```

The `SparkConnectExtensionDispatcher` is a `type_url`-indexed map of the plan-time handlers contributed by every loaded extension. The resolver calls it when it encounters a

Relation.extension, Command.extension, or Expression.extension payload; the section below details its shape.

The registry should itself be stored as a session extension so planning and codec code can reach it:

```
.with_extension(Arc::new(extension_registry.clone()))
```

The important thing is that the registry is not just a startup helper. It is a runtime service.

14.9 Function Registration

The current function maps are built-in and static. A future design can preserve fast built-in lookup while adding extension lookup.

One option is a layered function registry:

```
pub struct FunctionExtensionRegistry {
    scalar: HashMap<String, Arc<ScalarUDF>>,
    aggregate: HashMap<String, Arc<AggregateUDF>>,
    window: HashMap<String, Arc<WindowUDF>>,
    generators: HashMap<String, ScalarFunction>,
    table: HashMap<String, Arc<TableFunction>>,
}
```

Resolution order should be documented. I would choose:

```
temporary/session registered functions
-> extension functions
-> Sail built-ins
```

That makes explicit extension loading meaningful while preserving user-level session overrides. It also keeps Sail built-ins as the baseline.

Collision policy should be chosen at registration time, not hidden at lookup time. A useful default:

```
same extension registers same name twice: error
two extensions register same name: error unless an override policy is configured
extension overrides Sail built-in: allowed only with explicit override flag
session function overrides extension function: allowed, matching Spark-style
session behavior
```

The reason to avoid silent last-writer-wins is simple: distributed engines are hard enough to debug without guessing which implementation of ST_Distance ran on the worker.

14.10 Optimizer Rules

Extension optimizer rules need ordering.

The simplest design appends rules in extension registration order:

```
Sail required early rules
-> extension logical rules
-> Sail default logical rules
```

But the lakehouse optimizer already shows that some rules must run early. The `expand_row_level_op` rule is expected to run before built-in optimizers. A spatial extension may have the same need: merge a spatial predicate into a join before a general rule rewrites or pushes down expressions.

So the API should allow rule phases:

```
pub enum OptimizerPhase {
    BeforeSailDefaults,
    AfterSailDefaults,
    Final,
}

pub struct OrderedOptimizerRule {
    pub phase: OptimizerPhase,
    pub rule: Arc<dyn OptimizerRule + Send + Sync>,
}
```

For a first implementation, phase plus extension registration order is probably enough. More complex dependency constraints can wait until a real extension needs them.

14.11 Extension Physical Planners

DataFusion's extension planner chain is already the right mechanism:

```
DefaultPhysicalPlanner::with_extension_planners(extension_planners)
```

The design rule is:

```
extension planners
-> lakehouse planners
-> system table planner
-> Sail built-in extension planner
```

The exact placement of lakehouse planners is a policy decision. If lakehouse remains a built-in Sail capability, it can stay before third-party planners. If a third-party extension needs to override lakehouse behavior, that should require an explicit planner ordering option.

Every planner in the chain should follow the DataFusion convention:

```
if I recognize this node {
    Ok(Some(plan))
} else {
    Ok(None)
}
```

Only recognized but invalid nodes should produce errors.

14.12 Physical Codec Extensions

Distributed execution is where extension APIs often become too optimistic.

Sail serializes physical plans so workers can execute them. A physical extension node that exists only as an `Arc<dyn ExecutionPlan>` on the driver cannot magically cross the network.

The codec needs an extension registry:

```
pub trait PhysicalPlanCodecExtension: Send + Sync {
    fn name(&self) -> &'static str;

    fn try_encode(
        &self,
        node: Arc<dyn ExecutionPlan>,
        codec: &RemoteExecutionCodec,
    ) -> Result<Option<ExtensionPlanPayload>>;

    fn try_decode(
        &self,
        payload: &ExtensionPlanPayload,
        inputs: Vec<Arc<dyn ExecutionPlan>>,
        ctx: &SessionContext,
        codec: &RemoteExecutionCodec,
    ) -> Result<Option<Arc<dyn ExecutionPlan>>>;
}
```

The payload should include:

- extension name,
- extension version or ABI marker,
- node type name,
- serialized node bytes,
- child plan references or encoded child plans,
- optional schema and statistics metadata.

The worker must have a compatible extension loaded. If not, the error should be clear:

```
Cannot decode extension physical plan 'sedona::SpatialJoinExec'.
The worker session does not have extension 'sedona' version 'x.y.z' loaded.
```

The same problem applies to UDF re-resolution. Built-in functions can be rebuilt by name, but extension functions need their registry available on the worker.

14.13 Driver And Worker Symmetry

The extension registry must be available in both server sessions and worker sessions.

In local mode, that is easy. In cluster mode, it becomes a deployment contract:

```
driver process
  has extension registry A
  encodes plan requiring extension sedona
```

```
worker process
  must start with compatible extension registry A
  decodes sedona functions and physical nodes
```

This has operational consequences:

- Kubernetes worker images must include the same extension packages.
- Local-cluster workers must inherit the same extension loading configuration.
- Python extension discovery must happen for workers as well as the driver if Python extension code participates in execution.
- The plan codec should record required extension names so missing extensions fail before deep execution.

An extension manifest can help. A minimum useful form carries the extension identity, the plan-time `type_url` claims, and an optional native execution surface:

```
pub struct ExtensionManifest {
    pub name: String,
    pub version: String,
    pub spark_connect_relations: Vec<String>,    // type_urls
    pub spark_connect_commands: Vec<String>,
    pub spark_connect_expressions: Vec<String>,
    pub execution_surface: Option<ExecutionSurface>,
}
```

The “Versioning And ABI” section below details `ExecutionSurface` and explains why the two halves carry different version rules. The manifest is not only documentation. It is runtime compatibility data, and the driver should refuse to dispatch a plan that requires an extension manifest the workers do not also hold.

14.14 Python Entry-Point Discovery

The proposal’s Python story is:

```
[project.entry-points."pysail.extensions"]
sedona = "pysail_sedona:register"
```

At startup, `pysail` discovers installed entry points:

```
from importlib.metadata import entry_points

for ep in entry_points(group="pysail.extensions"):
    extension = ep.load()()
    register(extension)
```

The hard part is not Python discovery. `Sail` already does that for data sources. The hard part is what crosses the Python/Rust boundary.

If `register()` returns a Rust-backed extension object through `PyO3`, then the native types crossing the boundary are version-coupled:

- `Arc<dyn SailExtension>`,
- `DataFusion` UDF types,
- optimizer rule traits,
- physical planner traits,
- `Arrow` and `DataFusion` schema and expression types.

Rust has no stable ABI for arbitrary trait objects. That means `pysail-sedona` must be built against compatible versions of `pysail`, `Sail`, `DataFusion`, `Arrow`, and `PyO3`. This is acceptable if it is documented and enforced. It is dangerous if users only discover it through crashes.

A practical Python extension loading flow:

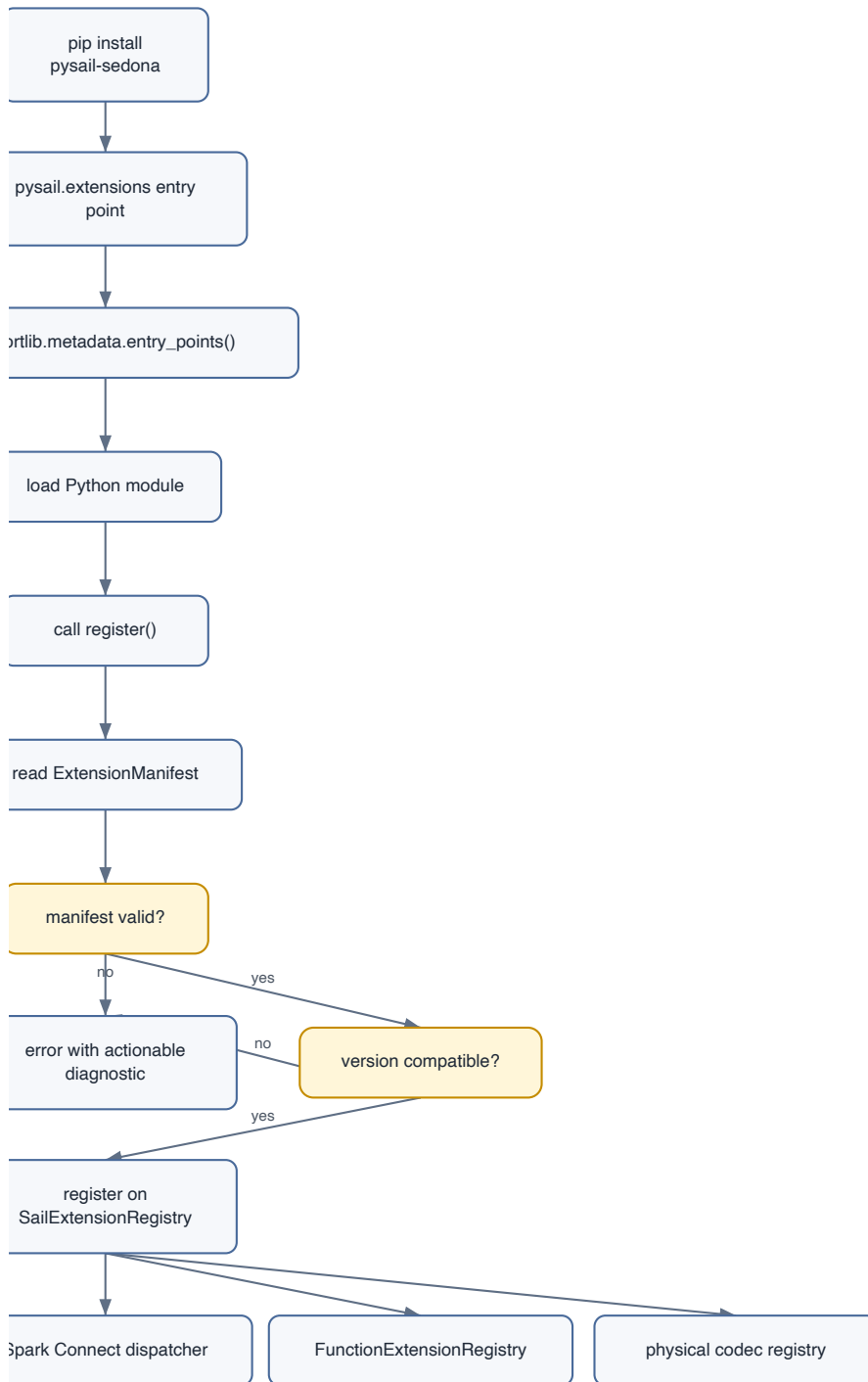


Figure 73: Flowchart 13.1

The extension loader should validate manifests before registering capabilities.

14.15 Spark Connect As The Plan-Time Extension Surface

This section develops the plan-time half of the two boundaries.

The plan-time channel needs to be stable across DataFusion and Arrow upgrades, hospitable to cross-language authors, and free of recompilation pressure when Sail releases a new version. Spark Connect already meets those needs.

Three facts make it the right channel. Every query already crosses it: PySpark, SQL, and DataFrame calls all serialize through Spark Connect protobuf or its companion messages. Protobuf is forward and backward compatible by construction, including for unknown fields, so an extension protocol bumped from v1 to v2 does not break older Sail servers in unrelated ways. And Spark Connect already defines extension hooks - `Relation.extension`, `Command.extension`, and `Expression.extension`, each typed as `google.protobuf.Any` - so Sail does not need new wire surface area, only a dispatcher.

14.15.1 Dispatcher Traits

A plan-time extension dispatcher has a small interface:

```
pub trait SparkConnectRelationExtension: Send + Sync {
    fn type_url(&self) -> &'static str;
    fn resolve(
        &self,
        payload: &[u8],
        ctx: &ResolverContext,
        inputs: Vec<spec::QueryNode>,
    ) -> PlanResult<spec::QueryNode>;
}

pub trait SparkConnectExpressionExtension: Send + Sync {
    fn type_url(&self) -> &'static str;
    fn resolve(
        &self,
        payload: &[u8],
        ctx: &ResolverContext,
        arguments: Vec<spec::Expression>,
    ) -> PlanResult<spec::Expression>;
}

pub trait SparkConnectCommandExtension: Send + Sync {
    fn type_url(&self) -> &'static str;
    fn resolve(
        &self,
        payload: &[u8],
        ctx: &ResolverContext,
    ) -> PlanResult<spec::CommandNode>;
}
```

Each handler claims a `type_url`. The resolver routes by `type_url` and falls through with a clear error when no handler matches. The error should name the missing extension so users diagnose installation problems before they diagnose plan errors:

No extension handler is registered for Spark Connect `Relation.extension` with `type_url` `'apache.sedona/SpatialJoin'`. Install `pysail-sedona`, or check that `'sedona'` appears in `sail.extensions.enabled`.

14.15.2 Two Resolution Patterns

There are two useful patterns for what a handler returns.

Pattern A decomposes the extension call into existing relations, expressions, and commands:

```
Relation.extension("example.com/JsonScan")
-> JsonScanExtension::resolve(...)
-> spec::QueryNode::Read { format: "json", path: ... }
-> normal Sail and DataFusion execution
```

The rest of Sail does not know an extension was involved. Pattern A extensions need no execution-time integration at all. They are pure plan-time additions. A surprising amount of useful behavior fits here: configurable readers, custom SQL-shaped commands, expression sugar that expands into well-known DataFusion expressions.

Pattern B emits a logical extension node and hands off to the execution-time half of the extension:

```
Relation.extension("apache.sedona/SpatialJoin")
-> SpatialJoinExtension::resolve(...)
-> spec::QueryNode::Extension { node: SpatialJoinNode { ... } }
-> Sedona optimizer rules
-> SpatialJoinExec
-> Sedona codec
```

Pattern B is what discussion #2001 implicitly assumed for everything. Pattern A is what makes Spark Connect dispatch worth its own ABI.

14.15.3 A Sketch In Python

A plan-time extension does not require a recompiled Sail server. From the PySpark side it looks like an ordinary client library that emits Spark Connect messages:

```
from pyspark.sql.connect.proto import Relation
from google.protobuf.any_pb2 import Any as AnyMessage
from pysail_geo_proto import JsonScan # extension's own proto

def json_scan(spark, path, schema=None):
    request = JsonScan(path=path, schema=schema or "")
    relation = Relation(extension=AnyMessage(
        type_url="example.com/pysail_geo/JsonScan",
        value=request.SerializeToString(),
    ))
    return spark._client._dataframe(relation)
```

The Sail server registers a corresponding handler when its extension loads. The two never share a Rust ABI. The protobuf is the only thing they agree on.

14.15.4 Federation Through Spark Connect

Spark Connect is also a complete query protocol, not only an extension channel. That makes a second pattern available.

A Sail driver can become a Spark Connect client of a separate extension server and delegate a subtree of a query. The extension server returns Arrow batches over the existing protocol. The two processes do not share memory, do not load each other's code, and do not need to agree on DataFusion versions:

```
driver Sail
  resolver dispatches Relation.extension
  -> spec::QueryNode::Extension { delegate_to: "spark://server:port" }
  -> physical plan with FederatedExec
  FederatedExec
    -> Spark Connect client to extension server
    -> stream Arrow batches back
```

This pattern is the strongest answer for execution-time isolation. It is also the most expensive: an extra network hop, an extra deployment, and a place to lose batches if the extension server crashes. It is appropriate for extensions that are themselves complex services - catalog metastores, vector search engines, production geospatial systems. It is overkill for a stateless scalar UDF.

14.15.5 Discovery And Registration

Plan-time extensions still need to register. The discovery mechanisms from earlier in this chapter apply directly:

```
configured or pip-discovered extensions
  -> SailExtension::spark_connect_relations()
  -> SailExtensionRegistry::spark_connect_dispatcher()
  -> resolver looks up type_url
```

A Python wheel-based extension can register a Python callback through PySpark and a server-side handler through `pysail.extensions` discovery. The Rust side of the handler can live in an extension's own crate, in a sidecar service, or in a federated Spark Connect server. The dispatch point is the same.

`type_url` is the new collision namespace and should follow the same strict policy as function names: two extensions claiming the same `type_url` is an error at registration, not a silent override at dispatch.

14.15.6 What This Buys And Costs

Compared to a pure Rust-trait surface, Spark Connect dispatch trades some ergonomics for substantial decoupling.

Costs:

- each extension call serializes through protobuf,
- handlers must hand-write decoding for their Any payloads,
- logical extension nodes that need to participate in optimization still require execution-time integration,
- `type_url` becomes a new namespace where extensions can collide.

In return:

- no Rust ABI for plan-time-only extensions,
- no PyO3 version coupling for Python extensions,
- a natural cross-language story, including non-Rust authors,
- an explicit federation pattern that the Rust trait cannot express,
- a wire format that already passes through every Sail query.

The right framing is not “Spark Connect *or* the trait.” It is “Spark Connect for the plan-time boundary, the trait for the execution-time boundary, and one manifest for both.”

14.16 A Sedona-Style Worked Design

Imagine a `sail-sedona` crate. It uses both boundaries: Spark Connect dispatch for the user-visible spatial `DataFrame` syntax, plus the full execution-time suite for the spatial join itself.

```
pub struct SedonaExtension {
    options: SedonaOptions,
}

impl SailExtension for SedonaExtension {
    fn name(&self) -> &'static str {
        "sedona"
    }

    fn configure_session(&self, config: SessionConfig) -> Result<SessionConfig> {
        Ok(config.with_extension(Arc::new(self.options.clone())))
    }

    // Plan-time boundary.
    fn spark_connect_relations(&self) -> Vec<Arc<dyn
SparkConnectRelationExtension>> {
        vec![Arc::new(SpatialJoinRelationHandler::new())]
    }

    fn spark_connect_expressions(&self) -> Vec<Arc<dyn
SparkConnectExpressionExtension>> {
        vec![
            Arc::new(StIntersectsExpressionHandler::new()),
            Arc::new(StDistanceExpressionHandler::new()),
        ]
    }

    // Execution-time boundary.
```

```

    fn register_functions(&self, registry: &mut FunctionExtensionRegistry) ->
Result<()> {
    registry.register_scalar("st_intersects", st_intersects_udf()?);
    registry.register_scalar("st_distance", st_distance_udf()?);
    registry.register_aggregate("st_union_aggr", st_union_aggr_udf()?);
    Ok(())
}

    fn logical_optimizer_rules(&self) -> Vec<Arc<dyn OptimizerRule + Send +
Sync>> {
    vec![Arc::new(MergeSpatialPredicateIntoJoin::new())]
}

    fn extension_planners(&self) -> Vec<Arc<dyn ExtensionPlanner + Send + Sync>>
{
    vec![Arc::new(SpatialJoinExtensionPlanner::new())]
}

    fn physical_codecs(&self) -> Vec<Arc<dyn PhysicalPlanCodecExtension>> {
    vec![Arc::new(SedonaPhysicalCodec::new())]
}
}

```

A query then flows like this:

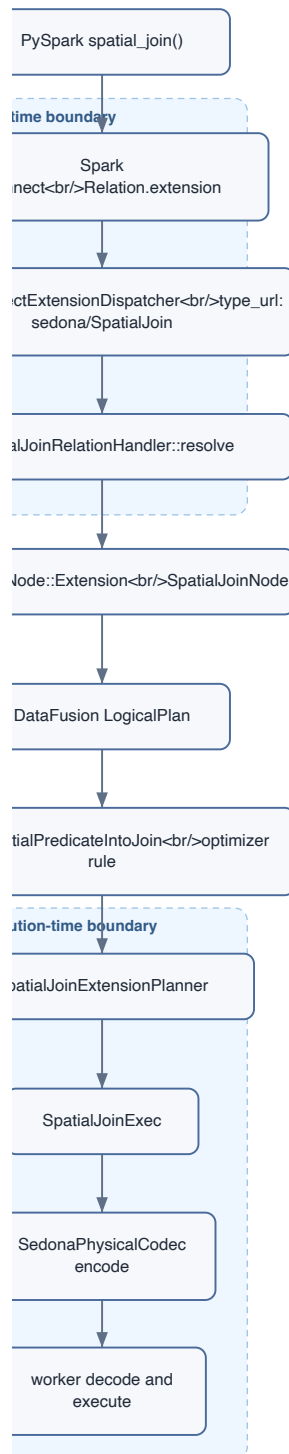


Figure 74: Flowchart 13.2

The two halves are visible. The plan-time half lets PySpark users call a `spatial_join(...)` `DataFrame` method that emits a `Relation.extension` payload; that payload becomes a `SpatialJoinNode` in the Sail spec. The execution-time half plans that node into `SpatialJoinExec` and encodes it for workers. Notice how much of Sail still does not need to know Sedona exists. It needs to know how to ask the Spark Connect dispatcher, function registries, and planner chains for contributions.

14.17 Table Formats As Extensions

Chapter 12 showed that table formats are already close to this model. A third-party format extension could do:

```
impl SailExtension for MyLakeExtension {
  fn register_table_formats(&self, registry: &TableFormatRegistry) ->
  Result<()> {
    registry.register(Arc::new(MyLakeTableFormat::new()))?;
    Ok(())
  }

  fn extension_planners(&self) -> Vec<Arc<dyn ExtensionPlanner + Send + Sync>>
  {
    vec![Arc::new(MyLakeRowLevelPlanner::new())]
  }

  fn physical_codecs(&self) -> Vec<Arc<dyn PhysicalPlanCodecExtension>> {
    vec![Arc::new(MyLakeCodec::new())]
  }
}
```

That would let a user write:

```
CREATE TABLE t USING mylake LOCATION '/warehouse/t' AS SELECT * FROM source;
```

The same extension might also register row-level commands, custom optimizer rules, or catalog provider factories.

14.18 Catalog Providers As Extensions

Catalogs should also be extension candidates.

The current session catalog construction selects providers from `AppConfig`. A future extension-aware design could let extensions register catalog provider factories:

```
pub trait CatalogProviderFactory: Send + Sync {
  fn catalog_type(&self) -> &'static str;
  fn create(
    &self,
    name: &str,
    properties: HashMap<String, String>,
    runtime: RuntimeHandle,
  ) -> Result<Arc<dyn CatalogProvider>>;
}
```


Configuration could then say:

```
[[catalog.list]]
type = "custom-rest"
name = "prod"
uri = "https://catalog.example"
```

The catalog manager does not need to know the concrete type. It only needs a factory that can produce an `Arc<dyn CatalogProvider>`.

14.19 Conflict Policy

The proposal lists open questions around collisions. A good default policy is strict:

Collision	Default behavior
Two extensions with same name	error
Two extensions register same function name	error
Extension function shadows built-in	error unless explicit override
Session function shadows extension function	allowed
Two planners claim same logical node	first successful planner wins, but log planner name
Two Spark Connect handlers claim same type_url	error
Two entry points share same name	error
Same SessionConfig extension type inserted twice	error unless explicit replace
Same table format name registered twice	error unless explicit override

Strict defaults make early development noisier and production behavior clearer. An administrator can always choose a permissive override mode later.

14.20 Enablement Policy

Extensions need two levels of enablement:

```
installed
-> available to Sail process
```

```
enabled
-> active in a session
```

For a server binary, enabled extensions may come from configuration:

```
[extensions]
enabled = ["sedona", "my-lake"]
```

For `pysail`, entry-point discovery can make installed extensions available, but the session should still know which ones are active. A useful SQL-level control might be:

```
SET sail.extensions.enabled = 'sedona,my-lake'
```

Per-session enablement matters because extensions can change planning behavior. A spatial optimizer rule, for example, may rewrite joins. Users should be able to run a session with it disabled for debugging.

14.21 Versioning And ABI

The two boundaries deserve two version stories.

Plan-time extensions inherit `protobuf`'s compatibility properties. An extension that only registers Spark Connect handlers needs no native ABI coupling at all, and Sail can load it against any compatible server build. The relevant version data is:

```
extension name
extension version
declared Spark Connect type_urls
Sail Spark Connect API version
```

Execution-time extensions must couple more tightly. A custom `ExecutionPlan` is linked against specific `DataFusion`, `Arrow`, and (for Python plugins) `PyO3` versions, and Rust has no stable ABI for arbitrary trait objects:

```
Sail extension API version
DataFusion version
Arrow version
PyO3 version, for Python-native plugins
declared codec type names
```

The `ExtensionManifest` introduced earlier in “Driver And Worker Symmetry” combines both. Its `execution_surface` field carries the native-ABI version data:

```
pub struct ExecutionSurface {
    pub sail_api_version: String,
    pub datafusion_version: String,
    pub arrow_version: String,
    pub pyo3_version: Option<String>,
    pub capabilities: Vec<ExtensionCapability>,
}
```

When `execution_surface` is `None`, the extension is plan-time only. The strict native ABI checks do not apply, the extension can be installed against any compatible Sail server, and it cannot contribute custom physical operators. When `execution_surface` is set, the loader performs exact-version or narrow-range checks before registering anything.

The loader should fail fast on incompatible versions and report which surface failed. A friendly error here is worth more than a mysterious decode failure later:

Cannot load extension 'sedona' 1.2.0.
Plan-time Spark Connect type_urls: OK.
Execution-time DataFusion version mismatch: extension built against 44.0.0, this Sail server uses 45.1.0. Install a sedona build matching the server, or use sail-sedona's federated mode.

For pure Python data sources, the compatibility story can be looser because Sail stores pickled Python classes and calls a defined data source interface. For native Rust DataFusion integrations, strict coupling is the honest answer. The two-boundary design makes that honesty selective rather than universal.

14.22 Security And Trust

Extensions are code. They can:

- execute Rust native code,
- execute Python code,
- access object stores through the runtime,
- inspect query plans,
- influence optimizer choices,
- run on distributed workers.

So extension loading should be treated like loading a database plugin or Spark jar, not like reading a harmless config file.

Practical policies:

- load extensions only from configured sources,
- log every loaded extension and version,
- expose loaded extensions through a system table,
- support disabling extension discovery in production,
- require explicit enablement for native extensions,
- make worker extension mismatches visible in job errors.

14.23 System Tables For Observability

Sail already has a system catalog. Extensions should be visible there.

Useful system tables:

```
system.extensions  
system.extension_functions  
system.extension_table_formats  
system.extension_planners  
system.extension_codecs
```

Example rows:

extension	capability	name	version	enabled
sedona	scalar_udf	st_intersects	1.0.0	true
sedona	planner	spatial_join	1.0.0	true
my-lake	table_format	mylake	0.3.0	true

This makes extension behavior inspectable from Spark SQL.

14.24 Implementation Roadmap

A safe implementation can land in stages.

14.24.1 Stage 1: Planner Composition Fix

Change the fallback extension planner so unknown nodes return `Ok(None)` instead of an internal error. Recognized but invalid Sail nodes should still error.

This is small, low-risk, and unblocks third-party planners in the chain.

14.24.2 Stage 2: Native `SailExtension` Registry

Introduce:

- `SailExtension`,
- `SailExtensionRegistry`,
- `ServerSessionFactory.with_extension(...)`,
- session extension storage for the registry.

Wire extension contributions into:

- session config,
- analyzer rules,
- logical optimizer rules,
- physical optimizer rules,
- extension physical planner chain,
- table format registry.

14.24.3 Stage 3: Spark Connect Extension Dispatch

Introduce the plan-time half of the boundary:

- `SparkConnectRelationExtension`,
- `SparkConnectCommandExtension`,
- `SparkConnectExpressionExtension`,
- `SparkConnectExtensionDispatcher` indexed by `type_url`,
- resolver routing for `Relation.extension`, `Command.extension`, and `Expression.extension` payloads,
- collision policy for `type_url` claims,
- diagnostic errors that name missing extensions, not missing message types.

This stage unblocks pattern-A extensions (plan-time decomposition) without requiring any of the execution-time work that follows. A toy “JsonScan” extension is enough to prove the dispatch path.

14.24.4 Stage 4: Function Extension Registry

Replace static-only function lookup with layered lookup:

```
catalog/session functions
  -> extension functions
  -> built-in functions
```

Add collision policy and tests for scalar, aggregate, window, generator, and table function registration.

14.24.5 Stage 5: Distributed Codec Registry

Add codec hooks for:

- extension physical plans,
- extension UDF and UDAF re-resolution,
- required extension manifest data in encoded plans.

Test with local-cluster execution, not only local execution. This is the execution-time half of the boundary; Stage 3 is the plan-time half.

14.24.6 Stage 6: Python Entry-Point Discovery

Add pysail.extensions discovery:

```
discover entry point
  -> load module
  -> call register()
  -> validate manifest
  -> add SailExtension
```

Prototype the risky part early: passing a native `Arc<dyn SailExtension>` across independently built PyO3 modules. Plan-time-only extensions can skip this risk because they cross the boundary as protobuf, not as Rust trait objects.

14.24.7 Stage 7: Observability And Operations

Add:

- system tables,
- startup logs,
- worker compatibility checks,
- per-session enable/disable,
- config controls for discovery.

14.25 Tests An Extension API Needs

The test suite should include an intentionally tiny extension crate. It does not need to be useful. It needs to exercise every hook.

Minimum test extension:

- Spark Connect `Relation.extension` handler that resolves to an existing DataFusion `MemoryExec` (pattern A),
- Spark Connect `Expression.extension` handler that rewrites to an existing scalar expression,
- Spark Connect `Relation.extension` handler that produces a logical extension node (pattern B),
- scalar UDF: `ext_add_one(x)`,
- aggregate UDAF: `ext_count_non_null(x)`,
- optimizer rule: rewrite a marker expression,
- logical extension node,
- physical planner producing a custom exec,
- codec for that custom exec,
- table format that reads a tiny in-memory or local file source,
- Python entry-point wrapper, if possible.

Test matrix:

Mode	What to verify
local, pattern A	Spark Connect dispatch resolves to existing DataFusion operators, no execution-side extension involved
local, pattern B	Spark Connect dispatch produces a logical extension node, physical planner and codec take over
local	function resolution, optimizer rewrite, physical planning
local cluster	codec encode/decode, worker function re-resolution
disabled extension	functions, planners, and Spark Connect handlers unavailable
collision	configured collision policy is enforced for both function names and <code>type_url</code> claims
missing worker extension	clear distributed error naming the extension
missing dispatcher	clear plan-time error naming the missing <code>type_url</code>
Python discovery	installed package registers extension on both boundaries

An extension API without distributed tests will look done before it is done.

14.26 A Mental Model For Future Contributors

When adding a new extension dimension, ask six questions:

1. Which boundary does it cross - plan-time, execution-time, or both?
2. Where is the capability registered?
3. Where is it used during planning?
4. Does it affect optimizer ordering?
5. Does it create physical objects that must cross the network?
6. How will a worker rebuild or execute it?

If the answer to question 1 is “plan-time only,” the extension can ride on Spark Connect dispatch and inherits protobuf compatibility. If the answer to question 5 is yes, the extension is part of the distributed execution contract. It needs codec support, version-matched workers, or a deliberate reason it can only run in local mode.

14.27 How The Previous Chapters Fit Together

Here is the whole book reduced to one extension-oriented map:

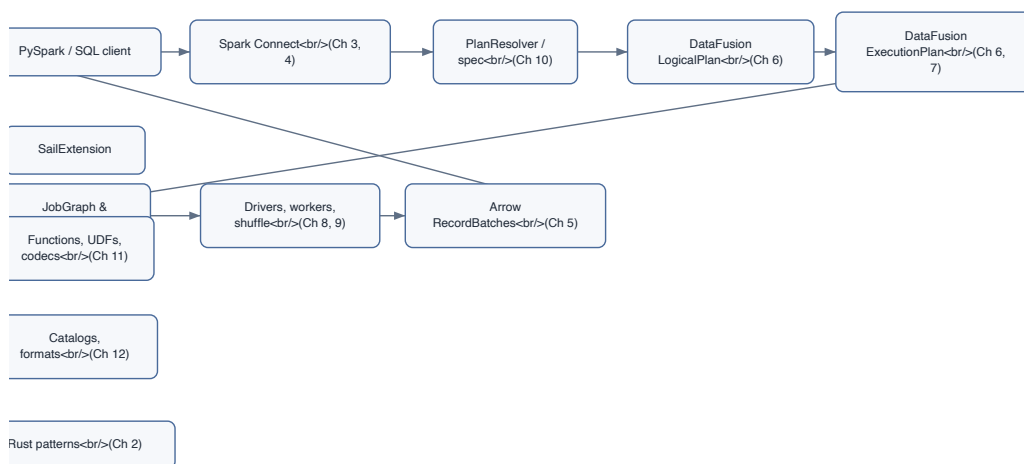


Figure 75: Flowchart 13.3

An extension architecture succeeds when both extension surfaces - plan-time through Spark Connect dispatch and execution-time through `SailExtension` contributions - are explicit, typed, ordered, versioned, and available consistently across driver and workers. The two surfaces share one `SailExtension` registration object, one manifest, and one observability story, but they cross the wire by different mechanisms because their stability requirements are different.

14.28 Final Takeaways

Rust gives Sail the tools for the execution-time half of this design: traits, Arc, typed session extensions, and explicit error handling. Arrow gives extensions a shared memory and schema

model. DataFusion gives them logical plans, optimizer rules, physical planners, execution plans, and UDF traits. Spark Connect gives them something the other layers cannot give: a wire-format ABI that is forward and backward compatible by construction, language-neutral, and already crossing every query.

The architecture of Sail is already close to an extension-friendly shape. The table format registry, Python data source discovery, session extensions, lakehouse planner chain, and physical codec all show pieces of the execution-time answer. `Relation.extension`, `Command.extension`, and `Expression.extension` provide the plan-time answer once Sail adds a dispatcher. Discussion #2001 asks Sail to make those pieces first-class and composable.

The final design principle is simple:

A Sail extension should be loaded once, registered clearly through one object, expressed at the plan-time boundary as a stable protobuf, executed at the execution-time boundary as native code, serialized explicitly between them, available on driver and workers, and observable from the session.

That is the difference between a convenient local hook and a real distributed query engine extension API.

Navigation: Previous: [Chapter 12, Catalogs, Lakehouse Tables, And File Formats](#) | [Reader Guide](#)